free study notes

Phase 5 Architecture, Reliability, Security & Observability

76 lessons · notes, diagrams and 456 interview questions

1 · Watch
Scan the QR code on a lesson to open its video.

2 · Read
Notes go deeper than the video: worked examples, trade-offs, mistakes.

3 · Practise
Answer the interview questions out loud before reading the answers.

Reliability and resilience

1 · Availability Calculations
2 · SLIs, SLOs and SLAs
3 · Error Budgets
4 · Timeouts
5 · Retries and Backoff
6 · Circuit Breaker
7 · Bulkhead Pattern
8 · Graceful Degradation
9 · Health Checks and Self-Healing
10 · Redundancy and Replication
11 · Failover Mechanisms
12 · Disaster Recovery
13 · RTO and RPO
14 · Backups and Restore Testing
15 · Multi-Region Architecture
16 · Cell-Based Architecture
17 · Chaos Engineering
18 · Static Stability

Architecture and microservices

19 · Monolith vs Microservices
20 · Modular Monolith
21 · Domain-Driven Design Basics
22 · Bounded Contexts
23 · Database per Service
24 · Sync vs Async Communication
25 · API Gateway Pattern
26 · Backend for Frontend
27 · Service Mesh
28 · Sidecar and Ambassador
29 · Strangler Fig Pattern
30 · Anti-Corruption Layer
31 · Hexagonal Architecture
32 · Clean Architecture
33 · Serverless
34 · Saga Orchestration
35 · CQRS and Event Sourcing in Microservices
36 · Distributed Tracing in Microservices
37 · Microservice Anti-Patterns
38 · Configuration Management

Security

39 · Authentication vs Authorization
40 · Session-Based Authentication
41 · JWT Authentication
42 · OAuth 2.0
43 · OpenID Connect and SSO
44 · RBAC and ABAC
45 · Service-to-Service Auth
46 · Encryption in Transit
47 · Encryption at Rest and KMS
48 · Secrets Management
49 · Hashing Passwords
50 · OWASP Top 10
51 · OWASP API Security
52 · Abuse Prevention
53 · Least Privilege and Zero Trust
54 · Threat Modeling
55 · Audit Logging
56 · Data Privacy and Retention
57 · Multi-Tenant Isolation
58 · Supply Chain Security

Observability and delivery

59 · Observability Fundamentals
60 · Structured Logging
61 · Metrics Types and Design
62 · Prometheus and Grafana
63 · Distributed Tracing
64 · OpenTelemetry
65 · Correlation and Trace IDs
66 · Log Aggregation
67 · Dashboards and Alerting
68 · Health Endpoints
69 · Root-Cause Analysis
70 · Incident Management
71 · Capacity Planning
72 · CI/CD Pipelines
73 · Rolling Deployments
74 · Blue-Green Deployments
75 · Canary Deployments
76 · Feature Flags

Free to share. Videos: youtube.com/@tapankumarbarik9454 · Notes and practice: backendroadmap.com

Availability Calculations

series multiplies, parallel adds nines

Watch the 5-min video

In one line: components in series multiply their availabilities, so every hard dependency lowers yours; redundant components in parallel fail only when all fail, so availability becomes $1 - (1 - A)^n$ — as long as the failures are really independent.

1 A promise you can't keep

Your service promises **99.99%**. It calls five services, each at **99.9%**. Even if your own code never fails, $0.999^5 \approx$ **99.5%**: about 44 hours of downtime a year against a promise of 53 minutes. You can't be more available than the things you depend on, unless you add redundancy or remove the dependency from the critical path.

2 What the nines mean

Availability	Downtime per year	Per month	Typical for
99% (two nines)	3.65 days	7.2 hours	internal tools, batch jobs
99.9% (three nines)	8.8 hours	43 minutes	most web products
99.99% (four nines)	53 minutes	4.3 minutes	payments, core APIs
99.999% (five nines)	5.3 minutes	26 seconds	telecom, critical infrastructure

Availability = uptime ÷ total time. Each extra nine cuts allowed downtime by 10×, and usually costs far more than 10× the engineering.

3 The two formulas

$$\text{series: } A = A_1 \times A_2 \times \dots \times A_n \quad \cdot \quad \text{parallel: } A = 1 - (1 - A)^n$$



Figure 1. Series dependencies multiply; parallel replicas multiply only the **failure** probabilities.

Series means a request needs every component, so you multiply. **Parallel** means one healthy copy is enough: the system is down only when all copies are down together, so you multiply the failure

probabilities (1 − A) and subtract from 1. Two 99% replicas give $1 - 0.01^2 = 99.99\%$. Redundancy adds nines; dependencies remove them.

WORKED EXAMPLE: THREE REPLICAS AT 99.9%

Each replica is down with probability 0.001. All three are down together with probability $0.001^3 = 10^{-9}$, so the replica tier is theoretically **99.9999999%** (nine nines). Put a 99.99% load balancer in front and the path is $0.9999 \times 0.999999999 \approx$ **99.99%**: the load balancer now sets the ceiling. And one bad deploy that hits all three replicas at once makes the real number far lower than nine nines.

4 Why real numbers are lower



Figure 2. The parallel formula assumes independent failures. These break that assumption.

- **Correlated failure:** replicas in one zone, built from one image, or changed by one deploy fail together.
- **Ceilings:** a hard dependency in series caps you at its availability, however many replicas you add elsewhere.
- **Recovery time:** downtime ≈ how often you fail × how long you take to recover (MTTR). Faster detection and rollback raise availability too.

HOW TO RAISE AVAILABILITY

- Remove dependencies from the critical path: make them asynchronous or optional.
- Add redundancy across **independent** failure domains: zones, then regions.
- Add fallbacks and caches so a failed dependency degrades the answer instead of breaking it.
- Cut time to detect and recover: health checks, automatic failover, fast rollback.

COMMON MISTAKES

- Promising four nines on top of three-nines dependencies.
- Counting replicas as independent when they share a zone, a deploy or a database.
- Forgetting the load balancer, DNS or gateway that sits in series with everything.
- Measuring uptime of servers instead of success of user requests (lesson 2).

5 Key terms

Availability: fraction of time (or requests) the system works

Nines: shorthand: 99.9% = three nines

Series: all components needed: multiply availabilities

Parallel: any one component enough: $1 - (1 - A)^n$

Failure domain: set of things that fail together

MTTR: mean time to recover

6 Interview questions

Q1. What does 99.9% availability allow? BASIC

About 8.8 hours of downtime a year, or roughly 43 minutes a month.

Q2. Your service depends on three services at 99.95% each. What is the best you can offer? BASIC

In series you multiply: $0.9995^3 \approx 99.85\%$, before counting your own failures.

Q3. Two replicas at 99% each behind a 99.99% load balancer. What's the availability? SCENARIO

Replicas: $1 - 0.01^2 = 99.99\%$. In series with the LB: $0.9999 \times 0.9999 \approx 99.98\%$.

Q4. How would you raise a service from 99.5% to 99.95%? SCENARIO

Find the weakest dependencies in the critical path; make some async or optional, add fallbacks, spread replicas across zones, and shorten detection and recovery.

Q5. Why don't three replicas give you nine nines in practice? TRICKY

Failures are correlated: shared zones, deploys, configs and dependencies take replicas down together, and the components in series cap the total.

Q6. Is availability better measured in time or in requests? TRICKY

Requests usually: a request-based SLI (good requests ÷ total) reflects what users feel, including partial outages that time-based uptime hides.

Next: SD-K-002 · SLIs, SLOs and SLAs backendroadmap.com

SLIs, SLOs and SLAs

measure what users feel, promise less than you aim for

Watch the 5-min video

In one line: an SLI measures user experience as good events ÷ valid events; an SLO is your internal target for it over a window (say 99.9% over 28 days); an SLA is a looser, external contract with penalties, so the gap between them gives you time to react before customers are owed money.

1 Healthy servers, unhappy users

The dashboard shows CPU at 40% and every host green. Meanwhile a third of checkouts fail, because a payment dependency returns errors that the host metrics never see. CPU, memory and "is the process up" describe machines, not users. Reliability is measured from the user's side, with **service level indicators**.

2 SLI, SLO, SLA

An **SLI** is a ratio: **good events ÷ valid events**, for example successful requests ÷ all requests, or requests served under 300 ms ÷ all requests. "Valid" excludes events you shouldn't count, such as health-check traffic or a client's own 400 errors. Common SLI types are **availability**, **latency**, **freshness** (how old the data is) and **correctness**.

An **SLO** is a target for an SLI over a **window**: "99.9% of requests succeed over a rolling 28 days". Engineering and product agree it together. Never aim for 100%: it is impossible, very expensive, and users can't tell 99.99% from 100% through their own Wi-Fi and mobile networks. An **SLA** is a contract that adds consequences, usually service credits, and is set **looser** than the SLO.

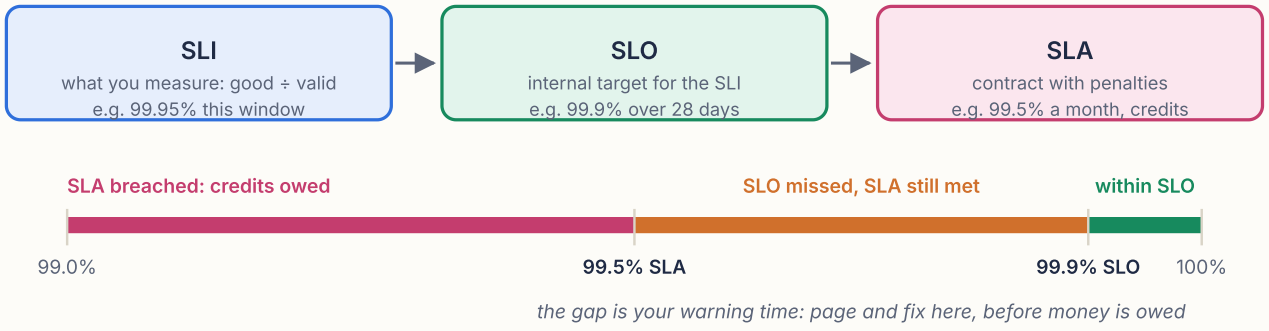


Figure 1. The SLI is the measurement, the SLO the internal target, the SLA the external promise. Missing the SLO is a warning; missing the SLA costs money.

3 Where and how you measure

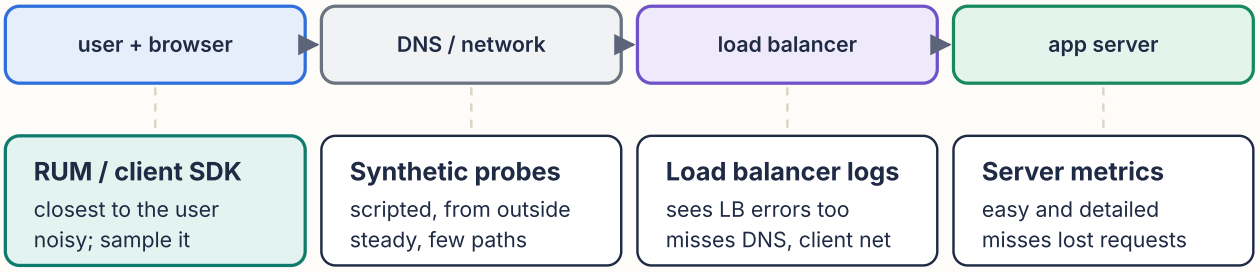


Figure 2. Each measurement point sees a different slice. Server logs can't count a request that never arrived.

- **Server-side** metrics are cheap and detailed but miss DNS failures, TLS errors and requests the load balancer rejected.
- **Load balancer** logs catch more, including 502/504s generated by the LB itself.
- **Client-side** measurement (real-user monitoring) and **synthetic probes** are closest to the real experience; most teams combine LB metrics for the SLO with probes for coverage.
- For latency, use **percentiles** (p99, p99.9), not averages: an average of 120 ms can hide 1% of users waiting 4 seconds. Better still, phrase it as a ratio: "fraction of requests under 300 ms".

4 Picking SLIs: a search API

SLI type	Good event	Valid events	Example SLO
Availability	search returns a non-5xx response	all search requests	99.9% / 28 days
Latency	answered in under 200 ms	all successful searches	99% / 28 days (i.e. p99 < 200 ms)
Freshness	new item searchable within 1 minute	all newly added items	99% / 28 days
Correctness	test query returns the expected top results	sampled golden queries	99.5% / 28 days

WORKED EXAMPLE: A CHECKOUT SLO

SLO: **99.9% of checkout requests succeed in under 500 ms, over 28 days**. The SLI counts a request as good only if it succeeds *and* is fast enough, so one SLI covers both availability and latency. With about 1,000,000 checkouts in the window, the SLO allows **1,000 bad ones** (0.1%). A checkout that succeeds in 2 seconds still counts as bad: to the user, a slow checkout feels broken. If the SLA promises 99.5%, the team has 4,000 more bad requests of warning before credits are owed.

COMMON MISTAKES

- Using CPU, memory or host uptime as SLIs: they describe machines, not users.
- Setting the SLA equal to the SLO, leaving no time to react before penalties.
- Averaging latency, which hides the slow tail that real users hit.
- Too many SLOs: pick 2–4 per user journey that you will actually act on (lesson 3).

5 Key terms

SLI: good events ÷ valid events, measured from the user's side

SLO: target for an SLI over a time window

SLA: external contract with penalties, looser than the SLO

Percentile (p99): value that 99% of requests are faster than

Rolling window: e.g. the last 28 days, recomputed continuously

Synthetic probe: scripted request sent regularly from outside

6 Interview questions

Q1. What is the difference between an SLO and an SLA? BASIC

An SLO is an internal reliability target for an SLI over a window; an SLA is an external contract with penalties such as credits, set looser than the SLO so you get warning time.

Q2. Why not set an SLO of 100%? BASIC

It's unachievable and very costly, it leaves no room for releases or maintenance, and users can't perceive the difference beyond their own network's reliability.

Q3. Pick SLIs for a search API. SCENARIO

Availability (non-5xx responses), latency (fraction under 200 ms, i.e. p99), freshness (new items searchable within a minute) and correctness (golden queries return expected top results), each as good ÷ valid events.

Q4. Servers look healthy but users complain. What do you change? SCENARIO

Define SLIs from the user's side: measure success and latency at the load balancer and with synthetic probes or RUM, so failures outside your servers are counted.

Q5. A checkout succeeds in 2 seconds. Is it a good event? TRICKY

Not if the SLI is "succeeds in under 500 ms": combining success and latency in one SLI counts slow successes as bad, because users experience them as failures.

Q6. Why are averages a poor latency SLI? TRICKY

A few very slow requests barely move the mean, so the tail that hurts users is hidden; percentiles or "fraction under threshold" expose it.

Next: SD-K-003 · Error Budgets backendroadmap.com

Error Budgets

the failure you're allowed, and how to spend it

Watch the 4-min video

In one line: the error budget is 1 − SLO, the amount of failure the SLO allows; you alert on how fast you burn it, and a pre-agreed policy says what happens when it runs out: ship freely while budget remains, freeze launches and fix reliability when it's gone.

1 Ship faster or slow down?

Product wants more releases. The reliability team wants fewer, because the last release broke checkout. Both argue from gut feeling. An **error budget** replaces the argument with a number both sides agreed on before the incident: the SLO from lesson 2 already says how much failure is acceptable.

2 The budget and the burn rate

```
budget = 1 − SLO    ·    burn rate = observed error rate ÷ (1 − SLO)
```

With a **99.9%** SLO the budget is **0.1%**: about **43 minutes** of total downtime in 30 days (0.001 × 43,200 min), or 1 million failed requests out of a billion. The budget is meant to be **spent**: on releases, experiments, migrations and planned maintenance.

Burn rate says how fast you're spending it. At 1× you use exactly the whole budget over the window. If 1% of requests fail against a 99.9% SLO, the burn rate is 0.01 ÷ 0.001 = **10×**, and a 30-day budget is gone in 3 days.



Figure 1. Burn rate is the slope. The same error rate is harmless or urgent depending on how quickly it empties the budget.

3 Alert on burn rate, with two windows

Raw error-rate alerts are noisy: 2% errors for one minute doesn't matter, 0.2% all week does. Alert instead on **burn rate**, using a long window to make sure the burn is significant and a short window (about 1/12 of the long one) so the alert stops soon after the problem is fixed. The Google SRE Workbook's recommended starting point for a 30-day, 99.9% SLO:

Action	Long window	Short window	Burn rate	Budget used when it fires
Page	1 hour	5 minutes	14.4×	2%
Page	6 hours	30 minutes	6×	5%
Ticket	3 days	6 hours	1×	10%

The video rounds this to "page at 14× over 1 hour, ticket at 3× over 6 hours"; the exact thresholds are a choice, the shape is what matters: **fast burns page a human, slow burns open a ticket**. Both conditions (long *and* short window) must be true to fire.

4 The error budget policy

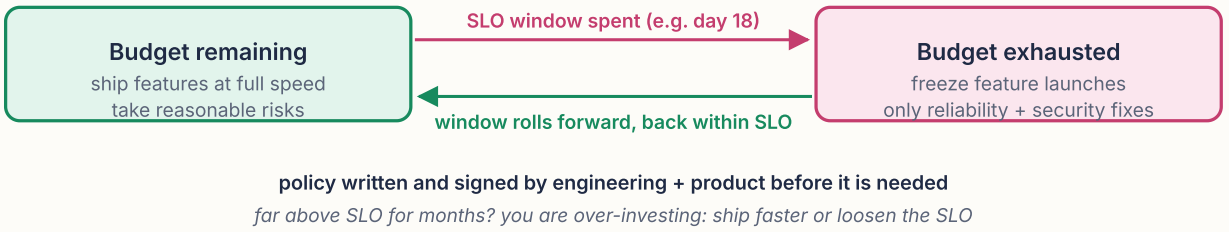


Figure 2. The policy is agreed in advance, so when the budget runs out the data decides, not the loudest voice.

- **Budget left:** teams release at normal speed and can take risks such as a big migration.
- **Budget spent:** feature launches freeze; only reliability fixes and security patches ship until the rolling window recovers.
- **Way above SLO:** you're over-investing in reliability, so release faster or spend the spare budget on riskier work.
- **Postmortems** are triggered by, and prioritised by, how much budget an incident consumed (lesson 70).

WORKED EXAMPLE: A LAUNCH FREEZE ON DAY 18

Checkout has a **99.9% SLO over 30 days**, so about 43 minutes of budget. A bad deploy causes 25 minutes of errors on day 9; a database failover adds 20 minutes on day 18. The budget is gone. As the policy says, feature launches stop and the team spends the rest of the window on **rollback automation** and **better canary checks** (lesson 75). As the bad days roll out of the 30-day window, budget returns and launches resume, with no meeting needed to decide it.

COMMON MISTAKES

- Writing the policy during the incident, when nobody can agree.
- Alerting on every error spike instead of burn rate, so on-call learns to ignore pages.
- Treating the budget as a target to avoid at all costs, instead of something to spend.
- No exception for security fixes in the freeze.

5 Key terms

Error budget: 1 − SLO: the failure the SLO allows in a window

Burn rate: error rate ÷ (1 − SLO); 1× empties it in exactly one window

Multi-window alert: fires only when a long and a short window both burn fast

Fast burn: high burn rate: page someone now

Slow burn: low but steady burn: open a ticket

Budget policy: pre-agreed actions when the budget is exhausted

6 Interview questions

Q1. What is an error budget for a 99.95% SLO over 30 days? BASIC

0.05% of requests, or about 21.6 minutes of full downtime (0.0005 × 43,200 minutes).

Q2. What does a burn rate of 10 mean? BASIC

You're failing ten times faster than the SLO allows; a 30-day budget would be exhausted in 3 days.

Q3. How do error budgets change engineering decisions? SCENARIO

With budget left, teams ship and take risks; when it's spent, launches freeze except reliability and security fixes, as the agreed policy says. Services far above their SLO can move faster.

Q4. How should you alert on an SLO? SCENARIO

On burn rate with multiple windows: page on fast burns (e.g. 14.4× over 1 h and 5 min), ticket on slow burns (e.g. 1× over 3 days, or 3× over 6 h).

Q5. Why add a short window to a 1-hour burn-rate alert? TRICKY

Without it the alert keeps firing for up to an hour after the fix, because the 1-hour average stays high; the short window makes it reset quickly.

Q6. A service has used none of its budget for six months. Good news? TRICKY

Not entirely: it suggests over-investment or an SLO that's too loose. Ship faster, run riskier experiments, or check that the SLI really reflects users.

Next: SD-K-004 · Timeouts backendroadmap.com

Timeouts as a Resilience Tool

a slow dependency is worse than a dead one

Watch the 4-min video

In one line: every remote call needs an explicit connect and request timeout, set a little above the dependency's p99 latency and shorter than the caller's own timeout, so a hanging dependency fails fast instead of holding threads until the whole service stops.

1 It didn't fail. It just stopped answering

The payment provider doesn't return errors: it accepts connections and never replies. Your checkout service waits, and waits. Within a minute every worker thread is stuck on a payment call, and the product page, which never touches payments, stops loading too. One missing timeout turned a slow dependency into a full outage.

2 What a timeout bounds

Timeout	Bounds	Typical value
Connect	establishing the TCP (and TLS) connection	short: 100 ms – 1 s inside a data centre
Read / socket	gap between bytes of the response	a little above the dependency's p99
Request (total)	whole call, including retries	fits inside the caller's budget
Deadline	absolute time the whole request must finish by	set at the edge, propagated inward

Defaults are dangerous. Python's `requests` and Go's `http.Client` wait forever unless you set a timeout, and an unanswered TCP connect on Linux takes about two minutes to give up. Always set them explicitly.

3 Why hanging calls take everything down

Every waiting request holds a thread, a connection, memory and perhaps a database transaction. By Little's law, threads in use = arrival rate × time each request takes. At 100 requests/s and 50 ms per call you need about 5 threads; if the call hangs for 30 s you'd need 3,000. Once the pool is full, new requests queue and fail, including requests for healthy endpoints. A **dead** dependency fails fast with "connection refused"; a **slow** one is more dangerous.

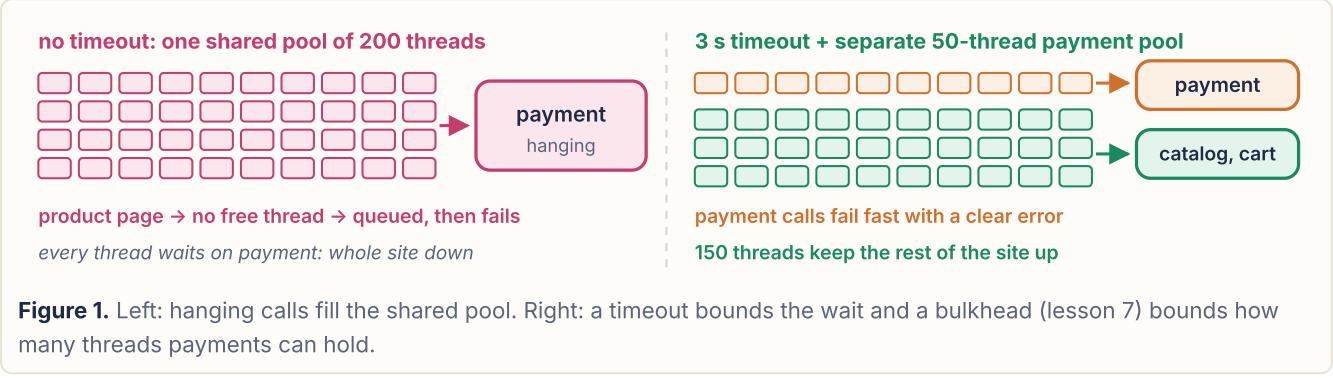


Figure 1. Left: hanging calls fill the shared pool. Right: a timeout bounds the wait and a bulkhead (lesson 7) bounds how many threads payments can hold.

4 Choosing values and layering them

Base the value on **measured latency**: a little above the dependency's p99 or p99.9. Too long and threads stay stuck; too short and you fail requests that would have succeeded, and trigger extra retries that add load. Review the values when latency changes.

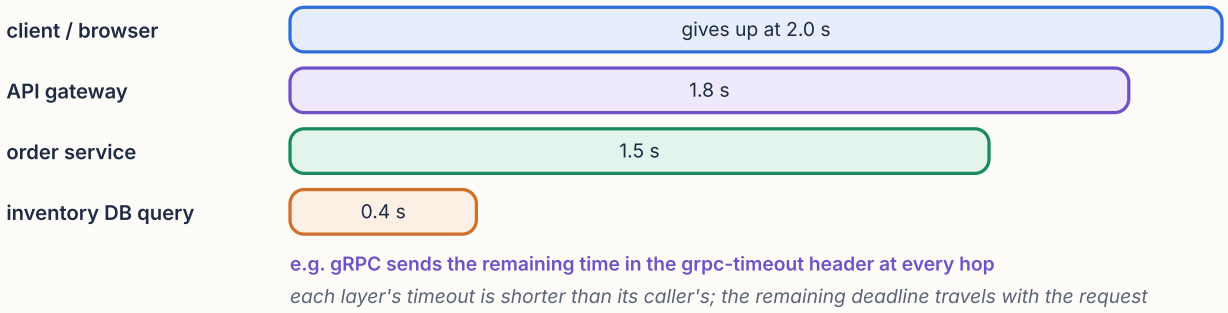


Figure 2. A timeout budget: work after the caller has given up is wasted, so inner timeouts must be shorter.

If the browser gives up after 2 s, the service shouldn't wait 10 s for a downstream call. Better than fixed values is a **propagated deadline**: the edge sets "finish by 12:00:02.000", and each hop checks the remaining time before starting work and passes it on. gRPC does this natively with deadlines.

WORKED EXAMPLE: A HANGING PAYMENT PROVIDER

A checkout service has **200 worker threads** and no timeout on its payment client. The provider starts hanging; in under a minute all 200 threads are blocked and the product page stops loading. Fix: `connect_timeout=0.5, timeout=3.0` plus a separate **payment pool of 50 threads**. At 10 payment calls/s, even if every one hangs for the full 3 s, only about 30 threads are busy. Payment fails fast with "payment temporarily unavailable", and the other 150 threads keep the site up.

TIMEOUTS WORK WITH OTHER PATTERNS

- **Retries** (lesson 5): only for idempotent calls, and counted inside the overall deadline.
- **Circuit breaker** (lesson 6): stop calling a dependency that keeps timing out.
- **Bulkhead** (lesson 7): a slow dependency can only exhaust its own pool.
- **Fallback** (lesson 8): decide what to return when the timeout fires.

COMMON MISTAKES

- Relying on library defaults, which are often infinite or minutes long.
- Setting only a connect timeout, so a slow response still hangs forever.
- Inner timeouts longer than outer ones, so work continues after the user has gone.
- Timeouts set once and never revisited as latency changes.

5 Key terms

Connect timeout: limit on establishing the connection

Request timeout: limit on getting the whole response

Deadline: absolute finish-by time passed along the call chain

Timeout budget: inner timeouts shorter than outer ones

Thread pool exhaustion: all workers blocked, new requests can't run

Little's law: in-flight requests = arrival rate × latency

6 Interview questions

Q1. What is the difference between a connect timeout and a request timeout? BASIC
The connect timeout bounds establishing the connection; the request (or read) timeout bounds waiting for the response once connected. You need both.

Q2. How do you choose a timeout value? BASIC
From measured latency: a little above the dependency's p99 or p99.9, and within the caller's remaining budget.

Q3. What happens to a service with no timeouts when a dependency hangs? SCENARIO
Requests pile up holding threads, connections and memory until the pools are exhausted; then the whole service stops responding, even for requests that never touch that dependency.

Q4. How should timeouts be set across layers? SCENARIO
As a budget: each layer's timeout shorter than its caller's, ideally with a deadline propagated with the request, as gRPC deadlines do.

Q5. Why is a slow dependency more dangerous than a dead one? TRICKY
A dead one fails immediately and frees resources; a slow one holds threads and connections for the full wait, so it exhausts pools and spreads the failure upstream.

Q6. Can a timeout that's too short hurt reliability? TRICKY
Yes: it fails requests that would have succeeded, and the retries it triggers add load to a dependency that's already slow.

Next: SD-K-005 · Retries and Backoff backendroadmap.com

Retries and Exponential Backoff in Practice

retry once, at one layer, slowly

Watch the 4-min video

In one line: retry only transient failures of idempotent operations, with capped exponential backoff and jitter, at a single layer, under a retry budget, so retries rescue brief blips instead of multiplying load on a dependency that is already struggling.

1 How a blip becomes an outage

The inventory database slows down for two seconds. Clients time out and retry. The gateway retries too, and so does the service. The database now receives many times its normal load at the moment it is weakest, can't recover, and a two-second blip becomes an hour-long **retry storm**.

2 What to retry

Failure	Retry?	Why
connection refused / reset, DNS blip	yes	transient; the request probably never ran
timeout	only if idempotent	the request may have succeeded on the server
503 Service Unavailable, 429 Too Many Requests	yes, later	overload; honour the `Retry-After` header
400, 401, 403, 404, 422	no	permanent: the same request fails the same way
non-idempotent POST (charge a card)	only with an idempotency key	otherwise a retry can charge twice

Idempotent means doing it twice has the same effect as once: GET, PUT, DELETE by design; a payment only if the server deduplicates on an **idempotency key** sent by the client.

3 Backoff, jitter and limits

$$sleep = random(0,\ min(cap,\ base \times 2^{attempt}))$$

Never retry immediately. **Exponential backoff** waits 100, 200, 400 ms... up to a cap. **Jitter** randomises each wait, so thousands of clients that failed at the same moment don't all retry at the same moment. The formula above is AWS's "full jitter". Limit attempts to two or three, and stop when the request's deadline (lesson 4) runs out.

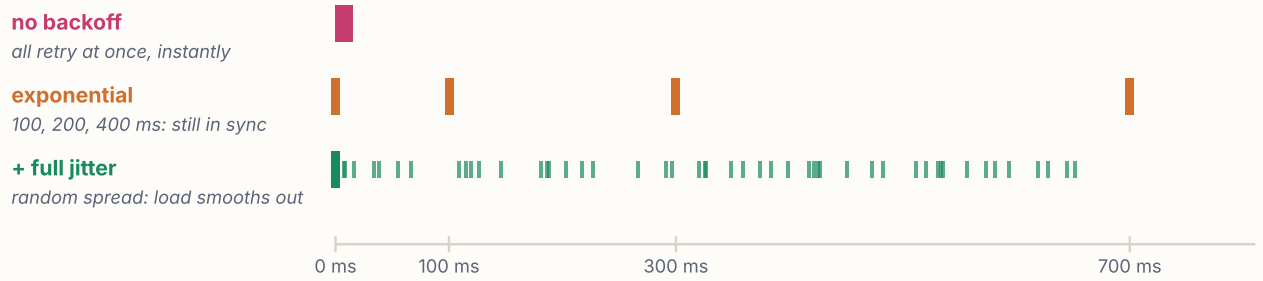


Figure 1. Backoff spaces retries out; jitter de-synchronises clients so retries arrive as a smooth trickle instead of waves.

4 Retry at one layer, under a budget

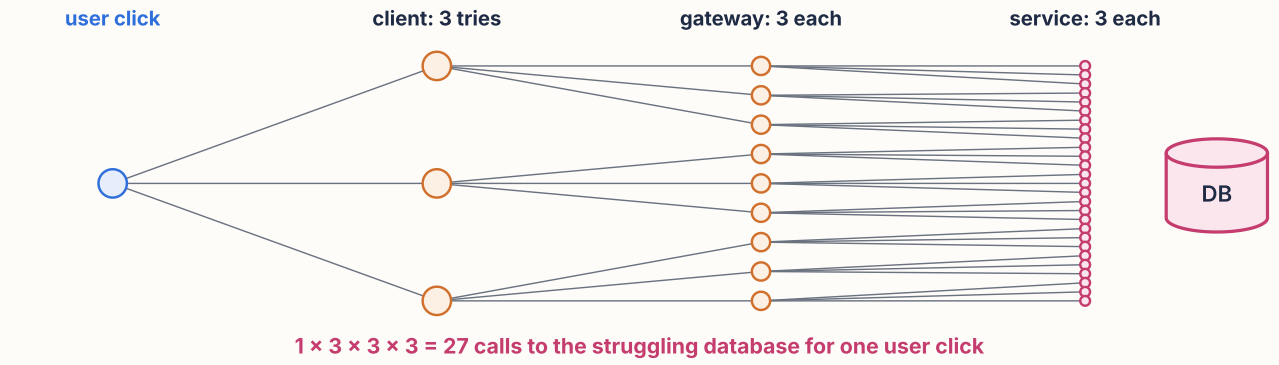


Figure 2. Each layer's retries multiply the layers below it. Three layers × three attempts = 27 calls.

Retry at **one layer**, usually the one closest to the failing call, and let the others pass errors through. Then add a **retry budget**: retries may be at most a small fraction of normal traffic, e.g. **10%**. When failures are rare, every failure gets retried; when most requests fail, the budget runs out and retries stop, so they can't double or triple the load. gRPC's retry throttling and Envoy's retry budgets implement this idea.

Combine with a **circuit breaker** (lesson 6): once it opens, calls fail fast with no retries at all. Servers can help with `Retry-After` or an explicit "overloaded, don't retry" signal.

WORKED EXAMPLE: THE 27× STORM

The inventory DB slows down. Client, gateway and service each make up to 3 attempts, so every click becomes up to **27 queries**. Fix: the client and gateway pass errors straight back; only the service retries, with backoff 100 → 200 ms plus jitter, at most 3 attempts inside a 1.5 s deadline, and a retry budget of 10%. During the incident the extra load on the database is now **at most 10%** instead of up to 27×, and it recovers.

COMMON MISTAKES

- Retrying 4xx errors, which will never succeed.
- Retrying payments or other side effects without an idempotency key.
- Backoff without jitter, so clients retry in synchronised waves.
- Retries configured independently in the SDK, the gateway and the service mesh (lesson 27).

5 Key terms

Transient failure: temporary error likely to succeed on retry

Idempotent: doing it twice = doing it once

Exponential backoff: wait doubles after each failed attempt, up to a cap

Jitter: random part of the wait to de-sync clients

Retry amplification: attempts multiplying across layers

Retry budget: cap on retries as a share of normal traffic

6 Interview questions

Q1. Which errors should you retry? BASIC

Transient ones (connection resets, timeouts, 503, 429) on idempotent operations; never permanent errors like 400 or 404.

Q2. Why add jitter to backoff? BASIC

Clients that failed together would otherwise retry together in synchronised waves; randomising the wait spreads the load.

Q3. How do you avoid retry storms? SCENARIO

Retry only transient errors on idempotent operations, use capped exponential backoff with jitter, retry at a single layer, enforce a retry budget (e.g. ≤ 10%) and add a circuit breaker.

Q4. A payment call timed out. Is it safe to retry? SCENARIO

Only if the call is idempotent: send an idempotency key so the provider returns the original result instead of charging again.

Q5. Why is retrying at every layer dangerous? TRICKY

Attempts multiply: three layers × three attempts = 27 calls to the failing dependency for one user request, exactly when it's weakest.

Q6. Why is a retry budget better than a fixed attempt count? TRICKY

A fixed count retries every failure even during a full outage, adding load; a budget allows retries when failures are rare and switches them off when they're widespread.

Next: SD-K-006 · Circuit Breaker backendroadmap.com

Circuit Breaker

stop calling what's certain to fail

Watch the 4-min video

In one line: a circuit breaker wraps calls to one dependency and, once failures cross a threshold, **opens** so calls fail instantly with a fallback; after a cool-down it goes **half-open** and lets a few trial calls decide whether to close again or stay open.

1 Waiting for a call that will fail

The recommendation service is down. Every product page still calls it, waits the full 2 s timeout, and then fails. Thousands of requests a second are spent on calls that are certain to fail, threads pile up behind the timeouts (lesson 4), and the struggling service gets no chance to recover because it is still being hammered. A circuit breaker, like the one in your house, cuts the circuit.

2 Three states

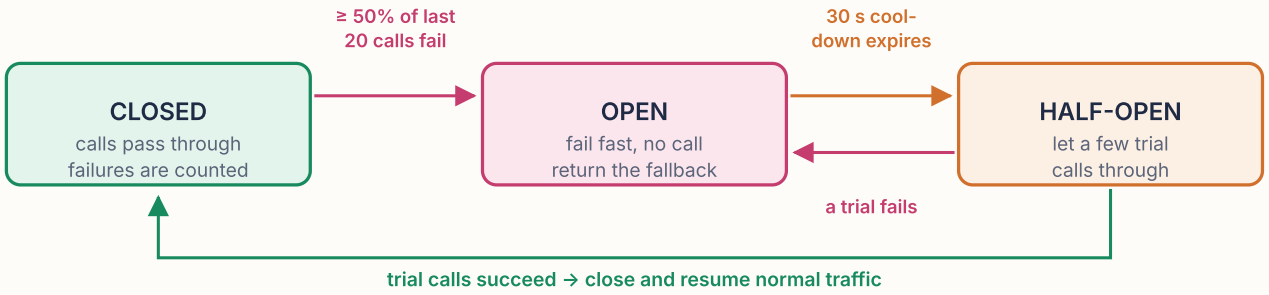


Figure 1. Closed → open on too many failures; open → half-open after a cool-down; half-open closes on success or reopens on failure.

- **Closed** (normal): calls go through; the breaker records each result in a sliding window.
- **Open**: once the failure rate crosses the threshold (e.g. 50% of the last 20 calls, or 5 consecutive failures), calls fail immediately without touching the dependency.
- **Half-open**: when the cool-down (e.g. 30 s) expires, a small number of trial calls are let through. If they succeed the breaker closes; if any fails it opens for another cool-down.

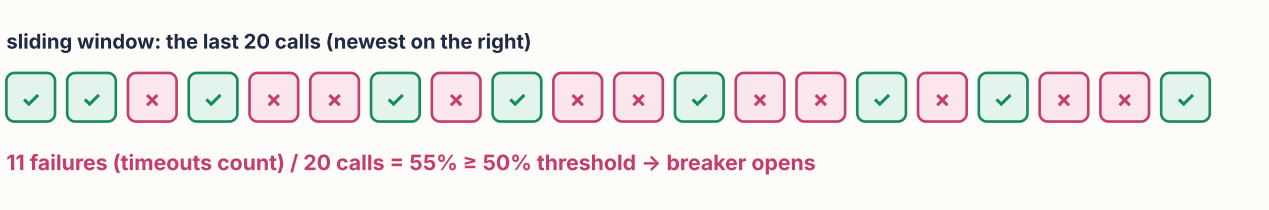


Figure 2. A count-based window. Real libraries also need a minimum number of calls before they judge, so 1 failure out of 1 call doesn't open the breaker.

Libraries name these settings explicitly. In **Resilience4j** (Java) they are `failureRateThreshold`, `slidingWindowSize`, `minimumNumberOfCalls`, `waitDurationInOpenState` and `permittedNumberOfCallsInHalfOpenState`; **Polly** (.NET) and **pybreaker** (Python) have equivalents, and service meshes such as Envoy/Istio offer outlier detection that ejects failing hosts (lesson 27).

3 Fallbacks, and how it differs from retries

When the breaker is open the caller needs an answer: **cached data**, a **default value**, an empty list, or a clear error the UI handles well (lesson 8). Failing in a millisecond keeps threads free and latency low.

	Retry	Circuit breaker
Assumes the failure is	brief: try again soon	lasting: stop trying for a while
Effect on the dependency	adds load	removes load, lets it recover
Scope	one request	all requests to that dependency
Together	retry 1–2 times inside the breaker; when the breaker is open, no retries happen at all (lesson 5)	

WORKED EXAMPLE: A FAILING RECOMMENDATION SERVICE

Recommendation calls start timing out. Once **50% of the last 20 calls** fail, the breaker opens. Product pages immediately show a **default list of best sellers** instead of waiting 2 s per request. After a **30 s** cool-down the breaker goes half-open and lets a few trial calls through; they succeed, the breaker closes, and personalised recommendations return. A metric and log line on every state change tells the team exactly when it opened and closed.

COMMON MISTAKES

- One breaker shared by several dependencies, so one failure blocks calls to healthy ones. Use one per dependency, sometimes per endpoint.
- Not counting timeouts as failures, so a hanging dependency never trips it.
- Treating 4xx client errors as failures, so bad input from one user opens the breaker.
- Writing your own instead of using a tested library, and emitting no metric when it opens.

4 Key terms

Closed: normal state: calls pass, results counted

Half-open: after the cool-down, a few trial calls probe recovery

Cool-down: how long the breaker stays open before trying again

Open: calls fail fast without reaching the dependency

Failure threshold: failure rate or count that opens the breaker

Fallback: what to return while the breaker is open

5 Interview questions

Q1. Explain the circuit breaker states. BASIC

Closed passes calls and counts failures; open fails fast once a threshold (e.g. 50% of the last 20 calls) is crossed; half-open lets trial calls through after a cool-down to decide whether to close or reopen.

Q2. What does the caller return when the breaker is open? BASIC

A fallback: cached or default data, an empty list, or a clear error that the UI handles gracefully.

Q3. How is a circuit breaker different from retries? SCENARIO

Retries assume a brief failure and try again soon; a breaker assumes a lasting failure and stops calling for a while. Use both: a couple of retries inside a breaker that stops all calls when failures persist.

Q4. One service calls payments, search and reviews. How many breakers? SCENARIO

At least three, one per dependency, so a reviews outage doesn't block payments; sometimes one per endpoint if they fail independently.

Q5. Why is a half-open state needed at all? TRICKY

Without it you'd either never re-close or send full traffic to a dependency that may still be broken; a few trial calls test recovery cheaply.

Q6. Could a breaker hurt availability? TRICKY

Yes, if it's too sensitive (no minimum call count, counting 4xx) it opens on noise and turns a healthy dependency into fallback-only. Tune thresholds and monitor state changes.

Next: SD-K-007 · Bulkhead Pattern

Bulkhead Pattern

watertight compartments for your resources

Watch the 4-min video

In one line: give each dependency, workload or tenant its own bounded pool of threads, connections or instances, so when one of them slows down or floods, it exhausts only its own compartment and everything else keeps working.

1 One leak shouldn't sink the ship

A ship's hull is split into watertight compartments, called bulkheads: if one floods, the others stay dry. Most services have one big shared compartment: one thread pool, one connection pool, one cluster for every customer. Then a single slow dependency, or a single noisy customer, can drown all of it.

2 How a shared pool spreads failure

A service has **100 threads** and calls inventory, pricing and reviews. Inventory slows from 50 ms to 5 s. Its calls hold threads 100× longer, until all 100 are waiting on inventory. Calls to the two healthy dependencies can't get a thread either: a failure in one place has spread everywhere. Split the pool into **40 / 30 / 30** and the slow dependency can fill only its 40; extra calls to it are rejected immediately, and pricing and reviews keep working.

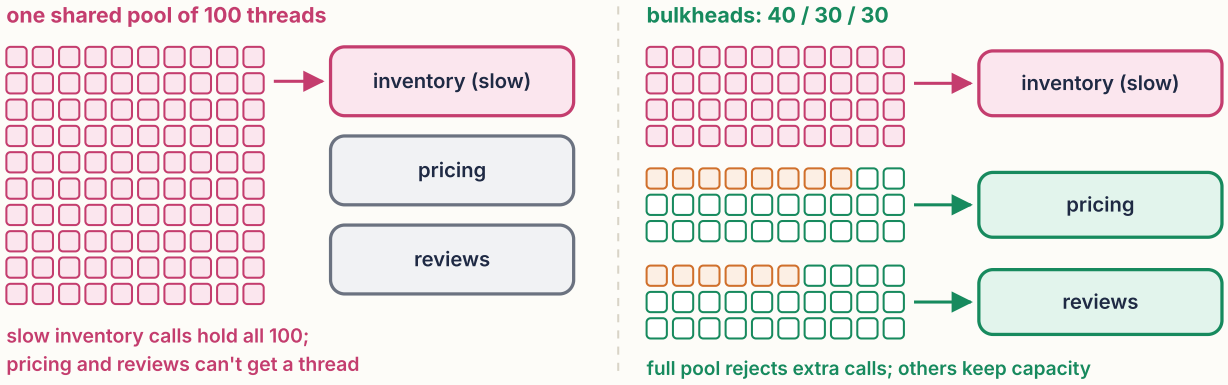


Figure 1. Left: one slow dependency occupies the shared pool. Right: it fills only its own compartment.

3 Bulkheads at every level

Level	What is isolated	Example
In-process	thread pool or semaphore per dependency	Resilience4j `Bulkhead` with max 40 concurrent calls
Connections	connection pool per database or downstream	reporting queries get their own pool, so they can't starve checkout
Instances	separate deployments or clusters per workload	critical API on one node pool, batch jobs on another
Tenants	capacity or concurrency per customer	per-tenant limits and queues; whole cells per group (lesson 16)

The goal is a small **blast radius**: how much of the system one failure can affect. The smaller the compartments, the smaller the radius. **Cell-based architecture** (lesson 16) takes this to the extreme: independent copies of the whole stack, each serving a slice of customers.

WORKED EXAMPLE: PREMIUM VS FREE TENANT POOLS

A report service shared one worker pool. A free-tier user submitted thousands of huge reports, and paying customers waited hours. Now premium tenants get a **dedicated pool of 30 workers**, free tenants share a **pool of 10**, and each tenant may run at most **2 jobs at once**. A flood of free jobs only lengthens the free queue, and even there one tenant can hold just 2 of the 10 workers. Premium reports keep finishing on time.

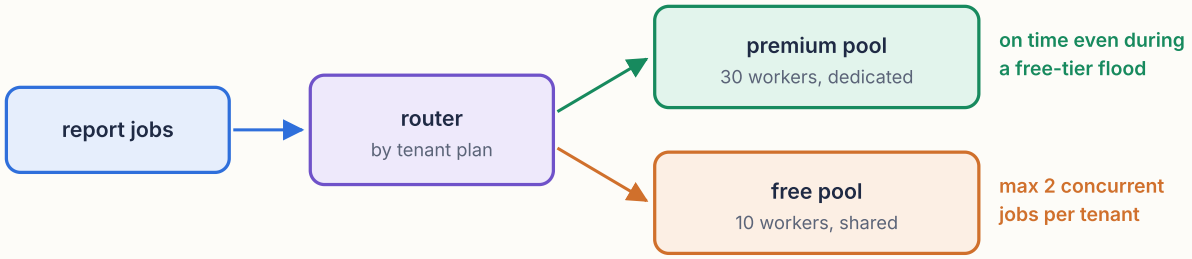


Figure 2. Two bulkheads: by plan (separate pools) and by tenant (a concurrency cap inside the shared pool).

COSTS AND PAIRINGS

- **Lower utilisation:** one pool may sit idle while another is full; that spare capacity is the price of isolation.
- **Sizing:** size each pool from measured load (Little's law: rate × latency, plus headroom) and revisit it.
- **Pair with timeouts** (lesson 4) that free threads, and **circuit breakers** (lesson 6) that stop calls to a dead dependency; the bulkhead caps the damage meanwhile.
- **Rejected calls** need a fallback or a clear error (lesson 8).

4 Key terms

Bulkhead: isolated, bounded pool of resources

Blast radius: how much of the system one failure can affect

Semaphore bulkhead: cap on concurrent calls, no separate threads

Thread-pool bulkhead: dedicated threads per dependency

Noisy neighbour: tenant whose load hurts others on shared resources

Cell: independent full copy of the stack for a slice of users

5 Interview questions

Q1. What is a bulkhead? BASIC

Isolating resources (thread pools, connection pools, instances, capacity) per dependency or tenant, so a failure in one compartment can't exhaust resources everywhere.

Q2. Give an example. BASIC

Separate worker pools for premium and free tenants with per-tenant limits, or a dedicated thread pool per downstream dependency, so one slow dependency only fills its own pool.

Q3. Checkout and an analytics export share one database connection pool. What do you do? SCENARIO

Give each its own pool (or a read replica for analytics), sized from measured load, so a heavy export can't take the connections checkout needs.

Q4. One tenant's spike slows down everyone. How do you fix it? SCENARIO

Per-tenant concurrency limits or rate limits, separate queues, and for big tenants dedicated capacity or their own cell.

Q5. Why not just make the shared pool bigger? TRICKY

A slow dependency fills any size of pool eventually; a bigger pool only delays the outage and adds memory and downstream load. Isolation, not size, contains it.

Q6. What is the main cost of bulkheads? TRICKY

Partitioned capacity is used less efficiently: one pool idles while another rejects work, and each pool must be sized and tuned.

Next: SD-K-008 · Graceful Degradation backendroadmap.com

Graceful Degradation and Fallbacks

less, but not nothing

Watch the 4-min video

In one line: decide in advance what every non-critical feature does when its dependency fails (stale data, a default, a static page, or simply hiding it) so a partial failure removes a feature instead of the whole page, and shed expensive features on purpose to protect the critical path.

1 Same failure, two outcomes

The personalisation service fails. Option one: the home page throws an error and nobody can shop. Option two: the home page shows popular items and most users never notice. The failure is identical; the design is different. **Graceful degradation** means serving reduced functionality instead of failing completely.

2 Critical vs non-critical

Classify features first. **Critical**: login, cart, checkout, payment: must work, and get the strongest protection. **Non-critical**: recommendations, reviews, recently viewed, live stock counts: can be simplified or dropped. Then, for each non-critical feature, write down its fallback, its timeout (lesson 4) and its circuit breaker (lesson 6).

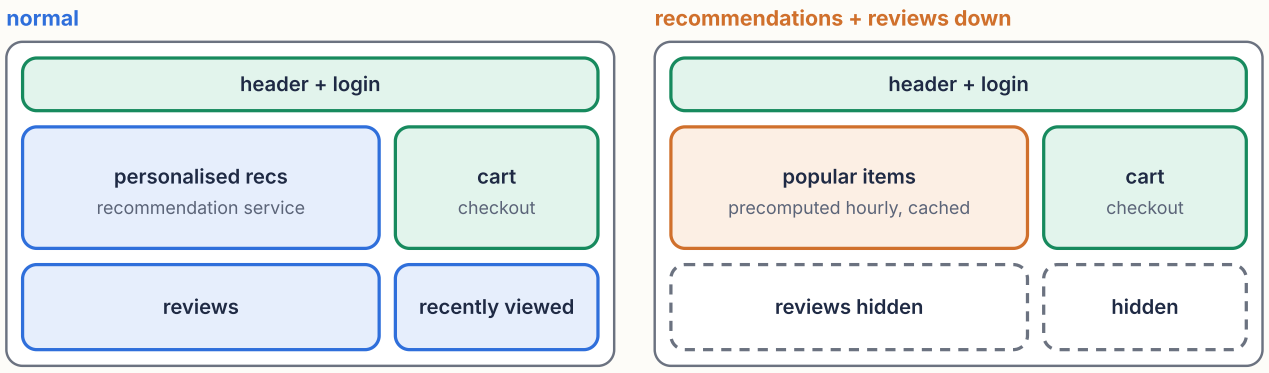


Figure 1. The page is composed of independent parts. When two dependencies fail, those parts degrade and the critical path is untouched.

3 Kinds of fallback

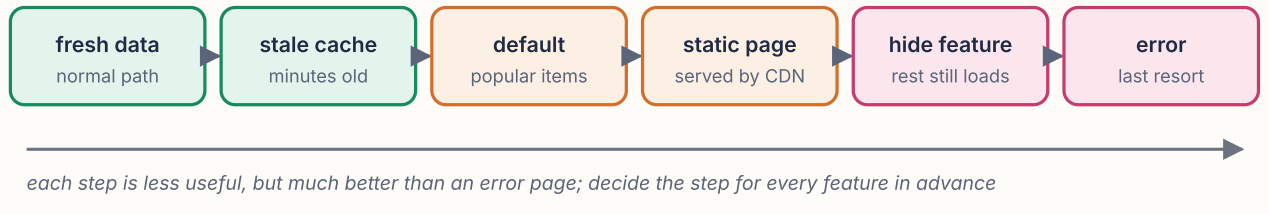


Figure 2. Fallbacks from best to worst. Most features should never reach the last box.

Fallback	Example	Watch out for
Stale / cached data	prices from a 5-minute-old cache	must be safe to be stale; never stale balances or stock at checkout
Default value	popular items, generic banner, default settings	precompute it; the default must not need the failed service
Static version	CDN-served static home page	keep it reasonably current
Hide the feature	no reviews section	page layout must cope with a missing block
Feature shedding	turn off search suggestions at peak	needs a fast switch: a feature flag (lesson 76), not a deploy

Feature shedding is degrading on purpose under load: during a sales peak, switch off search suggestions, personalised ranking and live inventory counts so checkout gets all the capacity.

4 "The cache cluster died. Now what?"

Without a plan every request falls through to the database, which was sized for the cache's 5% miss rate, not 100%, and collapses. A better design: **limit concurrent database reads** with a bulkhead (lesson 7) and reject the excess; serve **stale data from a small in-process cache**; **shed** non-critical features; return **defaults** where data isn't essential; and when the cache comes back, **warm it gradually** (request coalescing, ramped traffic) to avoid a stampede.

WORKED EXAMPLE: POPULAR ITEMS WHEN RECOMMENDATIONS FAIL

The home page calls the recommendation service with a **200 ms timeout** behind a circuit breaker. On timeout, error or open breaker it shows **popular items**, precomputed every hour by a batch job and cached locally, so the fallback needs nothing from the failed service. The page loads normally and sales dip slightly instead of dropping to zero. A `degraded_mode` metric shows how often the fallback serves; when the service recovers, personalised results quietly return.

COMMON MISTAKES

- Fallbacks never tested, so they fail the first time they run. Exercise them with chaos experiments and game days (lesson 17).
- A complex fallback that depends on another service, which can fail too.
- No metric for degraded mode, so a fallback silently serves for weeks.
- Degrading silently where users need to know: say "recommendations are temporarily unavailable".

5 Key terms

Graceful degradation: reduced functionality instead of total failure

Critical path: features that must work: login, cart, checkout

Stale data: older cached data served when fresh data is unavailable

Fallback: pre-planned response when a dependency fails

Feature shedding: turning off non-critical features under load

Cache stampede: many misses hitting the database at once

6 Interview questions

Q1. What is graceful degradation? BASIC
Serving reduced functionality (cached data, defaults, static pages, hidden features) instead of failing completely.

Q2. Name four kinds of fallback. BASIC
Stale cached data, a sensible default (e.g. popular items), a static CDN-served page, and hiding the feature.

Q3. How would your system degrade if the cache cluster died? SCENARIO
Protect the database with concurrency limits, serve stale data from local in-process caches, shed non-critical features, return defaults where data isn't essential, and warm the cache back gradually to avoid a stampede.

Q4. Black Friday traffic is 5× normal and checkout slows. What do you switch off? SCENARIO
Expensive non-critical features via feature flags: search suggestions, personalised ranking, live stock counts, so their capacity goes to checkout.

Q5. Why must a fallback be simple? TRICKY
It runs exactly when things are already failing; if it needs another service or complex logic, it can fail too and turn a partial outage into a full one.

Q6. Can you serve stale data everywhere? TRICKY
No: stale recommendations are fine, but stale balances, stock at payment or permissions can cause real harm; decide per feature.

Next: SD-K-009 · Health Checks and Self-Healing backendroadmap.com

Health Checks and Self-Healing

let the platform fix the 3 a.m. failures

Watch the 5-min video

In one line: liveness probes restart stuck containers, readiness probes take instances out of traffic without restarting them, startup probes protect slow starters; controllers then keep pushing the actual state toward the desired state, so routine failures heal without waking anyone.

1 Running, but not working

At 3 a.m. a service deadlocks. The process is still running and the port is open, so a simple "is it up?" check says yes, while every request hangs. Nobody is awake to restart it. A good platform notices and fixes this itself in under a minute: that's **self-healing**, and it starts with the right **health checks** (the endpoints themselves are covered in lesson 68).

2 Three probes, three questions

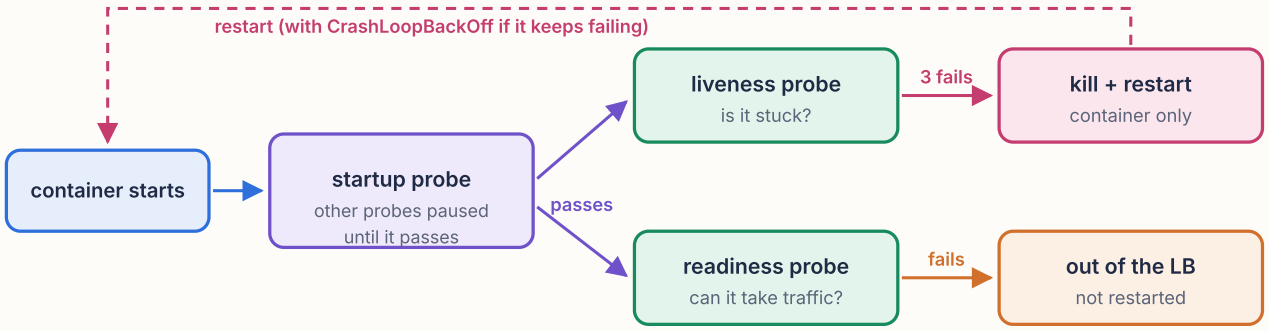


Figure 1. Kubernetes probes. Liveness failure restarts the container; readiness failure only removes the pod from Service endpoints.

Probe	Question	On failure	Should check
Liveness	is this process stuck?	container killed and restarted	only local: can the process answer at all
Readiness	can it serve traffic now?	removed from load balancing	what *this* instance needs: warm cache, loaded model
Startup	has it finished booting?	keeps waiting, then restarts	same endpoint as liveness, with a long allowance

Never check the database in a liveness probe. If the database goes down, every pod fails liveness and the platform restarts the whole fleet, which fixes nothing and adds a cold-start storm on top. Check shared dependencies in readiness carefully (if all pods go unready, nothing serves), or not at all.

3 Self-healing beyond restarts

- A crashed container is restarted by the kubelet; a dead node's pods are **rescheduled** on healthy nodes.
- Cloud auto scaling groups **replace** instances that fail health checks.
- Controllers run a **reconciliation loop**: desired state (5 replicas) vs actual (4), then fix the difference, forever.
- Restarts fix transient problems only. A pod that crashes on startup restarts with exponential backoff, 10 s, 20 s, 40 s... capped at 5 minutes, shown as `CrashLoopBackOff`; that needs an alert and a human.

4 Tuning the probes

`periodSeconds` sets how often the probe runs and `failureThreshold` how many failures in a row trigger action; Kubernetes defaults are 10 s and 3, with a 1 s `timeoutSeconds`. Too aggressive and a 2-second GC pause kills a healthy pod; too lax and a dead pod serves errors for minutes. Time to act ≈ period × threshold, so 10 s × 3 ≈ 30 s.

WORKED EXAMPLE: A DEADLOCKED JAVA SERVICE

Two threads wait on each other forever. The port is open, but `/healthz` runs on the same thread pool and stops answering. Liveness is `httpGet /healthz :8080, periodSeconds: 10, failureThreshold: 3`. After three failures, about 30 s, Kubernetes kills the container and starts a fresh one. Readiness failed too, so the pod was already out of the load balancer. Users saw a few slow requests, not an outage, and the rising restart count tells the team to find the bug.

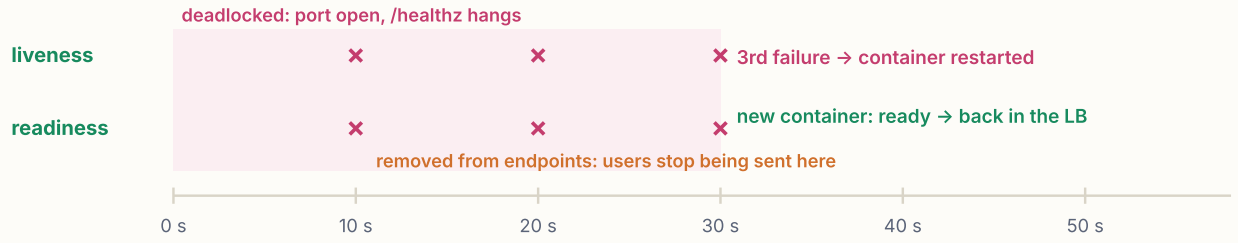


Figure 2. The deadlock heals itself in about 30 s, and readiness keeps users away from the stuck pod meanwhile.

COMMON MISTAKES

- Checking the database or other shared dependencies in liveness: one outage restarts the whole fleet.
- A liveness endpoint served by a separate thread, so it says "alive" while the worker pool is deadlocked.
- No startup probe for a slow-starting app, so liveness kills it before it finishes booting.
- Treating restarts as the fix: frequent restarts hide a bug that still needs an alert.

5 Key terms

Liveness probe: fails → container restarted

Readiness probe: fails → pod removed from traffic, not restarted

Startup probe: holds other probes until the app has booted

Reconciliation loop: controller drives actual state toward desired state

CrashLoopBackOff: restart delay doubling to a 5-minute cap

failureThreshold: consecutive failures before acting

6 Interview questions

Q1. What is the difference between liveness and readiness? BASIC

Liveness asks whether the process is stuck, and failure restarts the container; readiness asks whether it can serve now, and failure only removes it from the load balancer.

Q2. What is a startup probe for? BASIC

Slow-starting apps: liveness and readiness are paused until it passes, so the app isn't killed while still booting.

Q3. How do orchestrators self-heal applications? SCENARIO

They run liveness, readiness and startup probes, restart containers that fail liveness, pull unready pods from traffic, reschedule pods from dead nodes, and keep reconciling actual state toward desired state.

Q4. A pod is in CrashLoopBackOff. What does it mean and what do you do? SCENARIO

It crashes on start and Kubernetes is backing off restarts up to 5 minutes; read the logs of the previous container, fix the config or bug, and roll back if it came from a deploy.

Q5. Why shouldn't a liveness probe check the database? TRICKY

A database outage would fail liveness on every pod and trigger a fleet-wide restart that fixes nothing. Check dependencies in readiness, or not at all.

Q6. Can readiness probes cause an outage? TRICKY

Yes: if every pod checks a shared dependency and it fails, all pods go unready and nothing serves, even requests that didn't need it.

Next: SD-K-010 · Redundancy and Replication backendroadmap.com

Redundancy and Replication for Availability

copies only help if they fail separately

Watch the 4-min video

In one line: run at least N+1 (better N+2) copies, place them in different failure domains, usually three availability zones each able to carry the load without one zone, and replicate the data too, because copies that share power, a deploy or a dependency fail together.

1 Two copies, one building

You run two copies of every service and feel safe. Then one data centre loses power and both copies go down, because they were in the same building. Redundancy only adds availability when the copies fail **independently** (lesson 1's parallel formula assumes exactly that).

2 How many copies: N, N+1, N+2

N is what you need to handle peak load. **N+1** adds one spare, so you survive one failure. **N+2** survives a failure while another instance is down for a deploy, patching or maintenance, which happens every week. With only N+1, a routine deploy plus one crash already leaves you short.

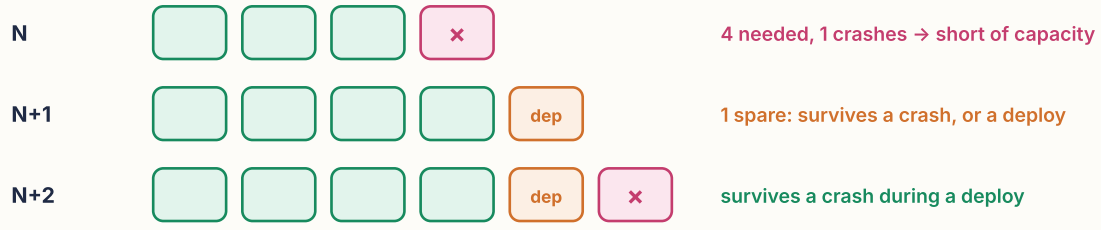


Figure 1. With N = 4: N+1 covers one event at a time; N+2 covers the common case of a crash during planned work.

3 Failure domains

A **failure domain** is a set of things that fail together because they share something. Redundant copies must sit in **different** domains.

Failure domain	Shared thing	Spread by
Host / rack	power supply, top-of-rack switch	anti-affinity, spread placement groups
Availability zone	building(s), power, cooling, network	instances in ≥ 3 zones
Region	a metro area, regional control plane	multi-region (lesson 15)
Software version	one bad deploy hits every copy	staged rollouts, canaries (lesson 75)
Config / dependency	one config push, one DNS provider, one DB	staged config, diverse or redundant dependencies

In the cloud, an **availability zone** is one or more data centres with independent power, cooling and networking, a few milliseconds from the others in its region. Use **three zones** and size each so the other two carry the full load: each zone needs 50% of peak, so you run 150% of N in total.

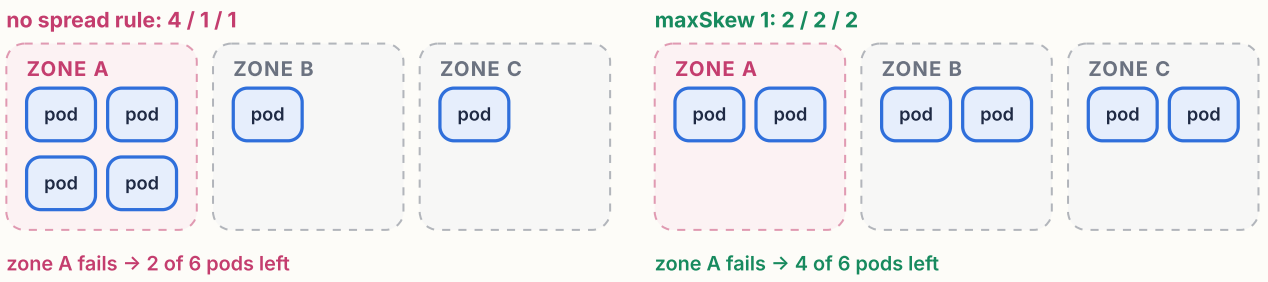


Figure 2. The same 6 pods. Without a spread rule the scheduler can pack a zone; losing it takes most of your capacity.

4 Correlated failures, and the data

The hidden danger is **correlation**: a bad deploy pushed everywhere at once, a shared dependency, even disks from the same batch wearing out together. Fight it with staged rollouts, diverse dependencies and independent copies of critical state.

Stateless services are easy to copy; **data needs replication**. Keep database replicas in other zones: **synchronous** to lose nothing when a zone fails, or **asynchronous** for lower write latency with a small loss window (lesson 13). Object stores such as S3 Standard already store data across multiple zones. Your availability is capped by the **least redundant** component on the request path.

WORKED EXAMPLE: SIX PODS ACROSS THREE ZONES

A web service runs 6 pods on Kubernetes. Without guidance the scheduler might put 4 in zone A. A **topology spread constraint** with `topologyKey: topology.kubernetes.io/zone` and `maxSkew: 1` forces 2 per zone. If zone A fails, 4 pods keep serving and the autoscaler adds more in B and C. A **PodDisruptionBudget** (e.g. `minAvailable: 5`) stops node maintenance from draining too many at once.

```
# Deployment pod spec: spread 6 replicas evenly over zones
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector: { matchLabels: { app: web } }
```

Figure 3. The spread rule from the example. `DoNotSchedule` leaves a pod pending rather than break the spread.

COMMON MISTAKES

- Replicas in one zone, rack or node, counted as independent.
- Running exactly N+1, so every deploy is a single-failure-from-outage window.
- Redundant app servers in front of a single database instance.
- Zones not sized to absorb a lost zone, so the survivors overload and fail too.

5 Key terms

N+1 / N+2: capacity for peak plus one or two spares

Availability zone: isolated data centre(s) within a region

Topology spread: Kubernetes rule to balance pods across zones

Failure domain: components that fail together

Correlated failure: one cause takes down several copies

PodDisruptionBudget: limit on pods taken down voluntarily at once

6 Interview questions

Q1. What is a failure domain? BASIC
A set of components that can fail together because they share something: power, a switch, a rack, a zone, a region, a software version or a config push. Redundant copies must sit in different ones.

Q2. How much redundancy do you need? BASIC
At least N+1, and N+2 to survive a failure during maintenance or a deploy, spread across at least three zones with each zone sized so the other two can carry the load.

Q3. You need 6 instances at peak across 3 zones. How many do you run? SCENARIO
Each zone must carry the load if one fails, so 2 surviving zones need 6: 3 per zone, 9 in total.

Q4. Your app servers are in three zones. What else could still take you down? SCENARIO
Anything shared: a single-zone database, one DNS or identity provider, a global config push or a deploy that reaches every zone at once.

Q5. Why is N+1 often not enough? TRICKY
Planned work regularly takes one instance out; a single crash at that moment leaves you below N, so N+2 covers the realistic case.

Q6. Does replicating data synchronously across zones remove all data-loss risk? TRICKY
No: it protects against losing a zone, but a bad write, bug or deletion is replicated instantly too; you still need backups (lesson 14).

Next: SD-K-011 · Failover Mechanisms backendroadmap.com

Failover Mechanisms

detect, decide, promote, redirect

Watch the 4-min video

In one line: failover detects that the primary is gone, confirms it, promotes a standby, fences the old primary and redirects clients; its downtime is the sum of those steps, and its data loss is whatever replication hadn't copied yet.

1 Noon, and the primary is gone

Your primary database dies at noon. A standby is ready in another zone. Two numbers matter now: **how long** until writes work again (your RTO, lesson 13), and **how much data** was lost (your RPO). Failover design is about both.

2 Four steps

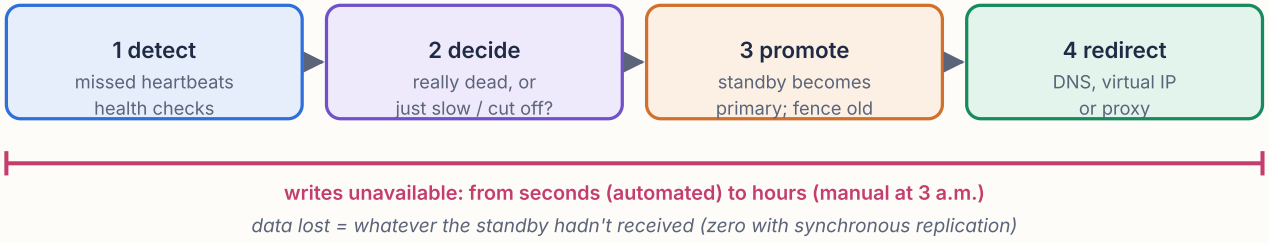


Figure 1. Failover time is detection + decision + promotion + redirection. Every step can be the slow one.

Automated failover is fast, often under a minute, but can act on a false alarm and **flap** back and forth. **Manual** failover lets a human confirm but takes minutes to hours. Many teams automate zone-level failover of a database and keep a human in the loop for region-level failover (lesson 15).

3 Detection and split brain

Detection is a trade-off: a short timeout (say 3 missed heartbeats at 1 s) reacts fast but fires on network blips; a long one is safe but extends the outage. The worst outcome is **split brain**: the old primary was only cut off, not dead, and keeps accepting writes while the new one does too.

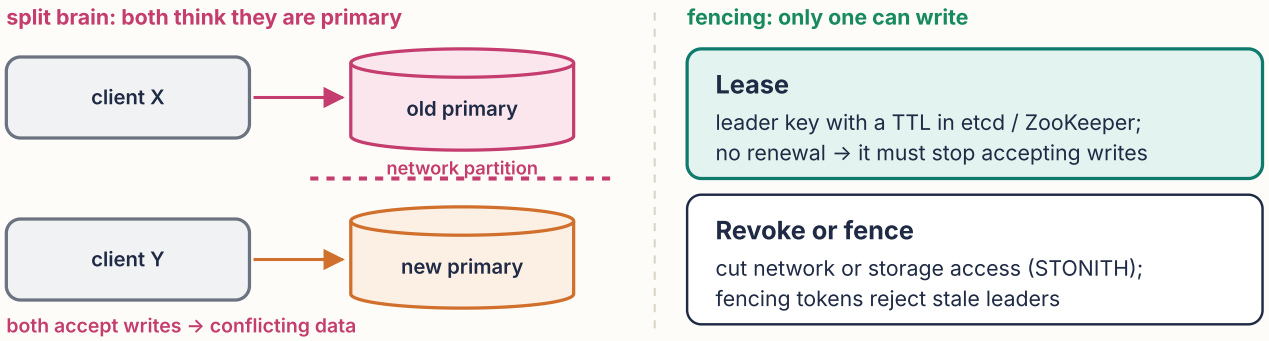


Figure 2. Fencing makes sure the old primary can't write after the new one takes over.

Tools build this in: **Patroni** for PostgreSQL keeps the leader key in etcd or Consul with a TTL; **Redis Sentinel** needs a quorum of sentinels to agree before promoting a replica; managed services such as

Amazon RDS Multi-AZ handle detection, promotion and fencing for you.

4 Redirecting clients, and what is lost

Redirection	How it works	Watch out for
DNS update	the endpoint name is repointed to the new primary	cached records: keep TTL short (e.g. 5–30 s), clients must re-resolve
Virtual IP	a floating IP moves to the new primary	usually limited to one network segment / zone
Proxy	HAProxy, PgBouncer, RDS Proxy always routes to the leader	the proxy itself needs redundancy
Client discovery	clients ask Sentinel / etcd who leads	every client library must support it

With **synchronous** replication every committed write is already on the standby, so nothing committed is lost. With **asynchronous** replication the standby lags; writes inside that lag window are lost. Either way, **in-flight transactions abort**, so clients must reconnect and retry with backoff (lesson 5).

WORKED EXAMPLE: MANAGED POSTGRES, MULTI-AZ

A managed PostgreSQL instance runs in multi-AZ mode with a **synchronous standby** in zone B. Zone A fails. The service detects it, promotes the standby and repoints the database's DNS name, which AWS documents as typically 60–120 seconds for RDS Multi-AZ. No committed data is lost. Open connections drop, so the app's connection pool reconnects with retries; users see a short burst of errors, then recovery. The team tests this regularly with a forced failover (reboot with failover).

COMMON MISTAKES

- No fencing, so a partitioned old primary keeps taking writes.
- Long DNS TTLs, or clients that resolve once at startup and never again.
- Apps that crash on a dropped connection instead of reconnecting with backoff.
- Never testing failover, then discovering a missing step during a real outage.

5 Key terms

Failover: moving the primary role to a standby

Split brain: two nodes both acting as primary

Lease: time-limited leadership that must be renewed

Promotion: making a replica the new writable primary

Fencing: preventing the old primary from writing

Replication lag: how far an async standby is behind

6 Interview questions

Q1. What are the steps of failover? BASIC

Detect the failure, decide it's real, promote the standby (and fence the old primary), then redirect clients to it.

Q2. Automated or manual failover? BASIC

Automated is fast but can act on false alarms and flap; manual is safer but slow. Automate the common case, keep a human for big decisions like region failover.

Q3. How long does failover take, and what can be lost? SCENARIO

Detection + decision + promotion + redirection: seconds to a few minutes for managed databases. Synchronous replication loses no committed writes; asynchronous loses the replication lag. In-flight transactions abort and must be retried.

Q4. After failover, half your app servers still write to the old primary. Why? SCENARIO

They cached the old DNS answer or hold old connections; use short TTLs, re-resolve on reconnect, and fence the old primary so its writes fail.

Q5. What's the biggest failover risk? TRICKY

Split brain: the old primary keeps accepting writes. Prevent it with fencing or leases.

Q6. Why not set the detection timeout to one second? TRICKY

Brief network blips or GC pauses would trigger failovers that cause more downtime and risk flapping than the blip itself.

Next: SD-K-012 · Disaster Recovery backendroadmap.com

Disaster Recovery Strategies

pay for the recovery speed you need

Watch the 4-min video

In one line: the four classic DR strategies (backup and restore, pilot light, warm standby, active-active) trade money for recovery speed and data loss; choose per system from its RTO and RPO, and only trust the one you have actually rehearsed.

1 A whole region goes dark

Not a server, not a zone: the whole region. Do you recover in ten minutes or in two days? That was decided months ago, when you chose a **disaster recovery** strategy and how much to pay for it. Zone redundancy (lesson 10) doesn't help when the entire region, its control plane or your account in it is unavailable.

2 Four strategies



Figure 1. From cheapest and slowest to most expensive and fastest. Each step keeps more of production running in the second region.

Strategy	In the DR region	RTO	RPO	Cost
Backup + restore	backups only; rebuild infra, then restore data	hours to a day	since last backup	lowest
Pilot light	replicated data, infra as code, app servers off	tens of minutes	seconds–minutes	low
Warm standby	scaled-down but working copy of production	minutes	seconds–minutes	medium
Active-active (multi-site)	full production, serving traffic	near zero	near zero	highest

Backup and restore is cheap because nothing runs, but you rebuild everything: infrastructure first, then hours of data restore. **Pilot light** keeps the core (data) always on and small; you start and scale the app tier during the disaster. **Warm standby** can take some traffic at once and is scaled up during failover. **Active-active** fails over almost instantly, but writes in two regions must be reconciled (lesson 15), which makes it the most complex option. These names and ranges follow AWS's DR whitepaper; other clouds describe the same ladder.

3 A plan you haven't tested is a hope

- Run **DR drills** that really fail over, measure the time, and compare it with your RTO and RPO (lesson 13).
- Keep **runbooks** current and **automate** the steps: during a real disaster people are stressed and make mistakes.
- Check the things people forget: DNS, secrets and certificates, quotas in the DR region, container images and third-party allow-lists.
- Keep backups in a separate account too, so the disaster can't be a deleted or compromised account (lesson 14).

WORKED EXAMPLE: PILOT LIGHT FOR AN INTERNAL EXPENSE APP

The app can tolerate **an hour** of downtime. A warm standby would roughly double its cost, so the team picks **pilot light**: the database replicates to a second region, and the app's infrastructure exists as **Terraform code** with **zero app servers** running. In a regional outage, a runbook launches the servers and switches DNS; the app is back in about **30 minutes**, inside its one-hour target, at a few percent of the cost of a second production. They drill it every quarter.

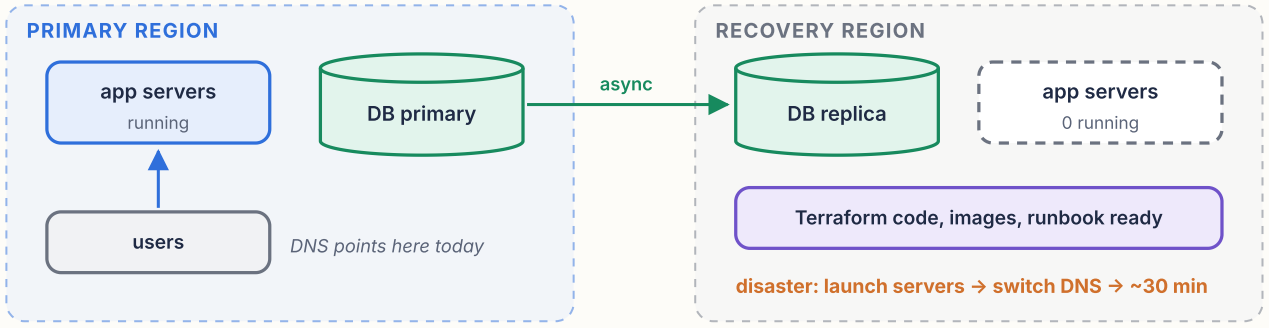


Figure 2. Pilot light: only the data is live in the recovery region; everything else is ready to start.

COMMON MISTAKES

- One strategy for every system, instead of matching each system's RTO/RPO.
- A DR region that depends on the primary region (its control plane, its DNS, its CI).
- Capacity assumptions: the DR region may not have quota or instances when everyone fails over.
- Never drilling, so the first real failover is also the first test.

4 Key terms

Disaster recovery (DR): restoring service after a large-scale loss

Backup and restore: rebuild from backups; cheapest, slowest

Pilot light: data live, app tier off until needed

Warm standby: small working copy, scaled up on failover

Active-active: every region serves traffic all the time

DR drill: a real, measured failover rehearsal

5 Interview questions

> **Q1. Name the four classic DR strategies.** BASIC
> Backup and restore, pilot light, warm standby and multi-site active-active, in order of rising cost and falling recovery time.

Q2. What's the difference between pilot light and warm standby? BASIC

Pilot light replicates data but keeps app servers off; warm standby runs a small working copy that can take traffic at once and is scaled up.

Q3. Compare DR strategies by cost and recovery time. SCENARIO

Backup and restore: cheapest, hours to a day. Pilot light: tens of minutes. Warm standby: minutes. Active-active: most expensive and complex, near-instant failover.

Q4. An internal app can be down for an hour. Which strategy? SCENARIO

Pilot light: replicate the data, keep infra as code, launch servers and switch DNS in about 30 minutes, at a fraction of a warm standby's cost.

Q5. How do you know your DR plan works? TRICKY

Run regular drills that really fail over, measure recovery against RTO and RPO, and fix the missing steps.

Q6. Why isn't active-active always the answer? TRICKY

It's the most expensive and complex: it needs multi-region data replication, conflict handling and capacity for full load in each region, which many systems don't justify.

Next: SD-K-013 · RTO and RPO backendroadmap.com

RTO and RPO

how long down, how much lost

Watch the 4-min video

In one line: RTO is the longest you may be down; RPO is the most data you may lose, measured in time. RPO drives how you copy data, RTO drives how you recover everything else, and lower numbers cost more, so set them per system from the business cost.

1 The two questions after every outage

How long were we down? How much data did we lose? Those answers have names. Agree them **before** the outage and they decide almost your whole recovery design (lessons 11–12) and its cost.

2 Definitions

RTO, recovery time objective: the maximum acceptable time from the start of an outage until service is restored. **RPO**, recovery point objective: the maximum acceptable data loss, measured in time. An RPO of one hour means you may lose up to the last hour of writes.

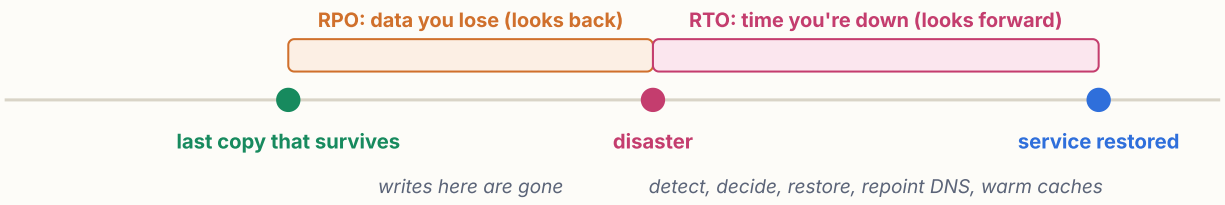


Figure 1. On one timeline, RPO looks backward from the disaster to the last copy that survives; RTO looks forward to recovery.

3 What each target forces you to build

Target	RPO needs (data)	RTO needs (everything else)
~24 hours	nightly backups	restore from backup, rebuild infra
~1 hour	hourly snapshots	pilot light: infra as code, data ready
minutes	continuous async replication or log shipping	warm standby, scripted failover
~zero	synchronous replication to another failure domain	active-active, automated failover

For RTO, count **every step**: detection, decision, restore or promotion, DNS changes, cache warm-up. A restore of a 2 TB database alone can take hours. RPO zero means each write is stored in two failure domains before it is acknowledged, which adds write latency (a cross-zone round trip on every commit).

4 Targets cost money, so set them per system

Synchronous replication slows every write; a hot standby doubles infrastructure. So targets belong to systems, not companies: payments might need **RPO 0, RTO minutes**; an internal reporting tool might accept **a day for both**. Tie each to the business cost of an hour down and an hour of lost data.

Then **test and monitor** them. Measure real restore times in drills, not estimates. Monitor **replication lag**: it is your real RPO right now. If lag reaches 10 minutes during a traffic spike, your 5-minute RPO is already broken; alert on it like an error rate.

WORKED EXAMPLE: AN RPO OF 5 MINUTES WITH NIGHTLY BACKUPS

The orders database has an RPO of **5 minutes** but only a 02:00 nightly backup. Lose it at 20:00 and up to **18 hours** of orders are gone. Fix: PostgreSQL continuous archiving, `archive_mode = on` and `archive_timeout = 60s`, ships the write-ahead log to object storage at least every minute, and a cross-region replica streams changes continuously. Worst-case loss is now about a minute (point-in-time recovery replays the WAL), and an alert fires if replica lag passes **3 minutes**.

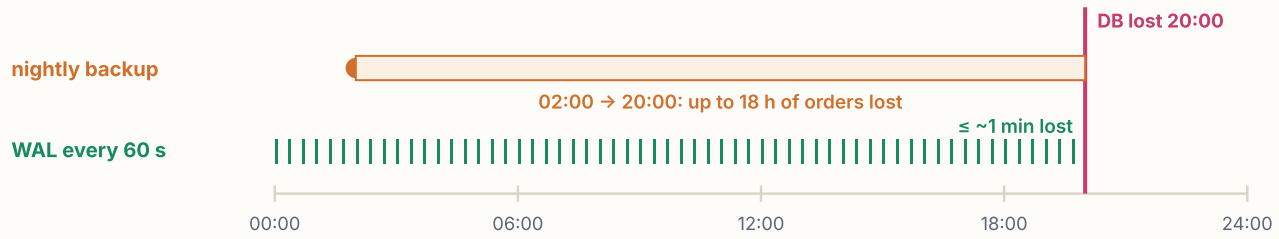


Figure 2. The same failure with two backup designs. The gap between the last surviving copy and the disaster is the data you lose.

COMMON MISTAKES

- Mixing them up: RTO is time down, RPO is data lost.
- One RTO/RPO for every system, usually too strict and too expensive.
- Promising an RPO while replication lag is not monitored.
- Counting only the restore, not detection, decision and DNS, in the RTO.

5 Key terms

RTO: maximum acceptable time to restore service

RPO: maximum acceptable data loss, measured in time

Synchronous replication: write acknowledged only once stored twice

WAL archiving: shipping the database's write-ahead log to storage

Point-in-time recovery: restore a backup, then replay logs to a moment

Replication lag: how far a replica is behind: your live RPO

6 Interview questions

Q1. What's the difference between RTO and RPO? BASIC

RTO is how long you can be down; RPO is how much data, measured in time, you can afford to lose.

Q2. What RPO do nightly backups give? BASIC

Up to 24 hours: anything written since the last backup is lost.

Q3. Your RPO is zero: what does the design need? SCENARIO

Synchronous replication to at least one other failure domain, so every acknowledged write already exists in two places, accepting extra write latency and cost.

Q4. The business wants RTO 15 minutes for a service restored from backups. Is it realistic? SCENARIO

Usually not: restoring and verifying a large database takes hours. You'd need a pilot light or warm standby with replicated data and scripted failover.

Q5. Your replica's lag hits 10 minutes during a spike. What does that mean for RPO? TRICKY

Your effective RPO is now 10 minutes; if the primary dies, that lag is lost. Alert on lag and fix capacity or throttling before it breaks the target.

Q6. Does RPO zero protect you from a bad DELETE? TRICKY

No: synchronous replication copies the mistake instantly. You need point-in-time recovery from backups and logs for that.

Next: SD-K-014 · Backups and Restore Testing backendroadmap.com

Backups and Restore Testing

a backup you never restored is a hope

Watch the 4-min video

In one line: keep full, incremental and log-based backups that give point-in-time recovery, store immutable copies in another region and account, and prove them by restoring automatically and regularly, because a backup only counts once you've restored it within your RTO.

1 Three years of backups, none usable

A company backed up its database every night for three years. Ransomware hits. They go to restore: the backups are empty, corrupted, or encrypted too, because the attacker had the same admin access. Nobody had ever tried a restore. A backup you have never restored is a hope, not a recovery plan.

2 Kinds of backup

Kind	Copies	Restore	Trade-off
Full	everything	one step	slow and large
Incremental	changes since the last backup	full + every incremental, in order	small and fast; long fragile chain
Snapshot	a disk or volume at a point in time (copy-on-write)	create a volume / instance from it	cheap and quick; often same account and region
Log archive (PITR)	base backup + continuous transaction log	restore base, replay log to any second	finest RPO; needs the whole log chain

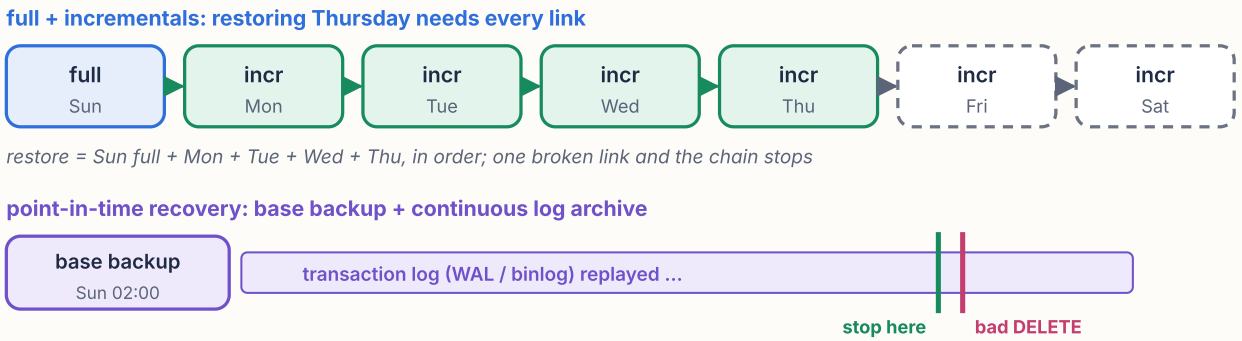


Figure 1. Top: an incremental chain. Bottom: point-in-time recovery replays the log up to just before the mistake.

Point-in-time recovery (PostgreSQL WAL archiving, MySQL binlogs, managed PITR in RDS or Cloud SQL) turns a daily recovery point into a per-second one: restore the base backup, then replay the log up to one second before someone ran the bad `DELETE` (lesson 13).

3 Backups must survive what production doesn't

- **3-2-1 rule:** three copies, on two kinds of media, one off-site. In the cloud: another region *and* another account.

- **Separate account:** one compromised admin key must not be able to delete production and backups together.
- **Immutable:** S3 Object Lock in compliance mode (or Azure immutable blob storage) means nobody, not even the root user or an attacker, can delete or change a backup before retention ends.
- **Keys:** back up and test access to the encryption keys; an encrypted backup without its key is noise.

4 Restore testing

Restores fail for hidden reasons: missing encryption keys, a changed schema, an expired IAM permission, or a backup job that silently stopped weeks ago. Restore **time** matters too: a 10 TB restore may take far longer than your RTO. The only proof is a real restore, done regularly and automatically.

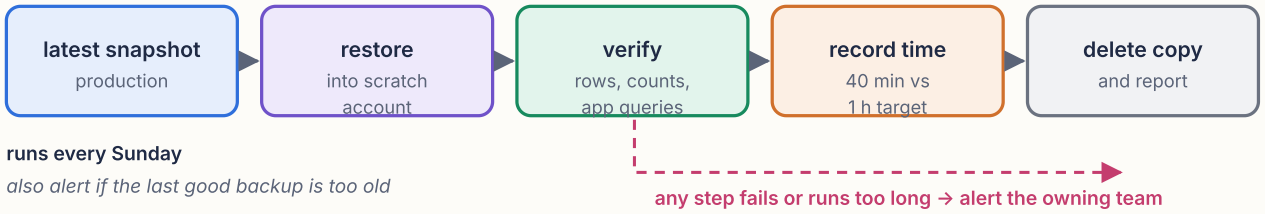


Figure 2. An automated weekly restore test. It measures restore time against the RTO and fails loudly.

WORKED EXAMPLE: THE SUNDAY RESTORE PIPELINE

Every Sunday a pipeline restores the latest production snapshot into a **scratch account**. It checks that the orders table has today's rows, compares row counts with production, and runs a few read-only queries from the app. It records the restore time, currently **40 minutes** against a **1-hour** target, then deletes the copy. One week the restore failed: a **key rotation** had broken access to older snapshots. They fixed it months before a real disaster would have found it.

COMMON MISTAKES

- Backups in the same account and region as production.
- Monitoring that the backup job ran, not that its output restores.
- Never measuring restore time, then discovering it's ten times the RTO.
- Treating replicas as backups: replication copies deletes and corruption instantly.

5 Key terms

Full backup: complete copy of the data

Incremental backup: only what changed since the last backup

Snapshot: point-in-time image of a volume, usually copy-on-write

PITR: base backup + log replay to any moment

3-2-1 rule: 3 copies, 2 media, 1 off-site

Object lock: write-once storage: no delete before retention ends

6 Interview questions

> **Q1. Full vs incremental backup?** BASIC
> A full backup copies everything and restores in one step; an incremental copies only changes and needs the full plus every incremental in order to restore.

Q2. What is point-in-time recovery? BASIC

Restoring a base backup and replaying the archived transaction log up to a chosen moment, e.g. just before a bad delete.

Q3. How do you protect backups from ransomware? SCENARIO

Keep immutable copies (object lock) in a separate account and region, with tight, separate access, so a compromised production admin can't touch them.

Q4. Design restore testing for a critical database. SCENARIO

An automated, scheduled restore into an isolated account, data checks (counts, recent rows, app queries), restore time recorded against the RTO, alerts on failure, then cleanup.

Q5. Why is a backup not a backup until restored? TRICKY

Restores fail for hidden reasons (missing keys, changed schemas, expired permissions, silently stopped jobs) and only a real restore proves the data is usable and the time fits your RTO.

Q6. We have synchronous replicas in three zones. Do we need backups? TRICKY

Yes: replicas protect against hardware and zone loss, but they replicate bad writes, deletes and ransomware encryption instantly.

Next: SD-K-015 · Multi-Region Architecture backendroadmap.com

Multi-Region Architecture

every region on its own, every record one home

Watch the 4-min video

In one line: going multi-region means choosing active-passive or active-active, accepting that cross-region latency makes synchronous writes slow, and avoiding write conflicts, usually by giving every user or tenant one home region, while keeping regions independent of each other.

1 Two questions from the CEO

The app runs well in one region. Then: "what if the whole region goes down?" (lesson 12), and "why are pages slow for customers in Asia?" Going multi-region answers both, but it changes almost everything about how you handle **data**.

2 Two shapes

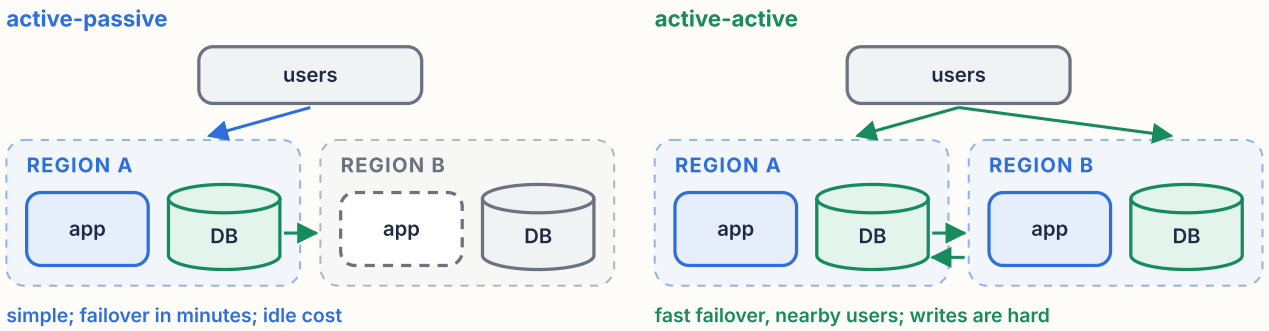


Figure 1. Active-passive keeps a standby region with replicated data; active-active serves traffic from every region at once.

Active-passive: one region serves all traffic, a second stands by with replicated data. Simpler, but failover takes minutes and the passive region is idle cost. **Active-active**: every region serves at once, so users get a nearby region and failover is just routing, but now two regions can write.

3 Why writes are hard

Regions are tens to hundreds of milliseconds apart (roughly 70–100 ms round trip between the US East Coast and Western Europe). **Synchronous** replication adds that to every write. **Asynchronous** replication is fast, but two regions can change the same record at once, and you get conflicts.

Approach	How	Cost
Home region per user/tenant	each record has exactly one writer region; reads may be local	re-homing on failure; cross-tenant features are harder
Synchronous / consensus	commit across regions (e.g. Spanner, CockroachDB)	cross-region latency on every write
Last writer wins	keep the write with the latest timestamp	silently drops a concurrent update
CRDTs / merge	data types that merge concurrent changes (counters, sets)	only fits some data; more complex

4 Home regions, isolation and sovereignty

The most common design avoids conflicts: each user or tenant has a **home region** where all its writes go. Data of different tenants never conflicts. Keep regions **isolated like cells** (lesson 16): no cross-region calls in the request path, so a failure in one can't spread. Home regions also make **data residency** natural, for example keeping EU customers' data in the EU when GDPR or a contract requires it.

WORKED EXAMPLE: A SAAS APP IN THREE REGIONS

The app runs in the US, Europe and Asia. At sign-up each tenant gets a **home region**, stored in a small, highly replicated global directory. A routing layer looks up the tenant and sends every request to its home region, where all reads and writes happen. Each region's data is replicated **asynchronously** to a backup region. If Europe fails, its tenants are re-homed to the backup, accepting the replication lag, a few seconds of data, as loss.

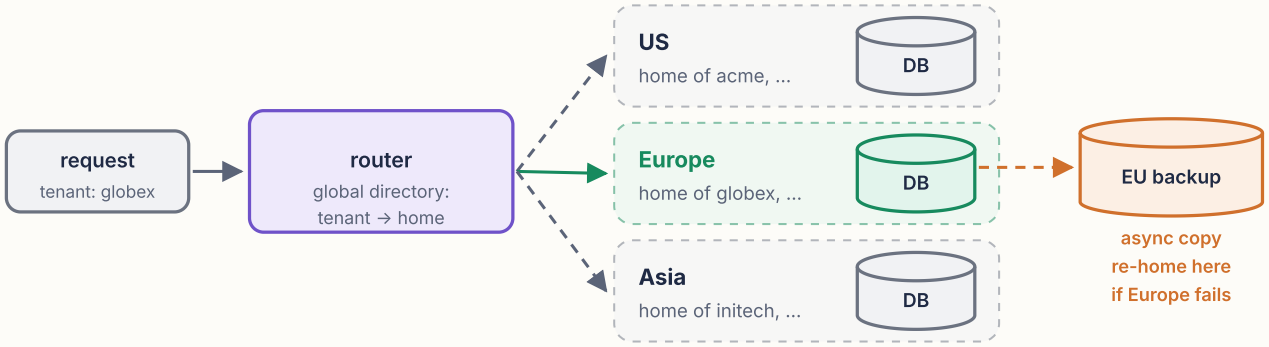


Figure 2. Routing by tenant: every tenant's data has one writer region, so there are no write conflicts to resolve.

THE COSTS

- Duplicate infrastructure, plus cross-region data transfer charges.
- Deploys, monitoring and on-call across regions; config drift between them.
- Global pieces (identity, DNS, the tenant directory) need their own careful design.
- Go multi-region for a real need: availability, latency or compliance, not because it sounds robust.

5 Key terms

Active-passive: one region serves; another stands by

Active-active: all regions serve traffic at once

Home region: the one region that owns a user's or tenant's writes

Write conflict: two regions change the same record concurrently

CRDT: data type whose concurrent updates merge automatically

Data residency: rules on where data may be stored

6 Interview questions

Q1. Active-passive vs active-active? BASIC

Active-passive serves from one region with a standby that takes over in minutes; active-active serves from all regions at once, giving low latency and fast failover but hard writes.

Q2. Why not replicate synchronously across regions? BASIC

Every write would wait for a cross-region round trip, tens to hundreds of milliseconds, and an unreachable region could block writes.

Q3. What changes when you go multi-region? SCENARIO

Cross-region latency makes synchronous writes slow, so you choose active-passive or active-active, design for replication lag and conflicts, isolate regions, and pay more in cost and complexity.

Q4. How do you handle writes in two regions? SCENARIO

Avoid conflicts with a home region per user or tenant (one writer region per record), or resolve them with last-writer-wins or CRDTs.

Q5. Your regions call each other's APIs for some requests. Problem? TRICKY

Yes: it couples their availability and latency, so one region's outage spreads to the other. Each region should serve its requests on its own.

Q6. What's the risk of last-writer-wins? TRICKY

Concurrent updates are silently lost, and clock skew between regions can make the 'last' write the wrong one.

Next: SD-K-016 · Cell-Based Architecture backendroadmap.com

Cell-Based Architecture

make every outage a small outage

Watch the 4-min video

In one line: split the system into independent cells, each a complete stack serving a subset of customers, behind a thin router, so a bug, overload or bad deploy hits one cell's customers instead of all of them; shuffle sharding shrinks the blast radius of a single bad customer even further.

1 Everyone down at once

A bad configuration change reaches the service and every customer goes down together. Now imagine the same bug hits **5%** of customers and everyone else never notices. That is the promise of cells: they limit the **blast radius**, the share of customers one failure can reach (lesson 7's bulkheads, applied to the whole system).

2 What a cell is

A **cell** is a complete, independent copy of the service stack: its own load balancer, app servers, database and queues. Each serves a subset of customers. Cells **share no state** and **don't call each other**. If one fails, from a bug, a poison request, overload or a bad deploy, only its customers are affected.

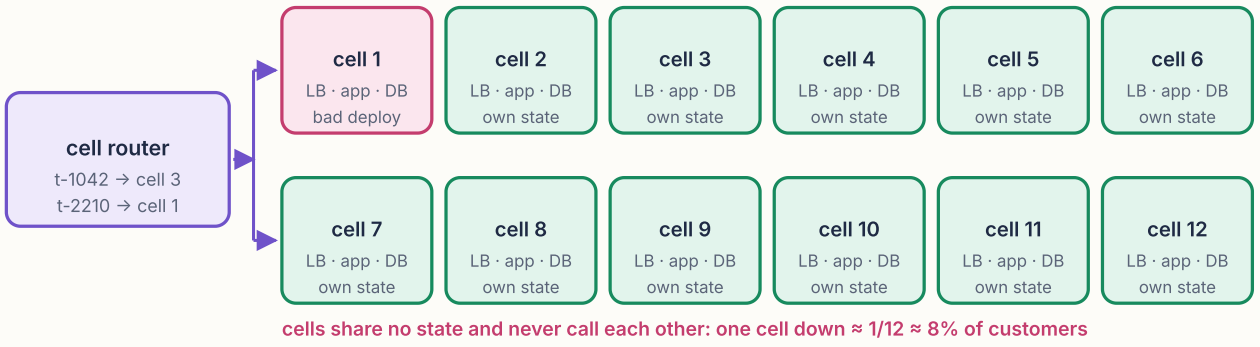


Figure 1. Twelve cells behind a router that maps each account to its cell.

Cells have a **fixed maximum size**, so you can load-test exactly that size, and you grow by **adding cells**, not by making one huge system bigger. The **cell router** in front only looks up the customer's cell (by account or tenant ID in a mapping table) and forwards the request. It is shared by everyone, so it must be very simple and very reliable: no business logic, and cached mappings so it keeps routing if the mapping store is down (lesson 18).

3 Safer deploys, and noisy neighbours

- **Staged rollout by cell:** deploy to one cell, watch its alarms, then a few more, then the rest. A bad change is caught while it affects one cell (lesson 75).
- **Isolation of big tenants:** move a noisy customer into its own cell so it can't hurt anyone else.
- **Migration** between cells is needed for rebalancing; design for it from the start.

4 Shuffle sharding

Instead of fixed groups, give each customer a **random combination** of workers, say 2 of 8. There are C(8,2) = 28 possible pairs, so two customers rarely share exactly the same pair. If one customer sends poison requests that crash its workers, other customers almost always keep at least one healthy worker. AWS describes using this technique for Route 53's name servers.

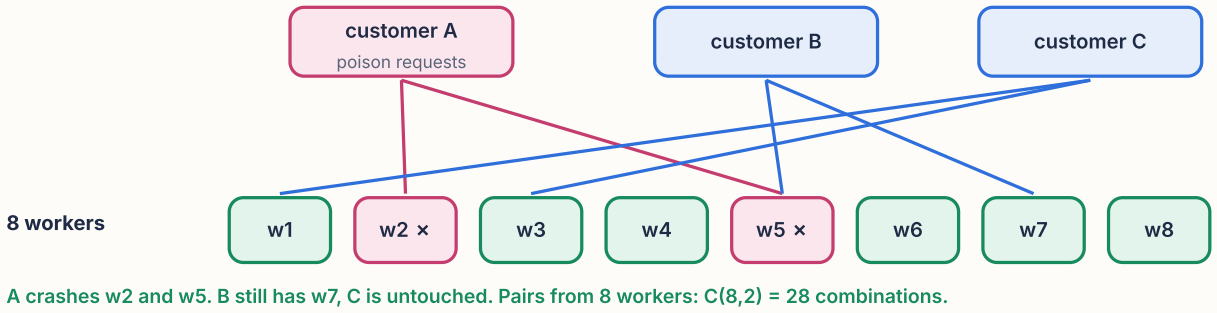


Figure 2. Customer A's poison requests take out its two workers. Only a customer with exactly the same pair would lose all its capacity.

Layout	One bad deploy reaches	One poison customer reaches
One shared fleet	100% of customers	100% of customers
12 cells	1 cell first: ≈ 8%	its cell: ≈ 8%
Shuffle shards (2 of 8)	depends on rollout	fully: only customers on the same pair (1 in 28 ≈ 3.6%); partly: those sharing one worker

WORKED EXAMPLE: TWELVE CELLS AND A CACHE BUG

A service runs as **12 cells**, each with its own load balancer, compute and database, sized for a few thousand customers. A cell router maps each account to a cell. A deploy contains a bug that corrupts a cache. It reaches **cell 1** first, alarms fire, and the pipeline rolls back automatically. About **8%** of customers (1 of 12) saw errors for a few minutes; the other 92% saw nothing, and the bug never reached the other cells.

TRADE-OFFS

- More infrastructure and some duplicated fixed cost per cell.
- Operations across many cells: deploys, dashboards and migrations must be automated.
- Cross-customer features (global search, reports) don't fit naturally.
- The router and the cell mapping become a critical shared component.

5 Key terms

Cell: complete, independent stack serving a subset of customers

Cell router: thin layer mapping each customer to a cell

Shuffle sharding: random combination of workers per customer

Blast radius: share of customers one failure reaches

Partition key: what decides the cell: account or tenant ID

Poison request: input that crashes whatever processes it

6 Interview questions

Q1. What is a cell? BASIC
A complete, independent copy of the service stack (load balancer, compute, data) serving a subset of customers, sharing no state with other cells.

Q2. Why must the cell router be simple? BASIC
Every request passes through it, so it is the one shared component; any complexity or dependency in it can take all cells down.

Q3. How does cell-based architecture limit outages? SCENARIO
A bug, overload or bad deploy affects only one cell's customers, and staged rollouts cell by cell catch problems while only one cell is exposed.

Q4. One huge tenant keeps overloading its cell. What do you do? SCENARIO
Move it to a dedicated cell (or a larger cell class) so its load can't affect the others.

Q5. What is shuffle sharding? TRICKY
Giving each customer a random combination of workers, so one bad customer's failure rarely takes out all of another customer's capacity.

Q6. Why give cells a fixed maximum size? TRICKY
You can test that exact size thoroughly and avoid the unknown scaling limits of one ever-growing system; growth means adding cells.

Next: SD-K-017 · Chaos Engineering backendroadmap.com

Chaos Engineering

break it on purpose, at a time you choose

Watch the 4-min video

In one line: chaos engineering runs controlled, hypothesis-driven experiments that inject real failures (killed pods, latency, errors, lost zones) and checks whether steady-state metrics hold, starting tiny with automatic abort conditions, so you find weaknesses before your customers do.

1 Have you ever seen your failover work?

The system has failover (lesson 11), retries (lesson 5) and circuit breakers (lesson 6). But nobody has watched them work. Many teams discover during a real outage that failover was misconfigured or a timeout was ten times too long. Chaos engineering finds those gaps **on purpose**, during working hours, with the team watching.

2 An experiment, not random breakage

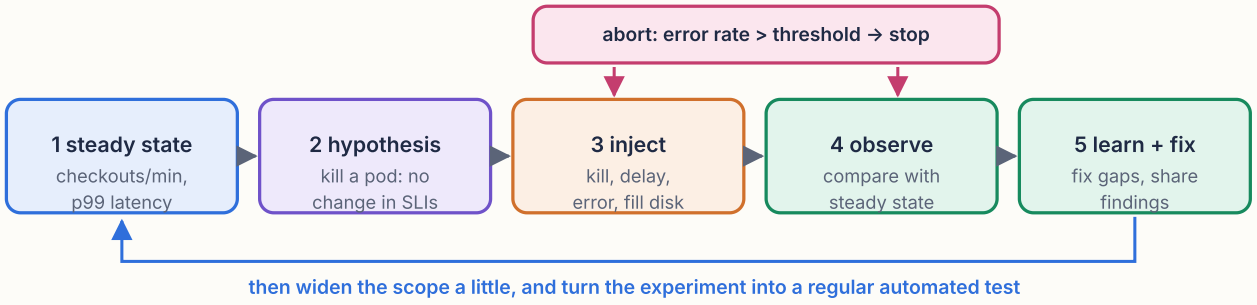


Figure 1. The chaos experiment loop. The abort condition is defined before anything is injected.

Define the **steady state** as user-facing metrics, e.g. successful checkouts per minute or p99 latency (your SLIs, lesson 2). State a **hypothesis**: "if we kill one instance, the steady state won't change". **Inject** a realistic failure and compare. If the metrics hold, confidence grows; if not, you've found a weakness cheaply.

Fault	Example injection	What it tests
Instance / pod loss	terminate a random pod or VM	redundancy, health checks, session handling
Network	add 300 ms latency or 5% packet loss to a dependency	timeouts, retries, circuit breakers
Dependency errors	make a downstream return 503s	fallbacks and graceful degradation
Resources	fill a disk, burn CPU, exhaust memory	alerts, autoscaling, limits
Zone / region	block a whole zone's traffic	failover, capacity of the remaining zones

3 Safety first: start small, widen slowly

Start with **one instance, in staging, in business hours**, with the team watching. Define **abort conditions** in advance, such as error rate above 1–2%, and stop automatically when they are crossed. Widen the scope only after each step passes.

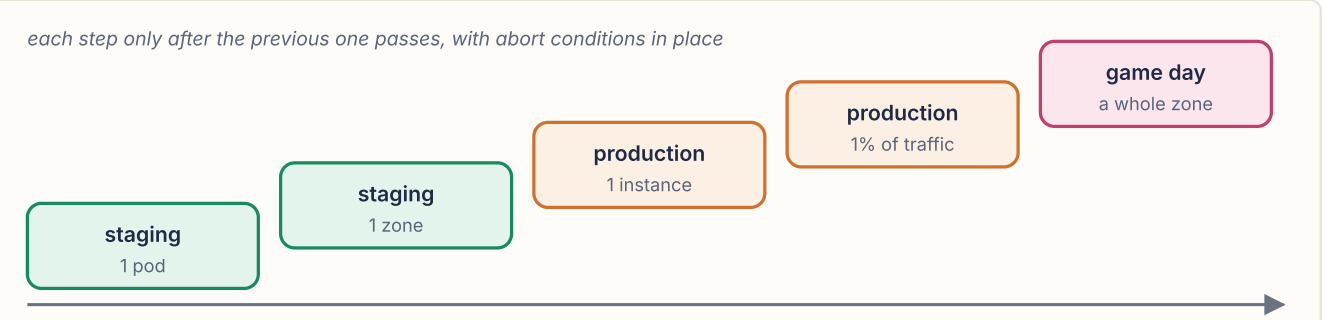


Figure 2. Growing the blast radius step by step.

A **game day** is a planned session where the team runs bigger experiments together, such as failing over a database or losing a zone, and practises the incident response (lesson 70). Tools: Netflix's **Chaos Monkey** randomly terminates instances; **LitmusChaos** and **Chaos Mesh** run experiments in Kubernetes; **AWS Fault Injection Service** injects faults into cloud resources with stop conditions tied to CloudWatch alarms; Azure has Chaos Studio.

Typical findings: a timeout that's too long, a retry storm, an alert that never fires, a stale runbook, a dependency nobody knew about. Each becomes a fix, results are shared, and the experiment becomes a **regular automated test** so the weakness can't quietly return.

WORKED EXAMPLE: KILLING A POD EVERY TUESDAY

A scheduled job kills **one random pod of each service** in staging every Tuesday. Hypothesis: requests keep succeeding and the pod is replaced within a minute. Abort condition: staging error rate **above 2%**. Week one, checkout failed: it kept user sessions **in memory**, so users on the killed pod were logged out. The team moved sessions to **Redis**; the experiment has passed every week since and now guards against the regression.

COMMON MISTAKES

- Starting in production with a big blast radius and no abort condition.
- No steady-state metric, so nobody can say whether the experiment passed.
- Running experiments at night or without the owning team watching.
- Finding weaknesses and not fixing them, or not repeating the experiment.

4 Key terms

Chaos engineering: controlled experiments that inject failures

Hypothesis: the expected result: steady state holds

Abort condition: threshold that stops the experiment automatically

Steady state: metrics that show normal, healthy behaviour

Blast radius: how much an experiment can affect

Game day: planned team exercise with bigger failures

5 Interview questions

Q1. What is chaos engineering? BASIC
Running careful, hypothesis-driven experiments that inject real failures to check that the system's steady state holds, and fixing what breaks.

Q2. What is a game day? BASIC
A planned session where the team injects bigger failures together, like a database failover or a lost zone, and practises the incident response.

Q3. How would you introduce chaos engineering safely? SCENARIO
Define steady-state metrics and a hypothesis, start with a tiny blast radius in staging during business hours, set automatic abort conditions, widen gradually, and turn findings into fixes and automated experiments.

Q4. Which experiment would test your timeouts and circuit breakers? SCENARIO
Inject latency or errors into one dependency, e.g. 300 ms delay or 503s, and check that callers time out, open the breaker and serve fallbacks while SLIs hold.

Q5. Isn't chaos engineering just breaking things randomly? TRICKY
No: each experiment has a hypothesis, a measured steady state, a limited blast radius and abort conditions; the goal is evidence, not damage.

Q6. Why run experiments in production at all? TRICKY
Staging differs in traffic, data, config and scale; some weaknesses only appear in production, so once staging passes, small controlled production experiments give real confidence.

Next: SD-K-018 · Static Stability backendroadmap.com

Dependency Management and Static Stability

keep working without changing anything

Watch the 4-min video

In one line: a statically stable system keeps serving when a dependency or the control plane fails, without having to change anything, because capacity is pre-provisioned and config, credentials and routes are cached as last known good; keep hard dependencies few and prove soft ones are soft.

1 Same outage, two outcomes

A cloud control plane has an outage: nobody can launch instances or change settings. One company's services keep running. Another's crash as soon as they restart, because they can't fetch their configuration at startup. The difference is **static stability**: the ability to keep working when dependencies fail, **without needing to react, scale or reconfigure**. For every dependency ask: if this goes down, do we keep serving?

2 Control plane vs data plane

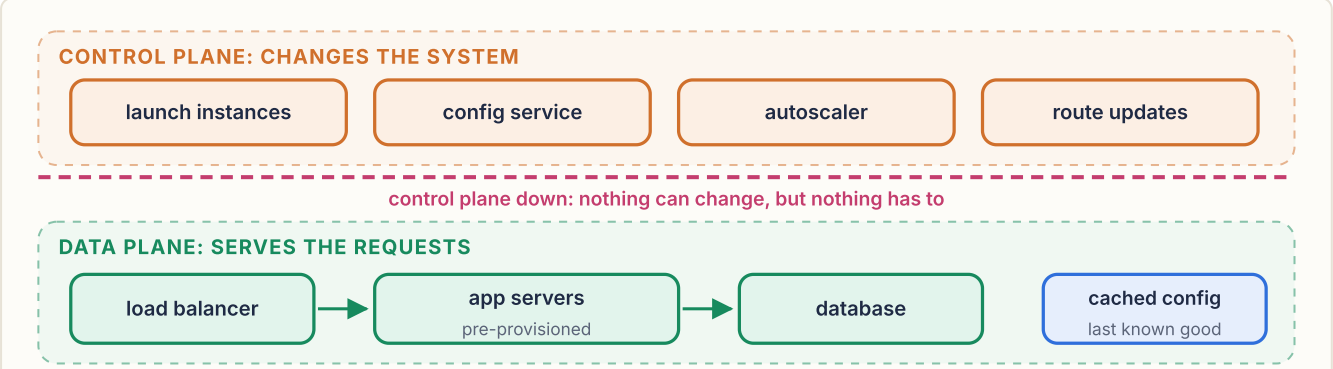


Figure 1. The data plane must not need the control plane to keep serving.

The **data plane** does the real work: serving requests, moving packets, reading data. The **control plane** changes the system: launching instances, updating config, changing routes. Control planes are more complex and change more often, so they fail more often. Design the data plane to keep running, with what it already has, while the control plane is down. AWS's Builders' Library describes this for EC2 and Availability Zones.

3 How to build it

- **Pre-provision capacity:** run enough in each zone that losing one needs no new launches mid-crisis (lesson 10's zone sizing). Launching is a control-plane action.
- **Cache locally** config, credentials, feature flags and service lists; keep using the **last known good** copy if refresh fails.
- **Push, don't pull per request:** distribute config to services ahead of time instead of fetching it on the request path.
- **Start without hard dependencies:** boot from cached values, report "degraded", refresh in the background.

Startup is the common trap. If a service refuses to start unless the config service, the flag service and the secrets store all answer, any one of their outages blocks every restart, deploy and autoscaling event, exactly when you need them.

4 Hard vs soft dependencies

Dependency	Hard or soft?	If it fails
main database (writes)	hard	writes fail; keep it highly available (lessons 10–11)
config / feature-flag service	soft	use cached last known good values
recommendations, analytics	soft	fallback or skip (lesson 8)
secrets store	soft at runtime	keep cached credentials until they expire; renew early
autoscaler / instance launch	soft if pre-provisioned	serve with existing capacity

Keep hard dependencies **few**, write them down, and **test** that the soft ones really are soft by switching them off, which is a chaos experiment (lesson 17).

WORKED EXAMPLE: CONFIG CACHED ON DISK

Every service fetches its configuration every **30 s** and writes it to a local `config.json`. On startup it reads the file first, so it can start immediately. One day the config service is down for an **hour**. Running services keep their last known good config, new pods start from the cached file, and autoscaling still works. Users see nothing; config *changes* simply wait until the service recovers.

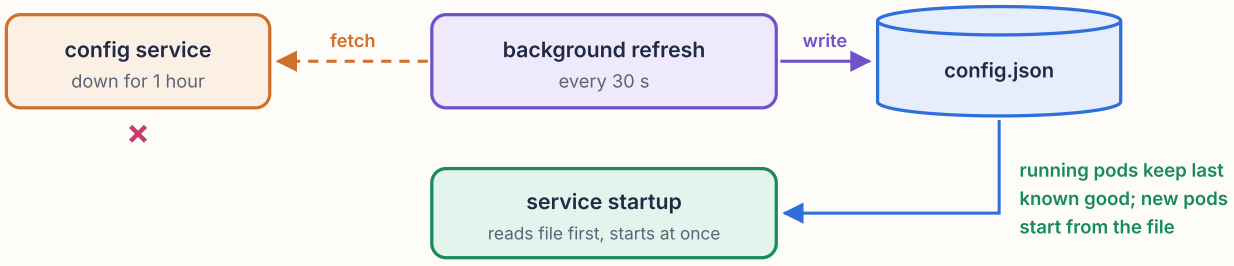


Figure 2. The config service is on the change path, not the serving path, so its outage only delays updates.

COMMON MISTAKES

- Services that must reach config, flags or secrets to boot.
- Relying on autoscaling or new launches to survive a zone loss.
- Fetching config or permissions on every request instead of caching.
- "Soft" dependencies that were never tested and turn out to be hard.

5 Key terms

Static stability: keeps working through failures without changing anything

Data plane: the part that serves requests

Control plane: the part that changes and manages the system

Last known good: most recent valid cached state

Hard dependency: can't serve without it

Soft dependency: can fail with a fallback

6 Interview questions

Q1. What is static stability? BASIC

A system's ability to keep working when a dependency or control plane fails, without making any changes, relying on pre-provisioned capacity and cached, last-known-good state.

Q2. What's the difference between the control plane and the data plane? BASIC

The data plane serves the actual requests; the control plane changes and manages the system. The data plane shouldn't need the control plane to keep running.

Q3. Your services can't start when the config service is down. How do you fix it? SCENARIO

Cache config to local disk on every refresh, start from the cached file, report degraded and refresh in the background.

Q4. How do you survive a zone loss if instance launches are failing? SCENARIO

Pre-provision: run enough capacity in each zone that the remaining zones carry full load without launching anything.

Q5. Why are control planes less reliable than data planes? TRICKY

They are more complex, change more often and do more coordination, so they fail more often; a data plane mostly repeats simple work.

Q6. Is caching last known good config always safe? TRICKY

Mostly, but it can keep serving a bad config or revoked credentials; add validation before accepting new config and a maximum age for sensitive values.

Next: SD-L-001 · Monolith vs Microservices backendroadmap.com

Monolith vs Microservices

independence costs a distributed system

Watch the 4-min video

In one line: a monolith is one deployable with fast in-process calls and real transactions; microservices buy independent deployment, scaling and team autonomy at the price of network failures, eventual consistency and a lot of operations work, so they pay off only when that independence is worth more than the overhead.

1 Forty services and a pager that never stops

Many teams start with microservices because big tech companies talk about them. Months later they have forty services, a message broker, and every release still needs five teams. Microservices solve real problems, but mostly problems of **people and deployment**, not code. Without those problems you pay the cost and get no benefit.

2 What each style gives you

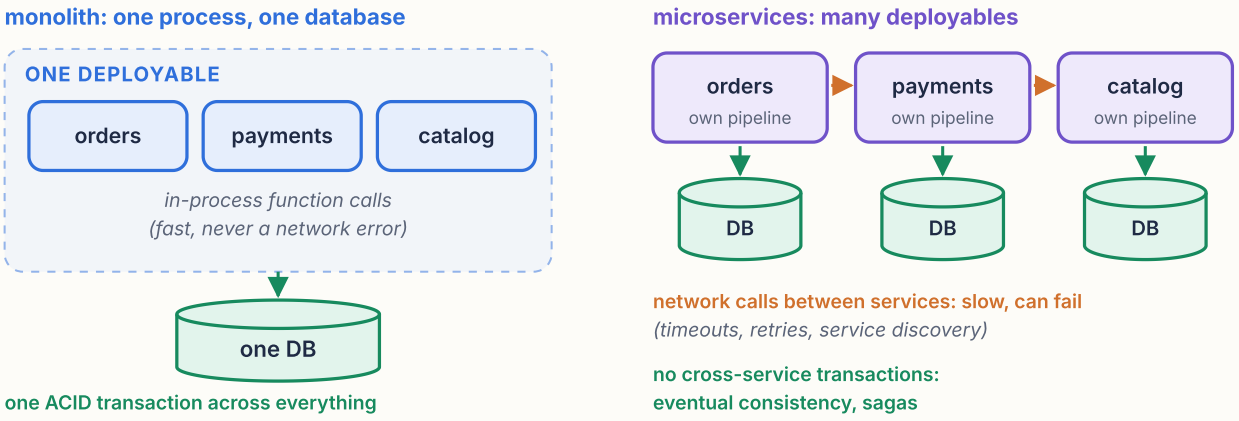


Figure 1. A monolith keeps calls and transactions local; microservices split code **and data**, so every hop is a network call.

A **monolith** is one deployable unit (often several identical copies behind a load balancer) and usually one database. Calls can't time out, one transaction keeps orders and stock consistent, and one stack trace shows the whole request.

Microservices split the system into services that each own their data and are deployed on their own, ideally by one team each. The big win is **independent deployment**: the payments team can ship ten times a day without a release train. Services can also be **scaled separately**, use different technology, and fail separately, so a bug in recommendations need not take down checkout.

3 What you pay

- **The network:** every call can be slow, fail, or succeed without you knowing. You now need timeouts, retries, circuit breakers and idempotency.
- **Data:** each service owns its database (lesson 23), so there are no joins or transactions across services. You accept eventual consistency and stale copies.

- **Operations and testing:** service discovery, a gateway, central logs, metrics and tracing, a pipeline per service, and contract tests for features that span services.

	Monolith	Microservices
Calls between parts	in-process, ~ns, can't fail on the network	HTTP/gRPC or messages, ~ms, can time out
Consistency	one ACID transaction	eventual; sagas for multi-step changes (lesson 34)
Deployment	all or nothing, one pipeline	per service, many pipelines
Scaling	scale the whole app	scale each service separately
Testing & debugging	one process, one stack trace	contract tests, distributed tracing

4 The distributed monolith trap and Conway's Law

A **distributed monolith** has separate services but keeps the coupling: they **share a database**, must be **deployed in lock-step**, or call each other in **long synchronous chains** (A → B → C → D on every request). You get the costs of microservices without the independence.

Conway's Law (Melvin Conway, 1968): a system's design mirrors the communication structure of the organisation that builds it. So service boundaries should match team boundaries (lesson 22). If every feature needs three teams, the boundaries are wrong.

WORKED EXAMPLE: SPLITTING OUT PAYMENTS FOR COMPLIANCE

An online store runs as a monolith. Card data (under PCI DSS rules) flows through the shared codebase and database, so the **whole** application is in audit scope. The team extracts one **payments service** with its own database (a card vault), network zone and pipeline. The monolith sends only an amount and order ID through a narrow API and gets back a payment token; it never sees card numbers. Audit scope shrinks to one small service, and the rest stays a monolith: one extraction for a real need, not a rewrite.

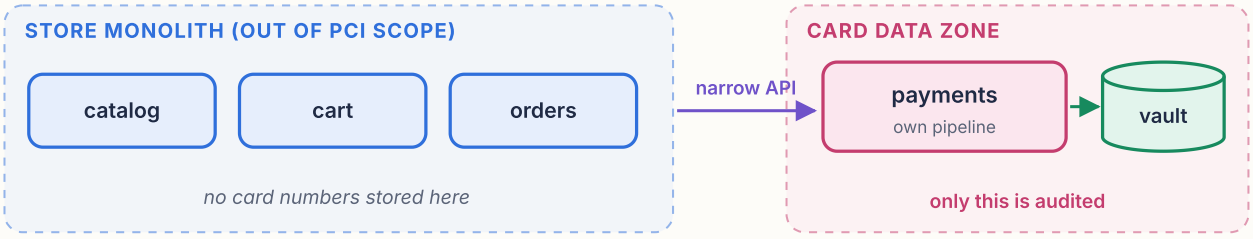


Figure 2. Only the payments service and its vault handle card data, so only they are in compliance scope.

WHEN NOT TO USE MICROSERVICES

- The team is small: one or two teams share one codebase easily.
- The domain is new or changing fast, so you don't yet know where the stable boundaries are.
- You lack strong automation: CI/CD, infrastructure as code, monitoring and tracing.
- No concrete pain yet. Start with a modular monolith (lesson 20) and split when a clear boundary, scaling need or team bottleneck appears (Fowler's "MonolithFirst").

COMMON MISTAKES

- Starting with microservices before the domain boundaries are known.
- Splitting the code but keeping one shared database: a distributed monolith.
- Splitting by technical layer instead of business capability.
- Adding services without tracing, per-service pipelines or on-call owners.

5 Key terms

Monolith: one deployable unit, usually with one database

Independent deployment: release one service without releasing others

Conway's Law: systems mirror the communication structure of the teams

Microservice: small, independently deployable service owning its data

Distributed monolith: separate services that are still tightly coupled

Eventual consistency: copies agree after a delay, not instantly

6 Interview questions

Q1. What is the main benefit of microservices? BASIC

Independent deployment: each team can build, test and release its service on its own schedule. Separate scaling and fault isolation follow from that.

Q2. What does a monolith give you that microservices don't? BASIC

Fast in-process calls that can't fail on the network, one ACID transaction across all data, and one thing to build, test, deploy and debug.

Q3. When should you NOT use microservices? SCENARIO

With a small team, an unclear or fast-changing domain, or weak automation and observability. Then the network, consistency and operations costs outweigh the benefits; start with a modular monolith.

Q4. Your store must pass card-data audits, and the whole monolith is in scope. What do you do? SCENARIO

Extract just payments into its own service, database, network zone and pipeline behind a narrow API, so only that service handles card data and the audit scope shrinks.

Q5. You have 30 services but every release still needs 5 teams. What's wrong? TRICKY

It's a distributed monolith: shared databases, lock-step deploys or boundaries that don't match teams. Fix the boundaries and data ownership, or merge services back.

Q6. How does Conway's Law affect service design? TRICKY

Systems mirror team communication, so service boundaries should match team boundaries; otherwise every feature needs cross-team coordination and the independence is lost.

Next: SD-L-002 · Modular Monolith backendroadmap.com

Modular Monolith

microservice discipline, no network

Watch the 4-min video

In one line: a modular monolith is one deployable split into modules with strong, **enforced** boundaries: each module owns its logic and tables and exposes a small public API, so you get clean code, in-process calls and real transactions now, and an easy extraction path later.

1 Microservices from day one?

A startup has six engineers and one product, and someone proposes microservices. That buys network calls, many databases and a lot of operations work before there is any team bottleneck to solve (lesson 19). The middle path is the **modular monolith**: the discipline of service boundaries without the distributed system.

2 One deployable, strong modules

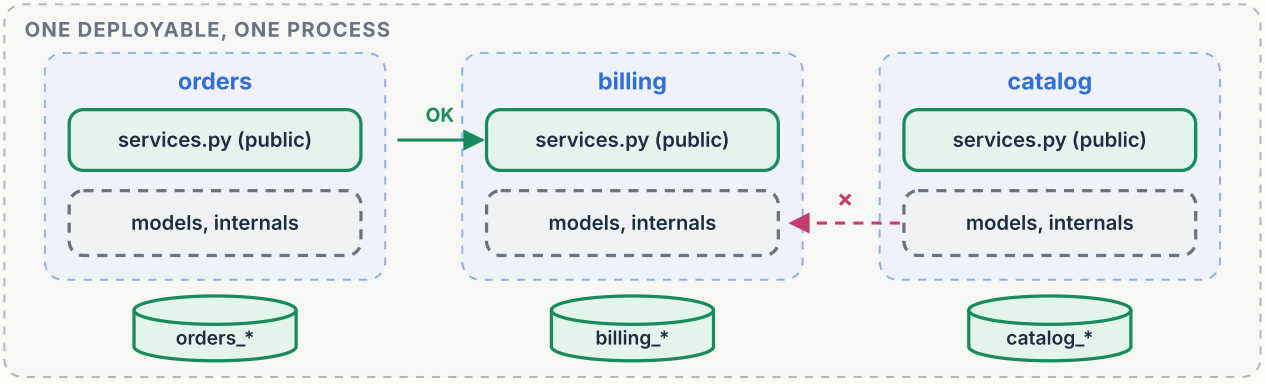


Figure 1. Modules talk only through each other's public **services** API; internals and tables are private.

Inside the single deployable, code is split by business capability: orders, billing, catalog. Each module:

- exposes a **small public interface** (a facade or `services` module of functions);
- keeps everything else private: models, helpers, its own tables (own schema or table prefix);
- is called with plain function calls, so calls are fast and can share **one database transaction** when they truly must.

Treat each module as a service that happens to live in the same process. Because its API is already narrow, it can later become a network API (REST or gRPC) with little change.

3 Enforce the boundaries

Boundaries that are only conventions decay into a **big ball of mud** within months. Enforce them with tooling:

- **Import rules in the build:** import-linter (Python), ArchUnit (Java), Packwerk (Ruby, built by Shopify), or separate projects/assemblies in .NET.
- **Architecture tests** that fail when one module queries another module's tables.

- **Per-module schema** or table prefix, ideally with separate database users so a stray query fails.
- **Code ownership** (e.g. a CODEOWNERS file) so each module has a responsible team.

Shopify runs one of the largest Ruby on Rails monoliths this way, with components and Packwerk checks rather than microservices.

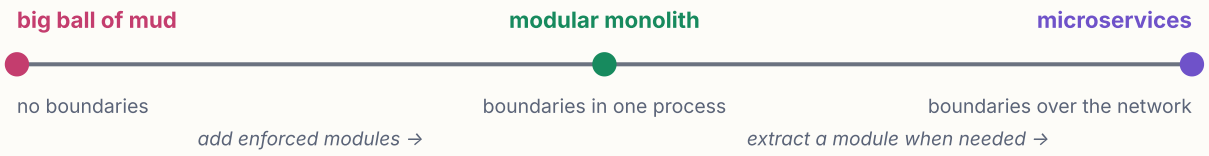


Figure 2. The modular monolith sits in the middle: real boundaries, still one process.

4 Why it suits a startup, and its limits

	Modular monolith	Microservices
Deploy	one pipeline, one artefact	one pipeline per service
Calls between modules	function calls	network calls (timeouts, retries)
Transactions	real ACID when needed	sagas, eventual consistency
Fault isolation	none: one leak hurts all modules	per service
Scaling	whole app only	per service
Release coupling	one bad change blocks the release	teams release independently

The bottom three rows are the real limits. They are usually fine early, and they tell you **when to extract** a module: a scaling hot spot, a module that needs its own release schedule or team, or very different runtime needs. Extracting a clean module is far easier than untangling a messy monolith.

WORKED EXAMPLE: DJANGO APPS WITH IMPORT RULES

A Django project has apps `orders`, `billing` and `catalog`. Each app exposes a `services.py`; everything else is internal. An import-linter contract in CI forbids `billing` from importing `orders.models`; it must call `orders.services.get_order_total(order_id)`. Each app keeps its own tables (Django already prefixes them, e.g. `orders_order`). When billing later gets its own team and release schedule, its calls to orders are already narrow, so they become HTTP calls and billing is extracted with few changes.

COMMON MISTAKES

- Boundaries by convention only, with no build rule or test to stop violations.
- Modules joining across each other's tables "just this once".
- Splitting by technical layer (controllers, models) instead of business capability.
- Extracting a module with no real reason, getting the network costs without a benefit.

5 Key terms

Modular monolith: one deployable with enforced internal module boundaries

Module API: small public interface; the rest is private

Big ball of mud: code with no real boundaries

Architecture test: automated test of dependency and data rules

Import rule: build check that forbids imports across module internals

Extraction: turning a module into a separate service

6 Interview questions

Q1. What is a modular monolith? BASIC

One deployable unit whose code is split into business modules with strong boundaries: each owns its logic and tables and is used only through a small public API.

Q2. What matters most to make it work? BASIC

Enforcing the boundaries with tooling (import rules, architecture tests, per-module schemas), not just conventions.

Q3. Why might a modular monolith be better than microservices for a startup? SCENARIO

It keeps one simple deployment, fast in-process calls and real transactions, while module boundaries keep the code clean; a module can be extracted later when a real need appears.

Q4. When would you extract a module into a service? SCENARIO

When it has a scaling hot spot, needs its own team and release cadence, has different runtime or compliance needs, or its failures must be isolated.

Q5. Two modules need to update data atomically. Is that allowed? TRICKY

Yes, through public APIs in one transaction, which is a key advantage; but note it as coupling that will need a saga if they are ever split.

Q6. What can a modular monolith not give you? TRICKY

Separate scaling, fault isolation (a memory leak hurts every module) and independent releases, because it is still one process and one deployment.

Next: SD-L-003 · Domain-Driven Design Basics backendroadmap.com

Domain-Driven Design Basics

one language, small aggregates

Watch the 4-min video

In one line: DDD models complex business rules with a shared **ubiquitous language**, entities and value objects, and **aggregates**: clusters with one root that enforces the rules, changed in one transaction, and linked to other aggregates by domain events with eventual consistency.

1 Lost in translation

The business says *booking*, the code says *ReservationRecord*, the database says *slot_entry*. Every translation is a chance for a bug and slows every conversation. Domain-driven design (Eric Evans, 2003) starts by fixing the language, then gives you a few building blocks for modelling rich business rules.

2 Ubiquitous language

Developers and domain experts agree on the words for business concepts and use exactly those words in conversation, documents and code. If the business says *order line*, the class is `OrderLine`. When the words change, the code changes too. The language is only valid inside one context, which leads to bounded contexts (lesson 22).

3 Building blocks

Block	What it is	Example
Entity	has an identity that stays the same while its data changes	Customer #42, Order #1001
Value object	no identity; defined by its values; immutable, replaced not edited	Money(25.00, EUR), Address
Aggregate	cluster changed as one unit, with one root as the only entry point	Order + its OrderLines
Domain event	a fact that happened, named in past tense	OrderPlaced, PaymentFailed
Repository	loads and saves whole aggregates, hiding the database	OrderRepository.get(id)

Two addresses with the same street and city are simply equal; two customers with the same name are not the same customer. That is the entity / value object difference in one sentence.

4 Aggregates and transactions

An **aggregate** is the unit of consistency. Its root enforces the **invariants**, the rules that must always be true, such as "total ≤ credit limit". Because outside code can only call the root, no one can bypass the rules by editing a child directly. Vaughn Vernon's rules of thumb:

- **One transaction changes one aggregate.** Keep aggregates small so transactions are short and conflicts rare.
- **Reference other aggregates by ID**, not by holding the object.

- **Use domain events** to update other aggregates afterwards, with eventual consistency.
- **One repository per aggregate**, not per table.

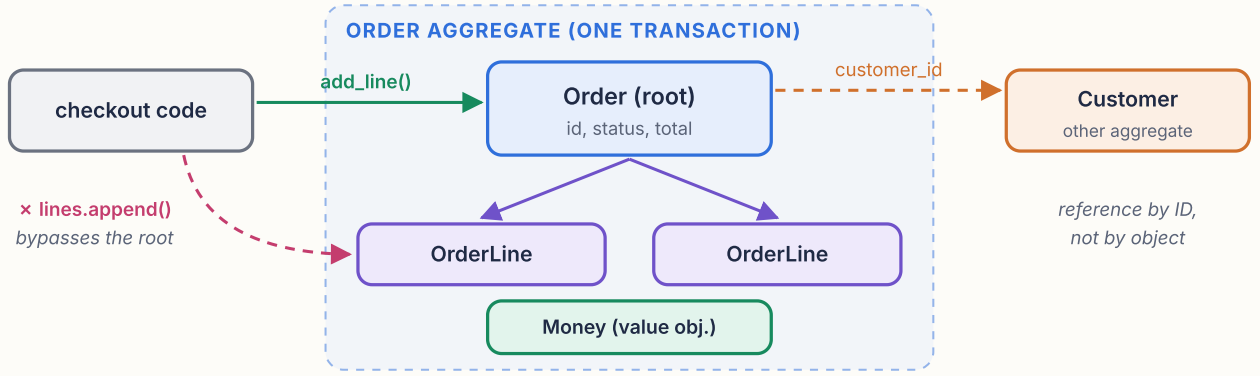


Figure 1. Outside code holds a reference only to the root. The root checks the invariants for everything inside.

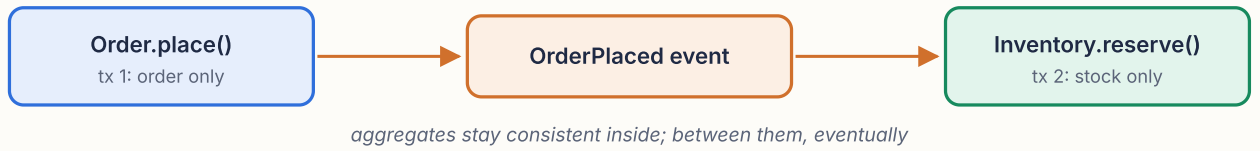


Figure 2. A change that spans aggregates becomes two transactions joined by a domain event.

Why it helps system design: aggregates show where transactions end; domain events show where services can be decoupled; language changes show where contexts and services split. DDD pays off when the business rules are rich; for simple CRUD it is overkill.

WORKED EXAMPLE: THE ORDER AGGREGATE

`Order` is the root; `OrderLine` s live inside. Invariants: the total can't exceed the customer's credit limit, and a shipped order can't change. Code can't call `order.lines.append(line)` ; it must call `order.add_line(product, qty)` . The root checks the status is not SHIPPED, computes the new total, rejects it if it exceeds the limit, and otherwise adds the line. `OrderRepository.save(order)` writes the order and its lines in **one transaction**. Reserving stock happens later, when inventory handles the `OrderPlaced` event.

COMMON MISTAKES

- Huge aggregates (Customer with all orders) causing lock contention and slow loads.
- Updating two aggregates in one transaction "for convenience".
- Anaemic models: getters and setters only, with the rules scattered in services.
- Using DDD tactics everywhere, even for simple CRUD screens.

5 Key terms

Ubiquitous language: shared words used in talk and code alike

Entity: object with a stable identity

Value object: immutable, compared by value

Aggregate root: the single entry point that enforces invariants

Invariant: business rule that must always hold

Domain event: past-tense fact, e.g. OrderPlaced

6 Interview questions

Q1. What is the difference between an entity and a value object? BASIC
An entity has an identity that persists as its data changes; a value object has no identity, is immutable and is equal to any other with the same values.

Q2. What is the ubiquitous language? BASIC
Words for domain concepts agreed by developers and domain experts and used unchanged in conversation, documents and code, within one bounded context.

Q3. What is an aggregate and why does it matter for transactions? SCENARIO
A cluster with one root that enforces its invariants; it is the unit of consistency, so one transaction should change one aggregate, and aggregates should be small.

Q4. Placing an order must also reserve stock. How do you model it? SCENARIO
Order and Inventory are separate aggregates: commit the order, publish OrderPlaced, and let inventory reserve stock in its own transaction, with a compensation if it fails.

Q5. Why reference other aggregates by ID instead of object? TRICKY
It keeps aggregate boundaries and transactions clear, avoids loading large object graphs, and lets the aggregates live in different services later.

Q6. What links aggregates if not transactions? TRICKY
Domain events such as OrderPlaced, handled asynchronously, giving eventual consistency between aggregates.

Next: SD-L-004 · Bounded Contexts backendroadmap.com

Bounded Contexts and Service Boundaries

where the language changes, split

Watch the 4-min video

In one line: a bounded context is a boundary inside which one model and one language apply; contexts follow business capabilities, own their data, and are the best guide to service boundaries: high cohesion inside, few dependencies outside.

1 What is a customer?

Ask billing and a customer is a payment method and invoices. Ask support and it is tickets and contact history. Ask marketing and it is segments and campaigns. One giant `Customer` model that pleases all three grows to dozens of optional fields, and every team must coordinate to change it. DDD's answer is not one model but several small ones, each valid in its own **bounded context**.

2 A bounded context

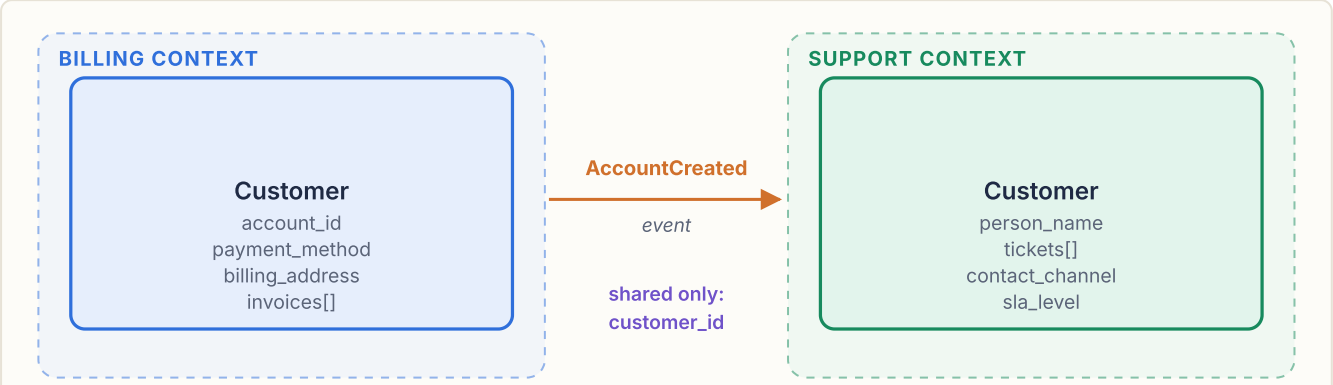


Figure 1. The same word is two concepts. Each context keeps its own model and data, linked only by an ID and events.

Inside a context, every term in the ubiquitous language (lesson 21) has one precise meaning. The same word in two contexts is really two concepts, and that is fine. Each context keeps a small, consistent model instead of a shared one that serves no one well.

3 Finding the boundaries

- **Business capabilities:** what the business does (billing, shipping, catalog, support), not how the code is layered.
- **Language changes:** where a word changes meaning, a boundary is near.
- **Team ownership:** one team should be able to own a context end to end (Conway's Law, lesson 19).
- **Data ownership:** each context owns its data; others read it through an API or events, never by writing its tables.

Talking to domain experts, or running an **event storming** workshop where the team lays out domain events on a timeline, reveals these seams quickly. Each context is a candidate service, though one service may hold several small contexts at first.

4 The context map

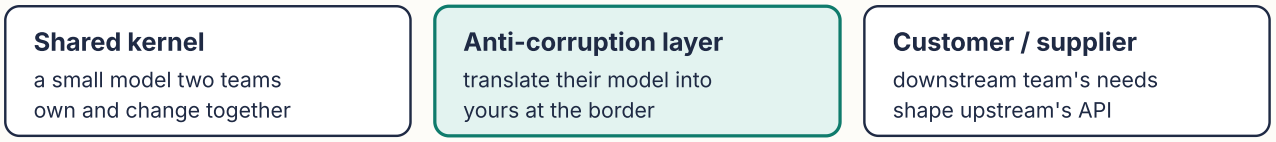


Figure 2. Common relations on a context map. Drawing them makes hidden dependencies visible.

Contexts still work together. A **context map** shows how: a **shared kernel** is a small model shared and changed jointly (use rarely); an **anti-corruption layer** translates another context's model so its concepts don't leak into yours (lesson 30); in a **customer/supplier** relation the upstream team plans for the downstream team's needs. A **conformist** simply adopts the upstream model as is.

5 Testing a boundary

Signal	Good boundary	Wrong boundary
A typical feature	changes one service	changes three services in lock-step
Calls per request	few, coarse-grained	chatty: many small calls
Data	owned by one service	two services write the same table
Split by	business capability	technical layer (a "validation service")

Two failure modes: boundaries **too fine** create chatty services that call each other on every request; boundaries **cut by technical layer** spread one business change across many teams. If two services constantly need each other's data or always change together, merge them or move the line.

WORKED EXAMPLE: BILLING VS SUPPORT

In **billing**, `Customer` = account ID, payment method, billing address, invoices. In **support**, `Customer` = person name, tickets, contact channel, SLA level. Each context stores its own customer table; the only shared value is `customer_id`. When an account is created, billing publishes `AccountCreated {customer_id, name, email}`; support consumes it and creates its own record with a default SLA level. Billing can rename or add fields freely, because support depends only on the event contract.

COMMON MISTAKES

- One shared canonical model for the whole company.
- Splitting by technical layer instead of business capability.
- Services so fine-grained that every request fans out to ten calls.
- Letting another context write your tables "to save an API".

6 Key terms

Bounded context: boundary within which one model and language apply

Context map: diagram of how contexts relate

Anti-corruption layer: translator that keeps a foreign model out

Business capability: something the business does, e.g. billing

Shared kernel: small model two teams share and change together

Event storming: workshop mapping domain events to find contexts

7 Interview questions

Q1. What is a bounded context? BASIC

A boundary within which one domain model and one ubiquitous language apply; the same word may mean something else in another context.

Q2. What is a context map? BASIC

A diagram of how bounded contexts relate: shared kernel, anti-corruption layer, customer/supplier, conformist, and which team adapts to which.

Q3. How do you decide service boundaries? SCENARIO

Follow business capabilities and bounded contexts: where language changes, where data is owned, where a team can work alone. Aim for high cohesion inside and few dependencies outside.

Q4. Billing and support both need customer data. How? SCENARIO

Each keeps its own customer model and table linked by customer_id; billing publishes AccountCreated and support builds its own record from it.

Q5. Two services always deploy together and call each other constantly. What does that tell you? TRICKY

The boundary is wrong: they are one context split in two. Merge them or move the line so most changes stay inside one service.

Q6. Is one bounded context always one microservice? TRICKY

No. A context is a modelling boundary; a service may contain several small contexts, but one context should not be split across services.

Next: SD-L-005 · Database per Service backendroadmap.com

Database per Service

own your data; share an API

Watch the 4-min video

In one line: each service owns a private database or schema that only it can read and write; others use its API or events, so schemas can change freely, at the price of no cross-service joins or transactions and some stale copies.

1 One renamed column

Two services share one database. The inventory team renames `qty` to `quantity`, and the order service, which queried that table directly, breaks in production. From then on every schema change needs a meeting. Shared tables are an **untyped, undocumented API** that every team depends on.

2 The rule

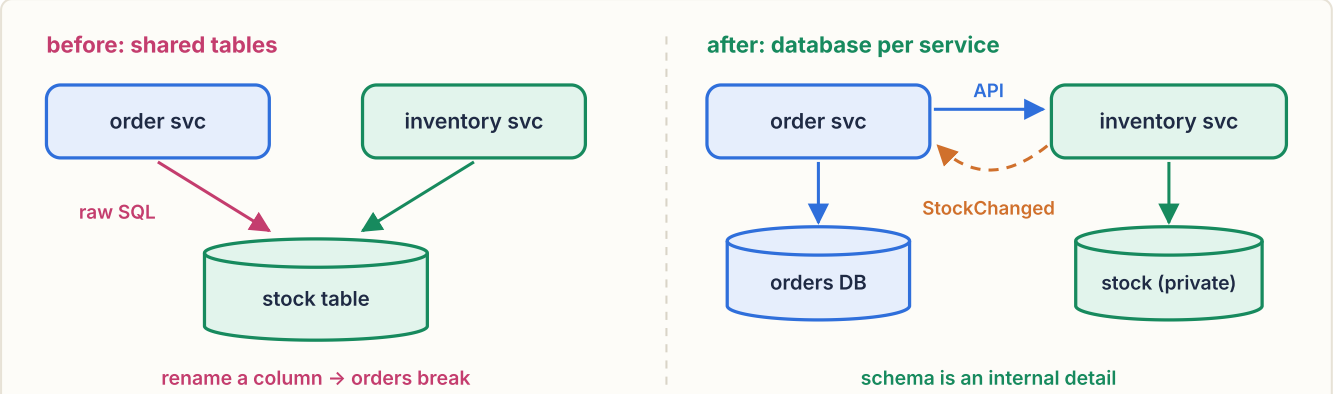


Figure 1. Before: another team's SQL is coupled to your columns. After: only the API and events are the contract.

Each service has a private database or schema. **Nobody else** queries its tables. Other services call its API or consume the events it publishes. The database becomes an implementation detail; the public contract is just the API and the event schemas, which can be versioned.

What you gain: a team can change its schema, indexes, or even its database type (Elasticsearch for search, Postgres for orders, Redis for sessions) without asking anyone. One service's heavy queries no longer slow the others, and each database scales on its own.

3 What it costs

Lost	Replacement
Joins across services	API composition (call both, merge in code) or a local read copy kept fresh by events
Transactions across services	sagas with compensating steps (lesson 34); outbox pattern for reliable events
Always-fresh data everywhere	eventual consistency: copies may be seconds stale
Ad-hoc SQL across all data	a separate analytics platform

To show an order with product names, either call orders and catalog and combine the results, or keep a small `product_name` copy in the order service, updated from `ProductRenamed` events. Read models built this way are the basis of CQRS (lesson 35).

4 Reporting across services



Figure 2. Analytics joins data far from production: services stream changes out, nobody queries their databases.

Don't let reports query every service's database. Each service publishes events, or a **change data capture** tool (such as Debezium, reading the database's write-ahead log) streams its changes into a warehouse or data lake. Analysts join there, and production stays fast.

5 Ownership, not infrastructure

Database per service is about **who may access** the data, not how many servers you run. Several services can share one database server for cost, if each has **its own schema and its own credentials**. Then the rule is enforced: the order service's user simply has no grant on the inventory schema. Without separate credentials it is just a convention, and conventions erode.

WORKED EXAMPLE: INVENTORY EXPOSES STOCK

Before: the order service ran `SELECT qty FROM stock WHERE sku = ?` against inventory's table. After: inventory exposes `GET /stock/{sku}/availability` and `POST /reservations {order_id, sku, qty}`, and publishes `StockChanged` events. At checkout the order service calls the reservation API (with a timeout and an idempotency key on `order_id`). The inventory team then splits `stock` into per-warehouse rows, and no other team notices.

COMMON MISTAKES

- Separate schemas but one shared database user, so anyone can still read anything.
- Reporting jobs reading production databases of every service.
- Chatty API composition on hot paths instead of a local read copy.
- Publishing events outside the database transaction, so they can be lost (use an outbox).

6 Key terms

Database per service: each service's data is private to it

Read copy / replica: local copy of another service's data kept fresh by events

Outbox pattern: write events to a table in the same transaction, then publish

API composition: query several services and join in code

CDC: change data capture: stream DB changes from its log

Integration database: anti-pattern: many apps share one schema

7 Interview questions

Q1. Why shouldn't services share a database? BASIC
Shared tables are a hidden contract: schema changes break other teams, deployments become coupled, and one service's load hurts the others.

Q2. How do other services get data they need? BASIC
Through the owning service's API or by consuming its events, possibly keeping a local read copy.

Q3. How do you report across services? SCENARIO
Stream events or CDC from each service into a warehouse or data lake and join there, never against production databases.

Q4. The order page needs product names from catalog. Options? SCENARIO
API composition at read time, or a local product-name copy in orders updated by catalog events; the copy is faster and survives catalog outages but may be slightly stale.

Q5. Can services share one database server? TRICKY
Yes, for cost, as long as each has its own schema and credentials so only the owner can reach its tables; ownership matters, not hardware.

Q6. How do you update two services' data consistently? TRICKY
Not with a distributed transaction: use a saga with compensations, and the outbox pattern so each step's events are published reliably.

Next: SD-L-006 · Sync vs Async Communication backendroadmap.com

Microservice Communication Styles

sync when you need the answer now

Watch the 4-min video

In one line: call synchronously (REST/gRPC) only when the caller needs the answer to continue, and protect that call with timeouts and circuit breakers; send everything else asynchronously through a broker, so services are decoupled in time. Decide per interaction, not per system.

1 One slow call breaks the chain

Checkout calls payment, which calls fraud, which calls scoring. Scoring gets slow, every caller's threads fill up waiting, and checkout times out. Yet some calls really do need an answer right now: you can't confirm an order without knowing the charge succeeded.

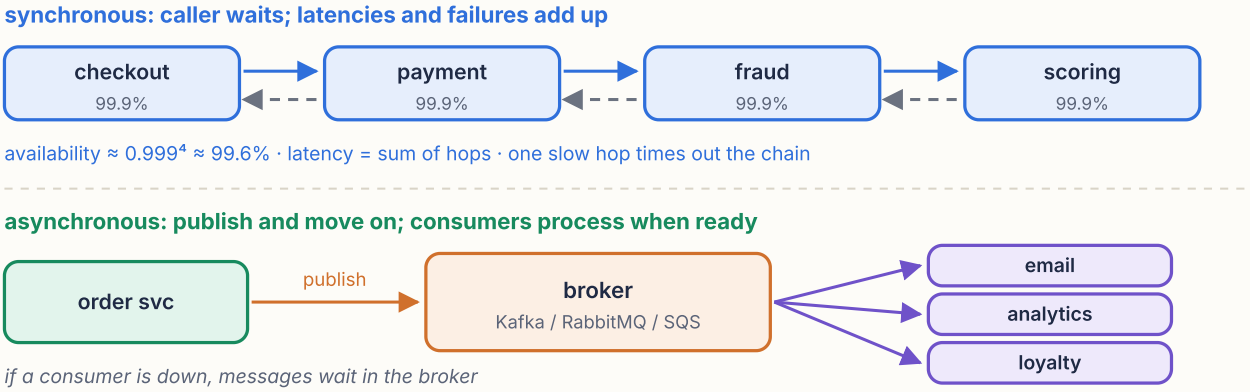


Figure 1. A synchronous chain multiplies availability and adds latency; a broker decouples sender and receivers in time.

2 Synchronous: request and response

The caller sends a request over **REST** (HTTP/JSON) or **gRPC** (HTTP/2, Protocol Buffers) and waits. It is simple, familiar and easy to debug, and fits queries and commands whose result the caller needs. The cost is **temporal coupling**: both services must be up at the same moment.

- Availability multiplies along a chain: five services at 99.9% give $0.999^5 \approx$ **99.5%** (lesson 1).
- Latency adds up; tail latency of the slowest hop dominates.
- Failures cascade unless every call has a **timeout**, limited **retries with backoff** and a **circuit breaker** (lessons 4–6).
- Keep chains short: ideally one or two synchronous hops per user request.

3 Asynchronous: messages through a broker

The sender publishes an **event** ("OrderConfirmed", a fact) or a **command** ("SendReceipt", a request to one receiver) to a broker such as Kafka, RabbitMQ, Amazon SQS/SNS or Azure Service Bus, then moves on. Receivers process when ready; if one is down, messages wait. This removes temporal coupling, absorbs spikes, and lets new consumers subscribe without changing the sender.

Patterns: in **choreography** services react to each other's events with no coordinator; in **orchestration** one service sends commands and tracks the steps (lesson 34). Request/response can even run over messaging with a **reply queue** and a **correlation ID**. The costs: eventual consistency, duplicate delivery (consumers must be idempotent), and harder debugging, so you need correlation IDs and tracing (lesson 36).

	Synchronous	Asynchronous
Coupling in time	both must be up	receiver can be down; messages wait
Result	immediate	later; eventual consistency
Load spikes	passed straight through	absorbed by the queue
Adding a consumer	caller must change	just subscribe
Debugging	simple call stack	needs correlation IDs and tracing

4 Choose per interaction

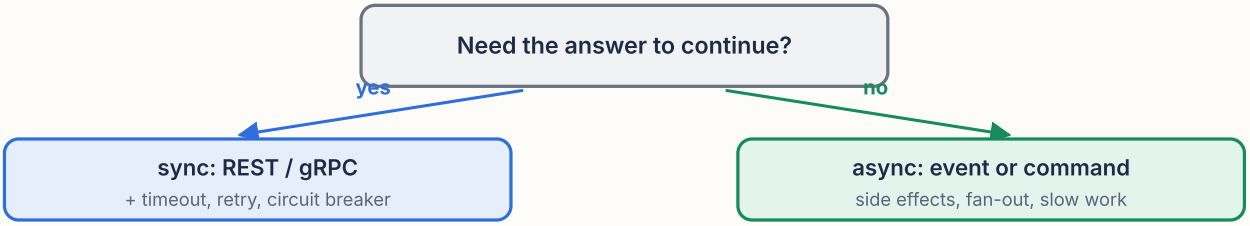


Figure 2. The key question. Most real systems mix both styles.

WORKED EXAMPLE: CHECKOUT AND RECEIPTS

The order service calls payment **synchronously** (gRPC, 2-second timeout, one retry with an idempotency key) because it must know whether the charge succeeded before confirming. Then it publishes `OrderConfirmed`. The email service consumes it **asynchronously** and sends the receipt; analytics and loyalty subscribe to the same event. If email is down for ten minutes, receipts are ten minutes late and checkout keeps working.

COMMON MISTAKES

- Long synchronous chains on the user's critical path.
- Sync calls with no timeout, so one slow service hangs everyone.
- Assuming exactly-once delivery: consumers must handle duplicates.
- Making everything async, even when the user is waiting for the answer.

5 Key terms

Temporal coupling: both sides must be available at the same time

Choreography: services react to events, no coordinator

Correlation ID: ID carried through a flow to link its messages

Event vs command: a fact for anyone vs a request to one receiver

Orchestration: one service directs the steps

Idempotent consumer: processing a message twice has no extra effect

6 Interview questions

Q1. What is temporal coupling? BASIC

Both caller and callee must be up at the same moment, as in a synchronous call; async messaging removes it because messages wait in the broker.

Q2. What is the difference between an event and a command? BASIC

An event states a fact that happened, for any number of subscribers; a command asks one specific receiver to do something.

Q3. When should services talk synchronously vs asynchronously? SCENARIO

Sync when the caller needs the answer now to continue, like authorising a payment, protected by timeouts and circuit breakers; async for side effects, fan-out and slow work.

Q4. Five services each at 99.9% are called in a chain. What's the availability? SCENARIO

About $0.999^5 \approx 99.5\%$, before counting the caller itself; make the non-essential calls async or optional to raise it.

Q5. Can you do request/response over a message broker? TRICKY

Yes: send the request with a reply-to queue and a correlation ID, and match the reply; it buffers load but the caller still waits for the result.

Q6. Why must async consumers be idempotent? TRICKY

Brokers usually deliver at least once, so redeliveries after crashes or timeouts happen; the consumer must detect or tolerate duplicates.

Next: SD-L-007 · API Gateway Pattern backendroadmap.com

API Gateway Pattern

one front door, no business logic

Watch the 4-min video

In one line: an API gateway is the single entry point for external clients: it routes requests, aggregates calls and handles cross-cutting concerns (TLS, auth, rate limits, logging), but business logic never lives in it.

1 Twenty addresses for one app

A mobile app talks to twenty microservices directly. It needs twenty hostnames, each service re-implements authentication, and one screen takes five round trips over a slow mobile network. When a service is split or moved, old app versions in the wild break. An API gateway puts one front door in front of everything.

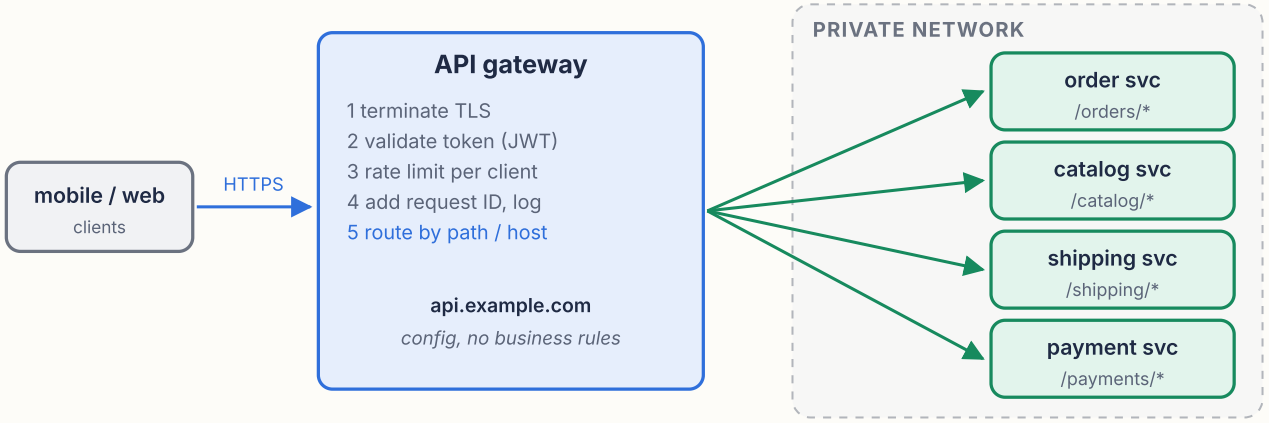


Figure 1. Every request passes the same checks once, then is routed by path to a private service.

2 Three jobs

1. Gateway routing. Clients call one public endpoint; the gateway routes by path, host or header (`/orders/*` → order service). Services can move, split or scale without clients noticing; the internal topology is hidden.

2. Gateway aggregation. One client request fans out to several services in parallel and the results are merged. The client makes one round trip instead of five, which matters on mobile networks with 100 ms+ round-trip times.

3. Gateway offloading. Cross-cutting concerns are done once: TLS termination, token validation, rate limiting and quotas, request IDs, access logs, CORS, response caching, request size limits. Products such as Kong, NGINX, Envoy, AWS API Gateway and Azure API Management provide these as configuration.

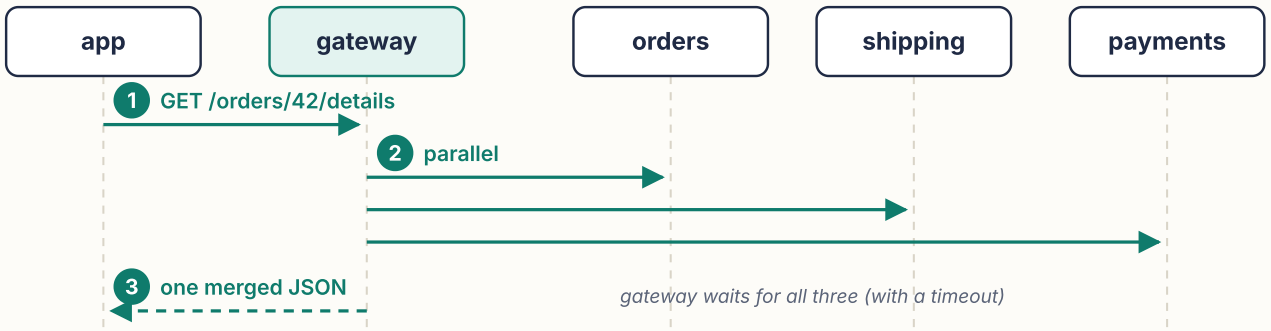


Figure 2. Aggregation: the slow part (several calls) happens inside the data centre, not over the mobile network.

3 What must never live in the gateway

Business logic. Pricing rules, discount checks, order workflows and data decisions belong in services owned by domain teams. Once they creep into the gateway it becomes a **shared monolith**: every team must change it, one team owns it, and it slows everyone down. A good gateway is boring: routing tables and policies in version control, not code full of rules. Note also that validating a token at the edge is **authentication**; whether this user may see order 42 is **authorisation** and stays in the order service.

Belongs in the gateway	Belongs in services
routing, load balancing, retries to healthy instances	domain rules: pricing, discounts, eligibility
TLS, token signature/expiry check	fine-grained authorisation (may user X see order 42?)
rate limits, quotas, request size limits	workflows and state changes
request IDs, access logs, metrics	validation of business data

4 Critical infrastructure

The gateway sits on every request, so it must be highly available (several instances across zones behind a load balancer), scaled out, monitored and changed carefully. Avoid one giant gateway owned by a bottleneck team: some companies run one gateway per domain or per client type, which leads to the BFF pattern (lesson 26).

WORKED EXAMPLE: TWENTY SERVICES, ONE ENDPOINT

All traffic enters through `api.example.com`. The gateway terminates TLS, checks the JWT's signature and expiry, applies a rate limit of, say, 100 requests per minute per client key (returning HTTP 429 when exceeded), adds an `X-Request-ID`, and routes by path. For `GET /orders/42/details` it calls orders, shipping and payments in parallel and returns one response. The twenty services have no public IPs; only the gateway is exposed to the internet.

COMMON MISTAKES

- Putting discount or workflow logic in gateway plugins or scripts.
- Treating gateway authentication as full authorisation.
- A single gateway instance or zone: a single point of failure.
- Aggregation with no per-call timeout, so one slow service delays every response.

5 Key terms

API gateway: single entry point for external clients

Gateway aggregation: fan out to several services, merge results

TLS termination: decrypt HTTPS at the gateway

Gateway routing: map path/host to an internal service

Gateway offloading: TLS, auth, rate limiting done once at the edge

HTTP 429: Too Many Requests: rate limit hit

6 Interview questions

Q1. What are the main jobs of an API gateway? BASIC

Routing requests to the right service, aggregating several calls into one response, and offloading cross-cutting concerns such as TLS, authentication, rate limiting and logging.

Q2. Why not let clients call services directly? BASIC

Clients would need every address and auth scheme, make many round trips, and break whenever services move; services would all re-implement the same edge concerns.

Q3. What logic should never live in the API gateway? SCENARIO

Business logic: domain rules, pricing, workflows and data decisions belong in team-owned services; otherwise the gateway becomes a shared monolith and bottleneck.

Q4. The order-details screen needs three services. How does the gateway help? SCENARIO

It calls orders, shipping and payments in parallel with timeouts and returns one merged response, so the client makes one round trip.

Q5. The gateway validates the JWT. Can the order service skip all checks? TRICKY

No: the gateway proves who the caller is, but the service must still check what they may do, e.g. that order 42 belongs to this user.

Q6. Isn't the gateway a single point of failure? TRICKY

It is a critical dependency, so run several stateless instances across zones behind a load balancer and keep it simple; splitting gateways per domain also limits blast radius.

Next: SD-L-008 · Backend for Frontend backendroadmap.com

Backend-for-Frontend (BFF)

one backend per client experience

Watch the 4-min video

In one line: a backend-for-frontend is a thin backend built for one client experience (web, mobile, partner), owned by that frontend's team, that aggregates the shared services and returns exactly the data the screen needs; business rules stay in the domain services.

1 One API that serves no one well

The web app shows rich dashboards. The mobile app needs tiny payloads over a slow network. A partner needs a stable, versioned API. One general-purpose API tries to serve all three: it grows optional fields, client-specific flags and special endpoints, and every change for one client risks breaking another. The **BFF** pattern (described by Sam Newman; popularised by SoundCloud and Netflix) gives each client experience its own backend.

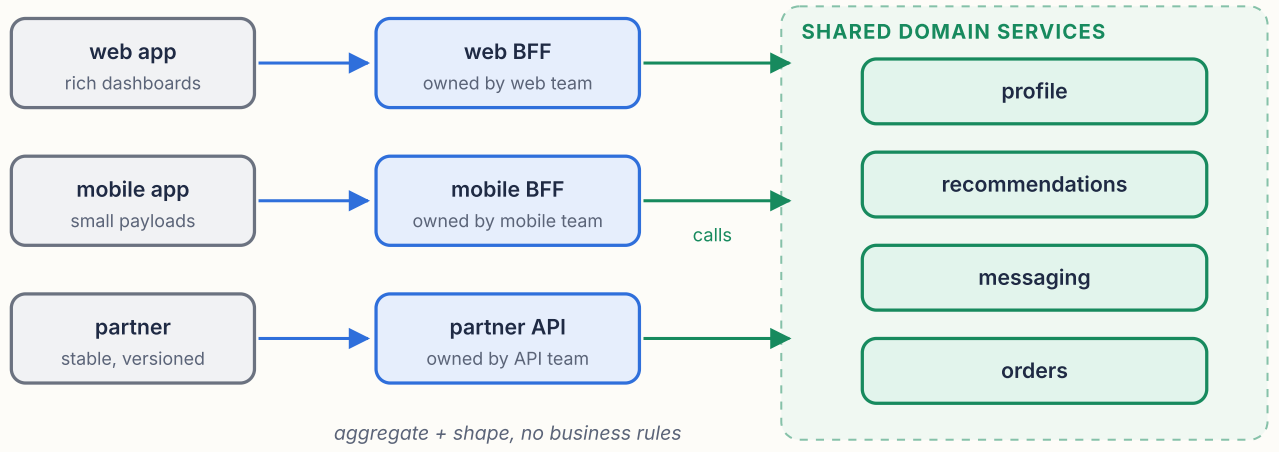


Figure 1. Each client gets its own thin backend, owned by the team that builds that client. All BFFs share the same domain services.

2 What a BFF does

- **Aggregates:** calls several services in parallel, server-side, where round trips take about a millisecond instead of 100 ms+ on mobile.
- **Shapes:** trims fields, flattens nested data, formats dates and money for the screen, and pages results the way the UI scrolls.
- **Adapts protocols:** e.g. gRPC to services, JSON or GraphQL to the client; handles client-specific auth flows and caching.

It is an API designed for **one** screen set, not for every possible consumer.

3 Ownership is the point

The mobile team owns the mobile BFF. When the home screen changes, they change the screen and its BFF and ship both together, with no ticket to a central gateway team and no risk to the web client. This matches Conway's Law (lesson 19): one team, one client, one backend.

	One general API / gateway	BFF per client
Payload	superset for every client	exactly what the screen needs
Change for one client	may break others	isolated to that BFF
Owner	central platform team	the frontend team
Number of backends	one	one per client type (more to run)
Risk	bloat, bottleneck team	duplicated logic across BFFs

BFFs and gateways work together: a thin gateway (lesson 25) in front still does TLS, token validation and rate limiting, then routes to the right BFF. **GraphQL** is the main alternative: one flexible endpoint where each client asks for exactly the fields it wants.

4 Costs

More services to build, deploy and monitor. Logic tends to get **duplicated** across BFFs ("is this order cancellable?" computed in web and mobile BFFs, differently). Keep such rules in the domain services and keep BFFs thin: aggregation and shaping only.

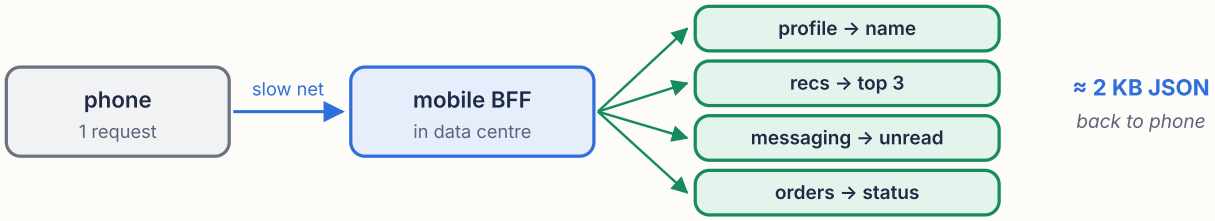


Figure 2. The mobile home screen: four calls inside the data centre, one compact response over the mobile network.

WORKED EXAMPLE: THE MOBILE HOME SCREEN

The screen shows the user's name, three recommendations, the unread message count and the current order status. The mobile BFF calls the profile, recommendation, messaging and order services **in parallel**, each with a short timeout. It keeps only the fields the screen shows and returns one payload of about **2 KB** instead of four full responses. If recommendations time out, it returns the rest with an empty list, so the screen still loads. Fewer bytes and one round trip mean faster loads and less battery and data use.

COMMON MISTAKES

- Business rules in BFFs, duplicated and drifting apart.
- A BFF per screen or per developer instead of per client experience.
- A BFF owned by a separate backend team, losing the main benefit.
- Sequential calls in the BFF where parallel calls would do.

5 Key terms

BFF: backend built for one client experience

Aggregation: combine several service calls into one response

Response shaping: trim and format data for a specific screen

General-purpose API: one API for all consumers

GraphQL: query language letting clients choose fields

Partial response: return what's available when one call fails

6 Interview questions

Q1. What is a backend-for-frontend? BASIC

A thin backend built for one client experience, owned by that frontend team, that aggregates shared services and returns data shaped for its screens.

Q2. How do a BFF and an API gateway relate? BASIC

They complement each other: the gateway handles edge concerns like TLS, auth and rate limits, then routes to the BFF that aggregates and shapes data.

Q3. Why use a BFF instead of one general gateway? SCENARIO

Each client gets an API shaped for its needs, aggregation happens server-side, and the frontend team ships its BFF independently; a general API bloats with every client's special cases.

Q4. Design the mobile home screen API. SCENARIO

A mobile BFF calls profile, recommendations, messaging and orders in parallel with timeouts, returns one ~2 KB payload, and degrades gracefully if one call fails.

Q5. What's the main risk of BFFs? TRICKY

Duplicated logic across BFFs that drifts apart; keep business rules in the domain services and BFFs thin.

Q6. When would you choose GraphQL over BFFs? TRICKY

When many clients need flexible field selection over the same data and you'd rather run one schema than many backends, accepting query-cost control and caching complexity.

Next: SD-L-009 · Service Mesh

Service Mesh

move networking out of the code

Watch the 4-min video

In one line: a service mesh puts a proxy (the data plane) beside every service instance and configures all of them centrally (the control plane), giving every service mTLS identity, retries, timeouts, traffic splitting and uniform telemetry without code changes, at the cost of extra latency, resources and operational complexity.

1 The same library in five languages

A company runs two hundred services in Java, Go, Python and Node. Security wants every connection encrypted. Operations wants consistent retries, timeouts and metrics. Asking every team to add and upgrade the same libraries in every language never quite works: versions drift and some teams never finish. A service mesh moves these concerns into infrastructure.

2 Two parts

- **Data plane:** a small proxy, usually **Envoy** (Linkerd uses its own Rust proxy), runs as a **sidecar** next to each instance (lesson 28). Traffic in and out of the app is redirected through it (on Kubernetes, via iptables rules).
- **Control plane:** e.g. Istio's `istiod`. It turns operators' policies into proxy config, pushes it out, and acts as the certificate authority.

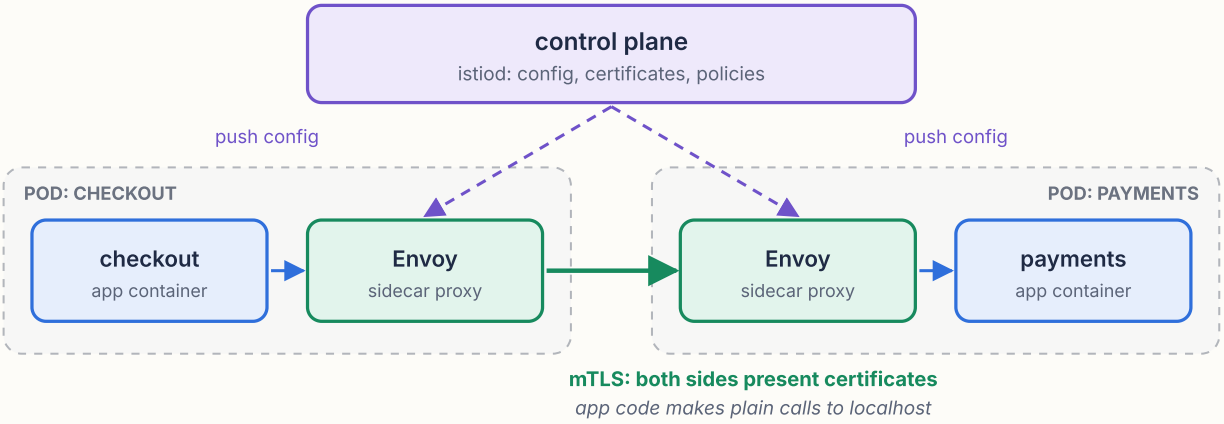


Figure 1. Every pod gets a sidecar proxy; all traffic flows proxy to proxy. The control plane configures every proxy and issues certificates.

3 What you get

Feature	What the proxies do
Security	issue each workload an identity certificate (SPIFFE IDs in Istio), encrypt all traffic with **mutual TLS**, rotate certs automatically, enforce "who may call whom"
Traffic management	timeouts, retries, load balancing, outlier ejection, canary and blue-green splits by weight or header
Resilience testing	inject delays or errors for a share of requests
Telemetry	request rate, error rate and latency for every service pair; propagate tracing headers (apps must still forward them)

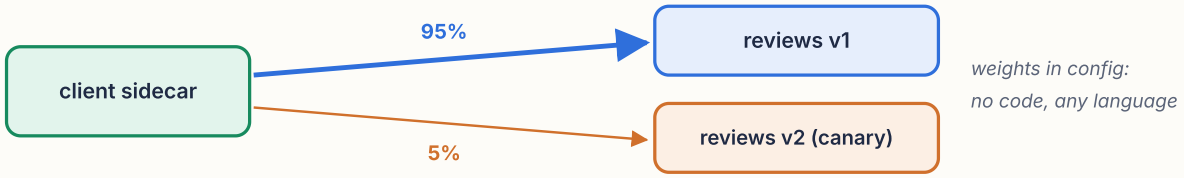


Figure 2. A canary release as configuration: the client's proxy sends 5% of requests to the new version.

All of this is configuration, identical across languages, audited in one place. Teams stop re-implementing retry logic, and dashboards for every service come almost for free.

4 What it costs

TRADE-OFFS

- **Latency:** two extra proxy hops per call (client and server side), typically around a millisecond or less each, but it adds up on long chains.
- **Resources:** CPU and memory for a proxy in every pod; at hundreds of pods this is real money.
- **Complexity:** the control plane must be run, upgraded and understood; misconfigured policies cause outages.
- **Debugging:** another layer between services. With a handful of services, a library or the platform's own features are simpler.

Newer "sidecarless" modes (Istio ambient mode, Cilium) move part of the proxy work to per-node components to cut this overhead.

WORKED EXAMPLE: STRICT MTLS IN ISTIO

A company runs Istio on Kubernetes. A mesh-wide `PeerAuthentication` with `mtls.mode: STRICT` makes every sidecar reject plain-text traffic, so every connection is encrypted and both ends are authenticated. An `AuthorizationPolicy` on the payments namespace allows requests only from the checkout service account. Certificates are short-lived and rotate automatically. The security team reviews two YAML files instead of two hundred codebases.

5 Key terms

Data plane: the proxies that carry traffic

Control plane: configures proxies, issues certificates

Envoy: widely used open-source L7 proxy

Traffic splitting: send a weighted share to another version

mTLS: TLS where both client and server present certificates

Ambient / sidecarless: mesh without a proxy per pod

6 Interview questions

Q1. What problems does a service mesh solve? BASIC

Consistent mTLS and service identity, traffic control (timeouts, retries, load balancing, canary splits, fault injection) and uniform telemetry, all without application code changes.

Q2. What are the data plane and the control plane? BASIC

The data plane is the set of proxies beside each service that carry traffic; the control plane configures them and issues certificates.

Q3. Security requires all service-to-service traffic encrypted and only checkout may call payments. How? SCENARIO

Enable strict mTLS mesh-wide so plain text is rejected, and add an authorisation policy allowing only checkout's identity to call payments.

Q4. You have 6 services in one language. Do you need a mesh? SCENARIO

Probably not: a shared library plus your platform's load balancing and TLS is simpler; adopt a mesh when service count, languages and security needs justify the overhead.

Q5. What does a service mesh cost? TRICKY

Extra latency and CPU/memory for every proxy, plus a complex control plane to operate, upgrade and debug.

Q6. Does a mesh give you distributed tracing for free? TRICKY

Partly: proxies emit spans, but the application must still forward trace headers from incoming to outgoing requests, or the spans won't join into one trace.

Next: SD-L-010 · Sidecar and Ambassador backendroadmap.com

Sidecar and Ambassador Patterns

a helper next to every app

Watch the 4-min video

In one line: a sidecar is a helper container that runs beside the app with the same lifecycle, network and disk, taking over cross-cutting work (logs, proxying, TLS, config) in any language; an **ambassador** is a sidecar for outbound calls and an **adapter** standardises the app's output.

1 The same plumbing in every service

Every service needs log shipping, a proxy, config refresh and metrics export. Building all that into each service, in each language, means duplicated work and slow upgrades. Instead, run a small helper next to the app that does it for you.

2 The sidecar

A **sidecar** runs next to the main container, on Kubernetes in the **same pod**. They share a lifecycle (they are scheduled, scaled and stopped together), share the network namespace (they talk over `localhost`) and can share volumes (they exchange files). The app often doesn't know the helper exists.

Why: cross-cutting concerns are separated from business code; the helper is written once, in any language, reused beside every service, and **upgraded independently** of the app. Typical sidecars: log shippers, TLS or service-mesh proxies such as Envoy (lesson 27), config reloaders, secret fetchers (e.g. the Vault Agent).

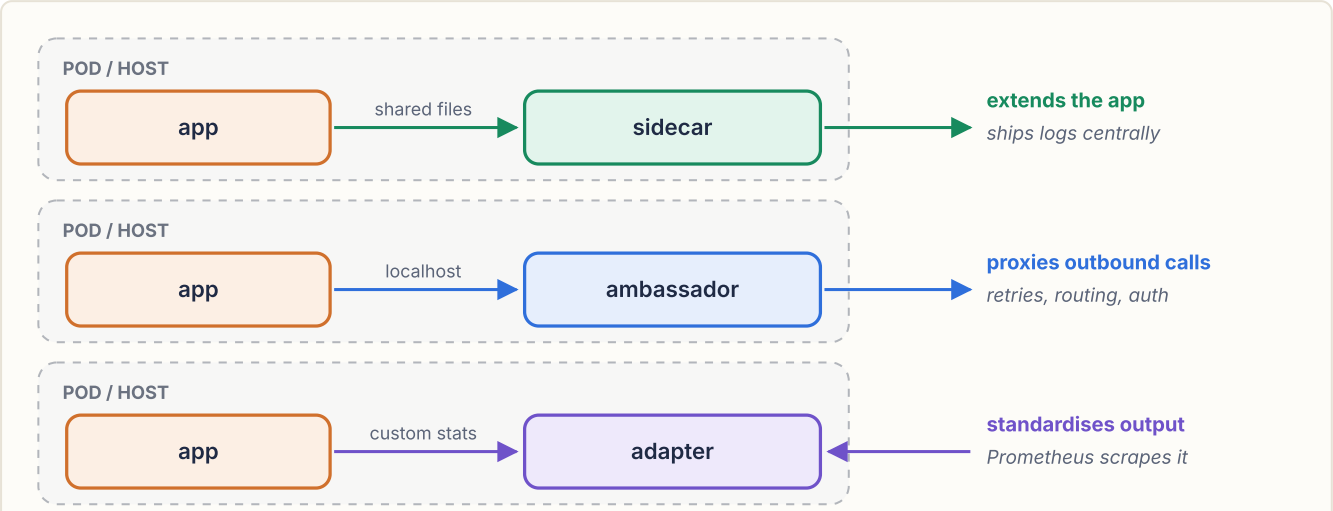


Figure 1. Same mechanism, three purposes: extend the app, represent it to the outside world, or translate its output.

3 Ambassador and adapter

An **ambassador** handles **outbound** connections. The app talks to `localhost`; the ambassador does the hard part of reaching remote services: retries, circuit breaking, routing, authentication and monitoring. It can hide a sharded database or a multi-region cluster behind one local address, so the app stays simple.

An **adapter** works the other way: it converts the app's output into the format the platform expects. A legacy app exposes custom metrics; an adapter (a Prometheus *exporter*) reads them and serves them at `/metrics` in Prometheus format, with no change to the app.

Pattern	Direction	Example
Sidecar	beside the app, general help	Fluent Bit log shipper, config reloader
Ambassador	app → outside world	local proxy that routes to the right database shard
Adapter	outside world → app's output	Prometheus exporter for a legacy app
Library instead	inside the process	small feature, one language, latency-critical

COSTS AND WHEN NOT TO USE ONE

- **Resources:** extra containers in every pod, each with CPU and memory requests.
- **Startup and shutdown order:** the app may need its proxy ready first (Kubernetes 1.29+ supports native sidecar containers to order this).
- **Latency:** an extra local hop for proxied traffic.
- **More moving parts** to debug. For a tiny feature, one language or latency-critical paths, a library is simpler.

WORKED EXAMPLE: LOG SHIPPING WITH FLUENT BIT

A Node app writes logs to files in a shared volume, `/var/log/app/*.log`. A **Fluent Bit** sidecar in the same pod tails those files, adds pod, namespace and service labels, and ships them to central logging. The app team changes no code to get logs indexed, and the platform team upgrades the shipper beside every service at once by changing one template.

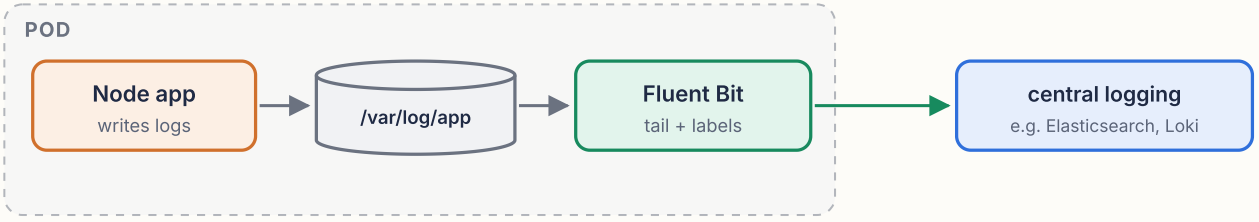


Figure 2. The app only writes files; the sidecar handles parsing, labelling and delivery.

4 Key terms

Sidecar: helper container sharing the app's pod and lifecycle

Adapter: sidecar that standardises the app's output

Shared volume: disk both containers can read and write

Ambassador: sidecar proxy for outbound connections

Pod: Kubernetes unit of co-scheduled containers

Exporter: adapter exposing metrics in Prometheus format

5 Interview questions

Q1. What is a sidecar? BASIC

A helper container running beside the main app in the same pod or host, sharing its lifecycle, network and optionally disk, that handles a cross-cutting concern.

Q2. How do ambassador and adapter differ? BASIC

An ambassador proxies the app's outbound calls (retries, routing, auth); an adapter converts the app's output, such as metrics or logs, into a standard format.

Q3. When would you use a sidecar? SCENARIO

When a cross-cutting concern (logging, proxying, TLS, config, secrets) should run beside many services in any language, with its own release cycle, without changing application code.

Q4. A legacy app exposes metrics in a custom format. How do you monitor it with Prometheus? SCENARIO

Run an adapter (exporter) sidecar that reads the custom metrics and serves them at /metrics in Prometheus format; the app is unchanged.

Q5. When should you not use a sidecar? TRICKY

When the overhead (CPU, memory, startup ordering, debugging) outweighs a small feature, or latency is critical; then a library is simpler.

Q6. Your app crashes on start because its proxy sidecar isn't ready. Fix? TRICKY

Order startup: use native sidecar containers (init containers with restartPolicy Always) or a readiness wait, and make the app retry its first connections.

Next: SD-L-011 · Strangler Fig Pattern backendroadmap.com

Strangler Fig Pattern

replace it slice by slice, never all at once

Watch the 4-min video

In one line: put a routing facade in front of the legacy system and move one slice of functionality at a time to new services, each move small, verified and reversible, until the old system has nothing left to do; never a big-bang rewrite.

1 Why not rewrite?

A ten-year-old monolith; everyone wants a rewrite. A big-bang rewrite means a year or more with no new features while the new system chases a moving target, then a cutover weekend that concentrates all the risk in one day. Many rewrites are abandoned. The **strangler fig** (named by Martin Fowler after a fig that grows around a host tree until the tree dies) replaces the system while it keeps running.

2 The facade

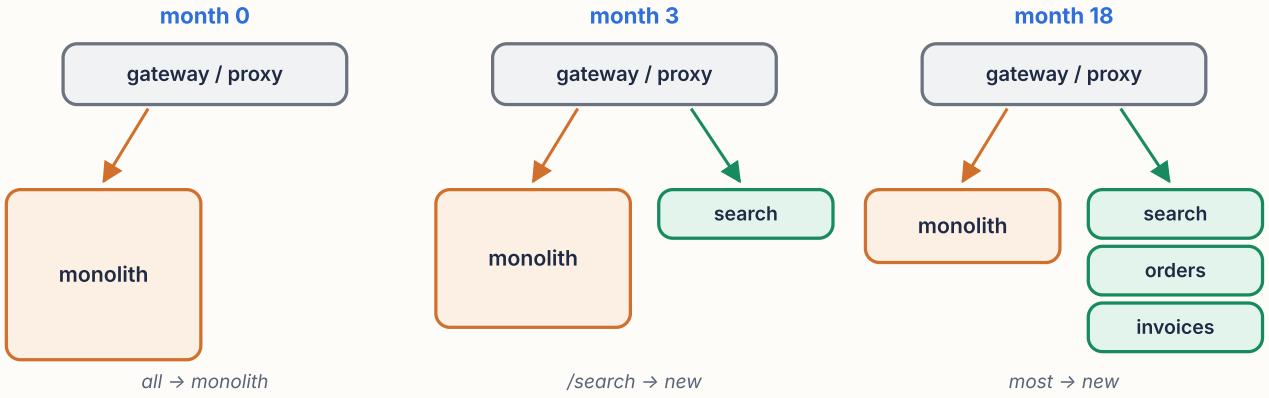


Figure 1. Clients always call the same URLs; only the routing rules behind the facade change.

Put a routing layer in front of the monolith: an API gateway, a reverse proxy such as NGINX or Envoy, or a load balancer with path rules. At first it sends everything to the monolith. Then you route one endpoint at a time to a new service. Clients never change URLs. Each move is a **routing change**, so it is reversible in seconds: if the new service misbehaves, flip the route back.

3 Choosing and verifying slices

- **Start small and loosely coupled:** a read-only endpoint like product search or catalog, not the tangled order core.
- **Ship in weeks, not months:** each slice delivers value on its own.
- **Parallel run for risky slices:** send each request to both old and new, serve the **old** answer, compare in the background and log every difference. Switch when differences stay at zero for a while.
- **Shift traffic gradually:** 1% → 10% → 50% → 100%, watching error rates and latency at each step.

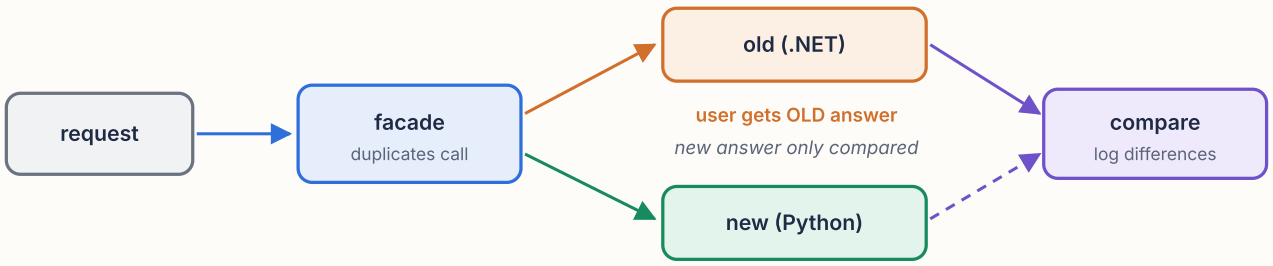


Figure 2. Parallel run ("dark launch"): users are served by the old code while the new code is checked against it.

4 Data: the hardest part

Step	Who owns the data	How the new service gets it
1. Read through	monolith	calls the monolith's API (via an anti-corruption layer, lesson 30)
2. Sync	monolith	own copy kept fresh by change data capture
3. Take over	new service	new service writes; sync back to the monolith if it still reads it
4. Clean up	new service	delete the old tables and code

Step 4 matters: if old code is never deleted, the monolith never shrinks and you run two systems for ever.

WORKED EXAMPLE: .NET MONOLITH TO PYTHON SERVICES

A gateway is put in front of a .NET monolith. First `/search` moves to `python-search-svc`, with a two-week parallel run comparing results; once differences are near zero, traffic shifts 1% → 10% → 100%. Then `/orders`, then `/invoices`. Each move is a routing change, so there is no downtime. After eighteen months only `/admin/*` still routes to the monolith.

COMMON MISTAKES

- Starting with the most tangled, most critical part.
- New services writing to the monolith's tables directly, creating a shared database.
- Never deleting migrated code, so the old system lives for ever.
- No rollback route: making a slice move irreversible.

5 Key terms

Strangler fig: incrementally replace a system behind a facade

Slice: a piece of functionality moved as one unit

Traffic shifting: move a growing percentage of requests

Facade / router: layer that decides old vs new per request

Parallel run: run old and new, serve old, compare results

Big-bang rewrite: replace everything in one cutover

6 Interview questions

Q1. What is the strangler fig pattern? BASIC

Incrementally replacing a legacy system by routing one slice of functionality at a time to new code behind a facade, until the old system can be removed.

Q2. Why not a big-bang rewrite? BASIC

It delays all value until the end, chases a moving target, and concentrates all the risk in a single cutover.

Q3. How would you migrate a large monolith with no downtime? SCENARIO

Put a routing facade in front, move one slice at a time, verify risky slices with a parallel run, shift traffic gradually, keep every step reversible, and delete old code once a slice is done.

Q4. Which slice do you move first? SCENARIO

Something small, valuable and loosely coupled, such as a read-only search or catalog endpoint, so the team learns the process with low risk.

Q5. How do you verify a rewritten pricing endpoint gives the same answers? TRICKY

Parallel run: call both, serve the old result, compare responses in the background and investigate every difference before shifting traffic.

Q6. The new service needs data the monolith owns. What now? TRICKY

Read through the monolith's API first, then keep a copy synced with CDC, then transfer ownership to the new service and remove the old tables.

Next: SD-L-012 · Anti-Corruption Layer

Anti-Corruption Layer

translate at the border, keep the domain clean

Watch the 4-min video

In one line: an anti-corruption layer (ACL) is a translation layer at one boundary that converts a legacy or external system's names, codes and structures into your own domain model and back, so the foreign model never leaks into your code.

1 KUNNR and status code 7

A new fulfilment service needs orders from a twenty-year-old ERP. The ERP calls a customer `KUNNR`, uses status `7` for shipped, stores weights in grams and dates as strings. If you use those models directly, every developer must learn what code 7 means, the ERP's quirks spread through your code, and your clean new domain slowly becomes the legacy system.

2 What an ACL is

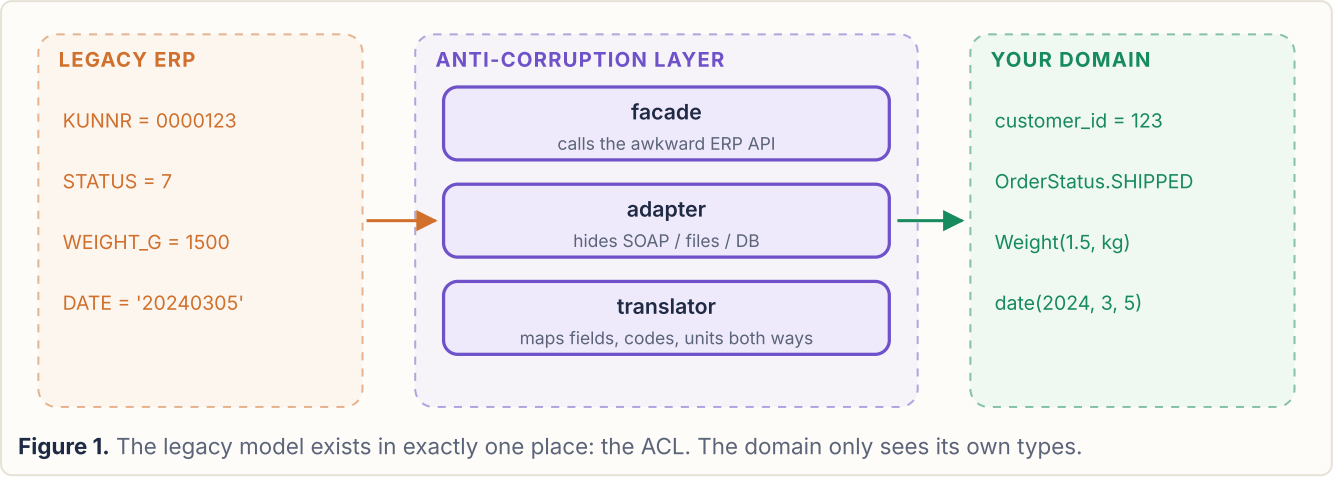


Figure 1. The legacy model exists in exactly one place: the ACL. The domain only sees its own types.

The ACL is a pattern from domain-driven design's context map (lesson 22). It sits between two bounded contexts whose models disagree and translates in both directions. Typically it has three parts:

- **Facade:** a simple interface over the legacy system's awkward API.
- **Adapters:** hide the transport: SOAP, nightly files, message formats, even a shared database.
- **Translators:** map fields, codes and units into domain objects, and back for writes.

It can be a module inside your service (often the adapter behind a port, lesson 31) or a small service of its own when several services need the same translation.

3 Why models leak, and what the ACL buys

Without an ACL	With an ACL
status code 7 checked in twenty files	`OrderStatus.SHIPPED` everywhere; code 7 in one mapping
ERP adds code 9: hunt for every switch statement	update one translator, add one test
ERP replaced: rewrite your service	replace the adapter; the domain is unchanged
legacy naming in new APIs and databases	your ubiquitous language (lesson 21) stays intact

The ACL pairs naturally with the **strangler fig** (lesson 29): new services talk to the monolith only through an ACL, and when the monolith is finally gone you delete the ACL and the domain is already clean.

COSTS AND WHEN TO SKIP IT

- **Extra code** to write, test and maintain.
- **Latency:** translation is cheap, but an ACL that makes one legacy call per field is slow. Batch calls and cache reference data.
- **Hidden data loss:** a legacy field with no clean equivalent can be silently dropped. Map it explicitly or fail loudly.
- **Skip it** when the other model already matches yours, or the integration is tiny and short-lived.

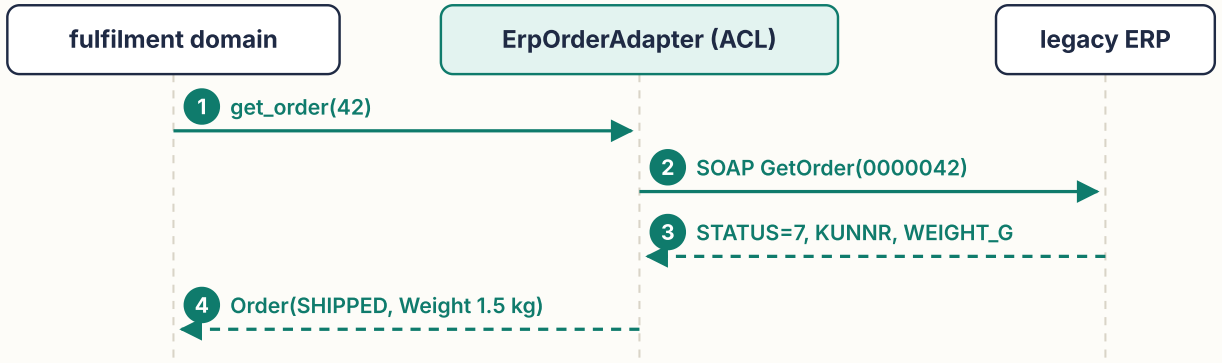


Figure 2. One read: the domain asks in its own terms; only the adapter speaks the ERP's language.

WORKED EXAMPLE: FULFILMENT READS ORDERS FROM THE ERP

An `ErpOrderAdapter` calls the ERP and returns domain objects. Its translator maps `STATUS = 7` → `OrderStatus.SHIPPED`, `KUNNR` → `customer_id` (stripping leading zeros), `WEIGHT_G = 1500` → `Weight(1.5, kg)`, and `'20240305'` → a real date. An unknown status code raises an error and an alert instead of guessing. The fulfilment domain never sees an ERP code; when the company replaces the ERP, only the adapter is rewritten.

4 Key terms

Anti-corruption layer: translator that keeps a foreign model out

Facade: simple interface over a complex system

Model leakage: foreign concepts spreading through your code

Translator / mapper: converts fields, codes and units

Adapter: hides transport such as SOAP or files

Upstream / downstream: who provides vs who consumes a model

5 Interview questions

Q1. What problem does an anti-corruption layer solve? BASIC

It stops a legacy or external model from leaking into your domain by translating names, codes and structures at one boundary.

Q2. What are the usual parts of an ACL? BASIC

A facade over the foreign API, adapters for the transport, and translators that map fields, codes and units in both directions.

Q3. Your new service must read orders from an ERP with cryptic codes. How? SCENARIO

Build an adapter with a translator that maps ERP codes and fields into domain types, fails loudly on unknown values, and is the only code that knows the ERP model.

Q4. How does an ACL help a strangler fig migration? SCENARIO

New services talk to the monolith only through the ACL, so their domain stays clean; when the monolith is gone, you delete the ACL.

Q5. When would you skip an ACL? TRICKY

When the other system's model already matches yours, or the integration is tiny; the extra code and latency then buy nothing.

Q6. Your ACL is slow. Why, and how do you fix it? TRICKY

Usually it makes many fine-grained legacy calls; batch requests, cache reference data such as code tables, and fetch only what the use case needs.

Next: SD-L-013 · Hexagonal Architecture backendroadmap.com

Hexagonal Architecture (Ports and Adapters)

the core defines the plugs

Watch the 4-min video

In one line: the domain core defines **ports** (interfaces in its own terms); **adapters** implement them for real technologies or call into the core; all source dependencies point inward, so infrastructure can be swapped and the core tested with fakes.

1 Tangled logic

A pricing rule reads HTTP headers, writes SQL inline and calls the OpenAI SDK directly. Now you must switch LLM provider, or simply write a unit test, and everything is tangled. **Hexagonal architecture** (Alistair Cockburn, 2005), also called **ports and adapters**, keeps the core separate from the outside world.

2 Core, ports and adapters

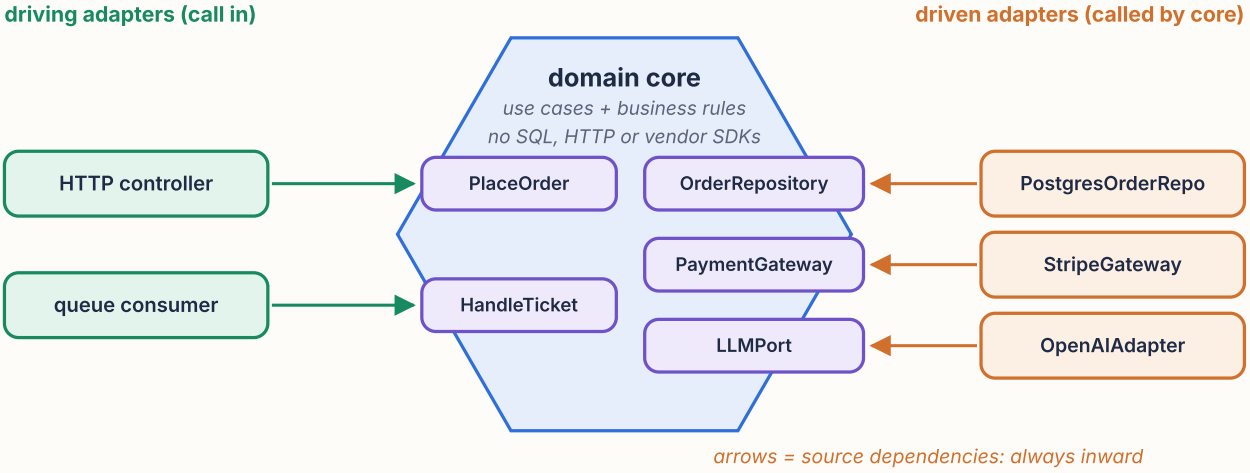


Figure 1. The core exposes and defines ports; adapters on both sides plug in. The six sides have no meaning: they just leave room for many ports.

- **Domain core:** business rules and use cases. Knows nothing about databases, web frameworks or vendors.
- **Ports:** interfaces the core defines in its own language: `OrderRepository.save(order)`, `PaymentGateway.charge(amount)`. Never "execute SQL" or "POST JSON".
- **Driving (primary) adapters** call into the core: an HTTP controller, a CLI command, a queue consumer, a test.
- **Driven (secondary) adapters** are called by the core: a Postgres repository, an email sender, a Kafka publisher, an LLM client.

3 Dependency inversion

The core defines the interface and the adapter implements it, so the **Postgres adapter depends on the core**, never the other way round. That is the D in SOLID (dependency inversion principle) applied to the

whole application. Control flows outward at run time (core → port → adapter → database), but source dependencies point inward. A **composition root** (the app's start-up code or a DI container) chooses which adapter plugs into each port.

4 Testing



Figure 2. Same core, different wiring. Only the composition root changes.

Plug in-memory fakes into the ports and the business logic is tested in milliseconds, with no database, network or API keys. Then test each real adapter with a few **integration tests** against the real technology (e.g. Postgres in a container). Fast tests where the logic is, slow tests only at the edges.

Use it when	Skip it when
the domain logic is rich	simple CRUD: forms in, rows out
you integrate several external systems	a prototype or short-lived script
you expect to swap infrastructure or vendors	the framework will never change and there is little logic

Don't add a port for everything. A good rule: one port per external system, or per reason to change.

WORKED EXAMPLE: SWAPPING THE LLM PROVIDER

A support assistant's core defines `LLMPort` with one method, `complete(prompt: str) -> str`. An `OpenAIAdapter` implements it. Later the company moves to **Azure OpenAI** for data residency. The team writes `AzureOpenAIAdapter(LLMPort)` and changes one line in the composition root. No business rule changes. Unit tests use a `FakeLLM` that returns canned answers, so they run offline and are deterministic.

COMMON MISTAKES

- Ports that mirror a vendor's API (`OpenAIChatCompletionPort`) instead of the core's need.
- Domain objects importing ORM base classes or framework annotations.
- An interface for every class, "just in case".
- Only fake-adapter tests, with no integration tests of the real adapters.

5 Key terms

Port: interface defined by the core in its own terms

Adapter: implements a port for a real technology

Driving adapter: calls into the core: HTTP, CLI, consumer

Driven adapter: called by the core: DB, email, LLM

Dependency inversion: details depend on abstractions, not the reverse

Composition root: start-up code that wires adapters to ports

6 Interview questions

Q1. Explain ports and adapters. BASIC
The core defines ports, interfaces in its own terms; adapters implement them for real technologies or drive the core; dependencies always point inward so infrastructure can be swapped.

Q2. What's the difference between driving and driven adapters? BASIC
Driving adapters call into the core (HTTP controller, queue consumer); driven adapters are called by the core through a port (database, email, payment API).

Q3. How does it help testing? SCENARIO
You plug in-memory fakes into the ports, so business logic is tested in milliseconds with no database, network or API keys; real adapters get a few integration tests.

Q4. You must move from OpenAI to Azure OpenAI. What changes? SCENARIO
Write a new adapter implementing the same LLM port and change the wiring in the composition root; the core and its tests don't change.

Q5. If the core calls the database at run time, how do dependencies point inward? TRICKY
The core calls an interface it owns; the database adapter implements that interface, so source code depends inward while control flows outward.

Q6. When is hexagonal architecture overkill? TRICKY
For simple CRUD services or prototypes with little logic, where many interfaces add ceremony without protecting anything.

Next: SD-L-014 · Clean Architecture backendroadmap.com

Clean Architecture

dependencies point inward

Watch the 4-min video

In one line: clean architecture arranges code in rings (entities, use cases, interface adapters, frameworks) under one rule: source-code dependencies point only inward, so business logic never depends on the web framework, the ORM or the database.

1 Database models everywhere

In a typical web project the ORM models show up in controllers, templates and business rules. Changing the framework or the database means rewriting everything, and testing a rule needs a running database. Robert C. Martin's **clean architecture** (2012) fixes this with one strict rule about which code may depend on which.

2 The rings

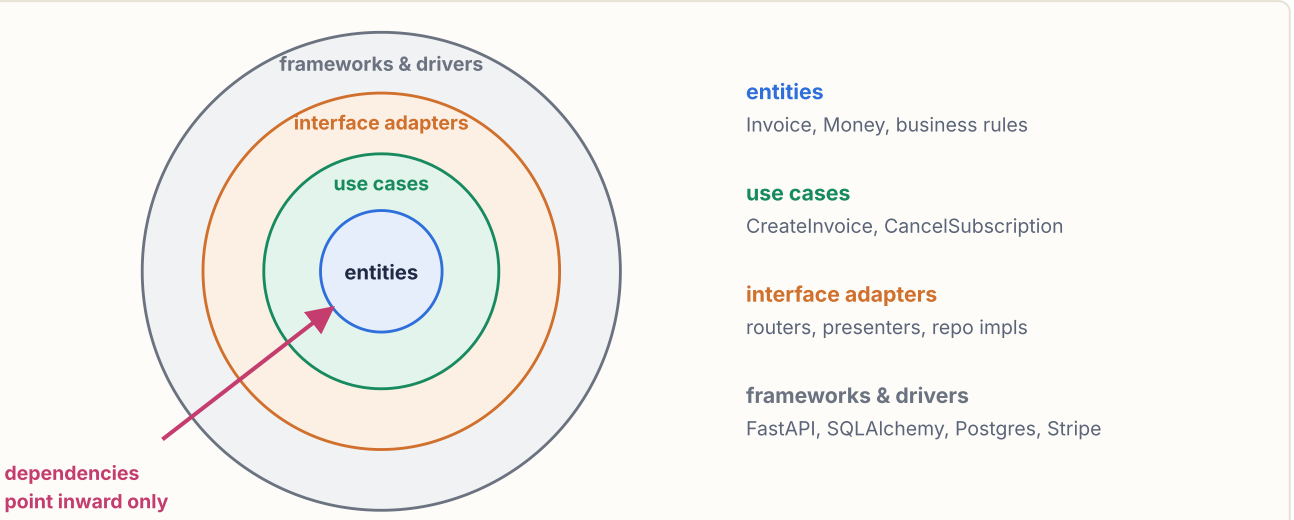


Figure 1. Four rings, one rule. Outer rings know about inner rings, never the reverse.

- **Entities:** core business objects and rules that would exist even without software.
- **Use cases:** application actions that orchestrate entities: *place order*, *cancel subscription*.
- **Interface adapters:** controllers, presenters, repository implementations: convert between use-case data and external formats.
- **Frameworks and drivers:** the web framework, ORM, database, message broker, external APIs.

3 The dependency rule

Source-code dependencies point **only inward**. An entity never imports the web framework; a use case never imports the ORM. Data crossing a boundary is passed as simple structures that belong to the inner ring (a command object, a DTO), not as ORM rows or HTTP requests.

How does a use case save without importing the database? The same trick as hexagonal architecture (lesson 31): the use case defines a `InvoiceRepository` interface, an outer ring implements it, and the

composition root wires the real implementation in at start-up. Control flows outward; dependencies point inward.

```
# adapters/http.py  (outer ring)
@router.post('/invoices')
def create(req: CreateInvoiceRequest):
    return CreateInvoice(repo).run(req.to_cmd())

# usecases/create_invoice.py  (inner ring: no FastAPI, no SQLAlchemy)
class CreateInvoice:
    def __init__(self, repo: InvoiceRepository): self.repo = repo
    def run(self, cmd): invoice = Invoice.create(cmd.customer_id, cmd.lines)
                        self.repo.add(invoice); return invoice.id

# infrastructure/sql_invoice_repo.py implements InvoiceRepository
```

Figure 2. Thin router, use case depending only on an interface, implementation in infrastructure.

4 Cousins: hexagonal, onion, clean

Style	Vocabulary	Distinctive idea
Hexagonal (Cockburn, 2005)	ports and adapters	driving vs driven sides
Onion (Palermo, 2008)	rings: domain model, domain services, application services	layers around the domain model
Clean (Martin, 2012)	entities, use cases, adapters, frameworks	explicit use-case classes; the dependency rule

All three put the domain in the middle and push infrastructure to the edges. Pick the vocabulary your team understands; the boundaries matter more than the names.

WHEN IT'S OVERKILL

- Small CRUD services and prototypes: many files for little logic, each field mapped three times (request → command → entity → row).
- Short-lived code, or a framework you will never replace.
- Use it for complex, long-lived business rules; you can apply it to one complex module and keep the rest simply layered.

WORKED EXAMPLE: A FASTAPI INVOICING SERVICE

Routers are thin: they parse the request and call a use case such as `CreateInvoice`. The use case depends on an `InvoiceRepository` interface, not on SQLAlchemy. `SqlInvoiceRepository` lives in an `infrastructure` package. Unit tests call `CreateInvoice` directly with an `InMemoryInvoiceRepository` and never start FastAPI or a database. An import-linter rule (lesson 20) fails the build if `usecases` imports `fastapi` or `sqlalchemy`.

5 Key terms

Dependency rule: source dependencies point only inward

Use case: one application action, e.g. CreateInvoice

Entity: core business object and rules

Interface adapter: converts between use cases and the outside

Composition root: start-up code that wires implementations

DTO: simple data structure crossing a boundary

6 Interview questions

Q1. What is the dependency rule? BASIC

Source-code dependencies only point inward: entities and use cases never import frameworks, ORMs, databases or UI code.

Q2. Name the four rings. BASIC

Entities, use cases, interface adapters, and frameworks and drivers, from the centre outward.

Q3. How does a use case save data without depending on the database? SCENARIO

It defines a repository interface; the infrastructure ring implements it, and the composition root injects the implementation at start-up.

Q4. How do you keep the rule from eroding? SCENARIO

Enforce it in the build with import rules or architecture tests (import-linter, ArchUnit), and keep framework types out of use-case signatures.

Q5. When is clean architecture overkill? TRICKY

For small CRUD services and prototypes, where the extra layers and mappings cost more than they protect.

Q6. How is it different from hexagonal architecture? TRICKY

Barely: both keep the domain at the centre with dependencies pointing inward; clean architecture adds named rings and explicit use-case classes.

Next: SD-L-015 · Serverless

Serverless Architecture

pay per call, mind the cold start

Watch the 4-min video

In one line: serverless (functions as a service) runs your stateless function when an event arrives, scales it out automatically and bills per request and duration; it fits spiky, event-driven, short tasks and fits poorly with long jobs, steady heavy traffic, strict latency or stateful connections.

1 No servers, no idle cost

What if you never patched a server, never configured autoscaling, and paid nothing when nobody used the app? That is the promise of **FaaS**: AWS Lambda, Azure Functions, Google Cloud Functions (now Cloud Run functions). You upload a function; the platform runs it when an event arrives and runs more copies in parallel as events pile up. Lambda bills per request and per millisecond of duration, scaled by the memory you allocate.

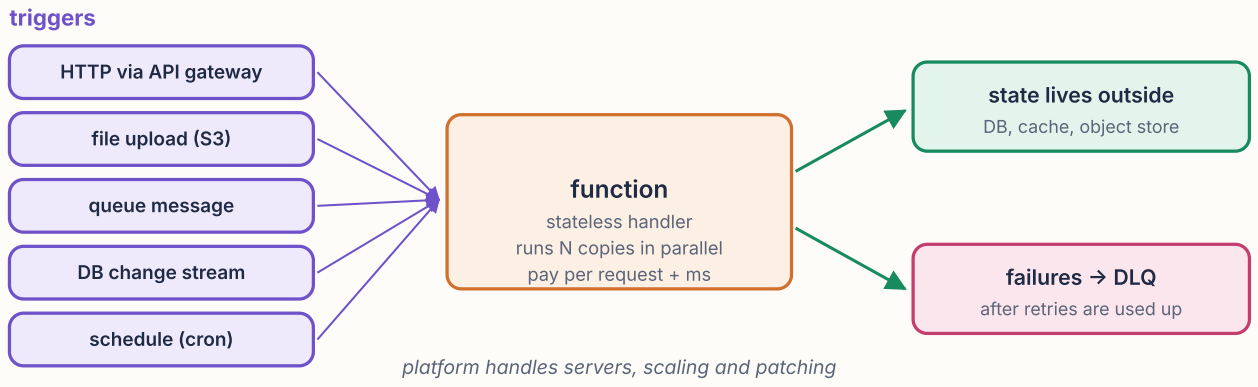


Figure 1. Events trigger stateless functions; state lives in managed services; failed events go to a dead-letter queue.

2 Triggers make it event-driven

An HTTP request through an API gateway, an object uploaded to storage, a message on a queue, a database change stream, a schedule. This makes serverless a natural fit for event-driven glue code and bursty work. Functions can be chained with queues between them to form pipelines, or coordinated by a workflow service such as AWS Step Functions.

3 The cold start

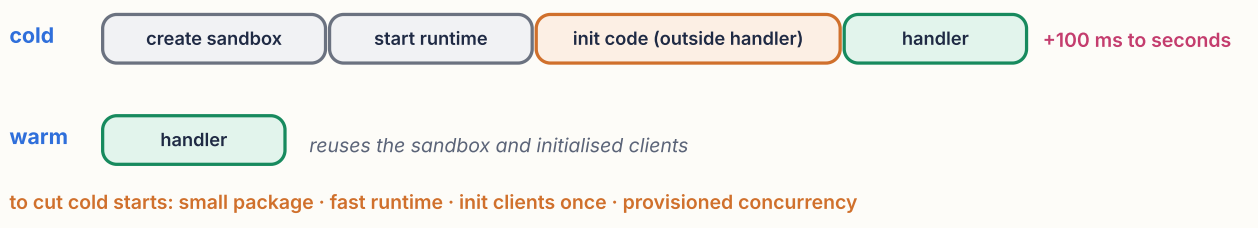


Figure 2. A cold start pays for sandbox creation, runtime start and your init code before the handler runs.

When no warm instance exists, the platform creates a sandbox, starts the runtime and runs your initialisation code. That adds from around 100 ms to several seconds, depending mostly on runtime and package size (large JVM or .NET apps are slowest without tuning). To reduce it: keep packages small, choose a fast-starting runtime, create SDK and database clients **outside the handler** so warm invocations reuse them, and use **provisioned concurrency** (pre-warmed instances, paid for while idle) on latency-sensitive paths.

4 Other limits

Limit	What it means	Mitigation
Stateless	local memory and disk don't survive between calls	keep state in a DB, cache or object store
Max duration	Lambda stops a run at 15 minutes	split work, or use containers or batch jobs
Concurrency	account limit (Lambda default 1,000 per region); fast scale-out can flood a database	reserved concurrency, queue in front, connection proxy (RDS Proxy)
Observability	many short-lived executions	structured logs, correlation IDs, tracing (lesson 36)

WHEN SERVERLESS IS A BAD FIT

- **Long-running jobs** beyond the duration limit.
- **Steady, high-volume traffic:** always-on containers or VMs are usually cheaper per request.
- **Strict low latency** that can't tolerate cold starts (unless you pay for provisioned concurrency).
- **State or persistent connections:** WebSockets held for hours, in-memory caches, local files.

Many teams mix both: containers for the core API, functions for event-driven side jobs.

WORKED EXAMPLE: PHOTO THUMBNAILS

A user uploads a photo to an S3 bucket. The `ObjectCreated` event triggers a Lambda function that downloads the image, creates small, medium and large thumbnails and writes them to a **different** bucket (writing to the same bucket would re-trigger the function in a loop). With no uploads the compute cost is zero; a burst of 10,000 uploads runs many copies in parallel, capped by reserved concurrency. Events that still fail after retries go to a dead-letter queue for inspection.

5 Key terms

FaaS: functions as a service: run code per event

Cold start: delay to create a new function instance

Provisioned concurrency: pre-initialised instances kept warm

Reserved concurrency: cap (and guarantee) for one function

Dead-letter queue: holds events that failed all retries

Trigger: event source that invokes a function

6 Interview questions

Q1. What is serverless / FaaS? BASIC

You deploy functions; the platform runs them on events, scales them automatically and bills per request and execution time, with no servers to manage.

Q2. What is a cold start? BASIC

The extra latency when no warm instance exists and the platform must create a sandbox, start the runtime and run initialisation code.

Q3. When is serverless a bad fit? SCENARIO

Long-running jobs, steady high-volume traffic where always-on servers are cheaper, strict low-latency paths, and workloads needing state or persistent connections.

Q4. How do you handle cold starts? SCENARIO

Keep packages small, initialise clients outside the handler, choose a fast-starting runtime, and use provisioned concurrency on latency-sensitive paths.

Q5. A traffic spike makes your Lambda functions take down the database. Why and how to fix? TRICKY

Functions scale out to hundreds of concurrent copies, each opening connections; cap reserved concurrency, put a queue in front, and pool connections (e.g. RDS Proxy).

Q6. Your S3-triggered function runs for ever and the bill explodes. Likely cause? TRICKY

It writes its output to the same bucket that triggers it, creating a loop; write to another bucket or filter the trigger by prefix.

Next: SD-L-016 · Saga Orchestration backendroadmap.com

Saga Orchestration in Microservices

durable workflows, not cron jobs

Watch the 4-min video

In one line: an orchestrated saga runs a multi-service business process as local transactions with compensations, and a workflow engine (Temporal, Step Functions, Durable Functions) gives it durable state, retries, timeouts and crash recovery, so you don't hand-build a fragile state machine.

1 The orchestrator crashed. Was the card charged?

An order needs payment, stock and shipping, in three services with three databases (lesson 23), so there is no single ACID transaction. Halfway through, the orchestrator crashes. Did payment succeed? Retry it, or refund? Hand-written orchestrators keep progress in state tables, poll with cron jobs and grow fragile fast.

2 Saga recap



Figure 1. A saga: each step commits locally; if a later step fails, earlier steps are undone by compensating actions.

A **saga** is a sequence of local transactions, each with a **compensating action** that semantically undoes it (a refund, not a rollback). In **choreography** services react to each other's events; in **orchestration** one orchestrator tells each service what to do and decides what happens next (lesson 24). The happy path is easy. The hard part is crashes, timeouts and retries in the middle, and running compensations in reverse order.

3 Durable execution

A workflow engine turns that into a platform. You write the workflow as ordinary-looking code: charge, then reserve, then ship. The engine records every step's result in an **event history**. If a worker crashes, another worker **replays** the history: completed steps return their recorded results instead of running again, and execution continues exactly where it stopped. A workflow can sleep for days and still resume.

Activities (the calls to services) get declarative **retry policies** with exponential backoff and **timeouts**. If a step finally fails, the workflow runs compensations for the completed steps. You can **query** a running workflow for its current step, which helps support teams.

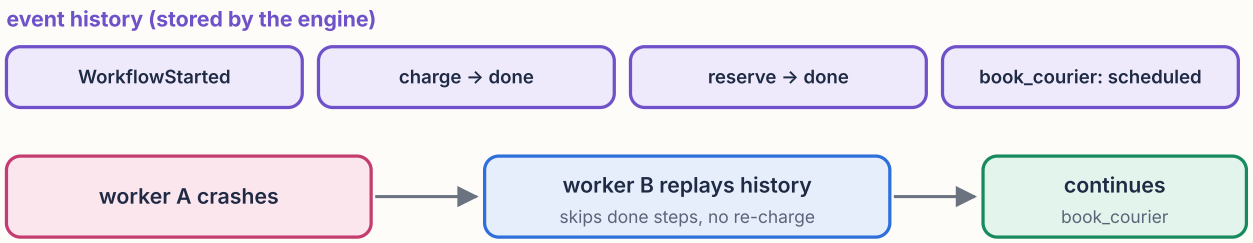


Figure 2. Replay: the history, not the worker's memory, is the source of truth.

```
@workflow.defn
class Fulfil:
    @workflow.run
    async def run(self, order):
        await workflow.execute_activity(charge, order, retry_policy=RETRY, start_to_close_timeout=T)
        try:
            await workflow.execute_activity(reserve, order, ...)
            await workflow.execute_activity(book_courier, order, ...)  # retried for up to 3 days
        except ActivityError:
            await workflow.execute_activity(release_stock, order, ...)   # compensations
            await workflow.execute_activity(refund, order, ...)
            raise
```

Figure 3. Temporal-style Python: the saga reads like normal code; the engine makes it durable.

	Hand-built orchestrator	Workflow engine
Progress after a crash	own state table, custom recovery code	replayed from stored history
Retries and timeouts	hand-coded, often inconsistent	declarative policy per activity
Waiting days	cron jobs polling tables	durable timers
Visibility	SQL queries on state tables	UI and queries per workflow

RULES AND TRADE-OFFS

- **Activities must be idempotent:** the engine may run them more than once (e.g. pass the order ID as an idempotency key to payment).
- **Workflow code must be deterministic:** no random numbers, direct clock reads or network calls inside it; use the engine's APIs, because replay must take the same path.
- **Versioning:** changing code while workflows are running needs explicit versioning or patching.
- **Another platform** to run or pay for, and keep business rules in the services, not all in the workflow.

WORKED EXAMPLE: ORDER FULFILMENT ON TEMPORAL

The workflow charges the card, reserves stock and books a courier, each as an activity with retries. The courier API is down; the activity retries with backoff for three days, then fails. The workflow releases the stock and refunds the payment, in reverse order. A worker restart on day two loses nothing, because the history is stored in Temporal's database; the new worker replays it and keeps waiting.

4 Key terms

Saga: sequence of local transactions with compensations

Orchestration: one coordinator directs the steps

Activity: a side-effecting step, retried by policy

Compensating action: business undo for a committed step

Durable execution: workflow state survives crashes via history

Deterministic workflow: same history replays to the same decisions

5 Interview questions

Q1. What is a saga? BASIC

A sequence of local transactions across services, each with a compensating action that undoes it if a later step fails.

Q2. Orchestration vs choreography? BASIC

Orchestration has one coordinator that commands each step and tracks state; choreography has services reacting to each other's events with no central controller.

Q3. Why use a workflow engine for sagas? SCENARIO

It provides durable state, retries with backoff, timeouts, crash recovery by replaying history, and compensation logic, instead of a fragile hand-built state machine.

Q4. The courier step fails after days of retries. What happens? SCENARIO

The workflow runs compensations for completed steps in reverse order: release stock, then refund the payment, and records the outcome.

Q5. Why must workflow code be deterministic? TRICKY

Recovery replays the history through the code; if the code makes different decisions (random values, clock reads), it diverges from the recorded history.

Q6. What do you need to watch out for? TRICKY

Idempotent activities, deterministic workflow code, and versioning for long-running workflows when the code changes.

Next: SD-L-017 · CQRS and Event Sourcing in Microservices backendroadmap.com

CQRS and Event Sourcing in Microservices

build the view from events

Watch the 4-min video

> **In one line:** to query across services you either compose their APIs at read time, or build a dedicated **read model** from their events: a projection shaped for the screen, fast and resilient but eventually consistent, and rebuildable by replaying the event history.

1 One screen, four databases

Support wants one screen with a customer's profile, orders, payments and tickets. That data lives in four services, each with its own database (lesson 23), so there is no join. How do you answer a query that spans services?

2 CQRS across services

CQRS (command query responsibility segregation) splits the **write side**, which handles commands and enforces rules, from the **read side**, which serves queries from models shaped for the screen. In microservices the read model doesn't have to live in the owning service. Each service publishes events when its data changes, and a separate **projection** service consumes them and maintains one denormalised view per customer. Queries hit that view, with no joins at query time.

Event sourcing goes further inside a service: the events themselves are the source of truth (an append-only log), and current state is derived by replaying them. You don't need event sourcing to build cross-service read models; published integration events are enough, as long as they are kept long enough to replay.

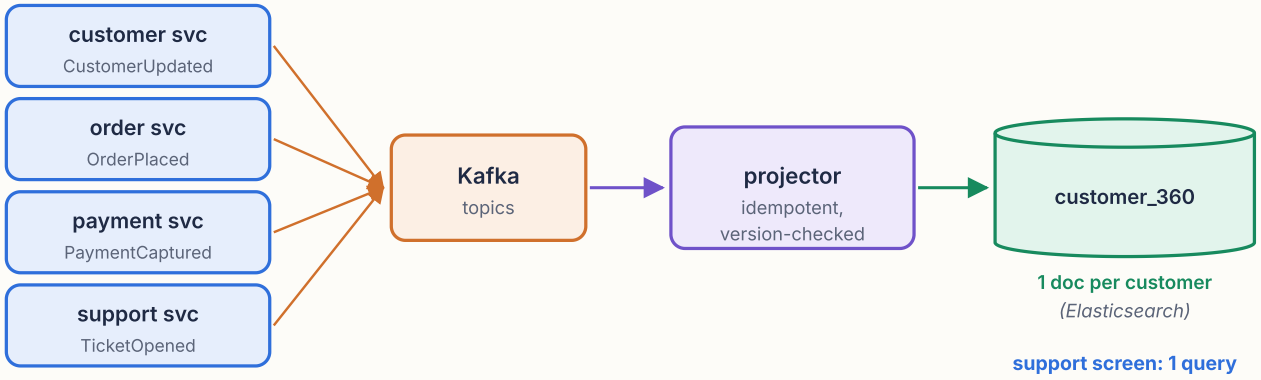


Figure 1. Services publish events; a projector folds them into one document per customer; the screen reads one document.

3 Composition or read model?

	API composition	Event-built read model
How	call all four services at query time and merge	projector maintains a view from events
Freshness	always current	eventually consistent (lag: ms to seconds)
Latency	slowest service + merge	one lookup
One service down	query fails or is partial	still served (slightly stale)
Cost	simple	broker, projector, extra store, rebuilds

Many teams start with composition and add read models for hot or complex queries.

4 Projections done right

- **Idempotent:** events can arrive twice; record processed event IDs or use upserts.
- **Ordering:** handle out-of-order events with per-entity version numbers; ignore an event older than the stored version.
- **Rebuilds:** to fix a bug or add a field, replay the history into a **new** table or index while the old one keeps serving, then switch. This needs events retained long enough (e.g. Kafka retention or compacted topics).
- **Operations:** monitor **projection lag** and show staleness if needed; version event schemas; remember the view is a copy and the owning services stay the source of truth.

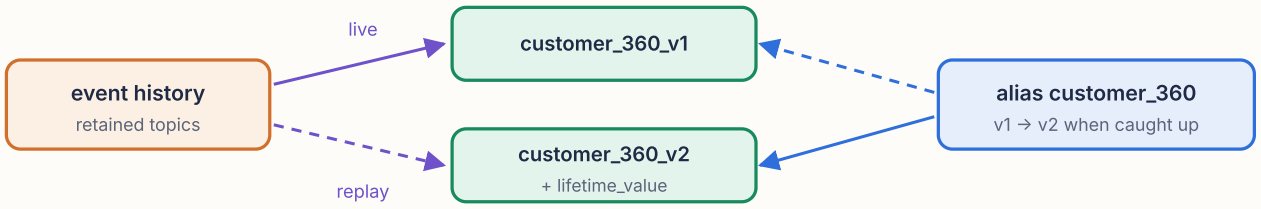


Figure 2. Safe change: build v2 from history beside v1, then move the alias. Users never see a half-built view.

WORKED EXAMPLE: THE CUSTOMER 360 VIEW

Customer, order, payment and support services publish events to Kafka. A projector writes one document per customer into Elasticsearch, so the support screen loads in a single call. To add a `lifetime_value` field, the team deploys a new projector that replays the retained topics into `customer_360_v2`. When its lag reaches zero they switch the `customer_360` alias from v1 to v2, and delete v1 a week later.

COMMON MISTAKES

- Treating the read model as the source of truth and writing to it.
- Non-idempotent projectors that double-count on redelivery.
- Event retention too short to rebuild a projection.
- Building read models for queries that simple composition would serve.

5 Key terms

CQRS: separate models for writes (commands) and reads (queries)

Event sourcing: events are the source of truth; state is derived

Projection lag: delay between an event and the updated view

Read model / projection: query-shaped view built from events

API composition: query several services and merge at read time

Replay: rebuild a view by reprocessing event history

6 Interview questions

Q1. What is CQRS? BASIC

Separating the write model, which handles commands and enforces rules, from read models shaped for queries, so each can be designed and scaled for its job.

Q2. Is event sourcing required for CQRS? BASIC

No: read models can be built from published integration events; event sourcing makes the event log the write side's source of truth.

Q3. How do you build a query that spans multiple services? SCENARIO

Either API composition at query time (simple, fresh, but slower and fragile) or a read model built from the services' events (fast and resilient, eventually consistent).

Q4. How do you change a read model safely? SCENARIO

Replay the event history into a new projection (new table or index) beside the old one, and switch traffic, e.g. an alias, once it has caught up.

Q5. An event arrives twice and another arrives out of order. How does the projector cope? TRICKY

It is idempotent (dedupe by event ID or upsert) and checks a per-entity version, ignoring events older than what it has applied.

Q6. Support sees an order the customer cancelled a second ago. Bug? TRICKY

Expected eventual consistency: the view lags the source; monitor lag, show a freshness hint, or read from the owning service for critical checks.

Next: SD-L-018 · Distributed Tracing in Microservices backendroadmap.com

Distributed Tracing Across Services

follow one request through every hop

Watch the 4-min video

In one line: a trace is one request's whole journey and a span is one timed unit of work in it; services pass the trace context (the W3C `traceparent` header) on every call and message, so all spans join into one tree that shows exactly where the time goes.

1 Four seconds, eight services, everyone says "not us"

Checkout takes four seconds and touches eight services. Every team says its service is fast. Logs show fragments, but nobody can see the whole journey. **Distributed tracing** connects every hop of one request into a single picture.

2 Traces and spans

A **trace** is the full journey of one request, identified by a 16-byte trace ID. A **span** is one unit of work inside it (an HTTP call, a DB query, a message handler) with a name, start time, duration, attributes, status and a **parent span**. Together the spans form a tree, drawn as a **waterfall** timeline: what runs in sequence, what runs in parallel, and which span is slow.

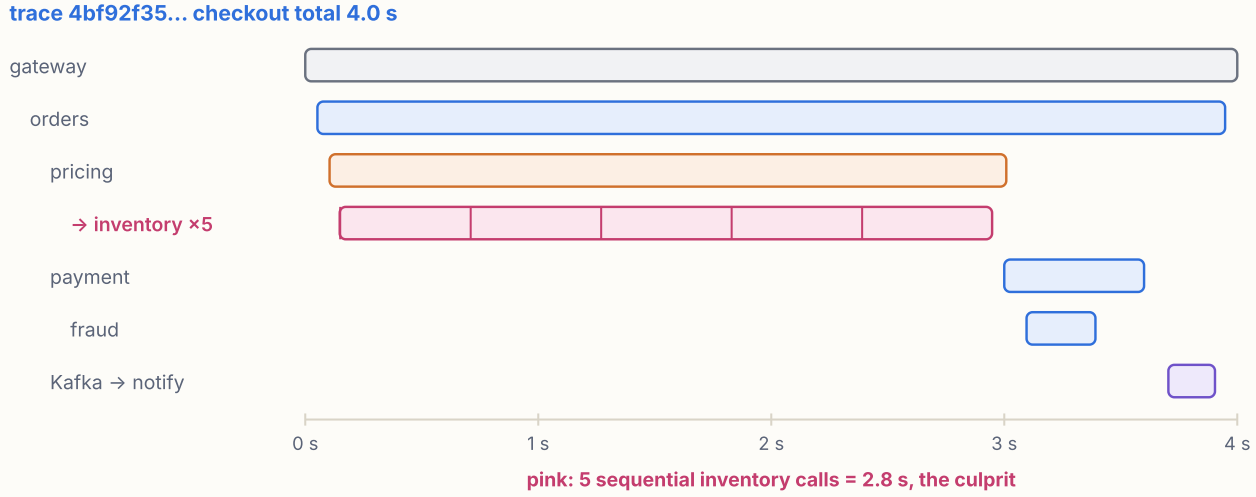


Figure 1. The waterfall from the worked example: pricing calls inventory five times in a row.

3 Context propagation

When service A calls B, it must pass the trace ID and its own span ID. The standard is the **W3C Trace Context** header `traceparent`: version, trace ID, parent span ID and flags (01 = sampled). B reads it, starts a child span, and passes its own context on. A companion `tracestate` header carries vendor-specific data.



Figure 2. The same context travels in HTTP/gRPC headers and in message headers, so the trace survives the asynchronous hop.

Across Kafka there is no live call. The producer writes `traceparent` into the **message headers**; the consumer extracts it and starts a span linked to the producer's span, even if it runs minutes later. OpenTelemetry's messaging conventions model this with parent/child or span links.

4 In practice: OpenTelemetry

- **Instrument** with OpenTelemetry SDKs and auto-instrumentation for frameworks, HTTP clients, DB drivers and Kafka clients; they create spans and propagate context for you (lesson 64).
- **Export** via OTLP, often through an OpenTelemetry Collector, to a backend such as Jaeger, Grafana Tempo, Zipkin or Azure Monitor.
- **Sample** to control cost: head sampling keeps a fixed share at the start; **tail sampling** in the Collector can keep every error and slow trace plus a small share of normal ones.
- **Add attributes** such as order ID and tenant, so traces are searchable, and put the **trace ID in every log line** to jump from a log to its trace (lesson 65).

WHERE TRACES BREAK

- A thread pool, async callback or background job that loses the current context.
- A custom HTTP client or proxy that doesn't forward `traceparent`.
- An uninstrumented service, leaving a gap and orphaned child spans.
- Inconsistent sampling decisions, so half a trace is kept and half dropped.

WORKED EXAMPLE: FINDING 2.8 SECONDS

An order passes the gateway, orders, pricing, inventory, payment, fraud, Kafka and notifications. The waterfall shows **pricing calling inventory five times in sequence**, about 0.56 s each, 2.8 s in total. The team replaces them with one batch call for all items, and checkout drops from 4.0 s to **1.2 s**. No log search would have shown the five calls as one pattern.

5 Key terms

Trace: all spans of one request, sharing a trace ID

traceparent: W3C header: version-traceid-parentid-flags

Tail sampling: decide what to keep after the trace completes

Span: one timed unit of work with a parent

Context propagation: passing trace context on every call or message

Span link: connects spans across async or batch boundaries

6 Interview questions

Q1. What is a span? BASIC

One timed unit of work (HTTP call, DB query, message handler) with a start, duration, attributes and a parent; spans together form the trace tree.

Q2. What does the traceparent header contain? BASIC

A version, the 16-byte trace ID, the 8-byte parent span ID and trace flags such as sampled, e.g. 00-<trace-id>-<span-id>-01.

Q3. How do you trace a request that crosses a Kafka topic? SCENARIO

The producer injects traceparent into the message headers; the consumer extracts it and starts a span linked to the producer's span, so the trace continues across the async hop.

Q4. Checkout is slow and each team says it's not them. What do you do? SCENARIO

Open traces for slow checkouts, read the waterfall to find the longest sequential spans, and fix that path, e.g. batching repeated calls.

Q5. Traces stop at one service even though it's instrumented. Likely causes? TRICKY

Context lost in a thread pool or async callback, a client that doesn't forward headers, or a proxy stripping them.

Q6. How do you keep tracing affordable but still catch problems? TRICKY

Tail sampling: keep all error and slow traces and a small percentage of normal ones, decided after the trace completes.

Next: SD-L-019 · Microservice Anti-Patterns backendroadmap.com

Microservice Anti-Patterns

all the costs, none of the benefits

Watch the 4-min video

In one line: most microservice pain comes from a few anti-patterns (the distributed monolith, shared databases, chatty calls, long synchronous chains, nano-services and unowned services) and the fix is the same: boundaries around business capabilities, private data, events over chains, and a team for every service.

1 Thirty services, five teams per release

A team splits its monolith into thirty services. A year later every release needs five teams to deploy together, one slow service brings down checkout, and nobody knows who owns half the code. They have the costs of microservices (lesson 19) and none of the benefits.



Figure 1. Six anti-patterns. The highlighted two are the most common causes of outages and release pain.

2 The anti-patterns

Distributed monolith. Separate processes, but tightly coupled: they must deploy together, a change in one forces changes in others, and they share libraries containing business logic. You get network latency and partial failure plus monolith coupling.

Shared database. Several services read and write the same tables, so no one can change a schema safely and one service's heavy query slows the rest (lesson 23). If two services must always change their schema together, they are really one service.

Chatty communication. One page needs dozens of calls between services, usually a sign that boundaries cut through one capability.

Long synchronous chains. Ten hops deep, latencies add and availabilities multiply: $0.999^{10} \approx 99.0\%$. One slow hop makes every caller wait until timeouts cascade (lessons 4 and 6).

Nano-services and missing ownership. Services of one function each carry more deployment and monitoring overhead than logic. Services with no team, no on-call and no approver rot. Test: if you can't explain a service in one sentence of business language, its boundary is probably wrong.

3 Symptoms and fixes

Symptom	Likely anti-pattern	Fix
releases need several services in lock-step	distributed monolith	redraw boundaries by bounded context (lesson 22); version APIs
schema change breaks another team	shared database	database per service; share via API or events
p99 latency explodes when one service slows	long sync chain	events for side effects, local caches, timeouts, circuit breakers
a feature touches 4+ services	wrong or too-fine boundaries	merge services that always change together
alerts nobody answers	no ownership	an owning team and on-call for every service

Sometimes the right fix is to merge back into a **modular monolith** (lesson 20). Measure progress with deployment frequency per service and the share of changes that touch more than one service.

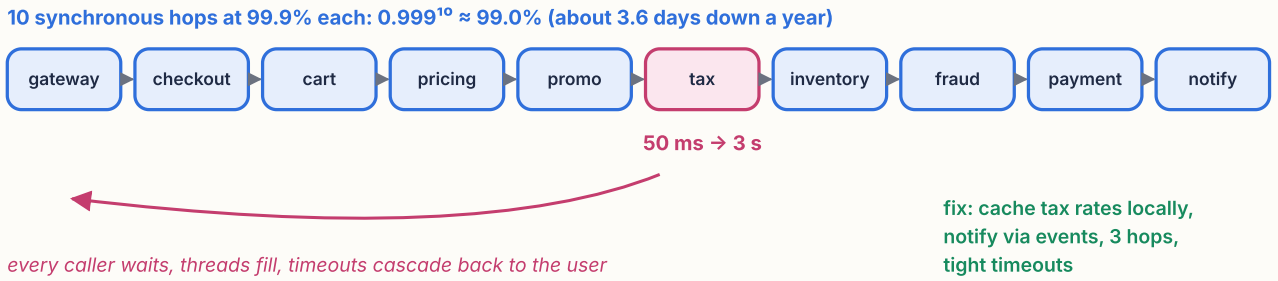


Figure 2. The worked example: one slow hop deep in a synchronous chain takes down checkout.

WORKED EXAMPLE: THE TAX SERVICE SLOWS DOWN

Checkout runs a ten-hop synchronous chain. One day the tax service goes from 50 ms to 3 s. Every upstream service waits, worker threads fill up, and timeouts cascade until checkout fails, although nine of ten services are healthy. The fix: cache tax rates locally in pricing (they change rarely), move notifications to events, and cut the critical path to three hops, each with a tight timeout and a circuit breaker.

4 Key terms

Distributed monolith: separate services that are still tightly coupled

Nano-service: service too small to justify its overhead

Lock-step deploy: services that must be released together

Chatty interface: many small calls where one would do

Cascading failure: one slow dependency exhausts its callers

Service ownership: a named team accountable for a service

5 Interview questions

Q1. What is a distributed monolith? BASIC

Services that run separately but are tightly coupled: deployed together, changed together, sharing a database or business libraries, so you pay the network costs without independence.

Q2. Why is a shared database an anti-pattern? BASIC

It is a hidden contract: schema changes break other services, deployments couple, and one service's load slows others.

Q3. What signs show you have a distributed monolith? SCENARIO

Services that must deploy together, changes that ripple across services, shared databases or business-logic libraries, chatty calls, and long synchronous chains.

Q4. What's the fix? SCENARIO

Draw boundaries around business capabilities, give each service its own data, prefer events over long synchronous chains, merge services that always change together, and give every service an owning team.

Q5. A 10-hop chain at 99.9% per hop. What's the availability, and why is latency worse than the average suggests? TRICKY

About $0.999^{10} \approx 99.0\%$; and the slowest hop's tail latency hits most requests, since every request touches every hop.

Q6. Is merging services ever the right answer? TRICKY

Yes: if two services always change and deploy together or are too small to justify their overhead, one service (or a modular monolith) is better.

Next: SD-L-020 · Configuration Management backendroadmap.com

Configuration Management for Services

same build, different config

Watch the 4-min video

In one line: keep config out of the code so one build runs everywhere; store it centrally, versioned and per environment; keep secrets in a secret store referenced from config; and change values (and feature flags) at run time, gradually and reversibly, without redeploying.

1 A two-second timeout, forty instances

A timeout must change from 2 s to 5 s. It is hard-coded, so you rebuild, test and redeploy forty instances. Meanwhile staging and production drift apart because someone edited a file on a server. Good configuration management makes such changes safe, fast and visible.

2 Config out of the code

The **twelve-factor app** rule: store config in the environment, separate from code. Config is everything that varies between deployments: URLs, timeouts, pool sizes, feature switches, credentials. The **same build artefact** runs in dev, staging and production; only the config differs. That keeps environments comparable and makes a tested build the one you ship.

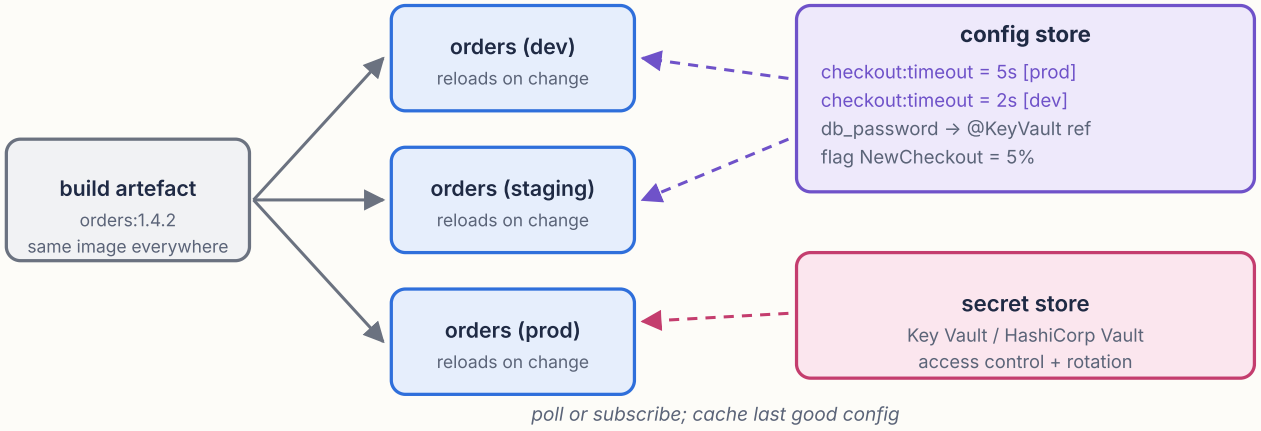


Figure 1. One image, three environments. Settings come from a labelled store; secrets are only referenced, never copied into config.

3 Where config lives

Option	Good for	Limits
Environment variables	few settings, set by the platform	change needs a restart; no audit trail
Config files in Git, applied by pipeline	reviewed, versioned changes (config as code)	change needs a pipeline run
Central store: Azure App Configuration, AWS AppConfig, Spring Cloud Config, Consul	many services; labels per environment; history and audit; dynamic reload	another dependency: cache values in case it's down
Secret store: Azure Key Vault, HashiCorp Vault, AWS Secrets Manager	passwords, API keys, connection strings	strict access policies; rotate regularly

Hierarchical keys such as `checkout:timeout` keep related settings together. Config may hold a **reference** to a secret (Azure App Configuration supports Key Vault references) but never the secret itself, and secrets never go into Git, even in a private repository (lesson 48).

4 Dynamic config and feature flags

To change values without redeploying, services **poll** the store or **subscribe** to change notifications and reload in memory. Validate new values (a timeout of 0 is a bug), roll changes out gradually like code, and cache the **last known good** config so a service can start even when the store is down. **Feature flags** are dynamic config for behaviour (lesson 76): turn a feature on for 5% of users, then everyone, or off in seconds.

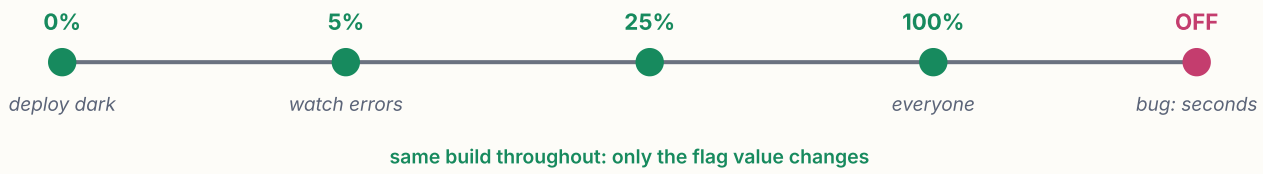


Figure 2. A flag rollout: exposure grows step by step, and the kill switch needs no deploy.

CONFIG DRIFT AND BAD PUSHES

- **Drift:** environments or instances end up different through manual edits. Keep config as code, apply it by pipeline, alert on differences.
- **Bad pushes:** a wrong value can cause an outage like bad code. Validate, stage the rollout, and keep one-click rollback to the previous version.
- **Undocumented settings:** record each key's meaning, default and owner.
- **Stale flags:** remove flags once a feature is fully released.

WORKED EXAMPLE: APP CONFIGURATION, KEY VAULT AND A FLAG

Services read settings from Azure App Configuration with a label per environment (`checkout:timeout = 5s` under `prod`) and the database password through a Key Vault reference. A new checkout flow ships behind the flag `NewCheckout`. The team enables it for 5% of users, watches error rates and conversion, then raises it to 100%. When a bug appears, they switch the flag off in seconds, with no redeploy, then fix it calmly.

5 Key terms

Twelve-factor config: config in the environment, not in code

Config store: central, versioned, labelled settings service

Dynamic config: values reloaded at run time without redeploy

Environment parity: one artefact; only config differs

Secret reference: pointer from config to a secret store entry

Config drift: environments silently diverging

6 Interview questions

Q1. What does "keep config out of the code" mean? BASIC

Everything that varies by environment (URLs, timeouts, flags, credentials) is supplied at deploy or run time, so the same build artefact runs everywhere.

Q2. Where should secrets go? BASIC

In a dedicated secret store with access control and rotation, referenced from config; never in code, config files or Git.

Q3. How do you change config without redeploying? SCENARIO

Keep it in a central store, have services poll or subscribe for changes and reload in memory, roll changes out gradually, and use feature flags for behaviour changes.

Q4. A new checkout flow is risky. How do you release it? SCENARIO

Ship it dark behind a feature flag, enable it for a small percentage while watching metrics, raise it step by step, and turn it off instantly if it fails.

Q5. The config store is down when a service starts. What happens? TRICKY

It should start from a cached last-known-good config or safe defaults rather than fail, and alert; otherwise the store becomes a single point of failure.

Q6. Staging passed but production broke on the same build. A likely cause? TRICKY

Config drift: production settings differ from staging; keep config as code applied by pipeline and compare environments automatically.

Next: SD-M-001 · Authentication vs Authorization backendroadmap.com

Authentication vs Authorization

who are you? vs what may you do?

Watch the 4-min video

In one line: authentication proves *who* the caller is, once, at the edge; authorization decides *what* that identity may do to *this specific resource*, on every request, in the service that owns the data.

1 Login worked. The bug is elsewhere

A user logs in, then calls `GET /orders/1042`. Order 1042 belongs to someone else, and the API returns it anyway. The password check was perfect; nobody checked **permission**. This is **Broken Object Level Authorization (BOLA)**, **API1** in the OWASP API Security Top 10 (2019 and 2023 editions), and the most common way real APIs leak data.

2 Two different questions

	Authentication (authN)	Authorization (authZ)
Question	Who are you?	May you do *this action* on *this resource*?
Input	password, passkey, one-time code, SSO assertion	verified identity + claims, the resource, the action
Output	an identity: session or token for user 42	allow or deny
How often	once per login; token verified per request	every request, every resource
Where	identity provider, gateway	the service that owns the data
HTTP on failure	401 Unauthorized (really “unauthenticated”)	403 Forbidden, or 404 to hide existence

Authentication produces an identity. Authorization *uses* that identity to make a decision, and it must make it again for every request, because each request touches a different resource.

3 Where each one lives

Authentication at the edge. A login service or identity provider (Entra ID, Okta, Auth0, Keycloak) checks the credentials and issues a session (lesson 40) or a signed token (lesson 41). The API gateway verifies that token on every request and rejects anonymous traffic early. The identity then travels inward as a user ID and claims; internal services never see passwords. The gateway must **strip** any identity headers that arrive from outside, so a client cannot simply send `X-User-Id: 1`.

Authorization close to the data. Only the order service knows who owns order 1042, so only it can decide. The gateway can do coarse checks that need no data; the fine-grained checks belong in the service.

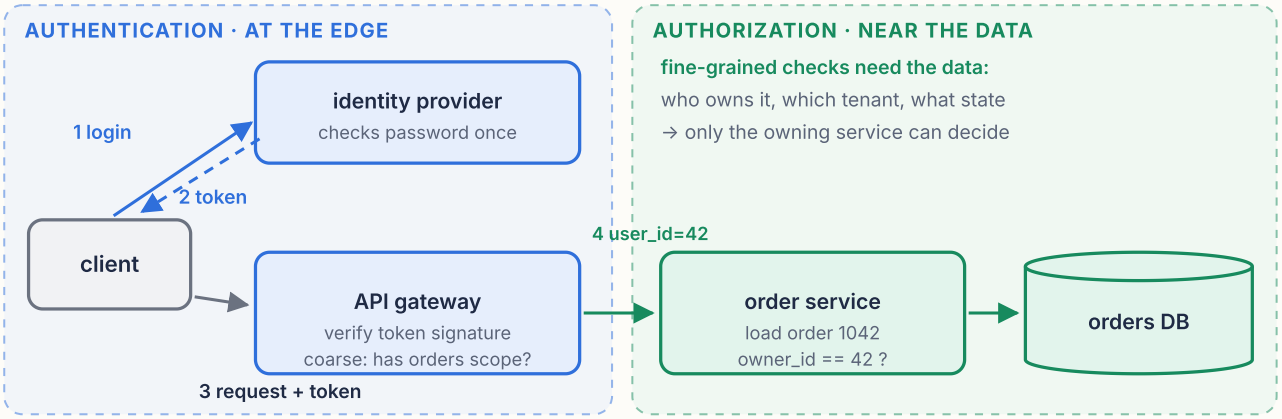


Figure 1. The identity provider checks credentials once; the gateway verifies the token on every call and forwards a trusted user ID; the order service, which can see `owner_id`, makes the final decision.

Gateway can check (coarse)

token valid: signature, issuer, expiry
scope present, e.g. orders:read
role claim, e.g. admin-only routes
rate limits per user or client

Service must check (fine-grained)

does user 42 own order 1042?
same tenant as the record?
is the record in an editable state?
field-level rules (hide cost price)

Figure 2. Split authorization by what each layer can see. Coarse checks at the gateway are an extra filter, never a substitute.

WORKED EXAMPLE: THE OWNERSHIP CHECK

User 42 signs in; the login service verifies the password and issues a token with `sub=42`. The user requests `GET /orders/1042`. The gateway validates the token's signature and expiry and forwards `user_id=42`. The order service runs `SELECT … WHERE id = 1042`, finds `owner_id = 77`, and returns **404 Not Found**. Authentication passed; authorization blocked the request. Better still, scope the query itself: `WHERE id = 1042 AND owner_id = 42`, so a missing check cannot leak a row.

4 Keep the jobs separate

GOOD PRACTICE

- Authenticate once, in one place, with a well-tested identity provider; never build your own password checks in every service.
- Pass a **verified** identity inward (signed token or gateway-set header on a trusted network), never raw passwords.
- **Deny by default**; authorize every request on the server against the specific resource, not just the endpoint.
- Centralise the rules in a policy function or engine (lesson 44) so every endpoint calls the same check.

COMMON MISTAKES

- Treating “has a valid token” as “may access every record”.
- Reading the role or user ID from the request body instead of from the verified token.
- Hiding a button in the UI and leaving the API endpoint open: anyone can call the API directly.
- Relying on unguessable IDs (UUIDs) instead of an ownership check; IDs leak through URLs, logs and shared links.

5 Key terms

Authentication: proving identity: who the caller is

Identity provider (IdP): service that authenticates users and issues tokens

BOLA / IDOR: accessing another user's object by changing its ID

Authorization: deciding what that identity may do

Claims: facts about the user in a token: sub, roles, scopes

Deny by default: anything not explicitly allowed is refused

6 Interview questions

Q1. Explain the difference between authentication and authorization, and where each is enforced. BASIC

Authentication proves who you are, usually once at the edge through an identity provider, with the gateway verifying the token per request. Authorization decides what that identity may do, checked on every request on the server, close to the data.

Q2. What is the difference between HTTP 401 and 403? BASIC

401 means the caller is not authenticated (missing or invalid credentials). 403 means the caller is known but not allowed; some APIs return 404 instead so they don't reveal that the record exists.

Q3. A tester changes /orders/1042 to /orders/1043 and sees another customer's order. How do you fix it? SCENARIO

Add an object-level check in the order service: load the order and compare its owner (and tenant) with the authenticated user, or scope the query by owner. Then add tests that call every endpoint with another user's IDs.

Q4. How does an internal service know which user made the request? SCENARIO

The gateway verifies the token and forwards the user ID and claims, either by passing the token on (each service verifies it) or in a header that the gateway sets after stripping any client-supplied copy. Services must not accept identity from anything the client controls.

Q5. What goes wrong if you mix up authentication and authorization? TRICKY

Logged-in users read or change other people's data: broken object level authorization, API1 in the OWASP API Security Top 10.

Q6. Do random UUIDs for order IDs fix BOLA? TRICKY

No. They make guessing harder, but IDs leak through links, logs and other APIs. Only a server-side check of ownership on every request fixes it.

Next: SD-M-002 · Session-Based Authentication backendroadmap.com

Session-Based Authentication

a random ID in a cookie, the state on the server

Watch the 4-min video

In one line: after login the server stores the session (user ID, expiry) in a shared store and gives the browser only a long random ID in a cookie; protect that cookie with HttpOnly, Secure and SameSite, block CSRF, and enjoy instant revocation.

1 HTTP remembers nothing

You log in, close the tab, and return an hour later still logged in. HTTP is stateless, so something must carry your identity between requests. The classic answer is a **server-side session**: simple, battle-tested, and the default in Django, Rails, Laravel, Spring and ASP.NET. The cookie that carries it is what attackers go after.

2 How a session works

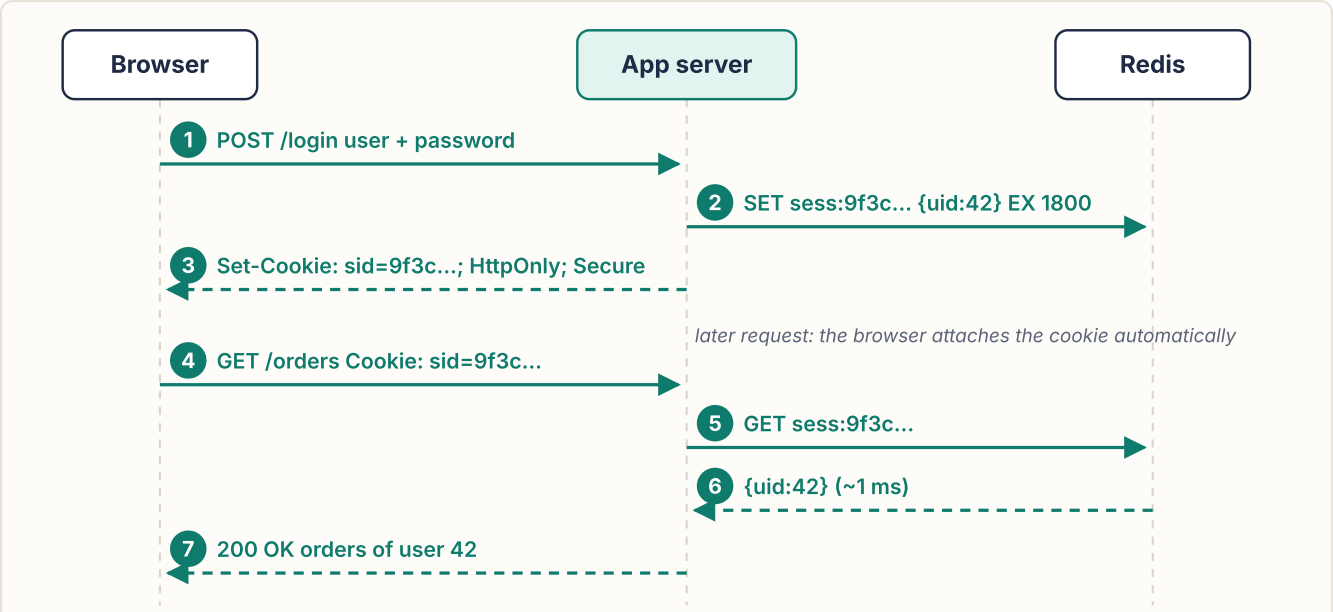


Figure 1. Login creates a session record in Redis and returns only its ID. Every later request carries the cookie, and the server looks the ID up.

The session ID must come from a cryptographically secure random generator and be long enough that guessing is hopeless: OWASP asks for at least **64 bits of entropy**; frameworks typically use 128 bits or more. The ID means nothing by itself; everything about the user stays on the server.

3 Where sessions live

In-process memory works for one server. Behind a load balancer, a user hitting a different server would look logged out, so either use sticky sessions (fragile: a server restart still logs users out) or, better, a **shared store** so every app server stays stateless. Redis is the usual choice: one key per session with a TTL, lookups around a millisecond. If Redis restarts with no persistence, every user is logged out, so run it with replication (and persistence if logouts would hurt).

4 Protect the cookie

```
Set-Cookie: __Host-sid=9f3c…; Path=/; HttpOnly; Secure; SameSite=Lax
```

- **HttpOnly**: JavaScript can't read the cookie, so an XSS bug can't steal the session ID (it can still act inside the page, so fix XSS too).
- **Secure**: sent only over HTTPS. The `__Host-` prefix additionally forces Secure, `Path=/` and no `Domain`, so subdomains can't overwrite it.
- **SameSite**: controls whether the cookie is sent on cross-site requests (table below).
- **Regenerate the ID at login** (and on privilege change) to stop **session fixation**, where an attacker plants a known ID before the victim logs in.

SameSite	Top-level link from another site (GET)	Cross-site POST, iframe, fetch	Use for
Strict	not sent	not sent	banking, admin consoles
Lax	sent	not sent	most session cookies; Chrome's default when unset
None (needs Secure)	sent	sent	embedded widgets, cross-site SSO; needs CSRF tokens

5 CSRF: the cookie rides along

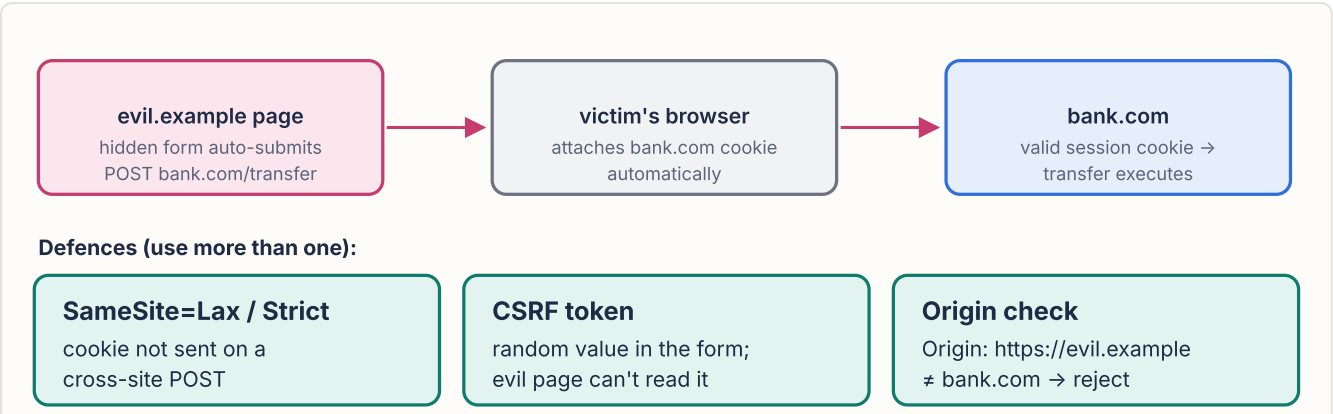


Figure 2. Cross-site request forgery: the attacker never sees the cookie; the browser sends it for them.

Because browsers attach cookies automatically, another site can make the victim's browser send an authenticated, state-changing request. Defend in layers: SameSite=Lax or Strict; a **CSRF token** (synchronizer token stored in the session and echoed in each form or header); and checking the `Origin` (or `Sec-Fetch-Site`) header on POST, PUT, PATCH and DELETE. Never change state on GET.

6 Revocation is the superpower

Logout deletes the record, and the cookie is useless at once. You can also log a user out of every device, expire idle sessions after, say, 30 minutes, and kill all sessions when the password changes. The price: a store lookup on every request, and the store must stay available. JWTs (lesson 41) make the opposite trade.

WORKED EXAMPLE: DJANGO SESSIONS IN REDIS

Point `CACHES` at a Redis cluster, then set `SESSION_ENGINE = "django.contrib.sessions.backends.cache"`, `SESSION_COOKIE_SECURE = True`, `SESSION_COOKIE_HTTPONLY = True`, `SESSION_COOKIE_SAMESITE = "Lax"`, and keep `CsrfViewMiddleware` on. Django's `login()` already rotates the session key. Any app server can serve any user, and logout works everywhere instantly.

COMMON MISTAKES

- Storing sessions in local memory behind a load balancer, so users get random logouts.
- Keeping the same session ID before and after login (session fixation).
- Setting `SameSite=None` for convenience and having no CSRF token.
- Logout that only deletes the cookie in the browser while the server-side session stays valid.

7 Key terms

Session ID: random, unguessable key to server-side state

HttpOnly: cookie hidden from JavaScript

SameSite: limits sending the cookie on cross-site requests

Session fixation: attacker plants a known ID before login

CSRF: another site triggers an authenticated request

Idle / absolute timeout: expiry after inactivity / after a fixed lifetime

8 Interview questions

Q1. How do you protect session cookies? BASIC

Use a long random ID with HttpOnly, Secure and SameSite=Lax or Strict, regenerate it at login, and expire and revoke sessions on the server.

Q2. What is CSRF? BASIC

Cross-site request forgery: another site makes the victim's browser send an authenticated request, because cookies are attached automatically. Block it with SameSite cookies, CSRF tokens and Origin checks on state-changing requests.

Q3. Users get logged out randomly after you scale from one server to four. Why? SCENARIO

Sessions live in each server's memory, so a request routed to another server finds no session. Move them to a shared store like Redis (or, as a stopgap, sticky sessions).

Q4. A user reports a stolen laptop. What do you do with sessions? SCENARIO

Delete all of that user's sessions in the store (track session IDs per user), which logs every device out at once, then force a password change.

Q5. HttpOnly is set. Is XSS now harmless? TRICKY

No. Script can't read the cookie, but it runs in the page, so it can still send requests with the session and read the responses. Fix XSS itself.

Q6. Does SameSite=Lax make CSRF tokens unnecessary? TRICKY

Not completely: Lax still sends the cookie on top-level GETs (so GETs must never change state), sibling subdomains count as same-site, and old browsers ignore it. Keep tokens or Origin checks too.

Next: SD-M-003 · JWT Authentication backendroadmap.com

JWT Authentication

a signed token any server can check

Watch the 5-min video

In one line: a JWT is a signed set of claims that any service can verify locally with the issuer's public key, so there's no session lookup; the price is that a token stays valid until it expires, so keep access tokens short and pair them with revocable refresh tokens.

1 Why stateless tokens?

Your API runs on fifty servers across three regions. A session lookup on every request (lesson 40) adds a network hop and a shared dependency in every region. A **JSON Web Token** (RFC 7519) carries the proof itself: if the signature checks out, the claims are true, with no database call. That is why JWTs are everywhere, and why they have sharp edges.

2 Three parts

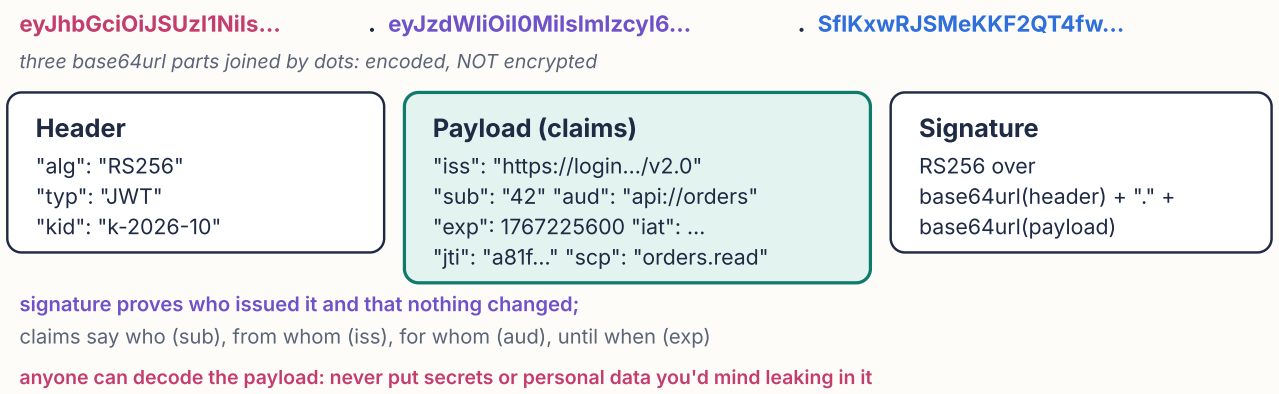


Figure 1. A JWT is `header.payload.signature`. The header names the algorithm and key ID; the payload holds the registered claims.

Registered claims (RFC 7519): `iss`, `sub`, `aud`, `exp`, `nbf` (not-before), `iat`, `jti` (unique token ID). Signed JWTs (JWS) are readable by anyone; JWE encrypts, but is rarely needed.

3 Verify without a lookup

The API (1) checks the signature using the algorithm **it expects**, (2) checks `iss` is the trusted issuer, (3) checks `aud` is this API, (4) checks `exp` / `nbf` with a small clock-skew leeway (about 30–60 s). Only then does it read `sub` and scopes. Verifying one signature takes microseconds, so a gateway can check every request.

4 HS256 vs RS256 / ES256

Algorithm	Key	Who can forge a token
HS256 (HMAC-SHA256)	one shared secret	every service that holds the secret to verify
RS256 (RSA, 2048-bit+)	private / public key pair	only the private-key holder (the IdP)
ES256 (ECDSA P-256)	key pair, smaller keys + signatures	only the private-key holder (the IdP)

Beyond a single service, use asymmetric keys: the identity provider signs, everyone else only verifies.

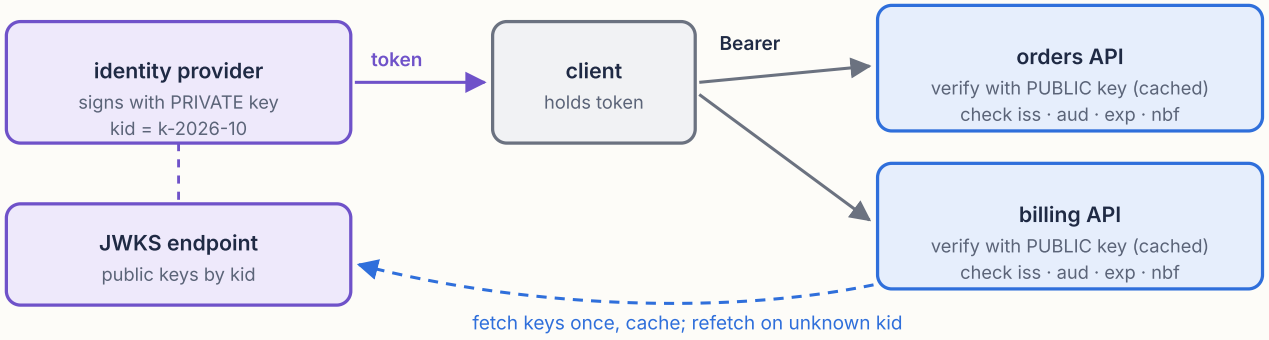


Figure 2. The identity provider signs with its private key and publishes public keys at a JWKS endpoint. APIs verify locally and cache the keys.

5 The revocation problem



Figure 3. A valid JWT stays valid until `exp`, even after logout or a stolen laptop.

So keep **access tokens short-lived** (5–15 minutes) and pair them with a **refresh token** that is stored securely, can be revoked at the identity provider, and is ideally **rotated** on each use (reuse of an old one signals theft). For emergencies, keep a small **deny list of `jti` values** (in Redis, with TTL = remaining lifetime) checked per request. That brings back a lookup, but a tiny one.

6 Key rotation and classic pitfalls

The token header names a `kid`; services cache the JWKS keys and refetch on an unknown one, so rotation needs no redeploy. Publish the new key before signing with it, and keep the old one until its last tokens expire.

COMMON MISTAKES

- Letting the token's header pick the algorithm (`alg: none`, algorithm confusion). Pin it: `algorithms=["RS256"]`.
- Skipping `aud`: a token issued for another API is then accepted by yours.
- Day-long access tokens, or tokens in `localStorage` where any XSS can read them.

WORKED EXAMPLE: FASTAPI VALIDATING ENTRA ID TOKENS

At startup the service reads the tenant's discovery document (`…/<tenant>/v2.0/.well-known/openid-configuration`) and caches the keys from its `jwks_uri` (PyJWT's `PyJWKClient`). Per request it takes the bearer token, picks the key by `kid` and calls `jwt.decode(token, key, algorithms=["RS256"], audience=API_ID, issuer=TENANT_ISSUER)`, which checks signature, `aud`, `iss` and `exp`. No call to Azure per request.

7 Key terms

Claims: statements in the payload: iss, sub, aud, exp, jti

kid: key ID in the header that picks the verification key

Refresh token: long-lived, revocable token used to get new access tokens

JWS: signed token; payload readable by anyone

JWKS: JSON Web Key Set: the issuer's published public keys

Deny list: revoked jti values checked until they expire

8 Interview questions

Q1. JWT vs sessions: what are the trade-offs? BASIC

JWTs are verified locally with no store lookup, so they scale across services and regions, but they're larger and can't be revoked before expiry. Sessions need a shared store lookup per request but revoke instantly. A common hybrid: session cookie for the browser, short-lived JWTs between back-end services.

Q2. How do you revoke a JWT? BASIC

Keep access tokens short-lived (5–15 min), revoke the refresh token, and for emergencies keep a deny list of token IDs (jti) checked on each request.

Q3. Ten microservices must verify user tokens. HS256 or RS256? SCENARIO

RS256 or ES256: the identity provider keeps the private key, services get public keys from JWKS. With HS256 every service holds a secret that can forge tokens.

Q4. How do you rotate signing keys without downtime? SCENARIO

Publish the new public key in the JWKS first, then start signing with it (new kid). Verifiers refetch on unknown kid. Remove the old key after the longest token lifetime has passed.

Q5. Can you put the user's role in the JWT and trust it? TRICKY

Yes, if the identity provider signed it and you verify the signature; but the role is frozen until expiry, so a demotion takes effect only when the token expires. Never trust a role the client sets.

Q6. What is the alg=none or algorithm-confusion attack? TRICKY

If a verifier lets the token header choose the algorithm, an attacker sends an unsigned token (alg none) or an HS256 token keyed with the public RSA key. Pin the expected algorithm and key type.

Next: SD-M-004 · OAuth 2.0 backendroadmap.com

OAuth 2.0

limited, revocable access without your password

Watch the 4-min video

In one line: OAuth 2.0 is a **delegation** protocol: the user approves limited scopes at the authorization server, and the app receives an access token, never the user's password; use the authorization code flow with PKCE for users and client credentials for services.

1 Sharing passwords

A photo-printing app wants your Google Drive photos. The old way: give it your Google password. Now it can read your email and change your password, and you can't take access back without changing the password everywhere. OAuth 2.0 (RFC 6749) fixes this: the app gets a token that is limited in **scope** and **time** and can be **revoked**.

2 Four roles

The **resource owner** is you. The **client** is the print app. The **authorization server** (Google's login and consent service) authenticates you, shows consent and issues tokens. The **resource server** (the Drive API) accepts tokens and enforces their scopes. The key idea: the client gets an access token, never your credentials.

3 The authorization code flow

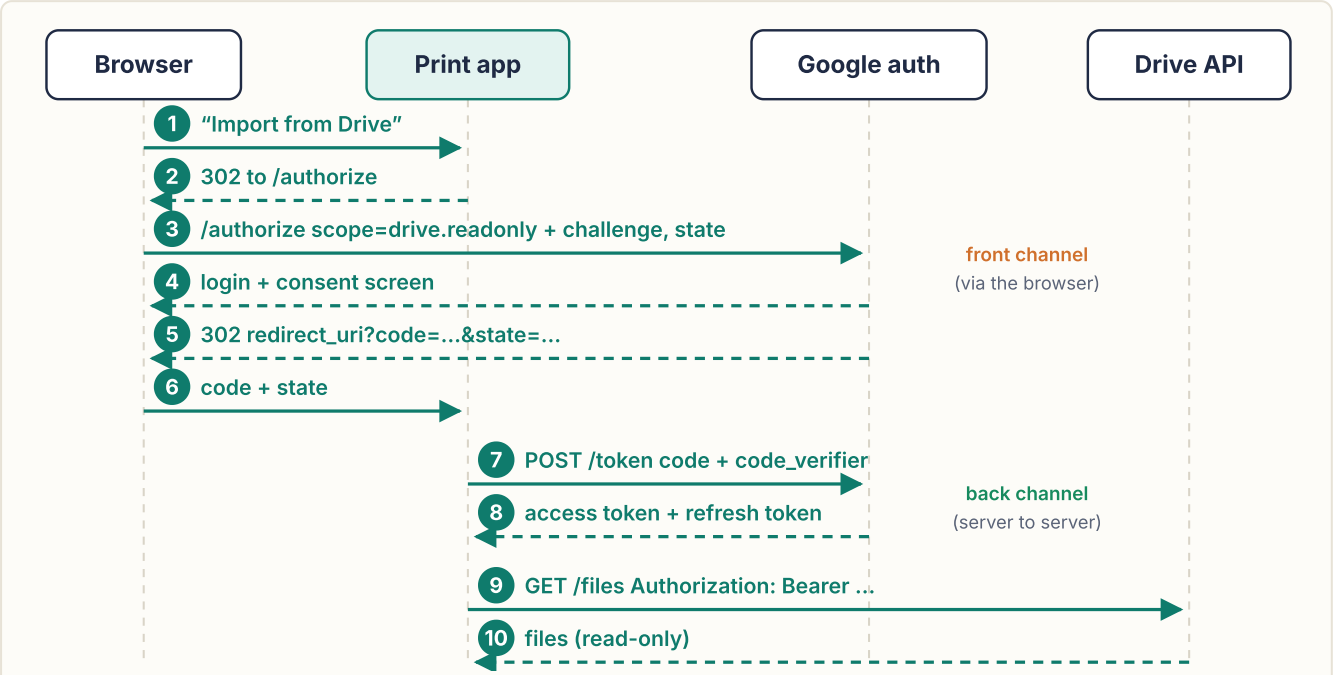


Figure 1. Authorization code flow with PKCE. Only a short-lived, single-use code passes through the browser; tokens travel on the back channel.

The app redirects the browser to the authorization endpoint with `response_type=code`, its `client_id`, a pre-registered `redirect_uri`, the `scope`, a random `state` (checked on return, against CSRF) and the PKCE challenge. After login and consent the server redirects back with a **code**: single-use, short-lived

(RFC 6749 recommends at most 10 minutes). The app's back end exchanges it at the token endpoint. The redirect URI must match exactly, or codes could be sent to an attacker's site.

4 PKCE

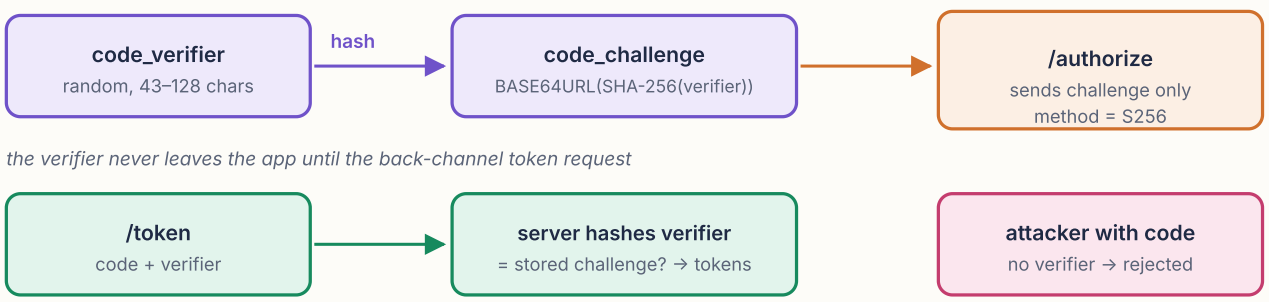


Figure 2. PKCE (RFC 7636, “pixy”): a stolen authorization code is useless without the verifier that only the real app holds.

On mobile, another app can register the same custom URL scheme and intercept the code; PKCE binds the code to the app that started the flow. Always use `S256`, not `plain`. The OAuth 2.0 Security Best Current Practice (RFC 9700, January 2025) recommends PKCE for **every** client, including confidential web apps with a client secret.

5 Scopes, refresh and revocation

Scopes limit what a token can do: `drive.readonly` instead of full Drive access, shown on the consent screen. Access tokens are short-lived (Google's last an hour); the app then sends its **refresh token** (`grant_type=refresh_token`) for a new one silently. The user can revoke the app at any time, which kills the refresh token.

6 Which flow?

Client	Flow	Notes
Web app with a back end	authorization code + PKCE	confidential client: also authenticates with a secret or certificate
SPA, mobile, desktop app	authorization code + PKCE	public client: can't keep a secret, so PKCE is essential
Service to service, cron job	client credentials	no user; the service gets a token for itself
TV, CLI, IoT (no browser)	device authorization (RFC 8628)	user approves on a phone with a short code
Implicit · password grant	don't use	implicit put tokens in the URL; password grant hands the app the password

WORKED EXAMPLE: IMPORTING FROM GOOGLE DRIVE

The print app stores a fresh verifier in the user's session and redirects to Google's authorization endpoint with `scope=…/auth/drive.readonly`, `code_challenge`, `code_challenge_method=S256` and `state`. The user approves; Google redirects back with a code. The app checks `state`, POSTs code + verifier to the token endpoint, and calls the Drive API with `Authorization: Bearer …`. The user can revoke it in their Google Account at any time.

COMMON MISTAKES

- Using an OAuth access token as proof of login: that's what OpenID Connect is for (lesson 43).
- Loose redirect URI matching (wildcards, prefixes), which lets attackers receive codes.
- Skipping `state` or PKCE; using the deprecated implicit flow in a new SPA.
- Requesting broad scopes “just in case”, or shipping a client secret inside a mobile app.

7 Key terms

Authorization code: single-use, short-lived code exchanged for tokens

Scope: named permission the token carries

Public vs confidential client: can't vs can keep a client secret

PKCE: verifier + hashed challenge that bind the code to the app

Refresh token: used at the token endpoint to get new access tokens

Client credentials: grant for a service acting as itself

8 Interview questions

Q1. What are the four OAuth 2.0 roles? BASIC

Resource owner (the user), client (the app), authorization server (issues tokens after login and consent) and resource server (the API that accepts tokens).

Q2. Which flow is used for service-to-service calls? BASIC

Client credentials: the service authenticates as itself with its client ID and a secret or certificate, and gets a token with no user involved.

Q3. Walk through the authorization code flow with PKCE. SCENARIO

The client creates a random verifier and sends its SHA-256 hash (the challenge) when redirecting the user to the authorization server. The user logs in and consents; the server redirects back with a short-lived code. The client's back end exchanges code + verifier at the token endpoint for tokens.

Q4. You're building a React SPA that calls your API. Which flow? SCENARIO

Authorization code with PKCE as a public client (no secret), or better a backend-for-frontend that does the flow server side and keeps tokens out of the browser.

Q5. Why was the implicit flow deprecated? TRICKY

It returned the access token in the URL fragment, where it leaks via history, referrers and logs and can't be bound to the client. Code + PKCE gives the same browser support without those leaks.

Q6. What does the state parameter protect against, and does PKCE replace it? TRICKY

state stops CSRF on the redirect (an attacker injecting their own code into your session). PKCE also prevents that when the server enforces it, but state is still used to carry and check request context.

Next: SD-M-005 · OpenID Connect and SSO backendroadmap.com

OpenID Connect and SSO

OAuth was never built for login

Watch the 5-min video

In one line: OpenID Connect adds a signed **ID token** to the OAuth 2.0 code flow, saying who logged in, when, and for which app; with discovery and one trusted identity provider, many apps get single sign-on.

1 OAuth isn't login

A developer logs users in with plain OAuth: the app gets an access token and assumes whoever holds it is the user. But an access token says what the bearer may *access*, not *who* they are, and it may have been issued for a different app (a malicious app can replay a token it got for itself). OAuth 2.0 (lesson 42) was built for delegation. **OpenID Connect** (OIDC Core 1.0, 2014) was built for login.

2 An identity layer on OAuth 2.0

The client adds the `openid` scope to a normal authorization code flow (with PKCE). The token response now includes an `id_token`: a signed JWT (lesson 41) issued **for this client**. The access token is still there, but only for calling APIs.

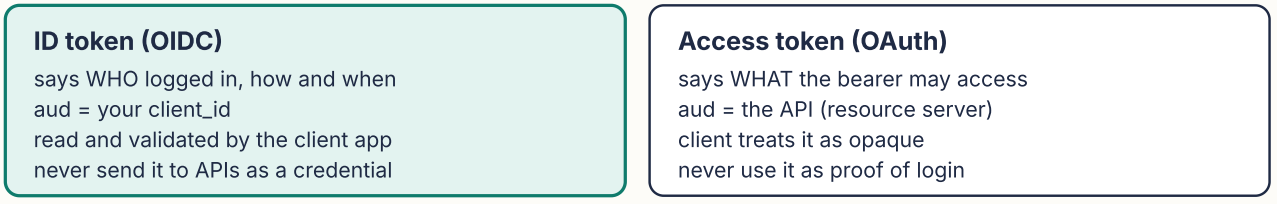


Figure 1. Two tokens, two audiences. Mixing them up is the classic OIDC mistake.

Validate the ID token: signature (keys from the provider's JWKS), `iss` = the expected issuer, `aud` contains your `client_id`, `exp` not passed, and `nonce` equals the value you sent at the start (stops replay). Identify the user by `iss` + `sub`, not by email: emails change and can be reassigned. Extra profile claims (`name`, `email`, `email_verified`) come in the token or from the **userinfo** endpoint.

3 Discovery

Every provider publishes `<issuer>/.well-known/openid-configuration`, a JSON document listing `authorization_endpoint`, `token_endpoint`, `userinfo_endpoint`, `jwks_uri` and supported scopes. A library needs only the issuer URL to configure itself, so moving from Google to Okta or Entra ID is mostly configuration.

4 Single sign-on

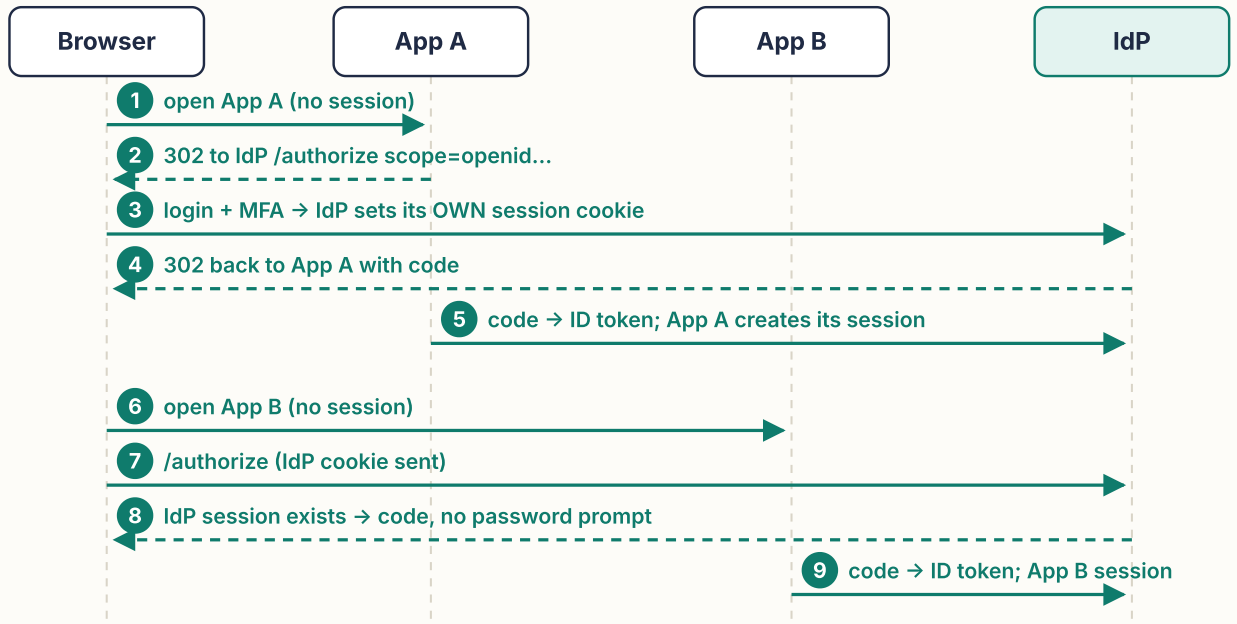


Figure 2. SSO: the identity provider keeps its own session. The second app redirects there, and the user comes straight back with a fresh ID token.

Many apps trust one identity provider. One password, one place for MFA and conditional access, and one place to disable a leaving employee. Each app still keeps its **own** session after login. Single *logout* is harder: OIDC has RP-initiated, front-channel and back-channel logout specs, but each app must implement them.

	SAML 2.0	OpenID Connect
Format	XML assertions, XML signatures	JSON, JWT (ID token)
Built on	browser redirects and POSTs	OAuth 2.0 authorization code flow
Good for	existing enterprise web apps	SPAs, mobile apps, APIs, new systems
Also gives API access?	no (separate mechanism)	yes, the access token from the same flow

WORKED EXAMPLE: ENTERPRISE SIGN-IN WITH ENTRA ID

The app is registered in Entra ID (formerly Azure AD) with its redirect URI. Login uses the code flow with PKCE and `scope=openid profile email`. The app validates the ID token: signature, `iss = https://login.microsoftonline.com/<tenant-id>/v2.0`, `aud` = its client ID, `nonce`, expiry. It keys the user on `tid` + `oid` (stable across the tenant's apps) and creates its own session cookie. Employees sign in once and reach every company app.

COMMON MISTAKES

- Using the access token, or a userinfo call alone, as proof of login.
- Accepting an ID token whose `aud` is another app's client ID.
- Skipping the `nonce` check, or matching users by email instead of `iss` + `sub`.
- Sending the ID token to APIs as a bearer token.

5 Key terms

OIDC: identity layer on OAuth 2.0 for login

nonce: random value tying the ID token to this login request

userinfo endpoint: returns profile claims for an access token

ID token: signed JWT about the login, aud = the client

Discovery document: /.well-known/openid-configuration

SSO: one IdP session signs the user into many apps

6 Interview questions

Q1. What does OIDC add on top of OAuth 2.0? BASIC

A standard, signed ID token that says who logged in and for which app (iss, sub, aud, nonce), plus the userinfo endpoint and discovery documents.

Q2. What is single sign-on? BASIC

Many apps trust one identity provider; the user logs in once there and is signed into every app without re-entering credentials.

Q3. Your company has 30 internal apps with separate logins. How do you consolidate? SCENARIO

Register each app with one IdP (Entra ID, Okta) using OIDC, or SAML for legacy apps; enforce MFA there, and disable accounts in one place.

Q4. Which ID token checks must your app perform? SCENARIO

Signature against the JWKS, issuer, audience equal to your client ID, expiry, and the nonce you sent.

Q5. Why is an OAuth access token not proof of identity for your app? TRICKY

It was issued for an API, may be opaque, and could have been obtained by another app and replayed to you; nothing binds it to your client.

Q6. Why key users on iss + sub instead of email? TRICKY

sub is the provider's stable, unique ID; emails change, can be reused, and may be unverified at some providers, which enables account takeover.

Next: SD-M-006 · RBAC and ABAC backendroadmap.com

RBAC and ABAC

who can do what, on which thing, when?

Watch the 4-min video

In one line: RBAC grants permissions through roles and suits coarse, stable access; ABAC writes rules over attributes of user, resource and context; ReBAC (Zanzibar) derives access from relationships like “Bob views the folder this doc is in”.

1 Who can do what, on which thing?

A user can edit a document. Can they share it? Can their manager see it? Can someone in another region open it? Every product ends up asking these questions. Lesson 39 said authorization happens on every request; this lesson is about **how you model the rules**, which decides whether they stay simple or turn into special cases that leak data or lock people out.

2 RBAC and where it breaks

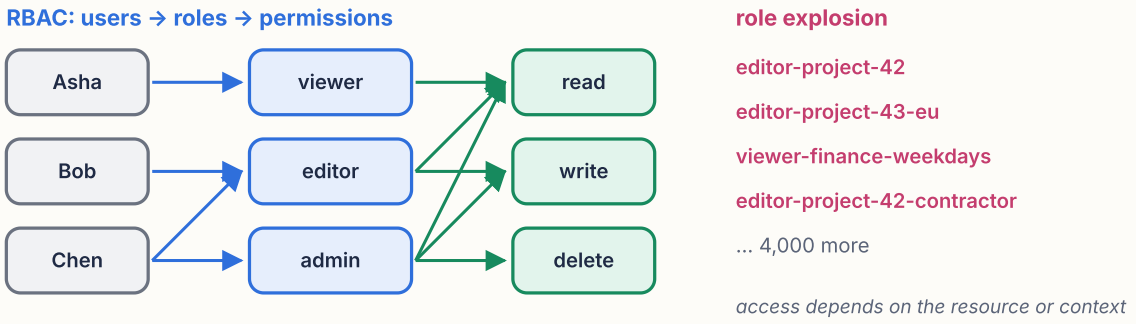


Figure 1. RBAC is a two-hop lookup. When access depends on *which* project, region or time, teams mint ever more specific roles.

Role-based access control: users get roles, roles carry permissions, and a check asks “does any of this user's roles grant `orders:write`?” It's simple, fast and easy to audit, ideal for internal tools and admin panels (Kubernetes RBAC, AWS IAM groups). It breaks when access depends on the **specific resource** (editor of *which* project?) or on **context** (business hours, region, device): that's **role explosion**.

3 ABAC: rules over attributes

Attribute-based access control (NIST SP 800-162) evaluates rules over attributes of the **subject** (department, clearance), the **resource** (owner, department, classification), the **action**, and the **environment** (time, IP, device posture). Example in Cedar:

```
permit(principal, action == Action::"edit", resource) when { principal.dept == resource.dept && context.managed_device };
```

Flexible, with no role explosion; but rules are harder to read, test and explain (“why was I denied?”).

4 Policy engines

Don't scatter `if` statements through the code. A **policy decision point** evaluates policy-as-code; services, the **enforcement points**, just ask “is this allowed?” **Open Policy Agent** (CNCF, Rego language)

is common for Kubernetes admission and API checks; **Cedar** (AWS, used by Amazon Verified Permissions) is designed to be analysable. Policies are versioned, reviewed and unit-tested. For speed, run the engine as a sidecar or library next to the service and cache decisions briefly.

5 ReBAC and Zanzibar

Sharing products (Google Drive, GitHub) need **relationship-based access control**. Google's Zanzibar paper (USENIX ATC 2019) stores relationship tuples `object#relation@subject` and answers checks by walking the graph. It reports trillions of tuples, millions of checks per second and p95 latency under 10 ms. Consistency tokens ("zookies") make sure a check never uses data older than a recent permission change.

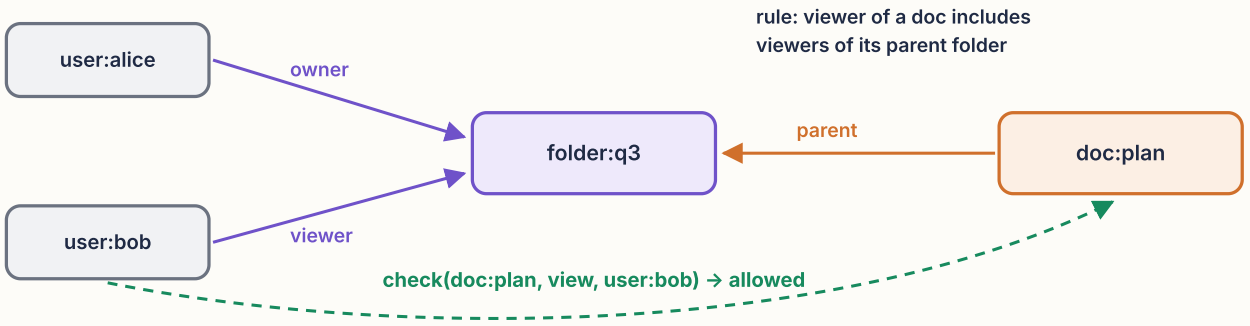


Figure 2. Bob was never granted the document; he can view it because he can view its parent folder.

	RBAC	ABAC	ReBAC
Access from	roles	attribute rules	relationship graph
Best for	admin panels, internal tools	context rules: region, time, device	sharing, folders, orgs, teams
Weakness	role explosion	hard to read and audit	needs a tuple store kept in sync
Tools	IAM, K8s RBAC	OPA, Cedar, XACML	SpiceDB, OpenFGA, Zanzibar

WORKED EXAMPLE: DOCUMENT SHARING WITH TUPLES

`folder:q3#owner@user:alice` · `folder:q3#viewer@user:bob` · `doc:plan#parent@folder:q3`. The schema says a doc's viewers include its parent's viewers. Bob opens the new doc: the check walks doc → parent folder → Bob is a viewer → **allowed**. Nobody wrote a role or rule for that document, and removing Bob's folder tuple revokes everything inside at once. SpiceDB and OpenFGA implement this model.

COMMON MISTAKES

- One role per customer or project until nobody knows what a role means.
- Checking only the role ("is editor") and not the resource ("editor of this project").
- Authorization logic copied into every service, slowly diverging.
- Untested policies: one bad rule silently allows or denies everything.

6 Key terms

RBAC: permissions granted through roles

ReBAC: access derived from relationships in a graph

PDP / PEP: policy decision point / enforcement point

ABAC: rules over subject, resource, action and environment attributes

Role explosion: too many narrow roles to manage

Relationship tuple: object#relation@subject, e.g. doc:plan#viewer@user:bob

7 Interview questions

Q1. When does RBAC break down? BASIC

When access depends on the specific resource, a relationship or context (time, region, device); teams create ever more specific roles until nobody understands them: role explosion.

Q2. What attributes does an ABAC rule use? BASIC

Subject (department, clearance), resource (owner, classification), action, and environment (time, IP, device posture).

Q3. Design permissions for a Google-Drive-like product. SCENARIO

ReBAC: store tuples for ownership, sharing, group membership and folder parents; checks walk the graph, so sharing a folder shares its contents. Use SpiceDB or OpenFGA rather than building it.

Q4. Ten services each implement their own permission checks. What do you change? SCENARIO

Move policies into a policy engine (OPA or Cedar) as versioned, tested code; services call it locally (sidecar or library) and enforce its decision.

Q5. How does Google Zanzibar model permissions? TRICKY

As relationship tuples (object#relation@subject) between users, groups and objects; a check follows the graph, with consistent, fast checks at huge scale.

Q6. What is the “new enemy” problem in Zanzibar? TRICKY

A check evaluated on stale data could let a just-removed user see content added after the removal. Zanzibar's zookies make checks at least as fresh as the content's version.

Next: SD-M-007 · Service-to-Service Auth backendroadmap.com

Service-to-Service Authentication

identity, not network location

Watch the 4-min video

In one line: give every workload a platform-issued identity, prove it on every call with mutual TLS or short-lived tokens, and authorize by identity, so there are no static secrets to leak and no trust in the network itself.

1 The shared API key problem

The orders service calls payments. How does payments know the caller really is orders, and not an attacker already inside the network? Many teams use a shared API key in a config file. It never expires, gets copied into repos, CI logs and laptops, and eventually leaks. Rotating it means coordinating every caller, so nobody does.

2 Zero trust

The old perimeter model trusted anything inside the firewall. **Zero trust** (NIST SP 800-207, Google BeyondCorp) drops that: every request proves who is calling, whether it comes from the internet or the pod next door, and is authorized by **identity**, not IP address. It limits lateral movement after one machine is breached (lesson 53).

3 Mutual TLS

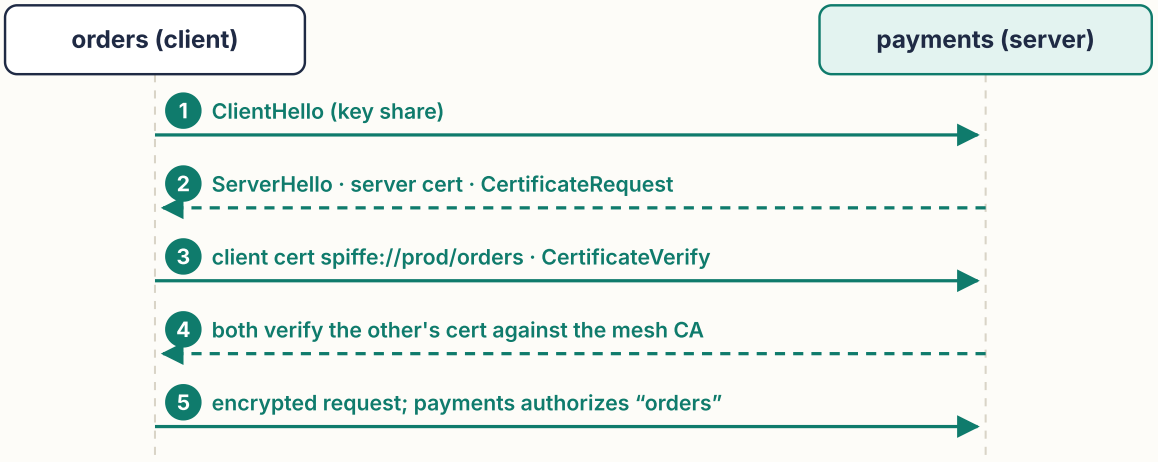


Figure 1. In mutual TLS the server also requests a certificate, so **both** sides are authenticated and the channel is encrypted.

In normal TLS only the server shows a certificate. In **mTLS** the client presents one too and proves it holds the private key. A service mesh (Istio, Linkerd) does this in sidecar or node proxies for every call, with no code change, and rotates the certificates itself (Istio's default workload certificates last 24 hours). Set Istio's `PeerAuthentication` to `STRICT` so plain-text calls are refused.

4 Workload identity

Certificates need names, and names must be issued safely. **SPIFFE** defines a workload ID such as `spiffe://prod/payments` and a credential called an SVID (an X.509 certificate or a JWT). **SPIRE** attests the workload (which node, which Kubernetes service account and namespace) before issuing short-lived SVIDs. Clouds offer the same idea as **managed identities** (Azure), IAM roles for service accounts (AWS) and Workload Identity (GCP): the platform vouches for the workload.

5 Short-lived tokens, no static secrets

Instead of a static key, the workload exchanges its platform identity for a token that expires in minutes to an hour. A leaked token is soon useless, and rotation is automatic because every token is new. The receiver validates signature, `aud` and `exp` on every call (lesson 41). Policies then say which identities may call which services: *orders may call payments; nobody else may*.

Approach	Proof of identity	Rotation	If it leaks
Shared API key	static string	manual, rare	works until someone notices
mTLS via mesh	X.509 cert from mesh CA	automatic, hours	key stays in the proxy; cert expires soon
Workload identity token	signed JWT, aud-bound	every token is new	valid minutes to an hour, one audience

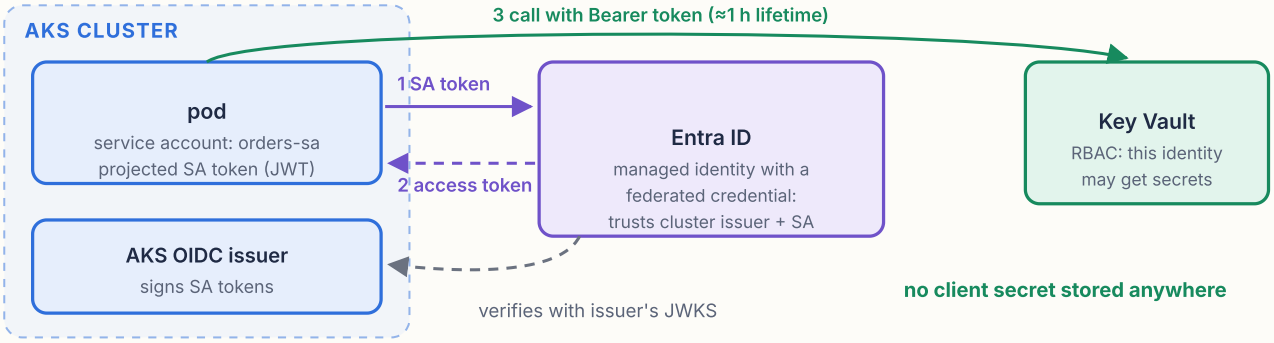


Figure 2. Azure Workload Identity: the cluster's OIDC issuer vouches for the pod; Entra ID swaps that for a short-lived token for Key Vault.

WORKED EXAMPLE: AKS POD READING KEY VAULT

The pod's Kubernetes service account is federated with a user-assigned managed identity in Microsoft Entra ID (issuer = the cluster's OIDC issuer URL, subject = `system:serviceaccount:orders:orders-sa`). The webhook mounts a projected service account token and sets `AZURE_CLIENT_ID`, `AZURE_TENANT_ID` and `AZURE_FEDERATED_TOKEN_FILE`. The Azure SDK sends that token as a client assertion, gets a short-lived Entra access token, and calls Key Vault, where only this identity has the “Key Vault Secrets User” role.

COMMON MISTAKES

- Trusting calls because they come from a private IP range.
- mTLS in PERMISSIVE mode forever, so plain-text calls still work.
- Authenticating services but not authorizing them: every workload may call every API.
- Tokens without an audience, so one service can replay its token to another.

6 Key terms

Zero trust: no implicit trust from network location

SPIFFE ID / SVID: workload name and its verifiable credential

Managed identity: cloud-issued workload identity with no secret

mTLS: both client and server present certificates

Attestation: platform checks what a workload is before issuing identity

Federated credential: trust another issuer's tokens for an identity

7 Interview questions

Q1. What is zero trust? BASIC

Never trusting the network: every request proves its identity and is authorized, whether it comes from the internet or the pod next door.

Q2. How does mutual TLS differ from normal TLS? BASIC

Both sides present and verify certificates, so the server also authenticates the client; the channel is encrypted as usual.

Q3. How do services authenticate each other without static secrets? SCENARIO

Give each workload a platform-issued identity (SPIFFE/SPIRE, cloud managed identity), then use mutual TLS and short-lived tokens validated (signature, audience, expiry) on every call, and authorize by identity.

Q4. A shared API key between two services leaked in a public repo. What now, and what next? SCENARIO

Revoke and replace it immediately and check logs for misuse. Then remove static keys: mesh mTLS or workload-identity tokens with per-caller authorization.

Q5. With mTLS everywhere, do you still need tokens? TRICKY

Often yes: mTLS proves which *service* is calling, but not which *user* the call is for. Propagate the user's token or a derived one for user-level authorization.

Q6. Why is a short-lived token safer than a long-lived key if both can leak? TRICKY

Exposure time: a leaked token stops working in minutes, is bound to one audience, and rotation needs no coordination because every token is new.

Next: SD-M-008 · Encryption in Transit backendroadmap.com

Encryption in Transit

private is not the same as safe

Watch the 5-min video

In one line: TLS gives confidentiality, integrity and server authentication (plus client authentication with mTLS); use it on every hop, including inside the private network, with TLS 1.2+ only, HSTS for browsers, and automated certificates.

1 Plain text behind the load balancer

The public site uses HTTPS, but the load balancer terminates TLS and forwards plain HTTP to the services: “it's a private network”. Then an attacker lands on one machine, or a misconfigured traffic mirror copies packets, and every password, token and customer record is readable. Encrypting only at the edge protects the easy part.

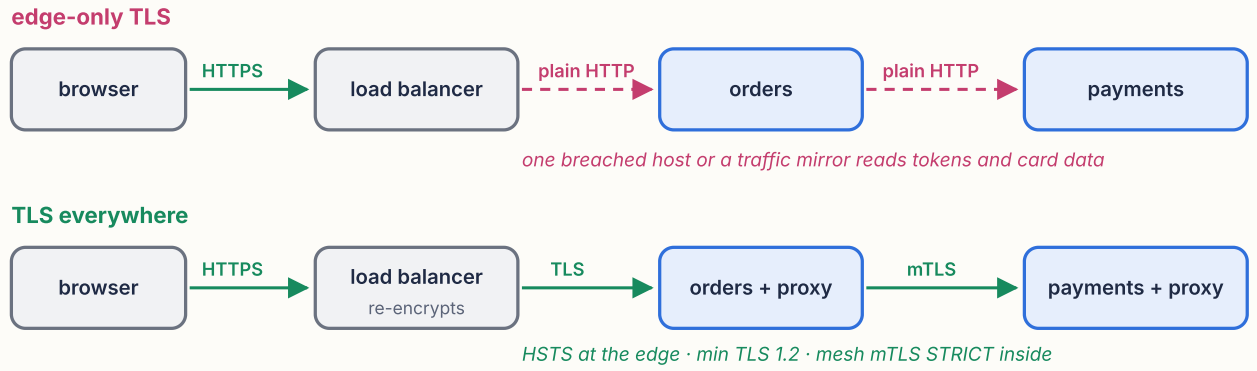


Figure 1. Terminating TLS at the edge leaves every internal hop exposed. Re-encrypt at the load balancer and use mesh mTLS between services.

2 What TLS gives you

- **Confidentiality**: symmetric encryption (AES-GCM or ChaCha20-Poly1305) of every record.
- **Integrity**: the AEAD tag detects any change in transit.
- **Server authentication**: a certificate chain to a trusted CA, plus a signature with the server's private key; the client checks the hostname.
- **Client authentication** only with mutual TLS (lesson 45).

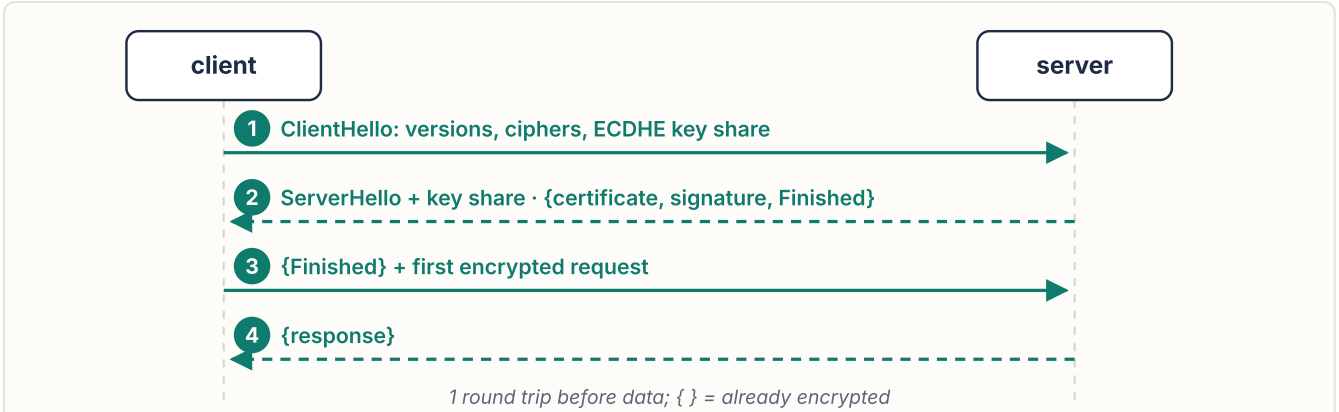


Figure 2. TLS 1.3 (RFC 8446) needs one round trip. The cost is mostly the handshake, so reuse connections (keep-alive, HTTP/2, pools).

3 Why encrypt internal traffic?

Private is not safe. A breached host can sniff its neighbours. Cloud networks are shared infrastructure. Insiders, debug proxies and packet captures see raw traffic. Compliance regimes (PCI DSS for card data, HIPAA for health data) require or strongly expect encryption in transit. The modern default is **TLS on every hop**, which zero trust assumes anyway (lesson 53).

4 HSTS for browsers

`Strict-Transport-Security: max-age=31536000; includeSubDomains; preload` tells the browser to use only HTTPS for this site for a year. That stops **SSL stripping**, where an attacker on public Wi-Fi intercepts the first plain-HTTP request and keeps the victim on HTTP. The first visit is still exposed unless the domain is on the browsers' **preload list** (hstspreload.org), which requires exactly these three directives.

5 Certificates, protocols and ciphers

TLS everywhere means certificates everywhere, and an expired certificate is a classic outage. Automate: ACME with Let's Encrypt (90-day certificates), cert-manager in Kubernetes, a mesh CA for short-lived internal certificates. Monitor expiry and alert weeks ahead. Public certificate lifetimes are shrinking (CA/Browser Forum: 200 days from March 2026, 47 days by 2029), so manual renewal no longer works. Keep private keys out of code and images.

Protocol	Status	Why
SSL 3.0, TLS 1.0, TLS 1.1	disable	weak constructions (POODLE, BEAST); TLS 1.0/1.1 formally deprecated by RFC 8996 (2021)
TLS 1.2	allow, with ECDHE + AES-GCM / ChaCha20	still needed by older clients; avoid static-RSA key exchange and CBC suites
TLS 1.3	prefer	only five AEAD suites, forward secrecy always, 1-RTT handshake

Forward secrecy (ephemeral ECDHE keys) means a server key stolen later can't decrypt traffic recorded earlier. TLS 1.3 makes it mandatory. Beware 0-RTT resumption in TLS 1.3: early data can be replayed, so allow it only for idempotent requests.

WORKED EXAMPLE: TLS 1.2+ ON ALL INTERNAL ENDPOINTS

The team sets a minimum of TLS 1.2 on the load balancer and API gateway (e.g. an AWS ALB security policy or Azure App Gateway SSL policy), and Istio `PeerAuthentication` mode `STRICT` for pod-to-pod traffic. An admission policy (OPA Gatekeeper or Kyverno) rejects any Service or Ingress exposing plain HTTP. A weekly scan (testssl.sh, sslyze) flags endpoints that still accept TLS 1.0/1.1 or weak ciphers, and expiry alerts fire 30 days ahead.

COMMON MISTAKES

- TLS at the edge only; plain HTTP behind the load balancer.
- Disabling certificate verification in internal clients (`verify=False`), which removes authentication entirely.
- Certificates renewed by hand and a calendar reminder.
- HSTS with a long max-age and `includeSubDomains` before every subdomain supports HTTPS.

6 Key terms

TLS: protocol for encrypted, authenticated connections

Forward secrecy: past sessions stay safe if the long-term key leaks

TLS termination: where TLS is decrypted, e.g. at the load balancer

AEAD: cipher mode that encrypts and authenticates, e.g. AES-GCM

HSTS: header forcing HTTPS for a domain

ACME: protocol for automated certificate issuance

7 Interview questions

Q1. What does TLS give you? BASIC

Confidentiality, integrity and server authentication, and with mutual TLS, client authentication too.

Q2. What is HSTS and what attack does it stop? BASIC

A response header telling browsers to use only HTTPS for the domain for max-age seconds; it stops SSL-stripping downgrades after the first visit (or from the first, if preloaded).

Q3. Why encrypt traffic inside a private network? SCENARIO

Private doesn't mean safe: a breached host, shared cloud infrastructure, insiders or misconfigured logging can expose plain traffic, and compliance often requires it.

Q4. Production went down because a certificate expired. How do you prevent a repeat? SCENARIO

Automate issuance and renewal (ACME, cert-manager, mesh CA), monitor expiry for every endpoint with alerts weeks ahead, and inventory where certificates live.

Q5. What is forward secrecy, and why does it matter? TRICKY

Each session uses ephemeral (EC)DHE keys, so stealing the server's private key later can't decrypt recorded traffic. TLS 1.3 requires it.

Q6. Isn't TLS everywhere too slow? TRICKY

On modern CPUs with AES instructions, record encryption is cheap; the handshake is the main cost, and connection reuse, session resumption and TLS 1.3's 1-RTT handshake make it small.

Next: SD-M-009 · Encryption at Rest and KMS backendroadmap.com

Encryption at Rest and Key Management

encryption is easy; the keys are the hard part

Watch the 4-min video

In one line: encrypt stored data at the right layer, keep the master keys in a KMS or HSM that never releases them, and use envelope encryption: data keys encrypt data locally, the KMS only wraps and unwraps those small keys.

1 Is the stolen data readable?

A backup tape goes missing. A disk from a decommissioned server is sold online. A storage bucket is accidentally public. The question each time: can the finder read the data? Encryption at rest makes stolen storage useless, **as long as the key wasn't stored next to it**.

2 Layers of encryption at rest

Layer	Example	Protects against	Doesn't protect against
Full disk / volume	BitLocker, LUKS, EBS encryption	lost or stolen drives	anyone who can log in to the running machine
Database (TDE)	SQL Server / Azure SQL TDE, Oracle TDE	stolen data files and backups	a user or app with SQL access
Object storage	S3 SSE-KMS, Azure Storage encryption	media theft at the provider	leaked credentials, public buckets
Application / field level	encrypt card or ID numbers before INSERT	DB admins, SQL injection dumps, leaked backups	a compromised app that holds the key

Lower layers are transparent and cheap (S3 encrypts every new object by default since January 2023); higher layers protect against more attackers but cost search and indexing. Use several.

3 Keys live in a KMS

If the key sits next to the data, a thief gets both. Keys belong in a **key management service**: AWS KMS, Azure Key Vault, Google Cloud KMS. Master keys never leave it; applications ask it to perform operations, access is controlled by IAM, and every use is logged. But a KMS call is a network round trip with request quotas and a small payload limit (AWS KMS `Encrypt` takes at most 4 KB), so you don't send gigabytes through it.

4 Envelope encryption

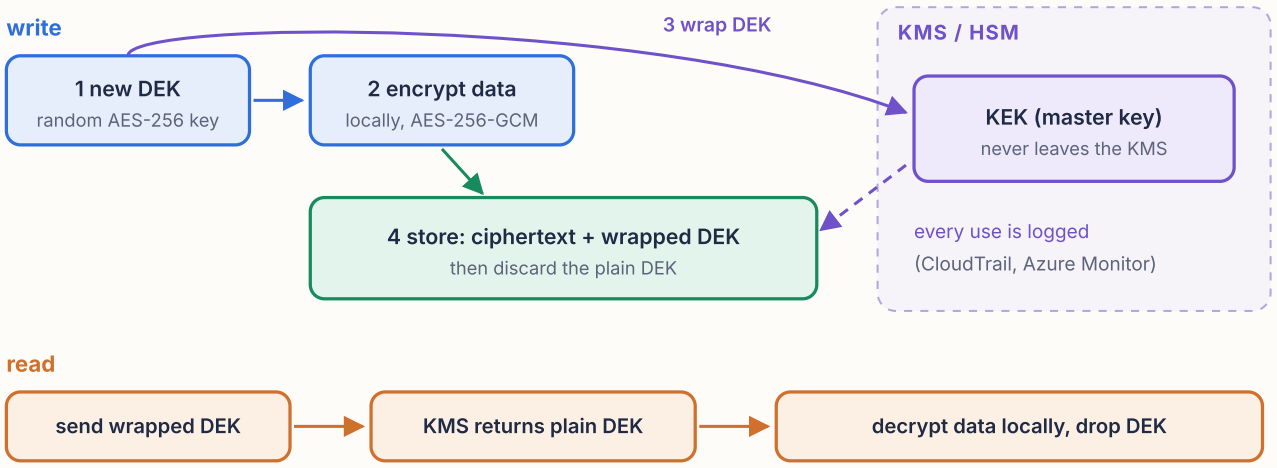


Figure 1. Envelope encryption: a data encryption key (DEK) encrypts the data locally; the key encryption key (KEK) in the KMS only wraps the DEK.

AWS's `GenerateDataKey` does steps 1 and 3 in one call, returning the plain DEK and the wrapped DEK. It is **fast** (bulk crypto is local, AES at GB/s), **scalable** (one DEK per object, file or tenant; cache plain DEKs briefly to cut KMS calls) and **cheap to rotate**: rotating the KEK means re-wrapping small DEKs, not re-encrypting terabytes. KMS-managed rotation keeps old key versions so old data still decrypts.

5 Customer-managed keys and HSMs

Customer-managed keys (CMK, sometimes BYOK) let a customer hold the master key in their own KMS account: revoke it, and the SaaS provider can no longer read their data. Many enterprises require this. **Hardware security modules** are tamper-resistant devices (validated to FIPS 140-2/140-3, often Level 3) for the highest-value keys: AWS CloudHSM, Azure Managed HSM. Rotate keys on a schedule (commonly yearly), and separate duties: the people who manage keys shouldn't also read the data.

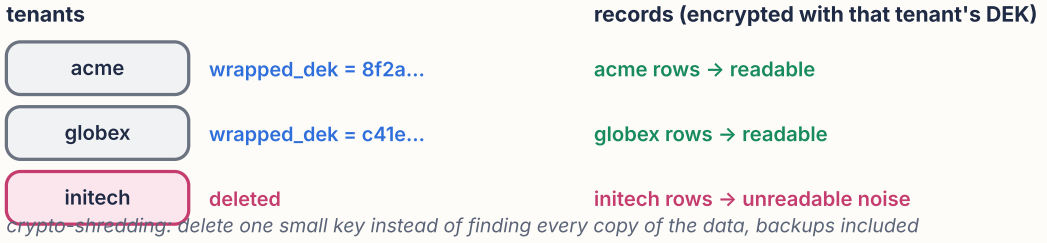


Figure 2. Per-tenant DEKs wrapped by one KMS master key. Deleting a tenant's wrapped key makes all their data unreadable at once.

WORKED EXAMPLE: PER-TENANT KEYS AND CRYPTO-SHREDDING

Each tenant row stores `wrapped_dek`, created with `GenerateDataKey` under the master key. Records are encrypted with AES-256-GCM using that tenant's DEK (with the tenant ID as associated data, so rows can't be swapped between tenants). When a tenant leaves, the service deletes the wrapped key: every record, replica and backup of theirs becomes unreadable. If one DEK ever leaks, only one tenant is exposed.

COMMON MISTAKES

- Keys in config files or in the same database as the data.
- Believing disk or TDE encryption protects against SQL injection or a stolen app credential; it doesn't.
- Deleting old key versions after rotation, so old backups can never be restored.
- Home-made crypto: ECB mode, reused GCM nonces, unauthenticated encryption.

6 Key terms

DEK: data encryption key; encrypts the actual data

KEK: key encryption key; wraps DEKs, stays in the KMS

KMS: managed service that stores keys and performs key operations

HSM: tamper-resistant hardware that holds keys

Customer-managed key: master key controlled by the customer

Crypto-shredding: destroying data by deleting its key

7 Interview questions

Q1. Explain envelope encryption. BASIC

Data is encrypted locally with a data encryption key; the DEK is wrapped by a key encryption key that never leaves the KMS, and only the wrapped DEK is stored with the data. To read, the KMS unwraps the DEK and decryption happens locally.

Q2. Why use envelope encryption? BASIC

Fast local encryption, per-object or per-tenant keys, and cheap master-key rotation: re-wrap the small DEKs, not the data.

Q3. The database volume is encrypted. Is customer data safe from a SQL injection dump? SCENARIO

No. Disk and TDE encryption are transparent to anyone querying the database. Use parameterised queries, and field-level encryption for the most sensitive columns.

Q4. An enterprise customer wants to be able to cut off your access to their data. How? SCENARIO

Customer-managed keys: their KEK lives in their KMS account and wraps their DEKs; if they revoke your access to it, you can't unwrap anything.

Q5. How do you rotate a master key without re-encrypting petabytes? TRICKY

With envelope encryption you only re-wrap DEKs under the new KEK version (or keep old versions for decryption and use the new one for new data).

Q6. How do you guarantee a deleted customer's data is gone from backups too? TRICKY

Crypto-shredding: encrypt each customer's data with their own key and delete the key; copies in backups become unreadable without editing the backups.

Next: SD-M-010 · Secrets Management backendroadmap.com

Secrets Management

never in code, never forever

Watch the 4-min video

In one line: keep secrets out of code, images and config; let each workload fetch them at runtime from a secret store using its own identity, prefer short-lived dynamic credentials, rotate the rest automatically, and scan every commit.

1 Leaked in minutes

A developer pushes a quick test to a public repo with a database password inside. Bots scan public GitHub continuously, and the credential is being tried within minutes. Secrets in code, config files and container images are one of the most common ways in: cheap to find, expensive to clean up. Deleting the line doesn't help; **Git history keeps it forever**, and every clone and fork has a copy.

2 Where secrets should (not) live

Location	Problem
Source code, Git history	copied to every clone, fork and CI cache; public repos are scanned by bots
Container image, plain config file	pushed to registries, readable by anyone who can pull the image
Environment variables	better, but leak via crash dumps, debug pages, child processes and `/proc`
Kubernetes Secret	only base64 in etcd unless encryption at rest is enabled; anyone with `get secrets` reads them
Secret store (Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager)	encrypted, per-secret access policy, audit log, versions, rotation

The application authenticates to the store with its **workload identity** (lesson 45), not with yet another secret, and fetches at startup or on demand. Platforms can also mount secrets as files (Secrets Store CSI driver, Vault Agent) or sync them (External Secrets Operator).

3 Dynamic secrets

Instead of one long-lived password shared by every instance, the store **creates credentials on request**. A leaked credential expires on its own, and every database session traces back to one instance. Cloud equivalents: IAM database authentication on RDS, or Entra ID tokens for Azure SQL, which remove the password entirely.

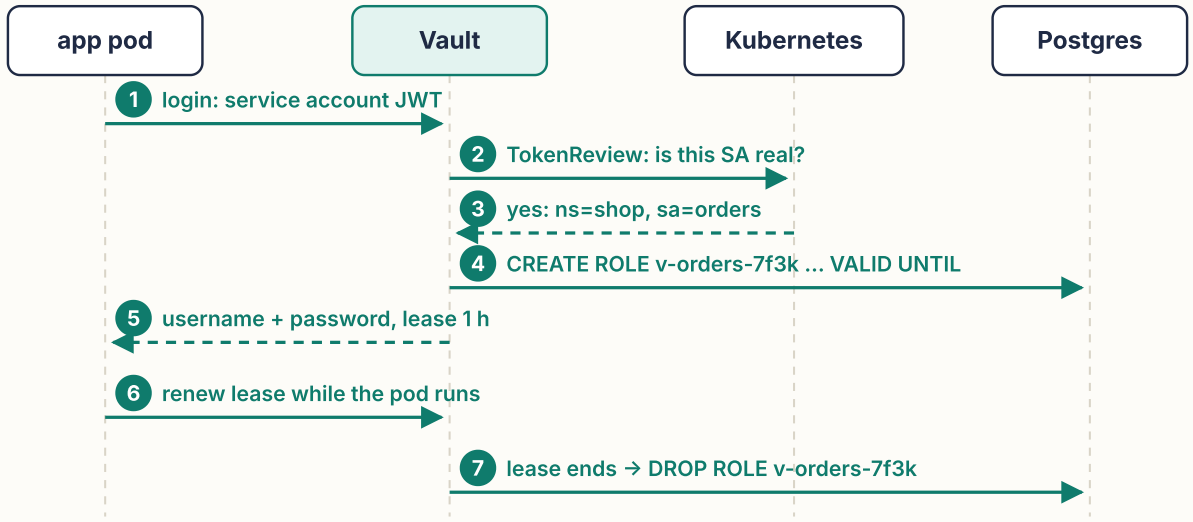


Figure 1. Vault's database secrets engine creates a unique database user per app instance with a one-hour lease, then deletes it.

4 Rotation without outages

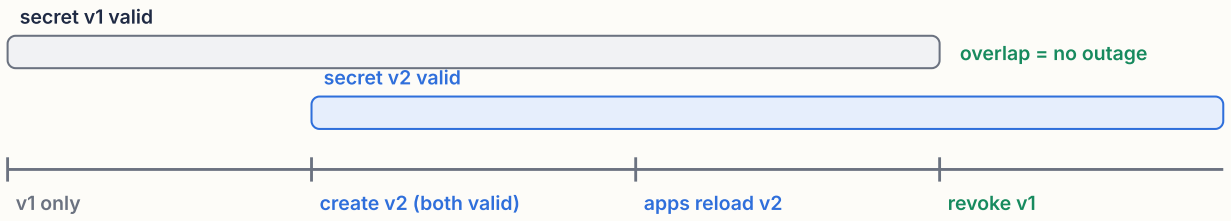


Figure 2. Rotate with two valid versions: create the new one, let every instance pick it up, then revoke the old one.

Static secrets (third-party API keys, legacy DB users) still exist, so rotate them on a schedule and automatically: AWS Secrets Manager runs a rotation function; the “alternating users” strategy keeps two DB users and flips between them. Apps should reload secrets without a restart, or the platform should roll pods gracefully. Track each secret's age and alert on old ones.

5 Catch mistakes early

Assume someone will slip. **Pre-commit hooks** (gitleaks, detect-secrets) stop secrets on the laptop; **GitHub secret scanning** with push protection blocks known key formats on push and alerts on history; CI scans images too. If a secret leaks: **revoke and rotate immediately**, then check its audit trail for use. Rewriting history is cleanup, not remediation.

WORKED EXAMPLE: VAULT ROTATING DATABASE CREDENTIALS

The pod authenticates to Vault with its Kubernetes service account (Kubernetes auth method). A Vault role maps `shop/orders` to a database role with `default_ttl=1h`. Vault creates `v-orders-7f3k` in Postgres and returns its credentials. The Vault Agent sidecar writes them to a shared in-memory file, renews the lease while the pod runs, and fetches new ones before the maximum TTL. Nobody sees the password, nothing is in Git, and deleted pods leave no valid credentials behind.

COMMON MISTAKES

- Deleting the leaked line in a new commit and calling it fixed.
- Bootstrapping the secret store with a static token baked into the image (“secret zero”).
- One shared credential for all services and environments, so nothing can be traced or rotated safely.
- Logging configuration or request headers, which prints the secrets you carefully protected.

6 Key terms

Secret: anything that grants access: password, API key, private key, token

Secret store: encrypted, audited service for secrets

Dynamic secret: credential created on demand with a short lease

Lease / TTL: how long a credential stays valid

Secret zero: the first credential needed to reach the store

Secret scanning: detecting keys in commits, history and images

7 Interview questions

Q1. How should an app get its database password? BASIC

Not from code, images or plain config: it authenticates to a secret store with its workload identity and fetches the credential at runtime, ideally a short-lived dynamic secret that is rotated automatically.

Q2. What else helps besides a secret store? BASIC

Least-privilege access policies per secret, audit logs, and secret scanning on every commit; if a secret leaks, revoke and rotate it immediately.

Q3. An AWS key was pushed to a public repo an hour ago and then deleted. What do you do? SCENARIO

Treat it as compromised: deactivate it now, issue a replacement, review CloudTrail for its use, then purge history and add push protection.

Q4. How do you rotate a database password used by 40 running pods with no downtime? SCENARIO

Create a second valid credential, roll or hot-reload the pods onto it, verify, then revoke the old one; or move to dynamic per-pod credentials.

Q5. How does the app authenticate to the secret store without a secret? TRICKY

Using an identity the platform vouches for: a Kubernetes service account token, cloud managed identity or instance role, verified by the store.

Q6. Are environment variables a safe place for secrets? TRICKY

Better than code, but they leak through crash reports, debug endpoints, child processes and logs. Mounted files or direct SDK fetches with short lifetimes are safer.

Next: SD-M-011 · Hashing Passwords backendroadmap.com

Hashing Passwords and Sensitive Data

slow, salted, and never reversible

Watch the 4-min video

In one line: store passwords only as slow, salted hashes from a password hashing function (Argon2id, scrypt or bcrypt) tuned to about 100 ms; for data you must get back, like card numbers, use tokenisation or encryption instead.

1 Ten million rows on a forum

A database leaks. Plain-text passwords: every account is gone in minutes, and on other sites too, because people reuse passwords. Plain SHA-256: most are cracked within days. Argon2id with sane parameters: attackers recover only the weakest few. The difference is a few lines of code.

2 Hash, don't store, and why SHA-256 fails

A hash is one-way: easy to compute, infeasible to invert. At login you hash what the user typed and compare. But attackers don't invert hashes; they **guess**: take a list of leaked passwords and dictionary words, hash each, compare. SHA-256 is built to be fast, and one modern GPU computes **billions** of SHA-256 hashes per second. Adding a salt doesn't change that speed. For passwords, fast is bad.

3 Password hashing functions

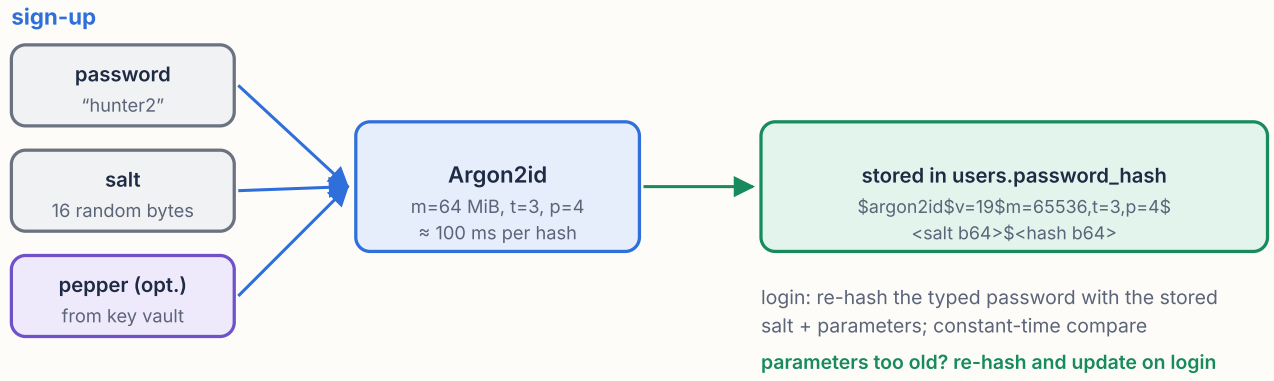


Figure 1. The stored string carries the algorithm, parameters and salt, so you can raise the cost later and upgrade hashes as users log in.

Function	Slows attackers by	OWASP minimum (Password Storage Cheat Sheet)
Argon2id (RFC 9106, recommended)	time + memory (memory-hard)	m = 19 MiB, t = 2, p = 1 (or more memory, fewer passes)
scrypt	time + memory	$N = 2^{17}$, r = 8, p = 1
bcrypt	time (cost factor)	cost 10 or more; input limited to 72 bytes
PBKDF2-HMAC-SHA256	iterations only	600,000 iterations; use when FIPS-140 compliance is required

Each guess should cost about **100 ms** and, for Argon2id and scrypt, tens of MiB of memory, which makes GPU and ASIC attacks expensive. The video's `m=64 MiB, t=3, p=4` is one of RFC 9106's recommended settings. Benchmark on your login servers; raise cost as hardware improves; limit concurrent hashing so a login flood can't exhaust memory.

4 Salt and pepper

A **salt** is random, unique per user (16 bytes is typical) and stored with the hash. Same password, different hashes: rainbow tables are useless and each hash must be attacked on its own. The libraries generate it for you. A **pepper** is one secret shared by all hashes and kept **outside** the database (key vault or HSM), applied as an HMAC; if only the database leaks, the attacker is missing it.

5 Data you need back: tokenise or encrypt

A card number for the next payment can't be hashed. **Tokenisation** keeps the real value in a vault and gives your systems a random token that means nothing elsewhere; restrict and log detokenisation. For values you only need to *look up* (national ID, email), use a keyed HMAC, not a plain hash: low-entropy inputs like 9-digit IDs can be brute-forced in seconds.

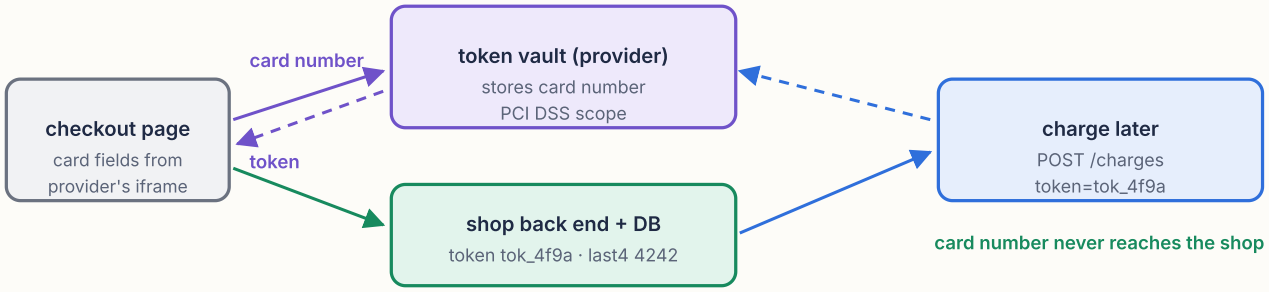


Figure 2. Saved cards via tokenisation: the shop stores a token and the last four digits; the provider's vault holds the card number.

WORKED EXAMPLE: ONE-CLICK CHECKOUT

The checkout page loads card fields from the payment provider (hosted iframe or SDK), so the card number goes straight to the provider's vault. The shop receives `tok_4f9a` and stores only `token, last4=4242, expiry, brand`. To charge again, the back end sends the token. With no card numbers stored, processed or transmitted by its own servers, most of the shop's systems fall outside PCI DSS scope.

COMMON MISTAKES

- MD5, SHA-1 or SHA-256 for passwords, even with a salt.
- Encrypting passwords (reversible) instead of hashing them.
- Writing your own scheme instead of a vetted library (argon2-cffi, bcrypt, libsodium).
- Logging request bodies on the login endpoint, which writes plain passwords to logs.

6 Key terms

Password hashing function: deliberately slow, tunable hash: Argon2id, scrypt, bcrypt

Salt: random per-user value stored with the hash

Pepper: shared secret kept outside the database

Memory-hard: needs lots of RAM per guess, hurting GPUs/ASICs

Rainbow table: precomputed hash → password lookup, defeated by salts

Tokenisation: swap a sensitive value for a meaningless token

7 Interview questions

Q1. How should passwords be stored? BASIC

Hashed with a slow, salted password hashing function, Argon2id (recommended) or bcrypt, with a cost tuned to roughly 100 ms, optionally plus a pepper kept outside the database.

Q2. Why not SHA-256? BASIC

It's designed to be fast: one GPU tests billions of guesses per second, so dictionary attacks against a leaked table succeed quickly.

Q3. You inherit a table of unsalted MD5 password hashes. How do you migrate? SCENARIO

Immediately wrap each: store Argon2id(MD5 hash) with a fresh salt, so no weak hash remains. On each login, verify and replace with a plain Argon2id hash of the password.

Q4. Your app must store card numbers for repeat payments. What do you do? SCENARIO

Don't store them: use the payment provider's tokenisation, keeping only the token and last four digits, which shrinks PCI DSS scope.

Q5. Why is a salted SHA-256 hash of a national ID number still weak? TRICKY

The input space is tiny (about 10^9 values), so the attacker hashes every possible ID with the salt in seconds. Use a keyed HMAC with the key in a vault, or encrypt.

Q6. What is a pepper, and where do you keep it? TRICKY

A secret added to every password hash (usually via HMAC) and stored outside the database, in a key vault or HSM, so a database-only leak can't be cracked offline.

Next: SD-M-012 · OWASP Top 10

OWASP Top 10

the same handful of mistakes, everywhere

Watch the 5-min video

In one line: the OWASP Top 10 lists the most critical web application risks from real data; most breaches chain two or three of them, and the fixes share one pattern: enforce on the server, keep data apart from code, and be secure by default.

1 Same mistakes, thousands of apps

Most breaches don't use clever new attacks. They reuse a handful of mistakes. The OWASP Top 10 (current widely used edition: 2021) ranks web application risk categories using data from hundreds of thousands of applications plus a community survey. Know it, and you can review any design with an attacker's eyes.

2021 category	Typical example	Main fix
A01 Broken Access Control	change an ID, read someone else's record	server-side checks, deny by default (lesson 39)
A02 Cryptographic Failures	plain-text or SHA-1 passwords, TLS 1.0	Argon2id, TLS 1.2+, keys in a KMS (lessons 46–49)
A03 Injection	SQL injection, OS command injection, XSS	parameterised queries, output encoding
A04 Insecure Design	password reset with no rate limit	threat modeling before code (lesson 54)
A05 Security Misconfiguration	default passwords, public buckets, debug on	hardened, automated config
A06 Vulnerable and Outdated Components	library with a known CVE	dependency scanning, fast patching
A07 Identification and Authentication Failures	credential stuffing, weak sessions	MFA, rate limits, proven IdP
A08 Software and Data Integrity Failures	unsigned updates, unsafe deserialization	signatures, trusted pipelines (lesson 58)
A09 Security Logging and Monitoring Failures	breach unnoticed for months	audit logs + alerts (lesson 55)
A10 Server-Side Request Forgery (SSRF)	server fetches the metadata endpoint	allowlist destinations, block internal IPs

2 A01 Broken access control

Users act outside their permissions: reading another user's record by changing an ID, or calling an admin endpoint whose button was only hidden in the UI. It moved to #1 in 2021; 94% of applications tested had some form of it. Fix on the server: deny by default, check ownership on every request, and write tests that try other users' IDs.

3 A03 Injection

vulnerable: string building

```
q = "SELECT * FROM users WHERE name = '" + name + "'"
name = x' OR '1'='1
→ SELECT * FROM users WHERE name = 'x' OR '1'='1' (every row)
```

safe: parameterised query

```
cur.execute("SELECT * FROM users WHERE name = %s", (name,))
```

SQL text is fixed and parsed first; the value travels separately to the driver, so x' OR '1'='1 is just a strange name that matches nobody

Figure 1. Injection mixes untrusted data into code. Parameterised queries keep them apart, so input can only ever be data.

The interpreter (SQL, shell, LDAP, the browser's HTML parser) can't tell data from code, so the attacker's text runs. Cross-site scripting is the same idea in the browser: fix it with context-aware output encoding (templating engines do this by default) plus a Content Security Policy. ORMs parameterise for you, unless you build raw string queries.

4 A02, A04, A05: crypto, design, configuration

Cryptographic failures expose data: plain-text passwords, fast hashes, old TLS, hard-coded keys. **Insecure design** is a flaw in the plan, like unlimited password-reset attempts or a coupon that can be applied twice, which careful coding can't fix: threat model early. **Misconfiguration**: default credentials, open storage buckets, stack traces in error pages, directory listing, permissive CORS.

5 A10 SSRF and the rest

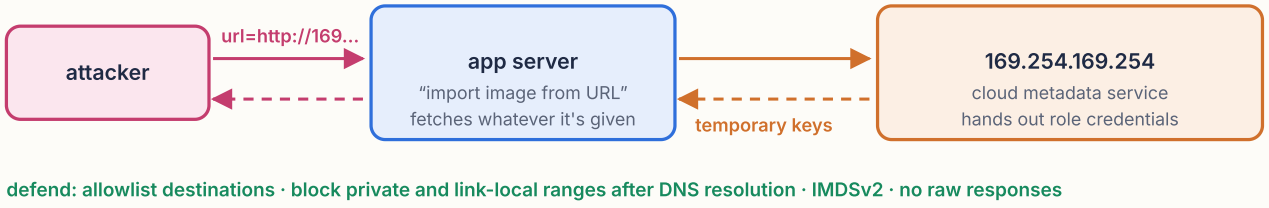


Figure 2. SSRF: the server fetches a URL the attacker chose, reaching internal services such as the cloud metadata endpoint. The 2019 Capital One breach used this path.

Vulnerable components: scan dependencies (Dependabot, Snyk, OWASP Dependency-Check) and patch continuously. The remaining categories cover authentication failures, integrity failures in software and data, and missing logging, so attacks go unnoticed. OWASP has since published a 2025 edition that keeps broken access control at #1, folds SSRF into it and adds software supply chain failures; the ideas are the same.

WORKED EXAMPLE: FIXING A SEARCH ENDPOINT

`/search?name=` built SQL by concatenation. Input `x' OR '1'='1` returned every user. The fix: `cur.execute("SELECT id, display_name FROM users WHERE name = %s", (name,))`. Also select only needed columns, run the app's DB user with read-only rights on that table, and add a test that sends quotes and `--` to every search parameter.

COMMON MISTAKES

- Escaping input by hand or with a blocklist instead of parameterising.
- Validating only in the browser; attackers call the API directly.
- Treating the list as a checklist to tick, not a guide for design reviews and tests.
- Fetching user-supplied URLs from servers with cloud credentials and no egress controls.

6 Key terms

OWASP: Open Worldwide Application Security Project

Parameterised query: fixed SQL with placeholders; values sent separately

SSRF: server tricked into requesting an attacker-chosen URL

Injection: untrusted input interpreted as code

XSS: attacker script runs in another user's browser

Insecure design: missing or flawed security control in the plan

7 Interview questions

Q1. Name the top OWASP risks and how you mitigate them. BASIC
Broken access control (server-side checks, deny by default), injection (parameterised queries, output encoding), cryptographic failures (strong hashing, modern TLS), insecure design (threat modeling), SSRF (allowlist destinations), misconfiguration and vulnerable components (hardening, dependency scanning).

Q2. What's the root cause of injection? BASIC
Mixing untrusted data with code, so the interpreter runs attacker input.

Q3. Your app lets users import a profile picture from a URL. What do you protect against? SCENARIO
SSRF: allow only http(s), resolve DNS and block private, loopback and link-local ranges (re-check after redirects), fetch from an isolated egress proxy, and enforce IMDSv2.

Q4. How do you prevent XSS in a server-rendered app? SCENARIO
Use a templating engine that auto-escapes by context, avoid raw HTML insertion, sanitise any rich text with a vetted library, and add a strict Content Security Policy.

Q5. Why can't “insecure design” be fixed by better code review? TRICKY
The control is missing from the design itself, such as no rate limit on password reset; correct code faithfully implements the flaw. It needs threat modeling and requirements.

Q6. Is input validation enough to stop SQL injection? TRICKY
No. Validation helps, but legitimate data can contain quotes (O'Brien). Parameterised queries make injection structurally impossible; validation is a second layer.

Next: SD-M-013 · OWASP API Security backendroadmap.com

OWASP API Security Risks

APIs fail in their own ways

Watch the 4-min video

In one line: APIs expose raw objects to scripts, so their top risks are object-level authorization (BOLA), weak authentication, returning or accepting too many properties, and unlimited consumption; fix them with per-object checks and explicit schemas.

1 Why a separate list?

APIs carry most traffic in modern apps. They return raw objects, accept raw JSON, and are called by scripts rather than browsers, so there is no UI to hide anything behind. OWASP keeps a separate **API Security Top 10**; the current edition is **2023**.

2023 risk	In one line
API1 Broken Object Level Authorization	no check that the caller may access *this* object ID
API2 Broken Authentication	weak login, unchecked or never-expiring tokens
API3 Broken Object Property Level Authorization	too many fields returned (excessive data exposure) or accepted (mass assignment)
API4 Unrestricted Resource Consumption	no rate, size, page or cost limits
API5 Broken Function Level Authorization	regular users can call admin functions
API6 Unrestricted Access to Sensitive Business Flows	bots automate buying, sign-ups, reviews
API7 Server Side Request Forgery	API fetches attacker-chosen URLs (lesson 50)
API8 Security Misconfiguration	verbose errors, permissive CORS, missing TLS
API9 Improper Inventory Management	forgotten old versions, test hosts, shadow APIs
API10 Unsafe Consumption of APIs	trusting data from third-party APIs

2 BOLA: the most common API bug

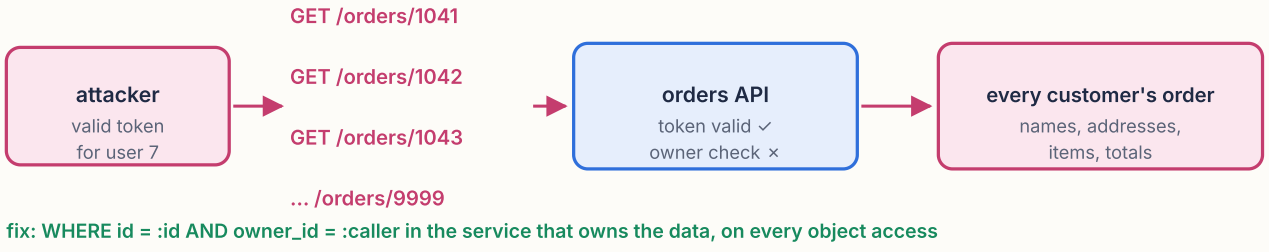


Figure 1. BOLA: authentication passes, so the attacker simply loops over IDs. Random IDs slow this down but don't fix it.

The API takes an ID from the path and returns the object without checking ownership (lesson 39). Fix it in the service that owns the data, on **every** access, including nested routes (`/users/7/orders/1042`), batch

endpoints and GraphQL resolvers. Automated tests should replay each request with a second user's token.

3 Broken authentication

Weak login flows, JWTs whose signature or `aud` is never checked (lesson 41), tokens that never expire, login and password-reset endpoints with no rate limit, and API keys sent in URLs where they end up in logs. Use a proven identity provider, validate every token fully, and keep tokens short-lived.

4 Too many properties: out and in

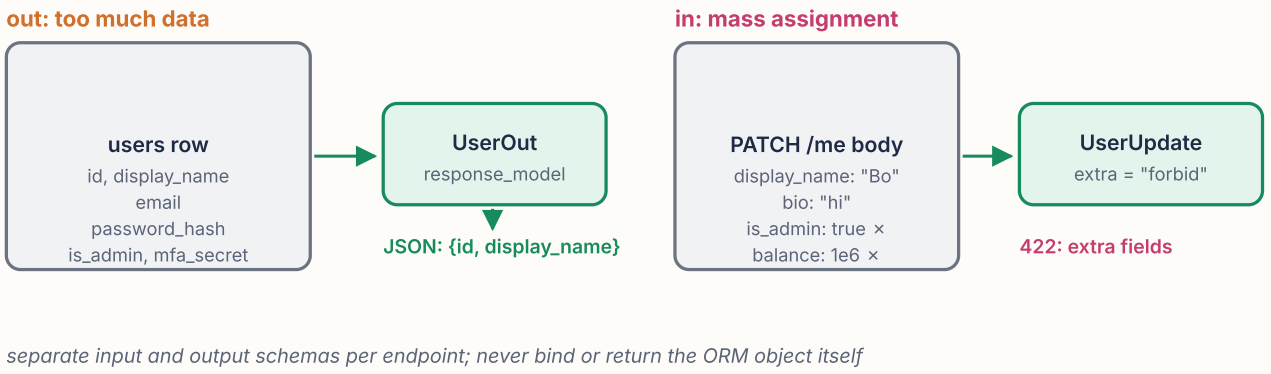


Figure 2. Excessive data exposure and mass assignment are two sides of API3:2023. Explicit schemas fix both.

Excessive data exposure: the API returns whole database objects and trusts the client to hide fields. The app shows a name; the JSON also has the email, password hash and internal flags, readable with any proxy. **Mass assignment** is the reverse: the API binds the request body onto the model, so a user adds `"is_admin": true` or a new balance. Rails, Django REST Framework and Spring all have this trap.

5 Limits and inventory

Unrestricted resource consumption: limit requests per user, key and IP (lesson 52), and also page sizes, upload sizes, query depth for GraphQL, and calls that cost you money (SMS, email). **Improper inventory**: forgotten `/v1` endpoints, staging hosts with production data, undocumented internal APIs. Keep a catalog generated from OpenAPI specs and gateway logs, and retire old versions on purpose.

WORKED EXAMPLE: FASTAPI RESPONSE AND INPUT MODELS

`class UserOut(BaseModel): id: int; display_name: str` is declared as `response_model=UserOut`, so FastAPI filters the returned object to exactly those fields. Updates use `class UserUpdate(BaseModel): display_name: str; bio: str` with `model_config = {"extra": "forbid"}`, so a body containing `is_admin` is rejected with 422. The handler then copies only validated fields onto the row. Over-exposure and mass assignment are blocked by design, not by memory.

COMMON MISTAKES

- Returning `user.__dict__` or the ORM object and filtering in the mobile app.
- Checking the user's role for the endpoint but not ownership of the object.
- Rate limiting only by IP, while attackers rotate through thousands of IPs.
- Leaving `/v1` running “for a few clients” without the fixes made in `/v2`.

6 Key terms

BOLA: API returns an object without an ownership check

BFLA: regular user can call an admin function

Excessive data exposure: response includes fields the client shouldn't see

Mass assignment: request sets fields it shouldn't, like is_admin

Response / input model: explicit schema of allowed output / input fields

Shadow API: endpoint missing from the inventory, so unpatched

7 Interview questions

Q1. What's the top API risk? BASIC

Broken object level authorization: always check that the caller may access that specific object, in the service that owns it.

Q2. What is mass assignment? BASIC

Binding a request body directly onto a data model, so a client can set fields it shouldn't (is_admin, balance). Prevent it with explicit input schemas that allow only safe fields and reject extras.

Q3. A mobile app shows only names, but a pentester finds emails and password hashes in the JSON. Fix? SCENARIO

Return a response model with only the fields that screen needs, never the database object; audit other endpoints the same way and remove hashes from any API layer.

Q4. How do you find APIs you didn't know you had? SCENARIO

Build an inventory from gateway and load-balancer logs, DNS records and OpenAPI specs; flag hosts and versions with traffic but no owner, then retire or secure them.

Q5. BOLA vs BFLA: what's the difference? TRICKY

BOLA is access to another user's *object* through a permitted function; BFLA is calling a *function* you shouldn't (e.g. DELETE /admin/users) at all.

Q6. Do UUIDs instead of sequential IDs fix BOLA? TRICKY

No. They stop easy enumeration, but IDs leak through links, logs and other responses. Only an authorization check on each object access fixes it.

Next: SD-M-014 · Abuse Prevention backendroadmap.com

Rate Limiting and Abuse Prevention

rate limits buy time; MFA removes the prize

Watch the 5-min video

In one line: defend in layers (edge DDoS protection and WAF, rate limits per IP, account and key, bot detection with challenges only for risky traffic), and remove the value of stolen passwords with breached-password checks, MFA and passkeys.

1 Credential stuffing

The login page gets 50,000 attempts an hour from 10,000 IP addresses, each trying a different email and password. The pairs come from other sites' breaches; people reuse passwords, so a small share succeed. Per-IP limits barely notice (five tries per IP) and per-account lockouts never fire (one try per account).

Attack	Pattern	Per-IP limit	Per-account limit	What really helps
Brute force	many passwords, one account, few IPs	catches	catches	slow hashing, delays, MFA
Password spraying	one common password, many accounts	catches if few IPs	misses	breached/common-password ban, MFA
Credential stuffing	leaked pairs, each account once, many IPs	mostly misses	misses	bot detection, MFA, passkeys

2 Layers

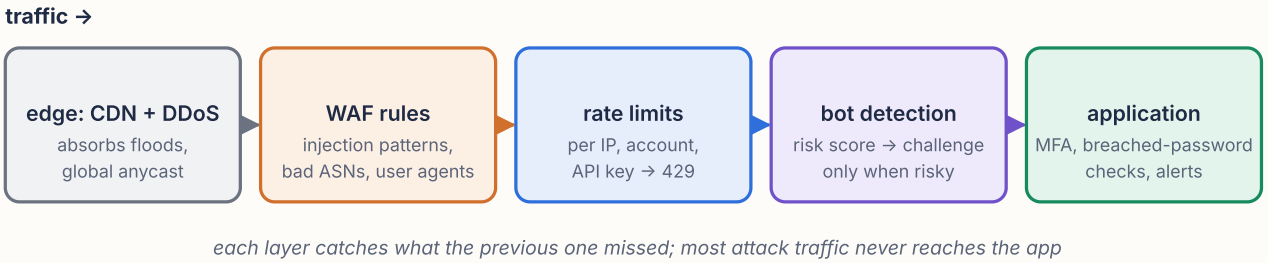


Figure 1. Abuse protection in layers, from the network edge to the application.

DDoS mitigation belongs at the edge: your servers can't absorb terabits per second. Cloudflare, AWS Shield or Azure Front Door spread traffic across a global anycast network and drop attacks before they reach you. A **web application firewall** adds managed rules (e.g. the OWASP Core Rule Set) for injection patterns, known bad user agents and suspicious networks.

3 Rate limits that work

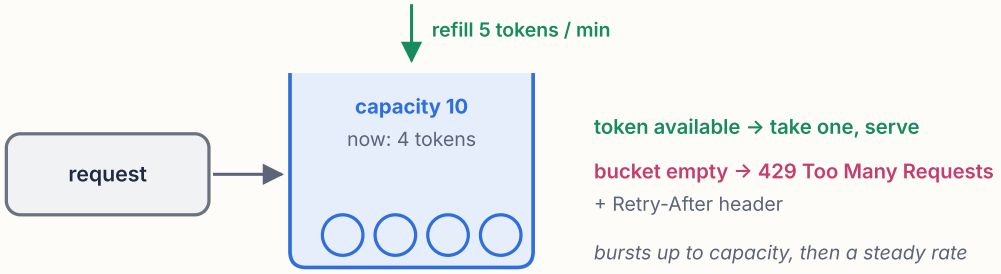


Figure 2. A token bucket allows short bursts and a steady average rate. Sliding-window counters in Redis are the common alternative.

Limit on several keys at once: **per IP** (cheap, catches simple scripts), **per account** (catches targeted guessing from many IPs), **per API key or tenant** (fairness and cost). Return **429** with `Retry-After`. For logins, prefer **progressive delays** and step-up checks over hard lockouts. NIST SP 800-63B requires limiting failed attempts per account (no more than 100 consecutive failures) and checking new passwords against breached and common-password lists.

4 Bots, CAPTCHAs and lockouts

Bot detection scores signals: request timing, TLS and browser fingerprints, JavaScript challenges, IP reputation (data-centre ranges, residential proxies). Show a CAPTCHA or managed challenge only when risk is high: CAPTCHAs annoy real users and paid solving farms beat them. **Hard lockout after five failures** lets an attacker lock out any user on purpose, a denial of service, and still misses stuffing.

5 Remove the prize

Rate limiting buys time. These take away the value of a stolen password: **MFA** stops the vast majority of stuffing logins; **breached-password checks** at sign-up and change (Have I Been Pwned's range API sends only the first 5 hex characters of a SHA-1 hash, k-anonymity); **passkeys** (WebAuthn) remove shared secrets entirely and can't be phished.

WORKED EXAMPLE: PROTECTING /LOGIN

Cloudflare sits in front with managed WAF rules and bot management. A rate-limit rule allows 5 requests per minute per IP to `/login`, then issues a challenge. Behind it, the app keeps a sliding window per account in Redis (`login:acct:{email}`) and adds a growing delay after failures. Logins from a new device need a second factor. During an attack the challenge rate spikes, most stuffing traffic never reaches the app, and an alert fires when the login failure rate jumps.

COMMON MISTAKES

- Only per-IP limits, against attackers with 10,000 residential proxy IPs.
- Hard account lockout that attackers use to lock out your users.
- Different error messages for “no such user” and “wrong password”, enabling account enumeration.
- Rate limiting /login but not password reset, sign-up or OTP verification endpoints.

6 Key terms

Credential stuffing: replaying leaked username/password pairs

Password spraying: one common password against many accounts

WAF: filters HTTP requests by attack signatures and rules

Token bucket: limiter allowing bursts up to a capacity

Progressive delay: longer wait after each failure

Passkey: WebAuthn key pair; phishing-resistant, no shared secret

7 Interview questions

Q1. How do you protect a login endpoint from credential stuffing? BASIC

Edge protection (WAF, bot management), rate limits per IP and per account, challenges only for risky traffic, progressive delays, breached-password checks, and MFA or passkeys.

Q2. Why not just lock accounts? BASIC

Attackers can trigger lockouts to deny service, and stuffing tries each account only a few times, so lockouts rarely fire.

Q3. Your API is scraped by bots using thousands of IPs. What do you do? SCENARIO

Require authentication and limit per account or API key, add bot management at the edge, cap page sizes, and watch for business-flow abuse (API6:2023).

Q4. Where should a distributed rate limiter keep its counters? SCENARIO

In a shared low-latency store such as Redis, using atomic increments or Lua scripts with TTLs; or at the edge/gateway so limits apply before your servers.

Q5. Token bucket vs fixed window: what's the difference? TRICKY

A fixed window allows double the limit across a window boundary; a token bucket (or sliding window) smooths that, allowing bursts only up to the bucket size.

Q6. Should the login form say “user not found”? TRICKY

No: different messages, or different response times, let attackers enumerate valid accounts. Return the same generic error and similar timing.

Next: SD-M-015 · Least Privilege and Zero Trust backendroadmap.com

Least Privilege and Zero Trust

limit the blast radius; assume breach

Watch the 4-min video

In one line: give every identity only the permissions it needs, for only as long as it needs them, and never trust a request because of where it comes from: authenticate, check context and authorize each request, segment the network and encrypt everything.

1 The bug wasn't the problem

An attacker gets into a small email service through a vulnerable library. That service's credentials can read every database and storage bucket in the account, so one small hole becomes a full breach. The real failure was **how much the compromised service was allowed to do**.

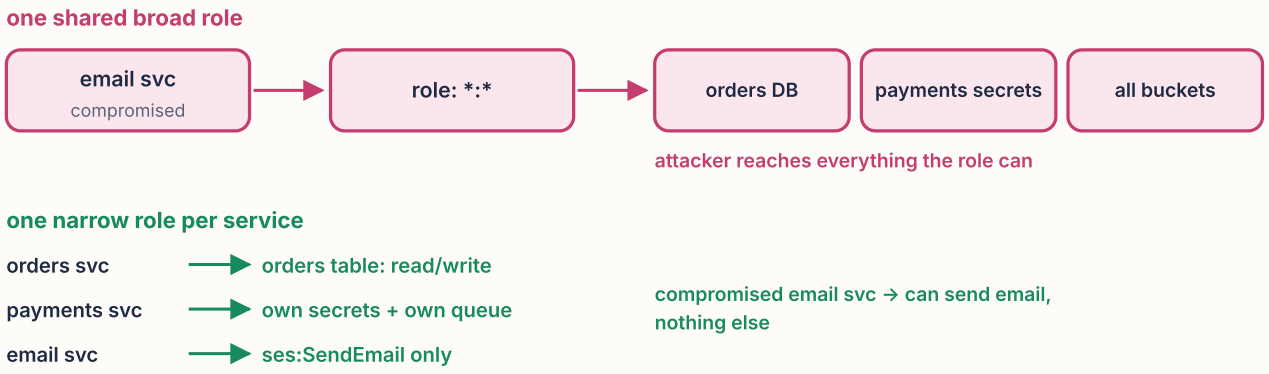


Figure 1. Least privilege doesn't stop the break-in; it limits the blast radius to what that one identity could already do.

2 Least privilege for services

- **One identity per service** (lesson 45), never shared keys, so permissions and audit trails are per service.
- **Narrow policies**: specific actions on specific resources, e.g. `dynamodb:GetItem, PutItem, Query` on `arn:aws:dynamodb:…:table/orders`. A wildcard (`"Action": "*", "Resource": "*"`) is a red flag in every review.
- **Remove unused permissions**: AWS IAM Access Analyzer, GCP IAM Recommender and Entra permissions reports find grants never used in 90 days.
- Apply it to CI/CD too: a pipeline that deploys one service shouldn't hold org-admin rights (lesson 58).

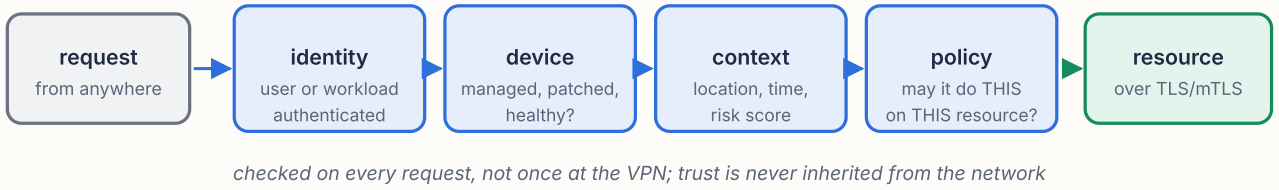
3 Least privilege for people

Standing admin rights are inherited by any stolen laptop or phished session. **Just-in-time access** (Azure PIM, AWS IAM Identity Center with temporary elevation, Teleport, Okta) grants elevated rights on request, for a short time, with approval and an audit trail (lesson 55). Keep a sealed **break-glass** account for emergencies, with alerts on every use.

4 Segment the network

Split the network into zones and allow only needed flows: web → API → database, nothing else crosses. An attacker in one zone can't freely scan the rest. In Kubernetes, start each namespace with a **default-deny NetworkPolicy** and allow specific pod-to-pod traffic; in the cloud, use security groups that reference each other rather than wide CIDR ranges.

5 Zero trust



checked on every request, not once at the VPN; trust is never inherited from the network

Figure 2. Zero trust: every request is authenticated, evaluated in context and authorized for the specific resource.

	Perimeter model	Zero trust (NIST SP 800-207)
Trust from	being inside the network / VPN	verified identity + device + context, per request
Internal traffic	often plain text	encrypted and authenticated (mTLS)
After a breach	attacker moves laterally freely	each hop needs its own authorization
Example	VPN to the office network	Google BeyondCorp: internal apps over the internet, no VPN

WORKED EXAMPLE: PER-SERVICE ROLES ON AN E-COMMERCE PLATFORM

Each microservice runs with its own IAM role. **orders**: read/write on the orders table only. **payments**: read its own secrets and consume its own queue. **email**: `ses:SendEmail` only. Network policies let only the API tier reach the database. When a library flaw gives an attacker code execution in the email service, they can send email, and nothing else; orders data stays out of reach, and the unusual calls show up in the audit trail.

COMMON MISTAKES

- Copying an admin policy “to get it working” and never narrowing it.
- Treating zero trust as a product to buy rather than per-request authentication and authorization.
- Permanent production access for every engineer.
- Flat networks where any pod can reach any database.

6 Key terms

Least privilege: only the permissions needed, only for as long as needed

Just-in-time access: temporary, approved elevation that expires

Zero trust: no implicit trust from network location; verify every request

Blast radius: what an attacker can reach from one compromise

Segmentation: zones with only the required flows allowed

Break-glass account: emergency access, sealed and alerted

7 Interview questions

Q1. What is least privilege? BASIC

Granting each identity only the permissions it needs, for only as long as it needs them.

Q2. What does zero trust mean in practice? BASIC

Never trust a request because of its network location: authenticate and authorize every request, give each service a least-privilege identity, segment the network, encrypt everything (mTLS) and assume breach.

Q3. Engineers need production DB access during incidents. How do you allow it safely? SCENARIO

Just-in-time access: request with a reason, approval, time-boxed credentials (e.g. one hour), session recording or audit logs, and automatic expiry.

Q4. A service's IAM role has s3:* on *. How do you fix it without breaking it? SCENARIO

Use access logs or an access analyzer to see which actions and buckets it really used, generate a narrow policy from that, deploy it, and watch for denials.

Q5. If you have zero trust, do you still need network segmentation? TRICKY

Yes, as defence in depth: segmentation limits what a compromised host can even reach, and covers services that don't yet authenticate every call.

Q6. Does least privilege prevent breaches? TRICKY

Not the initial compromise; it limits what that compromise can do and makes abnormal access stand out.

Next: SD-M-016 · Threat Modeling backendroadmap.com

Threat Modeling

ask the attacker's question before launch

Watch the 4-min video

In one line: draw how data flows and where it crosses trust boundaries, ask the six STRIDE questions of every element, then rank threats by likelihood and impact and fix the high ones before launch, cheaper on a whiteboard than after a breach.

1 The question nobody asked

A file-upload feature is ready: code reviewed, tests green. Nobody asked how it could be abused. **Threat modeling** asks that on purpose, at design time. Adam Shostack's four questions frame it: *What are we working on? What can go wrong? What are we going to do about it? Did we do a good job?*

2 Data flows and trust boundaries

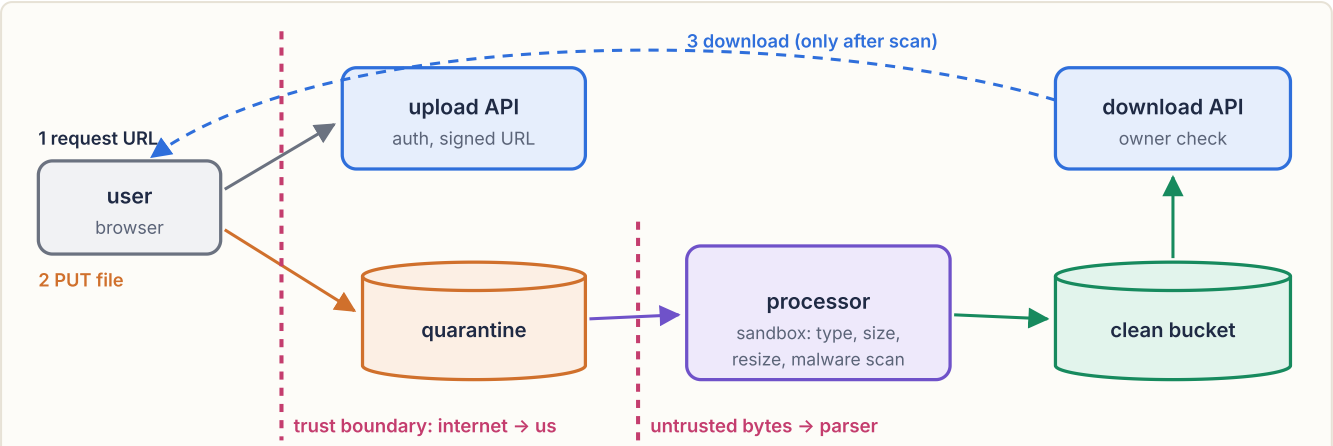


Figure 1. A data flow diagram of the upload feature. Dashed lines are trust boundaries, where untrusted input must be checked.

Draw **actors**, **processes**, **data stores** and **flows**. Then draw **trust boundaries**: where data moves between levels of trust (internet → API, your app → third party, user file → parser). Most attacks happen at those lines, so that's where validation and authentication belong. The **attack surface** is everything reachable: endpoints, parameters, file parsers, admin panels, queues, debug ports. Remove what you don't need.

3 STRIDE

Threat	Violates	Question	Upload example → mitigation
Spoofing	authentication	can someone pretend to be another user?	anonymous uploads → auth + short-lived signed URL
Tampering	integrity	can data be changed in transit or at rest?	file swapped after scan → immutable key, checksum
Repudiation	non-repudiation	can someone deny what they did?	"I never uploaded that" → audit log of uploader (lesson 55)
Information disclosure	confidentiality	can data leak?	public bucket, guessable names → private bucket, random keys
Denial of service	availability	can it be overwhelmed?	10 GB files, zip bombs → size limits, rate limits
Elevation of privilege	authorization	can someone gain rights?	parser exploit (e.g. ImageTragick) → sandboxed processing

Walk every element and flow in the diagram with these six questions. Other methods exist (attack trees, PASTA, LINDDUN for privacy), but STRIDE on a data flow diagram is the common interview answer.

4 Prioritize and record

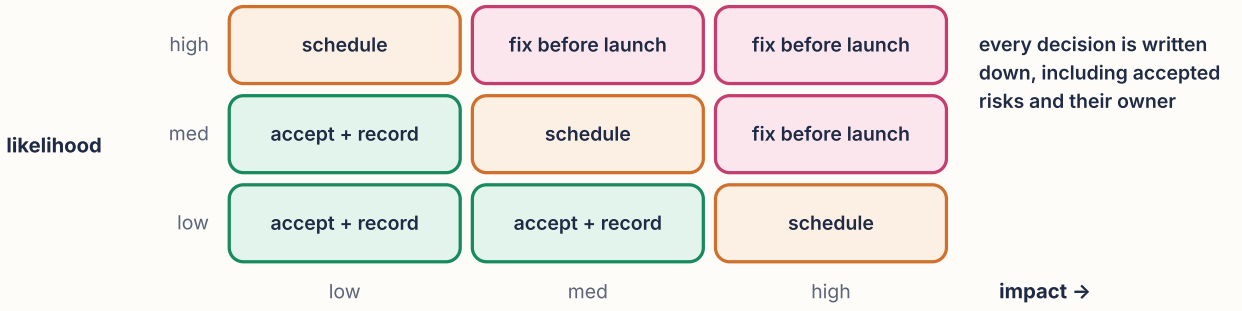


Figure 2. Rate each threat by likelihood and impact. A threat accepted on purpose is fine; a threat nobody noticed is not.

Keep it lightweight: an hour at a whiteboard with a developer, someone from security and someone who knows the product catches most big problems. Store the model with the design doc (OWASP Threat Dragon or the Microsoft Threat Modeling Tool help) and revisit it when the design changes: a new data store, integration or trust boundary.

WORKED EXAMPLE: THREAT MODEL OF A FILE UPLOAD SERVICE

S: require login; the API issues a signed upload URL valid for 5 minutes for one object key. **T/E**: the file lands in a quarantine bucket; a sandboxed worker (no network, no credentials, CPU and memory limits) checks magic bytes against an allowlist, enforces a size limit, re-encodes images, and runs a malware scan. **I**: private bucket, random keys, downloads only through an API that checks ownership, served with `Content-Disposition: attachment` from a separate domain. **D**: per-user upload rate and quota. **R**: audit every upload and download.

COMMON MISTAKES

- Threat modeling after launch, or only once, as a compliance document.
- Trusting the file extension or the client's `Content-Type`.
- Listing threats without owners, priorities or decisions.
- Modeling only the happy path and ignoring admin tools, queues and batch jobs.

5 Key terms

Threat model: structured list of what can go wrong and what you'll do

Data flow diagram: actors, processes, stores and flows

Trust boundary: where data crosses between trust levels

Attack surface: everything an attacker can reach

STRIDE: spoofing, tampering, repudiation, disclosure, DoS, elevation

Risk acceptance: a recorded, owned decision not to fix now

6 Interview questions

Q1. What does STRIDE stand for? BASIC

Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege; each violates authentication, integrity, non-repudiation, confidentiality, availability and authorization respectively.

Q2. How do you prioritize threats? BASIC

By likelihood and impact: fix the high ones before launch, schedule the medium ones, consciously accept the low ones, and record every decision.

Q3. Threat model a file upload feature. SCENARIO

Draw the data flow and the trust boundary where files enter, then apply STRIDE: authenticate uploads (signed URLs), validate type and size, process in a sandbox, store privately with random names, rate limit per user, and scan for malware before anyone can download.

Q4. Your team adds a third-party webhook integration. What changes in the threat model? SCENARIO

A new trust boundary: verify webhook signatures and timestamps (spoofing, replay), validate payloads, rate limit, and don't let it reach internal services (SSRF).

Q5. What is a trust boundary, and why focus there? TRICKY

A point where data or control passes between different trust levels; attacks enter there, so authentication and validation must happen there.

Q6. When should you redo a threat model? TRICKY

Whenever the design changes materially (new data store, integration, trust boundary or sensitive data), and after incidents that reveal a missed threat.

Next: SD-M-017 · Audit Logging

Audit Logging

who did what, to what, when, from where

Watch the 4-min video

In one line: an audit log is a structured, append-only trail of security-relevant actions (who, what, which resource, when, from where, result), kept apart from app logs, tamper-evident, readable by few, and retained as long as regulations require.

1 Who gave Bob admin?

An account got full admin rights last Tuesday and nobody remembers approving it. Who did it, from where, and was it really them? If the system can't answer, the investigation stalls and the auditor writes a finding. SOC 2, ISO 27001, PCI DSS (Requirement 10) and HIPAA all expect audit trails.

2 What to record

Log **security-relevant actions**: logins and failures, MFA changes, permission and role changes, data exports, configuration changes, deletions, access to sensitive records, and admin actions. Each entry is structured JSON with:

```
{"actor": "alice", "action": "role.granted", "target": "bob", "role": "billing-admin",
"tenant": "acme", "time": "2026-11-03T14:07:22Z", "ip": "203.0.113.7", "request_id": "r-81c",
"result": "ok"}
```

Use UTC timestamps from NTP-synchronised clocks and a correlation ID that links the action to its request trace (lesson 65). Record failures and denials too: a burst of denied requests is often the first sign of an attack. Log identifiers, **never secrets**: no passwords, tokens, keys or full card numbers.

3 Separate from application logs

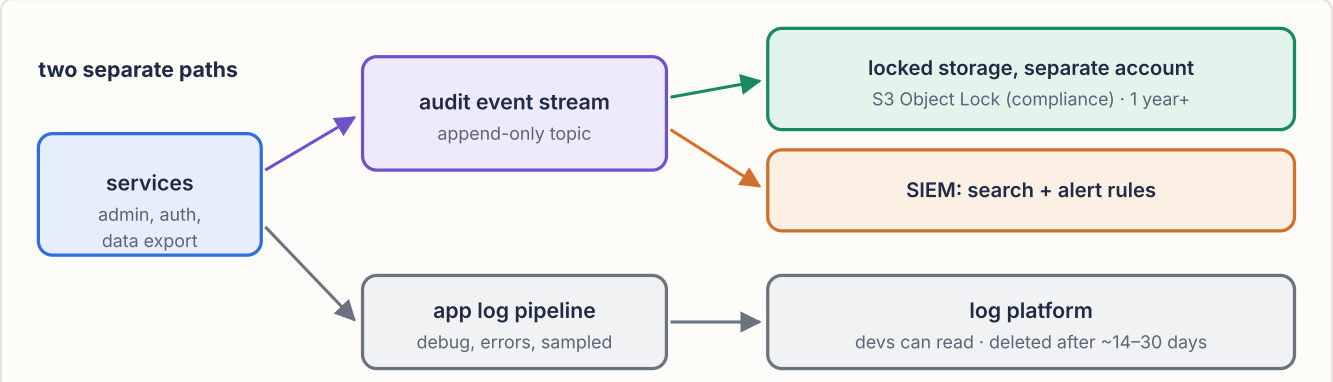


Figure 1. Audit events take their own path to write-once storage and a SIEM; ordinary app logs stay noisy, short-lived and widely readable.

	Application logs	Audit logs
Purpose	debugging, operations	accountability, investigations, compliance
Volume	high, often sampled	low, every event kept
Retention	days to weeks	months to years (PCI DSS: 12 months, 3 immediately available)
Who reads	developers, on-call	a small security / compliance group
Mutability	rotated and deleted freely	append-only, no one can edit or delete

4 Append-only and tamper-evident

If an attacker or rogue admin can delete entries, they cover their tracks. Write to **write-once storage**: S3 Object Lock in compliance mode (not even the root user can delete before retention ends), Azure immutable blob storage, or a separate account the application can only append to. Write asynchronously through a durable queue so auditing doesn't slow requests, but never drop events silently.

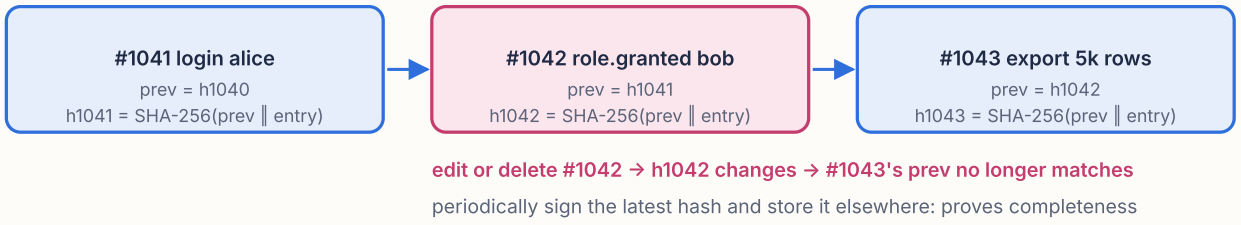


Figure 2. A hash chain makes tampering detectable: each entry includes the hash of the previous one.

Go further with a **hash chain**: each record stores the previous record's hash, so changing or removing any entry breaks verification. Periodically sign the latest hash and keep it elsewhere; that proves the log is complete up to that point. Then **alert** on suspicious patterns: admin grants outside business hours, many exports in an hour, audit-pipeline failures.

WORKED EXAMPLE: RECORDING ALL ADMIN ROLE CHANGES

When Alice grants Bob `billing-admin`, the IAM service writes the event above to an append-only stream before returning success. A consumer copies it to an Object Lock bucket in a separate security account (retention 2 years) and to the SIEM, where a rule alerts on admin grants outside 09:00–18:00 or by accounts that rarely grant roles. Months later, “who gave Bob admin?” is one query, answered in seconds, with a verifiable chain.

COMMON MISTAKES

- Audit events mixed into debug logs and deleted after 14 days.
- Admins who can edit or delete the audit trail.
- Logging tokens, passwords or full request bodies.
- Only logging successes, so failed and denied attempts are invisible.

5 Key terms

Audit log: trail of security-relevant actions for accountability

Append-only / WORM: write once, read many; no edits or deletes

Hash chain: each entry includes the previous entry's hash

Correlation ID: links an audit event to its request trace

SIEM: security information and event management: search + alerts

Retention period: how long records must be kept

6 Interview questions

Q1. What should an audit log contain? BASIC

Who did what, to which resource, when (UTC), from where, with what result, plus a correlation ID; structured, and never secrets like passwords, tokens or card numbers.

Q2. How do you keep it trustworthy? BASIC

Keep it separate from app logs, append-only on write-once storage in a separate account, hash-chained for tamper evidence, and readable only by a small security group.

Q3. Which events would you audit in a banking app? SCENARIO

Logins and failures, MFA and password changes, payee additions, transfers, limit changes, role and permission changes, data exports, and every admin action, including denials.

Q4. The audit pipeline is down. Should user actions fail? SCENARIO

For high-risk actions (admin grants, exports), yes, fail closed or queue durably; for the rest, buffer to durable local storage and alert. Never drop events silently.

Q5. Why isn't storing audit logs in the main database enough? TRICKY

Anyone with write access to that database, including an attacker with the app's credentials, can alter or delete them. They need separate, immutable storage.

Q6. How can you prove no audit entries were deleted? TRICKY

Hash-chain the entries and periodically sign and externally store the latest hash; verification recomputes the chain and detects any gap or edit.

Next: SD-M-018 · Data Privacy and Retention backendroadmap.com

Data Privacy and Retention

deleting a user means everywhere

Watch the 4-min video

In one line: collect only what you need, keep it only as long as you need it, and be able to find and erase a person's data everywhere it was copied, using a data map, deletion events that every owner confirms, enforced retention, and crypto-shredding for backups.

1 "Delete my account"

The app deletes the row from `users`. Done? The person's data is still in the search index, the analytics warehouse, the logs, three caches, the event stream and last month's backups. Under GDPR (Article 17, right to erasure) and India's DPDP Act 2023, it all has to go, normally within one month of the request.

2 The principles

- **Data minimisation**: collect only what the feature needs. Data you never stored can't leak.
- **Purpose limitation**: use it only for the purpose you stated.
- **Storage limitation**: keep it only as long as necessary.
- **Data subject rights**: access, correction, erasure, portability.

The laws apply to any company serving people in those regions. GDPR fines reach €20 million or 4% of worldwide annual turnover, whichever is higher; DPDP penalties reach ₹250 crore per breach.

3 Erasure in a distributed system

You can't delete what you can't find. Keep a **data map**: every store that holds personal data, which fields, and which team owns it. Keep it in code (schema annotations, a catalog generated in CI) so it doesn't go stale. Then make deletion an event every owner handles.

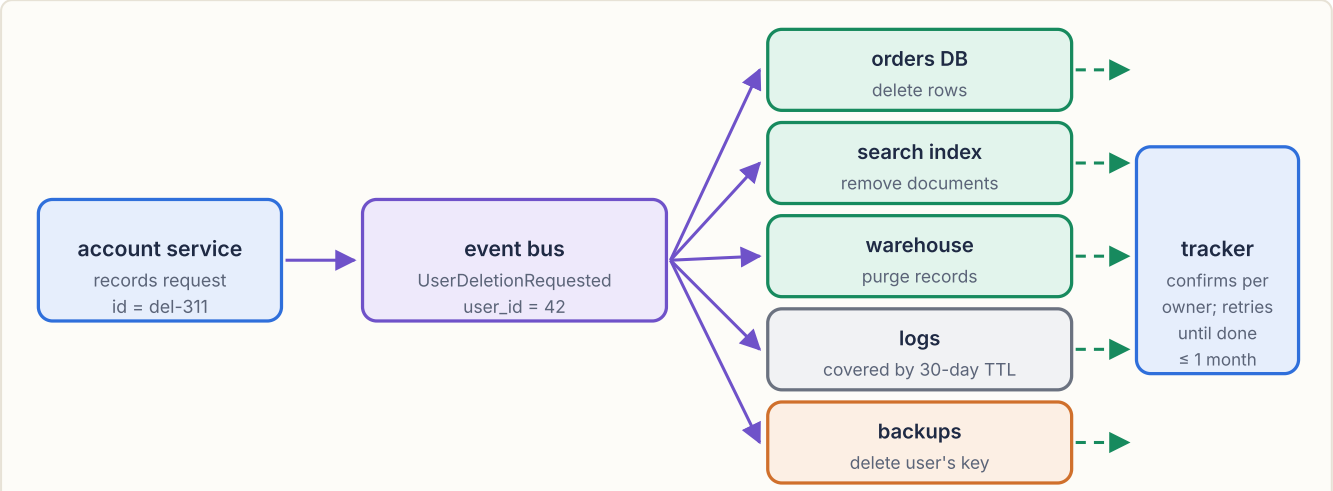


Figure 1. Deletion as an event: each owning service deletes and confirms; a tracker retries until every owner is done.

4 Backups and retention



or: keep backups short-lived, and replay pending deletions after any restore

Figure 2. Backups can't be rewritten row by row. Crypto-shredding (lesson 47) makes old copies unreadable instead.

Retention schedules make the rest automatic. Decide per data type, then enforce with TTLs and lifecycle rules, not good intentions:

Data	Example retention	Enforced by
raw application logs	30 days	log platform retention setting
session and analytics events	13 months, then aggregate	warehouse partition expiry
orders	while the account exists + 1 year	scheduled purge job
invoices	as tax law requires (often 6–10 years)	object storage lifecycle rule

Legal obligations can override erasure (you may have to keep invoices), so delete everything else and restrict access to what you must keep. Old data you no longer need is pure risk: it can only leak.

5 Anonymised vs pseudonymised

Pseudonymised data swaps names for tokens but can be re-linked with a key, so it is still personal data under GDPR. **Anonymised** data can't be linked back to a person by any reasonable means, and falls outside the law. That's harder than it looks: Latanya Sweeney showed that ZIP code, birth date and gender alone identify about 87% of Americans. Use aggregation, k-anonymity or differential privacy, not just dropping the name column.

WORKED EXAMPLE: DELETING A USER EVERYWHERE

The account service records request `del-311`, disables login at once, and publishes `UserDeletionRequested{user_id: 42}`. Orders, search and the warehouse consume it, delete, and publish confirmations; logs are covered by 30-day retention; backups by deleting user 42's key. A tracker shows per-owner status and retries failures until all confirm, well within one month. Each step is idempotent, so replays are safe.

COMMON MISTAKES

- Deleting from the primary database only, forgetting caches, indexes, warehouses and streams.
- Calling hashed emails or user IDs “anonymous”.
- Keeping everything forever “in case it's useful”.
- Restoring an old backup and silently resurrecting deleted users.

6 Key terms

Data minimisation: collect only what you need

Data map: inventory of where personal data lives and who owns it

Storage limitation: keep data only as long as necessary

Right to erasure: GDPR Art. 17: have your data deleted

Pseudonymisation: identifiers replaced, re-linkable with a key

Anonymisation: can't be linked back to a person at all

7 Interview questions

Q1. What's the difference between anonymisation and pseudonymisation? BASIC

Pseudonymised data replaces identifiers with tokens but can be re-linked with a key, so it's still personal data; anonymised data can't be linked back to a person at all.

Q2. What is data minimisation? BASIC

Collecting and keeping only the personal data a purpose needs, which also shrinks what a breach can expose.

Q3. How do you implement “delete my data” across a distributed system? SCENARIO

Keep a data map; publish a deletion event that every owning service handles and confirms, retrying failures; track completion; cover logs with short retention and backups with crypto-shredding.

Q4. How do you stop a backup restore from bringing deleted users back? SCENARIO

Keep a durable log of completed deletions and replay it after any restore, or rely on crypto-shredding so restored rows stay unreadable.

Q5. A user asks for erasure, but you must keep their invoices for tax law. What do you do? TRICKY

Erase everything else, keep only the legally required records, restrict access to them, and delete them when the legal period ends; tell the user why.

Q6. Is a SHA-256 hash of an email address anonymous? TRICKY

No. Anyone with a list of emails can hash them and match, so it's pseudonymous at best and still personal data.

Next: SD-M-019 · Multi-Tenant Isolation backendroadmap.com

Multi-Tenant Security Isolation

one missing WHERE clause away from disaster

Watch the 4-min video

In one line: pick an isolation model (silo, pool or bridge), take the tenant only from the authenticated identity, carry it into every query, cache key and limit, enforce it in the database with row-level security, and test for cross-tenant leaks in CI.

1 The worst bug you can ship

A SaaS serves 5,000 companies from one database. A developer forgets one `WHERE tenant_id = ?`, and company A sees company B's invoices. In multi-tenant systems, a cross-tenant leak is a breach of every customer's trust at once, and it takes only one missing line.

2 Three isolation models

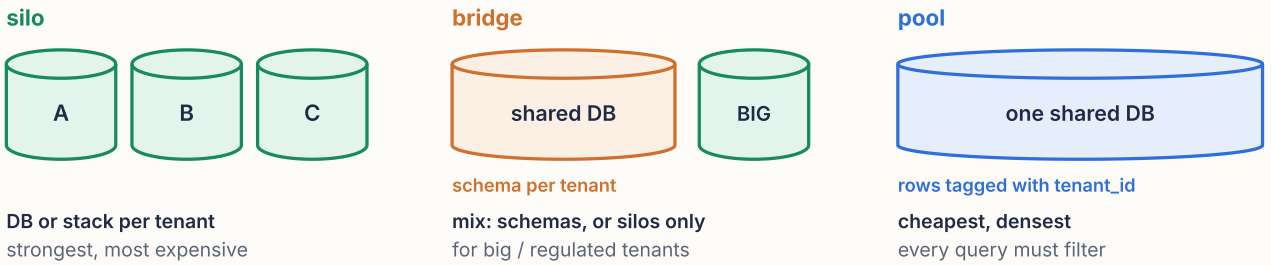


Figure 1. Silo, bridge and pool (the AWS SaaS Lens names). Most SaaS products pool small tenants and silo the largest.

	Silo	Bridge	Pool
Isolation	separate DB, account or cluster	schema per tenant, or mixed	shared tables + `tenant_id`
Cost, ops	highest; many deployments	medium; migrations per schema	lowest; one deployment
Leak risk	lowest	medium	highest; needs enforcement layers
Noisy neighbours	none	some	need quotas and limits

3 Tenant from identity, everywhere

In a pool, the tenant ID must come from the **authenticated identity** (a `tid` claim in the token, lesson 41), never from the request body, a header or a URL parameter the user controls. Resolve it once at the edge, pass it through every layer, and include it in every query, every cache key (`tenant:acme:invoice:1042`, or a shared cache serves one tenant's data to another), every queue message and every search filter.

4 Enforce it in the database

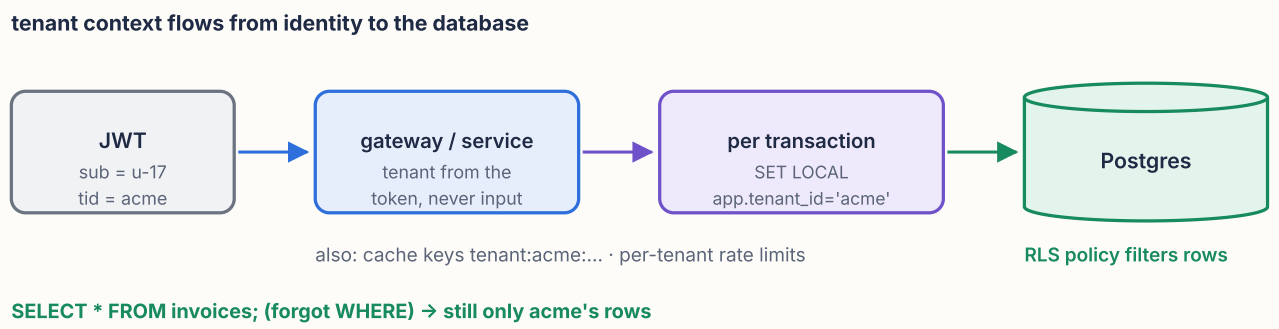


Figure 2. Row-level security makes the database apply the tenant filter, even when application code forgets it.

Don't rely on every developer remembering the filter. **Postgres row-level security** attaches a policy to each table; every query from the app's role is silently filtered. Two details matter: table owners and roles with `BYPASSRLS` skip policies (use `FORCE ROW LEVEL SECURITY` and a separate app role), and with connection pooling set the tenant with `SET LOCAL` inside each transaction so it can't leak to the next request.

5 Beyond rows

Per-tenant encryption keys (lesson 47) let you shred or lock one tenant independently, and support customer-managed keys. **Noisy neighbours**: one huge tenant can starve others of CPU, connections or queue capacity, so add per-tenant rate limits, quotas and fair scheduling. Large or regulated tenants may need dedicated infrastructure. And **test for leakage**: CI tests that log in as tenant A and request tenant B's resources by ID through every endpoint; alert on any cross-tenant access as a security event.

WORKED EXAMPLE: POSTGRES RLS ON EVERY TABLE

`ALTER TABLE invoices ENABLE ROW LEVEL SECURITY;`
`CREATE POLICY tenant_isolation ON invoices USING (tenant_id = current_setting('app.tenant_id'));`
The app connects as a role with `NOBYPASSRLS` that doesn't own the tables, and runs `SET LOCAL app.tenant_id = …` at the start of each request's transaction. The policy's `USING` clause also checks inserted and updated rows, so no one can write into another tenant. A forgotten filter now returns nothing extra.

COMMON MISTAKES

- Reading `tenant_id` from a request parameter or header.
- Cache keys without the tenant, so one tenant's cached response is served to another.
- Running the app as the table owner or a superuser, which silently bypasses RLS.
- Background jobs and admin tools that skip the tenant context entirely.

6 Key terms

Tenant: a customer organisation sharing the system

Silo / pool / bridge: dedicated / shared / mixed isolation models

Row-level security: database policies that filter rows per session

Tenant context: tenant ID carried from identity through all layers

Noisy neighbour: one tenant's load degrading others

Cross-tenant leak: data from one tenant visible to another

7 Interview questions

Q1. How do you guarantee one tenant never sees another's data? BASIC

Take the tenant from the authenticated identity, include it in every query and cache key, enforce it in the database with row-level security, use per-tenant keys where needed, and run cross-tenant leakage tests in CI.

Q2. Which isolation model would you pick? BASIC

Pool (shared tables + tenant_id) for cost, silo for the strongest isolation, or a bridge: pool most tenants and silo the largest or regulated ones.

Q3. An enterprise customer demands their data never shares a database with others. What do you offer? SCENARIO

A silo tier: dedicated database (or account/cluster) with its own customer-managed key, deployed by the same automation, priced accordingly.

Q4. One tenant's nightly import slows everyone down. What do you do? SCENARIO

Per-tenant quotas and rate limits, separate queues or worker pools for bulk jobs, and move very large tenants to dedicated capacity.

Q5. RLS is enabled but a test still sees all tenants' rows. Why? TRICKY

The connection's role owns the table or has BYPASSRLS (or is a superuser); owners skip policies unless FORCE ROW LEVEL SECURITY is set.

Q6. Why SET LOCAL instead of SET for the tenant setting? TRICKY

With pooled connections, a plain SET persists on the connection and can leak into the next request; SET LOCAL lasts only until the transaction ends.

Next: SD-M-020 · Supply Chain Security backendroadmap.com

Supply Chain Security

you ship far more code than you write

Watch the 4-min video

In one line: most of what you ship was written by strangers, so pin and scan dependencies, know what's inside each artifact (SBOM), prove where it came from (signatures and SLSA provenance), verify that at deploy time, and treat the CI pipeline as production.

1 Strangers' code in production

Your own code is a small fraction of what you deploy: open-source packages, base images, build tools and CI plugins make up the rest. Compromise one popular component and you're inside every company that installs it. **SolarWinds** (2020): attackers injected code into the Orion build system, so signed updates carried a backdoor to about 18,000 customers. **xz utils** (2024, CVE-2024-3094): a contributor spent two years earning maintainer trust, then shipped a backdoor targeting SSH; it was caught by chance before reaching most stable distributions.

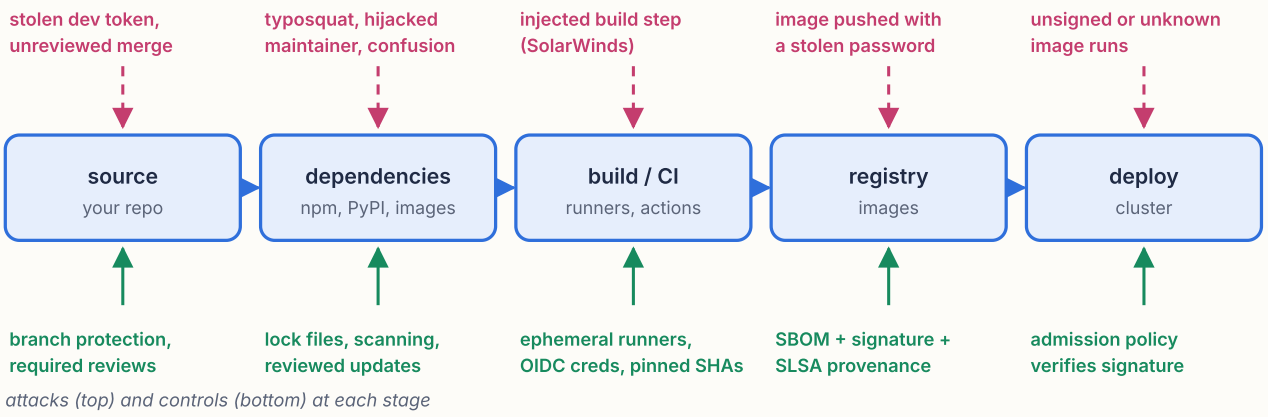


Figure 1. Each stage of the pipeline is an entry point. Controls at every stage make a single stolen credential insufficient.

2 Dependencies

Attack paths: a **hijacked maintainer account** publishes a malicious version; **typosquatting** (`reqeusts` vs `requests`); **dependency confusion**, where a public package with your internal package's name wins the resolver. Defences: **pin exact versions** with lock files (and hashes where supported, e.g. `pip --require-hashes`), scan for known CVEs, let Dependabot or Renovate open update PRs that humans review, use a private registry proxy with scoped names, and keep fewer dependencies.

3 Know what you ship: SBOM

A **software bill of materials** lists every component and version inside an artifact, in a standard format: **SPDX** (ISO/IEC 5962) or **CycloneDX** (OWASP). Generate it in CI for every build (Syft, Trivy) and attach it to the image. When the next Log4Shell lands, a query over SBOMs tells you in minutes which services include `log4j-core 2.14`, instead of days of grepping.

4 Prove where it came from

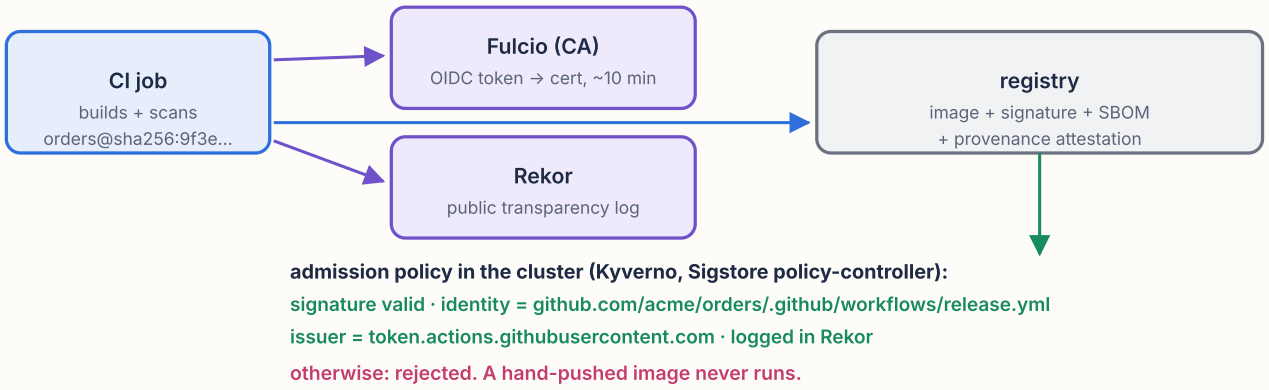


Figure 2. Keyless signing with Sigstore: the CI job's OIDC identity gets a short-lived certificate, and the signature is recorded in a public log.

Sign images in CI and verify before deploy, so the cluster refuses anything unsigned or signed by the wrong identity. **SLSA** (slsa.dev) adds **provenance**: a signed record of which source commit, build definition and builder produced the artifact. SLSA v1.0's build track runs from L1 (provenance exists) to L3 (hardened, isolated builder). A stolen registry password alone can no longer get code into production.

5 Harden the pipeline

- **Short-lived credentials**: OIDC federation from CI to the cloud instead of stored keys (lesson 48).
- **Pin third-party actions to a full commit SHA**, not a tag: in March 2025 the `tj-actions/changed-files` tags were repointed to code that dumped CI secrets.
- **Required reviews** and branch protection on main; no direct pushes, including for admins.
- **Ephemeral, isolated runners**, so one build can't poison the next; least-privilege tokens per job.

WORKED EXAMPLE: SIGNING IMAGES WITH COSIGN

After building and scanning `ghcr.io/acme/orders`, the release workflow runs `cosign sign ghcr.io/acme/orders@sha256:9f3e…`. Keyless mode exchanges the job's GitHub OIDC token for a Fulcio certificate valid for minutes and records the signature in Rekor; `cosign attest` attaches the CycloneDX SBOM and SLSA provenance. In the cluster, an admission policy verifies the signature, the expected workflow identity and issuer. An image pushed by hand, or by anyone else, is rejected. Sign and deploy by **digest**, not by mutable tag.

COMMON MISTAKES

- Floating versions (`^1.2`, `latest`) so builds change silently.
- Auto-merging every dependency update without review or tests.
- Long-lived cloud keys stored as CI secrets, readable by any workflow.
- Signing images but never verifying signatures at deploy time.

6 Key terms

Supply chain attack: compromise via a dependency, tool or pipeline

SBOM: list of all components and versions in an artifact

Typosquatting: malicious package with a look-alike name

Provenance: signed record of how and from what an artifact was built

SLSA: framework of build-integrity levels

Sigstore / cosign: keyless signing with short-lived certs + transparency log

7 Interview questions

Q1. What does an SBOM give you? BASIC

A list of every component and version in an artifact, so when a new CVE lands you know within minutes which services are affected.

Q2. Why pin dependencies with a lock file? BASIC

Builds become reproducible and nothing changes silently; a malicious new version can't slip in unless someone updates and reviews the lock file.

Q3. How do you secure your software supply chain? SCENARIO

Pin and scan dependencies, review automated updates, generate an SBOM per build, sign images in CI and verify at deploy, record SLSA provenance, and harden CI with short-lived credentials, pinned actions, required reviews and ephemeral runners.

Q4. A critical CVE is announced in a popular library. What do you do first? SCENARIO

Query SBOMs to find affected services and versions, prioritise internet-facing ones, patch or mitigate (WAF rule, config), rebuild and redeploy signed images, and verify.

Q5. Why pin a GitHub Action to a commit SHA rather than a version tag? TRICKY

Tags are mutable: whoever controls the action's repo can move v4 to malicious code. A full SHA is immutable, so you run exactly what you reviewed.

Q6. If the source code is reviewed, why isn't that enough? TRICKY

The build can inject code the source never contained (SolarWinds), and dependencies change underneath you. Provenance and signatures prove the artifact came from that reviewed source through a trusted builder.

Next: SD-N-001 · Observability Fundamentals backendroadmap.com

Observability Fundamentals

monitoring says that, observability finds why

Watch the 4-min video

In one line: monitoring checks the failure modes you predicted and tells you *that* something is wrong; observability lets you ask *new* questions of the logs, metrics and traces you already emit, so you can find out *why*, without shipping new code.

1 Every dashboard is green

It's 3 a.m. Checkout is slow, but only for some users. CPU, memory and error rate all look fine. Your dashboards answer the questions someone predicted when they built them. This problem is one nobody predicted. The test of an observable system: can you explain it from the outside, using data it already produces, without adding a log line and redeploying?

2 Monitoring vs observability

	Monitoring	Observability
Question	Is a known condition true? (up? error rate > 2%? disk full?)	Why is this happening? (questions you didn't plan)
Built from	predefined checks, thresholds, dashboards	rich, linked telemetry you can slice by any field
Good for	known-unknowns, paging on symptoms	unknown-unknowns, debugging novel failures
Output	an alert: *something is wrong*	an explanation: *what, where and why*

You need both. Monitoring is a subset of what good telemetry gives you: alerts (lesson 67) tell you to look; observability lets you find the answer.

3 The three pillars, and how they link

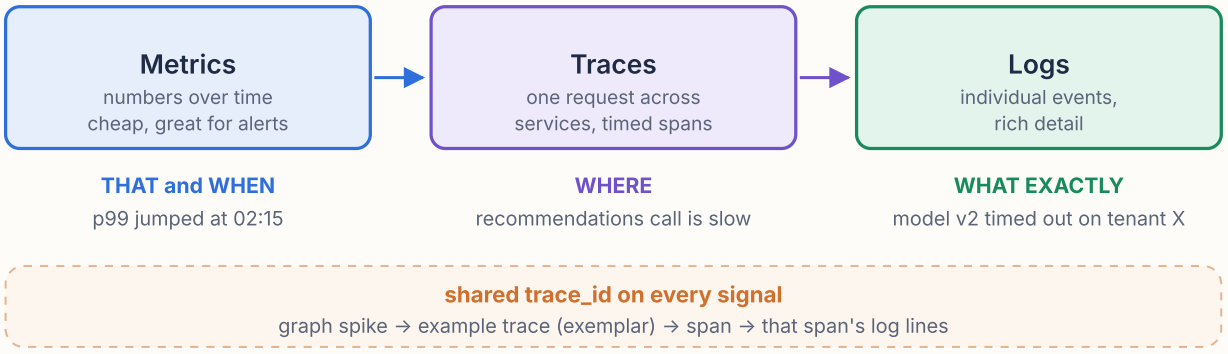


Figure 1. Each signal answers a different question. A shared trace ID turns three separate tools into one investigation.

Signal	What it is	Strength	Weakness	Common tools
Metrics	numeric aggregates over time (rate, p99)	cheap, fast queries, alerting	no per-request detail	Prometheus, Mimir (lesson 62)
Traces	one request's path as a tree of timed spans	shows where time went across services	usually sampled	Jaeger, Tempo (lesson 63)
Logs	timestamped records of single events	full detail and context	volume and cost	Loki, Elasticsearch (lesson 66)

Many teams add **wide events** (one rich record per request) and **continuous profiling** (where CPU and memory go, down to the function) as further signals. The value comes from **linking**: Prometheus *exemplars* attach a trace ID to a histogram sample, so you can click from a latency spike to a real slow trace, then from a span to the log lines that share its trace ID (lesson 65).

4 Cardinality: the limit on metrics

Cardinality is the number of distinct values a field can take. `status` has a handful; `user_id` has millions. A metrics database stores **one time series per unique combination of label values**, so label cardinalities multiply.

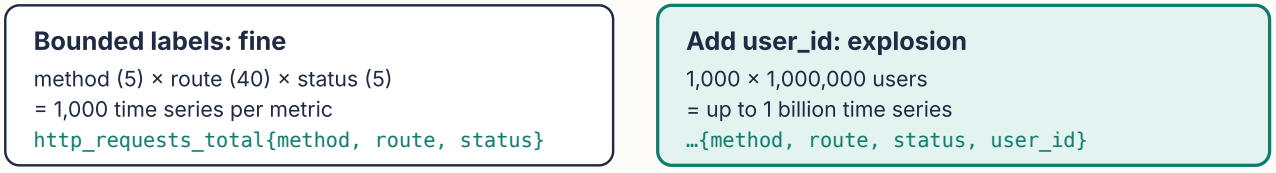


Figure 2. Labels multiply. One unbounded label turns a cheap metric into a memory and cost problem.

The rule: keep metric labels **bounded** (method, route template, status class, region). Put high-cardinality fields such as user ID, request ID and tenant in **logs and traces**, which store each event separately and are built to be searched by them.

5 Building observability in

- **Instrument once** with a vendor-neutral standard, OpenTelemetry (lesson 64): auto-instrumentation gives you HTTP, database and queue spans for free.
- **Emit structured, wide events** with context: tenant, region, service version, feature flag, build ID (lesson 60).
- **Propagate one trace ID** everywhere, including logs and async messages.
- **Keep it queryable** by dimensions you didn't plan: that is exactly what an unknown problem needs.

WORKED EXAMPLE: AN UNEXPLAINED LATENCY SPIKE

p99 checkout latency jumps from **300 ms to 2 s**, and no alert explains it. (1) Open the p99 graph and click an exemplar into a slow trace. (2) The waterfall shows almost all the time in one call: the **recommendations** service. (3) Compare slow traces with fast ones: every slow span carries the attribute `flag=new_ranking`, model version 2. (4) Turn the flag off; p99 returns to 300 ms. No new logging code was needed, because the events already carried the flag:

```
{"event": "checkout", "tenant": "acme", "version": "2.4.0", "flag": "new_ranking", "trace_id": "4bf9…"}
```

COMMON MISTAKES

- Equating observability with "more dashboards": dashboards only answer predicted questions.
- Putting user ID, request ID or full URLs in metric labels, then wondering why Prometheus runs out of memory.
- Three disconnected tools with no shared trace ID, so every investigation starts from scratch in each one.
- Thin events without context (no version, tenant or flag), so you can't slice by the thing that actually changed.

6 Key terms

Monitoring: checking predefined conditions and alerting on them

Telemetry: the logs, metrics, traces and profiles a system emits

Exemplar: a trace ID attached to a metric sample

Observability: ability to explain internal state from external outputs

Cardinality: number of distinct values of a field or label set

Wide event: one structured record per unit of work, with many fields

7 Interview questions

Q1. How is observability different from monitoring? BASIC

Monitoring checks known conditions and tells you something is wrong; observability lets you ask new questions about unknown problems from the telemetry you already emit, so you can find out why.

Q2. What are the pillars of observability? BASIC

Logs, metrics and traces (often plus events and profiles), linked by a shared trace ID, with high-cardinality detail kept in logs and traces rather than metric labels.

Q3. Users report slowness but every dashboard is green. What do you do? SCENARIO

Look at the user-facing symptom directly (p99 by route, region, tenant), jump to slow example traces, find the span that dominates, then compare its attributes with fast requests to see what they have in common.

Q4. A teammate wants a `customer_id` label on the request-latency metric. Your answer? SCENARIO

No: it creates one series per customer and can explode memory and cost. Keep customer ID on spans and logs, and use metrics labels like route and status class.

Q5. If you have great logs, why do you need metrics and traces? TRICKY

Logs are expensive to aggregate over time and don't show causal paths across services; metrics give cheap trends and alerts, traces show where time went in one request.

Q6. Is a system with 500 dashboards observable? TRICKY

Not necessarily: dashboards encode predicted questions. Observability depends on rich, high-cardinality, linked telemetry you can query in new ways.

Next: SD-N-002 · Structured Logging backendroadmap.com

Structured Logging

each log line is data, not prose

Watch the 4-min video

In one line: write every log line as machine-readable fields (usually JSON) with a level and automatic context such as request, tenant and trace IDs, so logs can be filtered, counted and joined across services; and never let secrets or sensitive personal data into them.

1 Regex archaeology at 2 a.m.

An incident is running. You need every error for one customer in the last hour. The logs say `user did something, failed, retrying`, each team in its own format. You write regular expressions that miss half the lines. Free text is written for a human reading one line; during an incident you need a machine to search millions.

2 Each log line is data

Free text	Structured (JSON)
`Order 991 failed for acme after 2300ms`	`{"level":"error","event":"order failed","order_id":991, "tenant_id":"acme","duration_ms":2300,"route":"/checkout"}`
search = regex, breaks when wording changes	search = query: `tenant_id=acme AND level=error`

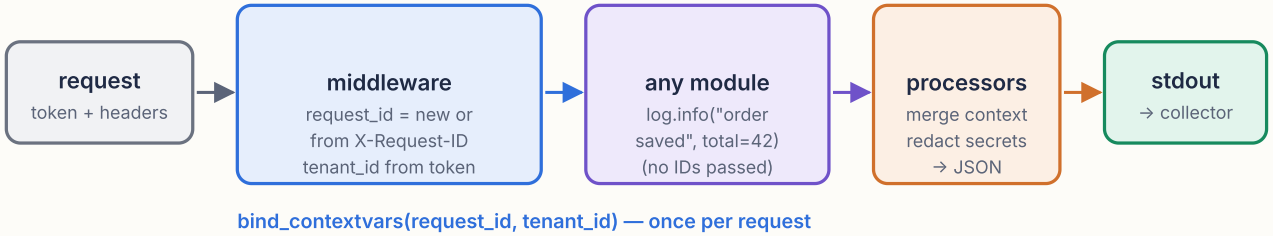
A good line has a **timestamp** (UTC, ISO 8601), a **level**, a short constant **event** name, and named fields. Keep the event name constant (`"order failed"`) and put the variable parts in fields, so identical events group together.

3 Levels, with discipline

Level	Means	Example	In production
DEBUG	developer detail	cache key computed	off; switch on per service when needed
INFO	normal, important event	request completed, job finished	on, often sampled for noisy paths
WARNING	unexpected, but coped	retry succeeded, fallback used	on
ERROR	a request or job failed	payment call failed after retries	on, always kept

If everything is ERROR, the level means nothing and alerts built on it become noise. A 404 from a user typo is not an error of *your* service.

4 Context on every line, automatically



Result on every line, from every module:

```
{"level": "info", "event": "order saved", "total": 42, "request_id": "r-81c", "tenant_id": "acme"}
```

Figure 1. Middleware binds the IDs once; every log call in that request, in any module, inherits them. Processors add level and time, drop secrets and render JSON.

Add **request ID, trace ID, tenant ID, user ID, service name and version** to every line using middleware and context variables, never by hand in each call. In Python, **structlog** (contextvars) or the standard `logging` module with a JSON formatter and a filter both work. With OpenTelemetry, the trace and span IDs can be injected too, so you can jump from a trace to its logs (lesson 65). Write to stdout and let the platform ship the stream (the Twelve-Factor App rule; lesson 66).

5 What never to log

Passwords, session tokens, API keys, JWTs, full card numbers, and raw request bodies that may contain them; also sensitive personal data beyond what you need (national IDs, health data). Logs are copied to many systems, kept for months and read by many people (OWASP Logging Cheat Sheet). Enforce it in the pipeline: a processor or filter that drops or masks known keys (`password`, `authorization`, `card`), so one careless line can't leak them. Prefer allow-lists of fields over logging whole objects.

6 Volume and cost

- **Sample** low-value logs (successful health checks, 200s on hot paths); keep **every** error.
- **Retention per level:** e.g. errors 30–90 days, debug a few days.
- **One rich line per request** with all context usually beats twenty thin lines.
- **Runtime-switchable** debug logging, so you can turn it on for one service during an incident without a deploy.

```
structlog.configure(processors=[merge_contextvars, add_log_level,
    TimeStamper(fmt="iso"), drop_secrets, JSONRenderer()])   # drop_secrets: yours

@app.middleware("http")
async def log_context(request, call_next):
    clear_contextvars()
    bind_contextvars(request_id=request.headers.get("x-request-id") or uuid4().hex,
                     tenant_id=tenant_from_verified_token(request))
    return await call_next(request)
```

Figure 2. The video's setup (structlog, names imported from structlog.contextvars and structlog.processors): JSON output plus FastAPI middleware that binds request and tenant IDs.

WORKED EXAMPLE: SUPPORT TICKET FROM TENANT ACME

The FastAPI middleware above generates (or accepts) a request ID and reads the tenant from the **verified** token, not from a client header the caller could forge. Every line in that request now carries both. Support receives "checkout failed around 14:00" from acme and runs `tenant_id=acme AND level=error` for 13:45–14:15. In seconds they see the failed requests, each with its `request_id`; searching one of those IDs pulls up that request's whole story across modules: a query, not a regex hunt.

COMMON MISTAKES

- Logging the whole request or user object "for debugging" and leaking tokens or personal data.
- Variable data in the message string, so identical events can't be grouped.
- Passing request IDs by hand through every function: someone forgets, and lines go missing from the story.
- Logging expected client errors (404, validation) at ERROR, drowning the real failures.

7 Key terms

Structured log: log line as named fields, usually JSON

Log level: severity: DEBUG, INFO, WARNING, ERROR

Context variables: per-request values merged into every line

Request ID: unique ID for one request, on every line

Redaction: dropping or masking sensitive fields in the pipeline

Sampling: keeping only a fraction of low-value logs

8 Interview questions

Q1. Why structured logs? BASIC

Machine-readable fields make logs searchable, filterable and joinable across services, especially with request and trace IDs on every line.

Q2. What should never be logged? BASIC

Passwords, tokens, API keys, full card numbers and other sensitive personal data; mask or drop them in the logging pipeline, not by convention alone.

Q3. How do you get the request ID onto every log line without changing every function? SCENARIO

Bind it once in middleware into context variables (structlog contextvars, or a logging filter reading a ContextVar); the formatter merges it into each line.

Q4. Your logging bill doubled. What do you cut first? SCENARIO

Sample successful high-volume lines like health checks, merge many thin lines into one per request, shorten retention for low levels, but keep all errors.

Q5. Should the tenant ID in logs come from the X-Tenant header? TRICKY

Not if a client controls it: take it from the verified token or session, or an attacker can make their activity appear under another tenant.

Q6. Is a 404 an ERROR? TRICKY

Usually not for your service: it's a correct response to a bad request. Log it at INFO or WARNING and keep ERROR for failures you must fix.

Next: SD-N-003 · Metrics Types and Design backendroadmap.com

Metrics Types and Design

the average hides the users who leave

Watch the 4-min video

In one line: use counters for things that happen, gauges for levels and histograms for latency; read latency as percentiles, not averages; measure RED for services and USE for resources; and keep labels bounded, because every label combination is a separate time series.

1 The average that lied

The dashboard says average latency is **120 ms**. But one request in a hundred takes **4 seconds**, and those users are leaving. Latency has a long tail, and an average blends the tail into the crowd. Choosing the right metric type, and reading it the right way, decides whether your graphs tell the truth.

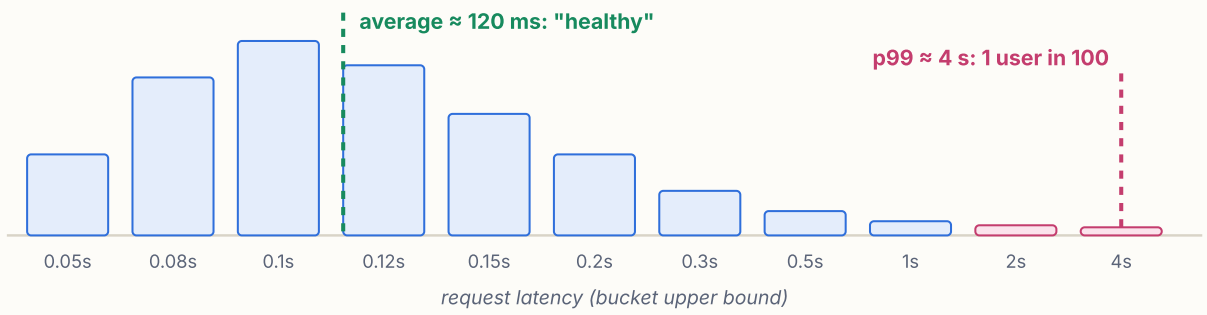


Figure 1. A long-tailed latency distribution. The mean sits in the crowd; p99 shows what the slowest 1% experience.

2 Four metric types

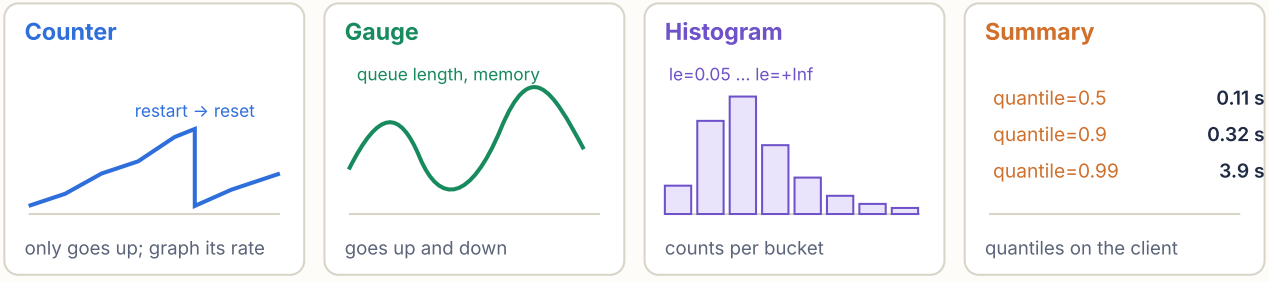


Figure 2. Counters only increase (they reset to zero on restart; `rate()` handles that). Histograms count observations per bucket; summaries ship precomputed quantiles.

Type	Use for	Read it as	Example name
Counter	events: requests, errors, bytes sent	rate per second	`http_requests_total`
Gauge	current level: memory, queue depth, connections	value now, min/max over time	`queue_messages`
Histogram	distributions: latency, payload size	percentiles from buckets	`http_request_duration_seconds`
Summary	quantiles for one instance	pre-computed p50/p99	(rarely the best choice)

3 Why histograms beat averages (and summaries)

A histogram counts observations into buckets. In Prometheus the buckets are **cumulative**: `le="0.3"` counts every request that took ≤ 0.3 s. From the buckets you estimate any percentile: p50 is the typical request, p99 the slowest one in a hundred. Crucially, bucket counts can be **summed across instances**, giving a true fleet-wide p99. A summary computes quantiles inside each process, and you **cannot average quantiles**: the mean of ten instances' p99 is not the fleet's p99. That is why histograms are usually preferred. The price: a percentile from buckets is an estimate, only as precise as the bucket boundaries near it.

4 What to measure: RED and USE

Method	For	Signals	Example
RED (Tom Wilkie)	request-driven services	**R**ate, **E**rrors, **D**uration	req/s, 5xx ratio, p99 per route
USE (Brendan Gregg)	resources: CPU, disk, pools, queues	**U**tilization, **S**aturation, **E**rrors	CPU 70%, run-queue length, disk errors

Google's "four golden signals" (latency, traffic, errors, saturation) are the same idea.

5 The cardinality problem

Each unique combination of label values is a **separate time series**, with its own memory and storage. Method × route template × status class is a few hundred series. Add `user_id` or `request_id` and you create millions: memory explodes, queries crawl, the bill jumps. And a histogram multiplies again by its bucket count (10 buckets = 10+ series per combination). Keep labels bounded; put unbounded IDs in logs and traces (lesson 59).

6 Design habits

- **Name with units**, base units: `http_request_duration_seconds`, `_bytes`, counters end in `_total`.
- **Route template, not raw path:** `/orders/{id}`, not `/orders/98123` (that is a hidden ID label).
- **Buckets around your targets:** if the SLO is 300 ms, have boundaries at e.g. 0.1, 0.3, 0.5 s.
- **Alert on symptoms** users feel (error ratio, p99), not on every internal number (lesson 67).

WORKED EXAMPLE: THE EXPORT ENDPOINT NOBODY NOTICED

A service records `http_request_duration_seconds{method, route, status_class}` with buckets 0.1 · 0.3 · 0.5 · 1 · 2.5 s. From that one histogram (PromQL in lesson 62): request rate, error ratio, and p50/p95/p99 per endpoint. `GET /orders/export` has a fine p50 but a **p99 of 8 s**; the service-wide average had hidden it for months, because export is a small share of traffic. It gets its own fix (streaming the export) and its own p99 alert.

COMMON MISTAKES

- Graphing average latency, or averaging per-instance p99s.
- Graphing a counter's raw value instead of its rate, so restarts look like drops.
- Raw URL paths or user IDs as labels: unbounded cardinality.
- Default buckets that don't bracket your SLO, so p99 can't be read accurately where it matters.

7 Key terms

Counter: monotonically increasing count; read as a rate

Gauge: value that can go up or down

Histogram: bucketed counts of observations; aggregatable

Percentile (p99): value 99% of observations are below

RED / USE: rate-errors-duration / utilization-saturation-errors

Time series: one metric name + one exact label set

8 Interview questions

Q1. Why are histograms better than averages? BASIC

They capture the whole distribution including the long tail, give percentiles like p99, and their buckets can be summed across instances (summaries can't).

Q2. What is the cardinality problem? BASIC

Every unique label combination is a separate time series, so unbounded labels like user ID or request ID explode memory, query time and cost.

Q3. Which metrics would you put on a new API service's dashboard? SCENARIO

RED per route: request rate, error ratio, p50/p99 latency from a histogram; plus USE for its resources: CPU, memory, connection-pool saturation.

Q4. Ten pods each report p99 = 200 ms via a summary. What is the fleet p99? SCENARIO

You can't tell: quantiles don't average. It could be well above 200 ms if one pod is slow; use a histogram and compute p99 over summed buckets.

Q5. Why does a counter reset not break your graphs? TRICKY

rate() and increase() detect a decrease in a counter and treat it as a reset, so the per-second rate stays correct.

Q6. Is a gauge of 'requests in the last minute' a good metric? TRICKY

No: it depends on scrape timing and can't be re-aggregated. Expose a counter and let the query compute rate over any window.

Next: SD-N-004 · Prometheus and Grafana backendroadmap.com

Prometheus and Grafana

pull, query, alert, draw

Watch the 4-min video

In one line: Prometheus pulls metrics from each service's `/metrics` endpoint, stores them as time series and evaluates PromQL rules; Alertmanager turns firing rules into grouped, routed notifications; Grafana draws dashboards from the same queries.

1 Fifty services, one metrics stack

You need metrics from fifty services, alerts that page the right people, and dashboards the whole team understands. The most common open-source answer is Prometheus to collect and query, and Grafana to show. Prometheus is a CNCF graduated project, and the pair is the default metrics stack in most Kubernetes clusters; the concepts (scrape, PromQL, rules, routing) carry over to managed versions.

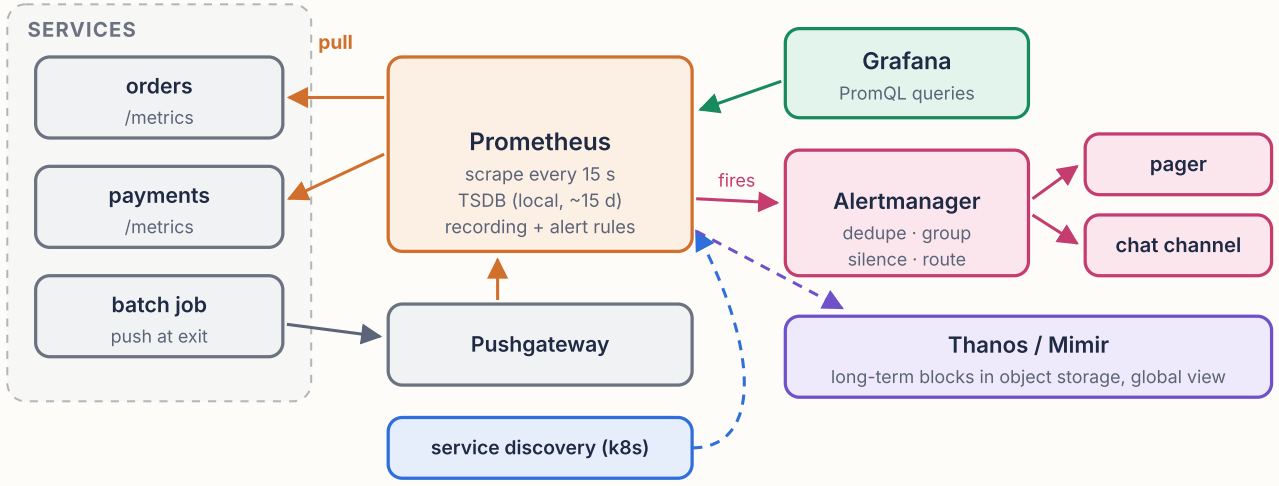


Figure 1. Prometheus scrapes targets found by service discovery, evaluates rules, sends alerts to Alertmanager; Grafana queries it; Thanos or Mimir keep long-term history.

2 Pull-based scraping

Each service exposes a plain-text endpoint, usually `/metrics`, listing its counters, gauges and histograms (lesson 61). On an interval (the config default is 1 minute; **15 s** is the common setting) Prometheus scrapes every target and stores the samples in its local time-series database (default retention 15 days). Targets come from **service discovery**, e.g. the Kubernetes API, so new pods are picked up automatically.

	Pull (Prometheus)	Push (StatsD, OTLP push)
Dead target	visible: `up == 0`	silence looks like "no traffic"
Debugging	open `/metrics` in a browser	inspect what the agent received
Short-lived jobs	need the Pushgateway	natural fit
Behind NAT / firewalls	server must reach target	target only needs outbound

The Pushgateway is only for batch jobs that exit before a scrape: it holds their last values. It is not a general push pipeline.

3 PromQL: rate, sum by, recording rules

A counter is turned into a per-second **rate** over a window: `rate(x[5m])`. Use a window of several scrape intervals (commonly ≥ 4×). `sum by (service)` aggregates away every other label. Expensive queries become **recording rules**: evaluated on a schedule and stored as a new series (`job:http_errors:ratio5m`), so dashboards and alerts stay fast.

```
# error ratio per service, last 5 minutes
sum by (service) (rate(http_requests_total{code=~"5.."}[5m]))
  / sum by (service) (rate(http_requests_total[5m]))

# p99 latency per route (the Grafana panel in the example)
histogram_quantile(0.99,
  sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))

# alerting rule
- alert: HighErrorRatio
  expr: job:http_errors:ratio5m > 0.02      # recording rule, precomputed
  for: 10m
  labels: {severity: page}
  annotations: {runbook_url: https://wiki/runbooks/high-errors}
```

Figure 2. The three queries you will write most: error ratio, p99 from a histogram, and an alerting rule with a duration and a runbook.

4 Alerting rules and Alertmanager

An alerting rule is a PromQL condition plus a duration: `for: 10m` means the condition must hold for 10 minutes before it fires, which filters blips. Prometheus sends firing alerts to **Alertmanager**, which **deduplicates** (two Prometheus replicas, one notification), **groups** related alerts (`group_by: [service]`, defaults `group_wait 30s`, `repeat_interval 4h`), applies **silences** during maintenance and **inhibitions** (suppress "high latency" when "cluster down" fires), and **routes** by label: `severity=page` to the pager, warnings to a chat channel (lesson 67).

5 Grafana and long-term storage

Grafana queries Prometheus and draws time series, heatmaps (great for histograms) and single stats. **Variables** such as `$service` and `$region` let one dashboard serve every service. Keep dashboards as JSON **in Git**, next to the service. One Prometheus server is a single node with limited local history, so for a global view across clusters and months of data, teams add **Thanos** or **Grafana Mimir**, which store blocks in object storage (S3, GCS, Azure Blob) and query across them.

WORKED EXAMPLE: A P99 LATENCY PANEL

The service exposes `http_request_duration_seconds` as a histogram. The panel query (Figure 2): take `rate()` of the `_bucket` series over 5 minutes, `sum by (le, route)` (you must keep `le`, the bucket boundary, or the quantile can't be computed), then `histogram_quantile(0.99, …)`. Result: one p99 line per route. A red threshold at **500 ms** shows which routes break the target; the same expression, as a recording rule, feeds the alert.

COMMON MISTAKES

- `sum by (route)` without `le` before `histogram_quantile`: the result is meaningless.
- `rate()` over a window shorter than two scrapes, giving gaps or empty results.
- Using the Pushgateway for long-running services: you lose `up` and keep stale values forever.
- Alerts with no `for:` duration, so every 15-second blip pages someone.

6 Key terms

Scrape: Prometheus pulling /metrics from a target

PromQL: Prometheus query language

Alertmanager: dedupes, groups, silences and routes alerts

Service discovery: finding targets automatically (e.g. Kubernetes)

Recording rule: precomputed query stored as a new series

Thanos / Mimir: long-term, global Prometheus storage

7 Interview questions

Q1. How does Prometheus collect metrics? BASIC

It pulls them: it scrapes each target's /metrics endpoint on an interval (often 15 s), with targets found through service discovery; short-lived jobs use the Pushgateway.

Q2. Write a PromQL query for error rate. BASIC

```
sum(rate(http_requests_total{code=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))
```

Q3. Two Prometheus replicas scrape the same targets for HA. Will on-call get two pages? SCENARIO

No: both send to Alertmanager, which deduplicates identical alerts (and an Alertmanager cluster gossips so it notifies once).

Q4. Dashboards with 30 days of p99 across 50 services load slowly. Fix? SCENARIO

Precompute the per-service histogram aggregations with recording rules, and serve long ranges from Thanos or Mimir with downsampling.

Q5. Why is pull useful for detecting failures? TRICKY

Prometheus knows which targets should exist, so a target that stops answering produces `up == 0`; with push, silence looks like no traffic.

Q6. Why can't you just average per-pod p99 in Grafana? TRICKY

Quantiles don't average. Sum the histogram buckets across pods by `le`, then apply histogram_quantile once.

Next: SD-N-005 · Distributed Tracing

Distributed Tracing

spans, propagation, and which traces you keep

Watch the 4-min video

In one line: a trace is a tree of timed spans sharing one trace ID, stitched together by context propagated in headers; because keeping every trace is expensive you sample, and the choice between head and tail sampling decides whether you keep the slow and failed requests you actually need.

1 Two seconds, six hops, one slow call

A request takes 2 seconds. It touched a gateway, three services, a cache and a database. Somewhere one call is slow. Logs from six places with no common thread won't tell you which. A trace will, if you understand how it is built and how much of it you keep. (Lesson 36 covered tracing from the architecture side; this lesson goes inside the data.)

2 The data model: traces and spans

A **trace** is the whole journey of one request, identified by a 128-bit **trace ID**. It is made of **spans**: one timed operation each (an HTTP handler, an outgoing call, a DB query). A span records name, start, duration, status, attributes (e.g. `db.statement`, `http.response.status_code`), events, its own **span ID** and its **parent span ID**. Parent links turn spans into a tree, drawn as a waterfall.

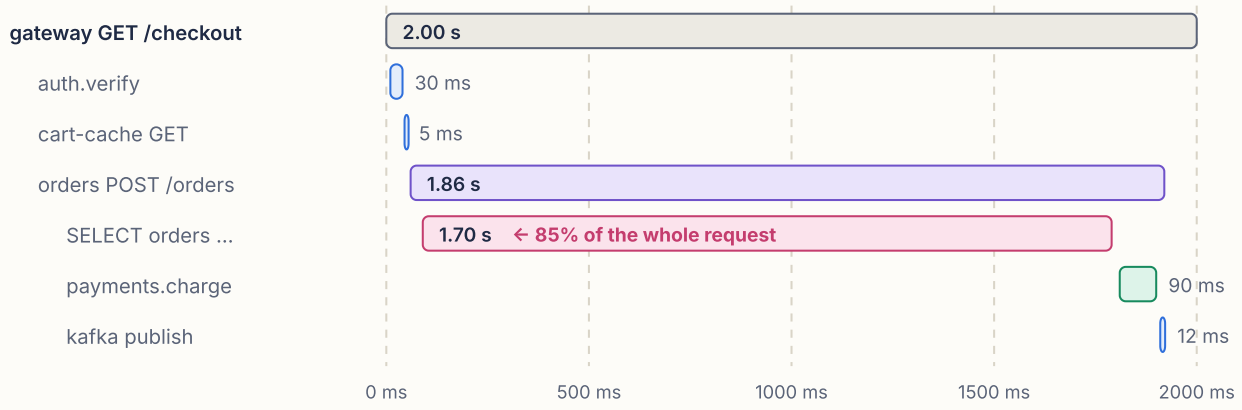


Figure 1. A trace waterfall. Indentation is the parent/child tree; bar length is duration. One DB span dominates the 2-second root span.

3 Context propagation

When a service calls another, it sends the trace ID and its current span ID in a header, standardised as the W3C **traceparent** header: `00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01` = version, trace ID (32 hex), parent span ID (16 hex), flags (`01` = sampled). The receiver starts a child span under that parent. Inside a process the current span lives in **context** (thread-local or async context), so instrumented libraries attach their spans automatically. Queues and background jobs need it carried in message headers (lesson 65).

4 Sampling: head vs tail

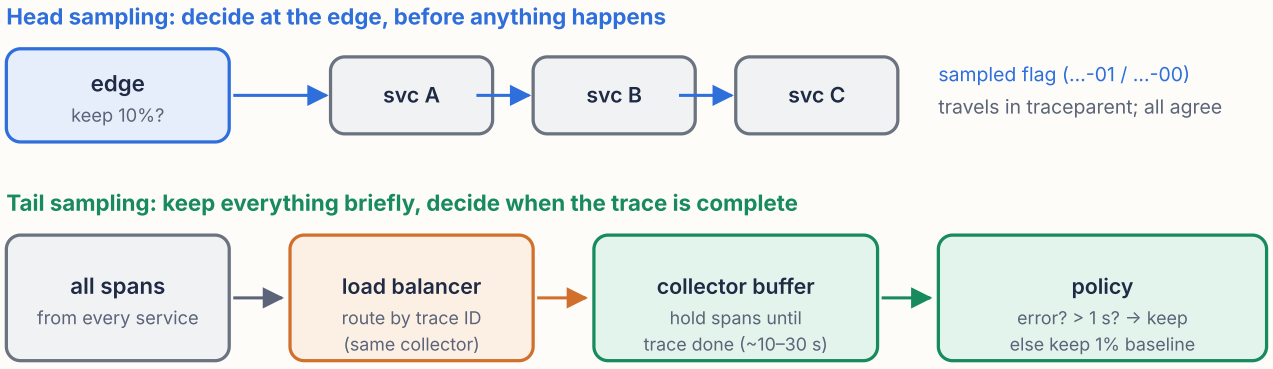


Figure 2. Head sampling decides blind but cheaply; tail sampling decides with full knowledge but must buffer every span, grouped by trace ID.

	Head sampling	Tail sampling
Decided	at the first span, before the outcome is known	after the trace completes
Keeps	a random X% (e.g. 10%)	errors, slow traces (> threshold), plus a small baseline
Cost	low: unsampled spans are never exported	high: every span sent to collectors and held in memory
Consistency	decision travels in the flags, all services agree	all spans of a trace must reach the same collector
Weakness	throws away most rare, interesting traces	more infrastructure; late spans can miss the decision

With 10% head sampling, an error that hits 0.1% of requests leaves you about one example trace per 10,000 requests. Tail sampling (e.g. the OpenTelemetry Collector's `tail_sampling` processor behind a `loadbalancing` exporter that routes by trace ID) keeps nearly all of them. Many teams combine both: light head sampling to cut volume, tail policies to keep what matters.

5 Trace backends

Open source: **Jaeger**, **Grafana Tempo** (object-storage based, cheap), Zipkin. Managed: Azure Monitor Application Insights, AWS X-Ray, Google Cloud Trace, Datadog, Honeycomb. Most accept OpenTelemetry data (lesson 64), so you can switch without re-instrumenting. Look for search by attribute, comparing slow and fast traces, service maps, and jumping from a span to its logs.

WORKED EXAMPLE: THE MISSING INDEX

A checkout request takes 2 s. In the waterfall (Figure 1) the root span is 2 s; most children are short, but one span, `SELECT … FROM orders WHERE customer_id = ?` from the orders service, takes **1.7 s**. Its attributes show the statement and table. `EXPLAIN` confirms a full table scan: `customer_id` has no index. After adding one, the span drops to **20 ms** and the request to about 300 ms.

COMMON MISTAKES

- One un-instrumented hop (a proxy, a queue) that drops the header: the trace splits into two unrelated traces.
- Head sampling at 1% and then wondering why there is no trace for the failed request.
- Putting full SQL with literal values or personal data into span attributes.
- Tail sampling without trace-ID-aware routing, so each collector sees only fragments.

6 Key terms

Trace: all spans of one request, sharing a trace ID

Span: one timed operation with parent link and attributes

traceparent: W3C header: version-traceid-parentid-flags

Head sampling: keep/drop decided at the start

Tail sampling: keep/drop decided after the trace completes

Waterfall: timeline view of a trace's span tree

7 Interview questions

Q1. What links spans into a trace? BASIC

A shared trace ID and parent span IDs, carried between services in propagation headers such as W3C traceparent.

Q2. Head vs tail sampling: what are the trade-offs? BASIC

Head sampling is cheap, simple and consistent but decides blindly, so it drops most slow and failed traces; tail sampling keeps errors and slow traces but needs collectors that buffer every span, routed by trace ID.

Q3. Traces show the frontend span but nothing downstream. Why? SCENARIO

Context isn't propagated: a hop (proxy, async queue, custom HTTP client) isn't instrumented or strips traceparent, so downstream spans start new traces.

Q4. Tracing every request costs too much. How do you cut cost without losing incidents? SCENARIO

Use tail sampling: keep all errors and requests above a latency threshold plus a 1–5% baseline; optionally add light head sampling to cut raw volume first.

Q5. Why must tail sampling route by trace ID? TRICKY

The decision needs the whole trace; if spans of one trace land on different collectors, none sees enough to decide, and the kept trace is incomplete.

Q6. A child span is longer than its parent. Is that a bug? TRICKY

Not necessarily: async work (fire-and-forget, a queued job) can outlive the parent; otherwise suspect clock skew between hosts.

Next: SD-N-006 · OpenTelemetry backendroadmap.com

OpenTelemetry

instrument once, send anywhere

Watch the 5-min video

In one line: OpenTelemetry is the vendor-neutral standard for telemetry: one set of APIs, SDKs and semantic conventions for traces, metrics and logs, one wire protocol (OTLP), and a Collector that receives, processes and exports data, so the backend becomes a config choice, not a code change.

1 Three vendors, three agents

Your team uses one vendor for logs, another for metrics and a third for traces. Each needs its own agent and library wired into every service. Switching vendors means re-instrumenting everything, so nobody switches. OpenTelemetry (OTel, a CNCF project formed in 2019 from the merger of OpenTracing and OpenCensus) fixes that: instrument once, send anywhere.

2 What OpenTelemetry standardizes

- **API**: what your code and libraries call (start a span, record a metric). Safe to depend on; a no-op without an SDK.
- **SDK**: the implementation per language: sampling, batching, exporting.
- **OTLP**: one protocol for all signals, over gRPC (port 4317) or HTTP (port 4318).
- **Semantic conventions**: standard attribute names, so data looks the same from every language.
- **Collector**: a separate process that receives, processes and exports telemetry.

It is supported by every major vendor and cloud. OTel does not store or display data: that is still the job of a backend.

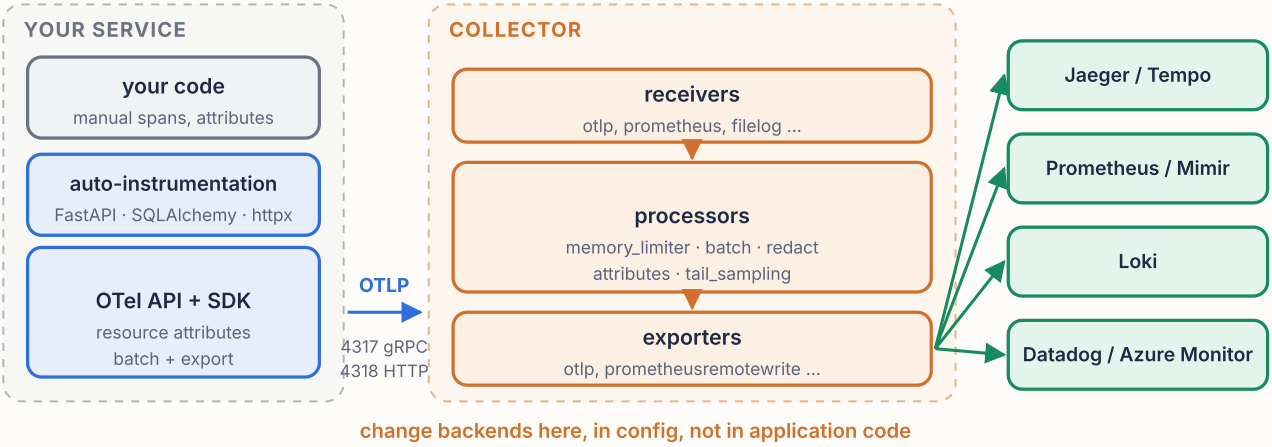


Figure 1. The SDK exports OTLP to a Collector; the Collector's pipeline of receivers, processors and exporters fans out to any backend.

3 Auto vs manual instrumentation

	Auto-instrumentation	Manual instrumentation
How	wrappers around popular libraries, no code changes	API calls in your code
Gives you	server spans, DB queries, HTTP clients, queues; propagation	business steps ("pricing"), domain attributes (plan, tenant)
Effort	install packages, start with a wrapper or agent	a few lines where it matters

Start with auto; it gets you surprisingly far. Then add manual spans and attributes where the questions are about your domain.

4 The Collector

Receivers accept OTLP and other formats (Prometheus scrape, Jaeger, log files). **Processors** batch, limit memory, drop noisy attributes, **redact secrets** and **sample traces** (tail sampling, lesson 63). **Exporters** send to Jaeger, Tempo, Prometheus, Loki, Datadog, Azure Monitor, often several at once. Run it as an **agent** on each node (low-latency hand-off, adds host metadata), as a central **gateway** (sampling, keys and routing in one place), or both.

5 Semantic conventions and resources

Semantic conventions fix names such as `service.name`, `http.request.method`, `http.response.status_code` and `db.system`. Every library in every language uses them, so one dashboard or alert works across Python, Java and Go services. **Resource attributes** describe the emitter: always set `service.name`, `service.version` and the environment, or everything shows up as `unknown_service`. Upgrade SDKs regularly: the project and its conventions move fast.

```
pip install opentelemetry-distro opentelemetry-exporter-otlp
opentelemetry-bootstrap -a install          # adds fastapi, sqlalchemy, httpx instrumentations
OTEL_SERVICE_NAME=orders \
OTEL_RESOURCE_ATTRIBUTES=service.version=2.4.0,deployment.environment=prod \
OTEL_EXPORTER_OTLP_ENDPOINT=http://collector:4317 \
opentelemetry-instrument uvicorn app:app

# one manual span where only you know what matters
tracer = trace.get_tracer(__name__)
with tracer.start_as_current_span("pricing") as span:
    span.set_attribute("customer.plan", plan)
    price = compute_price(cart, plan)
```

Figure 2. The video's FastAPI + SQLAlchemy + httpx setup: zero-code instrumentation via environment variables, plus one manual span.

WORKED EXAMPLE: INSTRUMENTING A FASTAPI SERVICE

Install the distro and the instrumentation packages, start the app under `opentelemetry-instrument`, point OTLP at the collector (Figure 2). Every request now becomes a trace: a server span for `POST /orders`, a child span per SQL query, and a client span per outgoing httpx call, with `traceparent` passed to the next service. One manual span wraps the pricing logic and tags the customer's plan. There is no vendor code in the app; switching from Jaeger to Azure Monitor is a collector config change.

TRADE-OFFS

- **Collector vs direct export:** a collector adds a hop and something to run, but centralises sampling, redaction and backend keys.
- **Auto-instrumentation** is quick but generic and adds some overhead; manual spans cost code but carry business meaning.
- **Maturity varies** by signal and language: traces are the most mature; check the status of logs and metrics for your SDK.
- **Vendor features** beyond OTel (profilers, special agents) can bring back lock-in: use them deliberately.

6 Key terms

OpenTelemetry (OTel): open standard and tooling for traces, metrics, logs

Collector: receives, processes and exports telemetry

Resource: attributes of the emitter, e.g. service.name

OTLP: OTel's wire protocol (gRPC 4317, HTTP 4318)

Semantic conventions: standard attribute names

Auto-instrumentation: library wrappers that create spans without code changes

7 Interview questions

Q1. What problem does OpenTelemetry solve? BASIC

It replaces vendor-specific agents and libraries with one open standard (APIs, SDKs, OTLP) for traces, metrics and logs, so you instrument once and send the data to any backend.

Q2. What does the Collector do? BASIC

It receives telemetry, processes it (batching, filtering, redaction, sampling) and exports it to one or more backends, so those decisions live in config instead of application code.

Q3. You must move from Jaeger to a managed backend with zero app changes. How? SCENARIO

If apps export OTLP to a collector, add the new exporter in the collector config, run both in parallel to compare, then remove the old one.

Q4. Traces show up as 'unknown_service'. Why? SCENARIO

The resource attribute service.name isn't set; set OTEL_SERVICE_NAME (and version and environment) on every service.

Q5. Is OpenTelemetry a backend? TRICKY

No: it produces, processes and transports telemetry. Storage, querying and dashboards come from Jaeger, Tempo, Prometheus or a vendor.

Q6. Why do semantic conventions matter if every service already sends data? TRICKY

Without shared names, the same field is called five things in five languages, so cross-service queries, dashboards and alerts break.

Next: SD-N-007 · Correlation and Trace IDs

Correlation and Trace IDs

one ID, carried everywhere

Watch the 4-min video

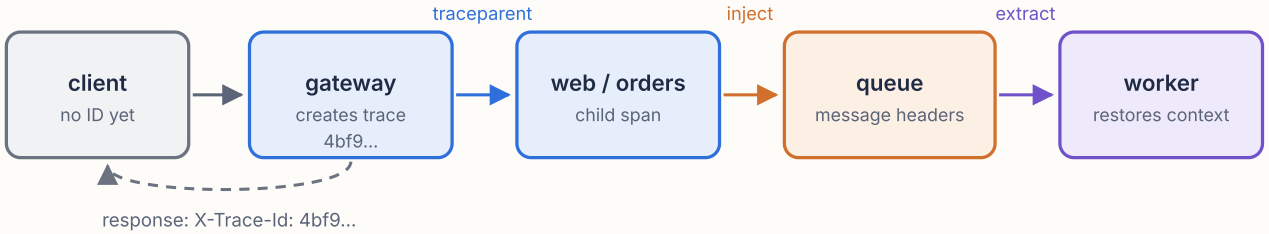
In one line: create one trace ID at the edge, propagate it on every HTTP/gRPC call, queue message and background job (the W3C `traceparent` header), and write it into every log line; then one search shows a request's whole journey, in order.

1 Which lines belong to that payment?

A customer reports a failed payment at 10:42. Five services write thousands of lines a second. Without a shared ID you grep by timestamp and hope, and the lines from concurrent requests interleave. One ID, carried everywhere, turns this into a single query.

2 Correlation ID, request ID, trace ID

A **correlation ID** (or request ID) is a unique value created where a request enters your system. Every service passes it on and every log line includes it. Today the standard carrier is the W3C **traceparent** header, which holds the **trace ID** for the whole request, the parent span ID and a sampled flag (lesson 63). OpenTelemetry creates and forwards it automatically on HTTP and gRPC calls (lesson 64). If you also have a legacy request ID, log both; return the trace ID to the client in a response header.




```
ONE SEARCH: TRACE_ID=4BF9...
10:42:01 web trace=4bf9 span=a1 POST /orders 202
10:42:03 worker trace=4bf9 span=c7 send_invoice start
10:42:04 worker trace=4bf9 span=c7 SMTP timeout · retry 1
10:42:09 worker trace=4bf9 span=d2 invoice sent
```

Figure 1. The trace ID created at the gateway follows the request through HTTP, a queue and a worker, and lands on every log line.

3 Linking logs to traces

Configure the logger to add `trace_id` and `span_id` from the current context to every structured line (lesson 60). Then you can jump from a slow span to its exact log lines, or from an error log to the full trace, in tools such as Grafana (Loki ↔ Tempo) or Kibana ↔ APM. No more matching timestamps by eye.

```
class TraceContextFilter(logging.Filter):
    def filter(self, record):
        ctx = trace.get_current_span().get_span_context()
        record.trace_id = format(ctx.trace_id, "032x")   # 32 hex chars, as in traceparent
        record.span_id = format(ctx.span_id, "016x")
        return True
# or: LoggingInstrumentor().instrument(set_logging_format=True)
```

Figure 2. A logging filter that copies the current trace and span IDs onto each record; OTel's logging instrumentation can do the same for you.

4 Async boundaries: where the trail goes cold



Figure 3. HTTP propagation is automatic once instrumented. These three need explicit care, or the instrumentation for your queue library.

5 Rules for IDs

Rule	Why
Validate client-supplied IDs (format, length) or start a new trace and **link** the old one	untrusted input can inject garbage into logs or hijack sampling
Random and meaningless (128-bit trace ID)	no user data or sequence numbers to leak or guess
Show it to users on error pages and in API error bodies	support asks for one ID and hands it straight to engineering
Same field name everywhere (`trace_id`)	one query works across all services

WORKED EXAMPLE: A CELERY TASK KEEPS THE TRACE

The web request has trace ID `4bf9…`. It enqueues `send_invoice`. The OpenTelemetry Celery instrumentation writes `traceparent` into the task's message headers on publish; the worker extracts it and starts the task span as a child. A logging filter (Figure 2) adds the trace ID to every record, so the worker's lines carry the same ID as the web request. One search for `4bf9` shows the request, the task, its SMTP timeout, the retry and the success, in order (Figure 1).

COMMON MISTAKES

- A new ID generated in every service instead of propagating the incoming one.
- Forgetting the queue: the consumer starts a fresh trace and the story splits in two.
- Putting the trace ID in a **metric label** or a Loki label: unbounded cardinality (lessons 59, 66).
- Trusting any `X-Request-ID` from the internet without validation.

6 Key terms

Correlation ID: one ID shared by everything a request causes

Trace ID: 128-bit ID of the whole trace (32 hex chars)

traceparent: W3C header carrying trace ID, parent span ID, flags

Inject / extract: write / read context into carrier headers

Span link: reference to a related trace without a parent relationship

Context: the current span, carried per thread or task

7 Interview questions

Q1. How do you follow one request through logs from five services? BASIC

Generate a trace ID at the edge, propagate it with the W3C traceparent header across every call, queue and job, and include trace_id (and span_id) in every structured log line; one search then shows the whole journey.

Q2. How do you keep the trail across a message queue? BASIC

Inject the trace context into the message headers when publishing and extract it in the consumer before processing, usually via the queue library's OTel instrumentation.

Q3. Support says 'the customer got an error'. How do you make that actionable? SCENARIO

Show the trace ID on error pages and in API error responses; support collects it and engineering searches logs and traces by it directly.

Q4. A nightly batch job processes 10,000 orders. One trace or many? SCENARIO

Usually one trace per job run (or per item for long jobs), with span links back to the triggering request or schedule, so traces stay a manageable size.

Q5. Should you accept an incoming traceparent from a public client? TRICKY

Accept only well-formed values, and many systems start a new trace at the edge and link the client's one, so outsiders can't force sampling or pollute your traces.

Q6. Why not just use timestamps to correlate logs? TRICKY

Concurrent requests interleave and clocks skew between hosts; only a shared ID reliably groups lines from one request.

Next: SD-N-008 · Log Aggregation backendroadmap.com

Log Aggregation

every log line in one searchable place

Watch the 4-min video

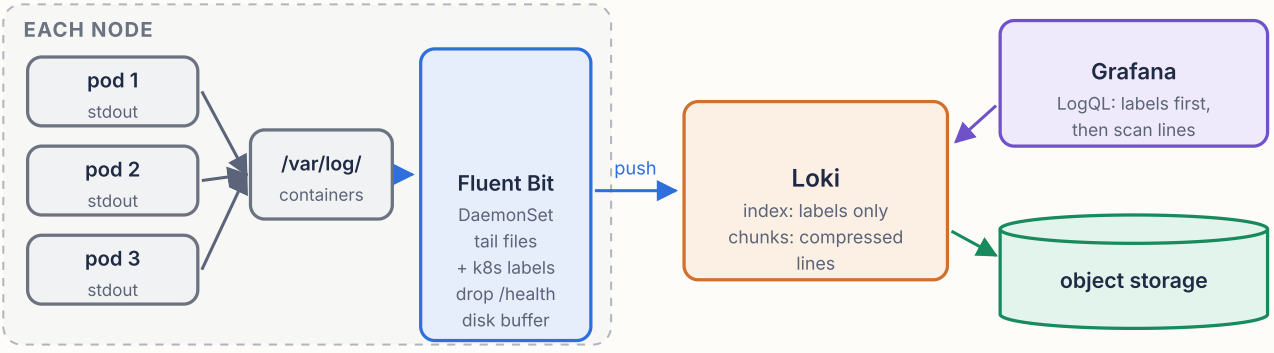
In one line: a shipper on every node collects, enriches, filters and buffers logs and sends them to a central store; choose how that store indexes (full text vs labels), cut volume at the source and tier retention, because at scale logs cost more than you expect.

1 Pods die, and their logs die with them

Fifty pods each write logs to their own node's disk. A pod crashes, Kubernetes replaces it, and its logs vanish. During an incident you're running `kubectl logs` pod by pod. Log aggregation puts every line, from every pod, in one searchable place that outlives the pods.

2 The pipeline: ship, store, search

Apps write structured JSON to **stdout** (lesson 60); the container runtime writes it to files on the node. A **shipper** runs on every node (a Kubernetes DaemonSet): **Fluent Bit**, Vector, Logstash or the OpenTelemetry Collector. It tails the files, parses, adds metadata (namespace, pod, container), filters noise and forwards to a central **store** that indexes for **search**.




```
{namespace="shop", service="orders"} |= "4bf92f35" # label selector, then line filter
```

Figure 1. Kubernetes logs to Loki: Fluent Bit tails container logs, enriches and filters them, buffers to disk and pushes to Loki, which keeps chunks in object storage.

3 The big choice: how the store indexes

Elasticsearch (ELK): full-text index
every word of every line is indexed
fast arbitrary search, aggregations
index can rival the raw data in size; RAM-heavy

Grafana Loki: label index
only a few labels indexed (namespace, app)
lines compressed in cheap object storage
filter by labels, then brute-force scan

Figure 2. Full-text indexing pays at write time for fast search; label indexing keeps writes and storage cheap and pays at query time.

	Elasticsearch / OpenSearch	Grafana Loki	Managed (Splunk, CloudWatch, Azure Monitor)
Search	any word, fast	fast if labels narrow it first	varies
Cost driver	index size, RAM, SSD	bytes scanned per query	GB ingested and retained
Ops work	high (shards, mappings, cluster)	moderate	low, more money
Best for	heavy ad-hoc search, analytics	Kubernetes logs, search by service + trace ID	teams buying time

In Loki, labels must be low-cardinality (namespace, app, level). A trace ID or user ID as a label creates a stream per value and breaks it; search for those with a line filter instead.

4 Controlling cost

Logging cost grows quietly: one debug line in a hot loop can cost more than the service. **At the source:** INFO in production, one rich line per request, sample repetitive successes. **In the shipper:** drop health-check and readiness-probe lines, throttle floods. **Find your ten noisiest sources** (by bytes per day) and fix those first. Then **tier retention**:

Tier	Typical age	Storage	Example streams
Hot	last few days	fast, indexed	everything you debug incidents with
Warm	weeks	cheaper nodes or object storage	app INFO and ERROR
Archive	months to years	object storage, cold classes	audit logs for compliance (lesson 55)

Set retention per stream: debug for days, audit for a year or more. Old logs are rarely read but always billed.

A ROBUST PIPELINE

- **Buffer to disk** in the shipper (Fluent Bit `storage.type filesystem`), so a slow store doesn't drop logs or block apps.
- **JSON logs**, so fields need no fragile regex parsing.
- **Redact** secrets and personal data before logs leave the node.
- **Monitor the pipeline itself:** alert on shipper retries, dropped records and ingestion lag.

WORKED EXAMPLE: KUBERNETES LOGS TO LOKI

Fluent Bit runs as a DaemonSet and tails `/var/log/containers/*.log`. Its **kubernetes** filter adds namespace, pod and container; a **grep** filter drops `/health` lines; a **throttle** filter caps repetitive floods. It pushes to Loki, which stores compressed chunks in object storage with retention of **14 days**. In Grafana, `{namespace="shop", service="orders"} |= "4bf92f35"` narrows by labels, then finds the trace ID (lesson 65), at a fraction of the cost of a full-text index.

5 Key terms

Shipper: per-node agent that collects and forwards logs

DaemonSet: Kubernetes object running one pod per node

Full-text index: index on every term of every line (Elasticsearch)

Label index: index on a few labels only (Loki)

LogQL: Loki query language: selector, then filters

Retention tier: hot, warm or archive storage by age

6 Interview questions

Q1. What does a log shipper do? BASIC

It runs on every node, collects logs, parses, enriches and filters them, buffers to disk and forwards them to central storage.

Q2. How do you control logging costs at scale? BASIC

Cut volume at the source (levels, sampling), drop noise like health checks in the shipper, use label-indexed storage such as Loki for most logs, and tier retention per stream.

Q3. ELK or Loki for a 200-service Kubernetes platform? SCENARIO

Loki if most searches start from service, namespace and a trace ID and cost matters; Elasticsearch if you need fast free-text search and analytics across all fields.

Q4. The log store is down for 20 minutes. What happens to logs? SCENARIO

With disk buffering, shippers queue and retry, then catch up; without it, they drop records or exert back-pressure on apps.

Q5. Why not make trace_id a Loki label? TRICKY

Each label value creates a separate stream; an unbounded label explodes the index and chunk count. Use a line filter or structured metadata instead.

Q6. Why do apps write to stdout instead of shipping logs themselves? TRICKY

It decouples the app from the logging backend: the platform can change shippers or stores, and a crashing app never blocks on a remote log server.

Next: SD-N-009 · Dashboards and Alerting backendroadmap.com

Dashboards and Alerting

page on symptoms, diagnose with causes

Watch the 4-min video

In one line: page a human only for urgent, actionable, user-facing symptoms, ideally an SLO burn rate, and link a runbook; everything else becomes a ticket or a dashboard panel, because a pager that cries wolf gets ignored when the real outage comes.

1 Paged at 3 a.m. for nothing

Your phone buzzes: CPU at 85% on one server. Users are fine. It happens the next night, and the next. By the time a real outage arrives, the team has learned to ignore the pager. Alert fatigue isn't a people problem; it's a design problem.

2 Symptoms, not causes

	Symptom	Cause
Examples	checkout errors, p99 above target, failed logins	high CPU, growing queue, one replica down, disk 80%
Hurts users?	always	maybe never (redundancy, headroom)
Use for	**paging**	**diagnosis** on dashboards; tickets if trending to trouble

Users don't care about your CPU; they care whether checkout works. One exception: causes that predict certain, imminent harm (disk full in 2 hours) can deserve a page.

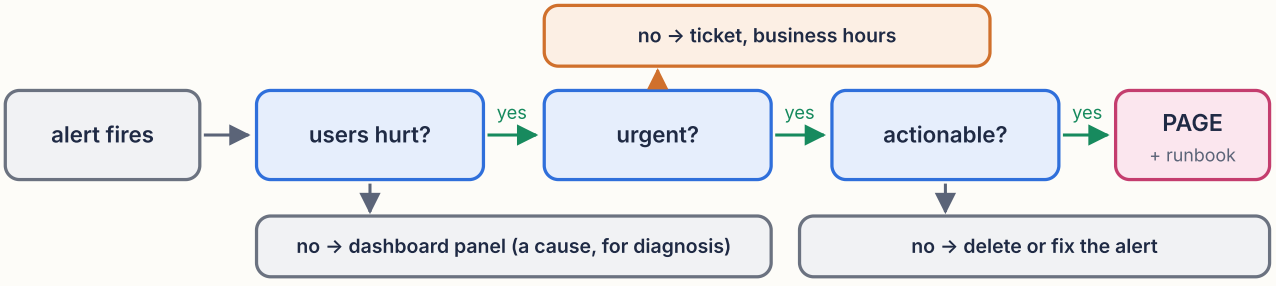


Figure 1. Three questions decide whether an alert may wake someone. Most "no" answers mean ticket, dashboard or delete.

3 SLO burn-rate alerts

A fixed threshold ("errors > 1%") is either too noisy or too slow. Instead alert on **burn rate**: how fast you're spending the error budget (lessons 2–3). Burn rate 1 spends the 30-day budget in exactly 30 days. With a 99.9% SLO the budget is 0.1% errors, so burn 14.4 means a 1.44% error rate, spending **2% of the month's budget in one hour**: page now.

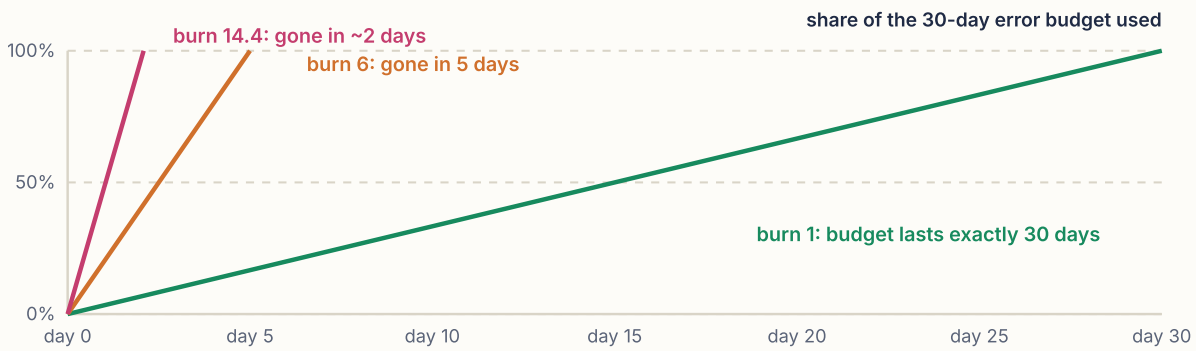


Figure 2. Budget consumption over a 30-day window for different burn rates. Time to exhaustion = 30 days ÷ burn rate.

Severity	Long window	Short window	Burn rate	Budget spent
Page	1 hour	5 minutes	14.4	2%
Page	6 hours	30 minutes	6	5%
Ticket	3 days	6 hours	1	10%

These are the Google SRE Workbook's recommended multiwindow pairs. The alert fires only if **both** windows exceed the rate: the long window proves it's significant, the short window proves it's *still happening*, so the alert also resets quickly after a fix. The result is fast and quiet.

4 Page vs ticket, and runbooks

Page only for urgent, user-facing problems that need action now; everything else is a ticket for business hours. If an alert fires and nobody needs to do anything, **delete it**. Every paging alert links a **runbook**: what the alert means, how to check impact, first mitigation steps (roll back, flag off, fail over), and who to escalate to. Review runbooks after each incident (lesson 70) and fix the steps that were wrong.

5 Dashboards are for diagnosis

- **Top row:** the four golden signals for the service: traffic, errors, latency (p50/p99), saturation.
- **Below:** dependencies (calls out, their errors and latency), then resources (CPU, memory, pools, queues).
- **One overview per service,** same layout everywhere, templated by service and region (lesson 62).
- **Link panels** to matching traces and logs; delete dashboards nobody opens.

WORKED EXAMPLE: CHECKOUT PAGES ONLY ON SLO BURN

Checkout has a **99.9% success SLO**. Rule 1: burn rate > 14.4 over **both** the last 1 h and the last 5 min (error ratio > 1.44%) → page on-call. Rule 2: burn rate > 1 over 6 h → open a ticket. Both alerts link the checkout runbook and dashboard. When one flaky node hits 95% CPU, nobody is paged: the load balancer routes around it and customers aren't failing. The CPU panel on the dashboard is where you'll look if checkout's burn rate ever does rise.

COMMON MISTAKES

- Paging on causes (CPU, memory, one pod restart) that don't hurt users.
- Alerts with no runbook link, so the 3 a.m. responder starts from zero.
- Single short window thresholds that flap on every blip; or only long windows that page hours late.
- Never deleting alerts: each one adds noise, and the team silences the channel.

6 Key terms

Symptom alert: fires on what users experience

Multiwindow alert: long and short windows must both exceed the rate

Alert fatigue: so many noisy alerts that people ignore them

Burn rate: rate of error-budget spend; 1 = exactly on budget

Runbook: steps for responding to a specific alert

Golden signals: latency, traffic, errors, saturation

7 Interview questions

Q1. What makes a good alert? BASIC

It fires on a user-facing symptom, ideally an SLO burn rate; it's urgent and actionable; and it links a runbook. Everything else becomes a ticket or a dashboard panel.

Q2. What is alert fatigue? BASIC

Too many noisy or non-actionable alerts, until people start ignoring pages, including the real one. Fix it by deleting or demoting alerts nobody acts on.

Q3. Your 99.9% service has a 0.5% error rate for the last hour. Page? SCENARIO

Burn rate = 0.5% ÷ 0.1% = 5: below both paging rates (14.4 and 6), so no page. It's far above 1, so the ticket rule fires if it persists; at this rate the month's budget is gone in 6 days, so treat the ticket as urgent.

Q4. On-call gets 40 pages a week. How do you fix it? SCENARIO

Review every alert: keep symptom alerts with runbooks, convert cause alerts to dashboards or tickets, move to burn-rate rules, and delete what nobody acted on.

Q5. Why use two windows instead of one? TRICKY

The long window shows enough budget burned to matter; the short window confirms it's still happening, so you page fast on real problems and stop soon after recovery.

Q6. Should 'disk 90% full' ever page? TRICKY

Only if it predicts certain user impact soon (e.g. full within hours at current growth); otherwise it's a ticket. Alert on time-to-full, not the percentage.

Next: SD-N-010 · Health Endpoints backendroadmap.com

Health Endpoints

live, ready, health: three different questions

Watch the 4-min video

In one line: `/live` says the process isn't stuck, `/ready` says this instance can serve right now, and `/health` is a detailed report; keep readiness about local essentials, cheap and cached, so one shared-dependency blip can't pull every instance out of service.

1 Trivial endpoints, real outages

A new pod starts and the load balancer sends it traffic at once, but its database pool isn't connected yet: the first hundred requests fail. Or a health check that queries the database every second, from every pod, becomes that database's biggest load. Health endpoints look trivial; the details decide whether they prevent outages or cause them. (Lesson 9 covered health checks and self-healing; this lesson designs the endpoints.)

2 Three endpoints, three questions

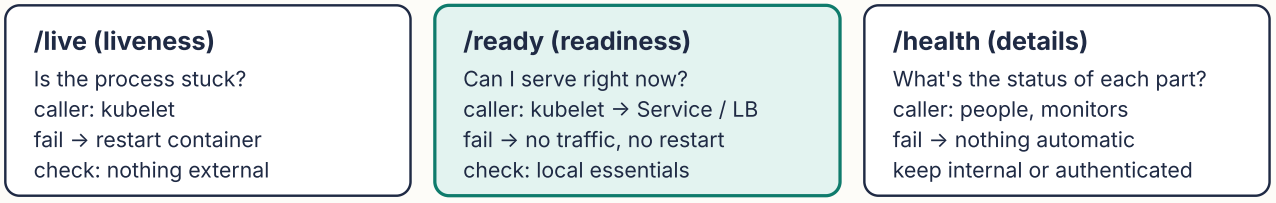


Figure 1. Different callers poll each endpoint and take different actions. Mixing them up (e.g. checking the DB in liveness) causes restart storms.

In Kubernetes these map to **liveness**, **readiness** and **startup** probes. Defaults: `periodSeconds: 10`, `timeoutSeconds: 1`, `failureThreshold: 3`. A failed liveness probe **restarts** the container; a failed readiness probe only removes the pod from Service endpoints. A **startup probe** holds off the other two until slow initialisation (e.g. loading a model) finishes, so liveness doesn't kill a pod that is merely starting.

3 What readiness should check

What *this instance* needs to do its job: DB pool connected, caches warmed, config loaded, model in memory. Return **200** when ready, **503** when not. On shutdown (`SIGTERM`), flip to not-ready first, keep serving in-flight requests while the load balancer stops sending new ones, then exit before the grace period (`terminationGracePeriodSeconds`, default 30 s) runs out.

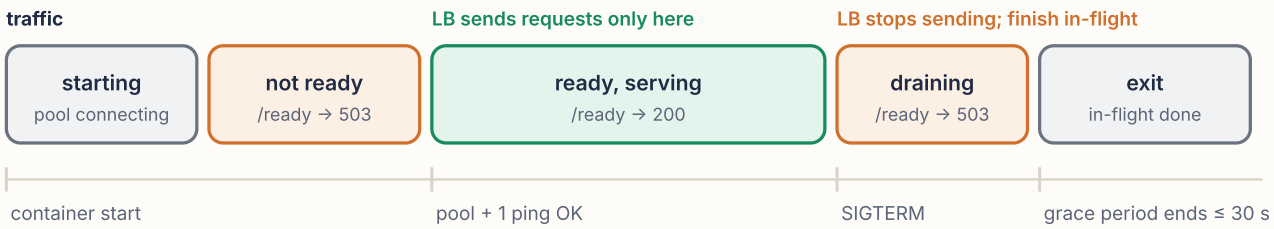


Figure 2. Readiness gates traffic at both ends of a pod's life: no requests before it's warm, none after it starts draining.

4 Shared dependencies: don't amplify outages

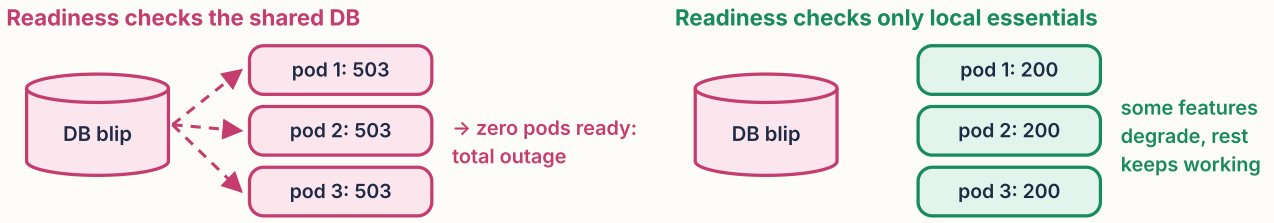


Figure 3. If readiness fails whenever a shared dependency fails, every pod goes unready at once and a partial outage becomes a total one.

Check only what is critical to *this* instance and that a restart or reroute could fix. If the database is down for everyone, removing all pods helps nobody: better to stay ready, return fast errors or fallbacks for affected features (lesson 8), and let symptom alerts (lesson 67) fire.

5 Cheap, cached and private

- **Cheap:** no real query per probe. Use a pool ping or reuse the result of recent real traffic.
- **Cached:** refresh dependency status in the background every few seconds; the endpoint returns the cached answer.
- **Short timeouts** inside the check, so a slow dependency can't hang the endpoint past the probe's own 1 s timeout.
- **Private details:** versions, hostnames and failing dependencies help attackers. Public answer = status code only; details on an internal port or behind auth.

	/live	/ready	/health
Checks	event loop responsive	pool, warm cache, config	each dependency, version
Response	200 / failure	200 or 503	JSON per component
Exposure	cluster-internal	cluster-internal	internal port or authenticated

WORKED EXAMPLE: FASTAPI THAT NEVER SERVES COLD

A lifespan/startup hook creates the DB pool, runs one `SELECT 1`, then sets `READY`. `@app.get("/ready")` returns `Response(status_code=200 if READY.is_set() else 503)`; `/live` just returns 200. Kubernetes points `readinessProbe` at `/ready` and `livenessProbe` at `/live`, so no request ever hits a cold pod. On SIGTERM the app clears `READY` and drains. A detailed `/health`, with per-dependency status, is served on an internal admin port.

COMMON MISTAKES

- Checking the database in **liveness**: a DB blip restarts every pod, which then all reconnect at once.
- One `/health` used for everything, so liveness, readiness and humans all get the same (wrong) answer.
- No readiness gate, so pods receive traffic before pools and caches are ready.
- Detailed health JSON on the public internet.

6 Key terms

Liveness probe: is the process alive and not stuck; failure restarts it

Startup probe: delays other probes during slow startup

503: Service Unavailable: the not-ready answer

Readiness probe: can it serve now; failure removes it from rotation

Draining: finishing in-flight work after leaving rotation

Grace period: time between SIGTERM and SIGKILL (default 30 s)

7 Interview questions

Q1. What should a readiness endpoint check? BASIC

Whether this instance can serve right now: local essentials like its DB pool, warm caches and loaded config, using cheap, cached checks with short timeouts; not every blip of a shared dependency.

Q2. Why not expose detailed health publicly? BASIC

It leaks versions, hostnames and internal topology. Keep the public answer to a status code; put details behind authentication or on an internal port.

Q3. Pods restart in a loop whenever the database is slow. Why? SCENARIO

The liveness probe checks the database, so DB slowness looks like a dead process. Make liveness local only; use readiness or fallbacks for dependencies.

Q4. Users see errors during every deploy. What do you check? SCENARIO

That readiness gates traffic until warm-up completes, and that on SIGTERM the app fails readiness and drains in-flight requests before exiting.

Q5. Should readiness fail when a shared database is down? TRICKY

Usually not: all pods would go unready together, turning partial impact into a total outage. Stay ready and degrade the affected features.

Q6. Why would you need a startup probe if you have liveness? TRICKY

Slow starters (big models, cache warm-up) would fail liveness and be killed before they finish; the startup probe disables liveness until startup succeeds.

Next: SD-N-011 · Root-Cause Analysis backendroadmap.com

Root-Cause Analysis

timeline, what changed, narrow, test

Watch the 4-min video

In one line: to debug a failure you've never seen, build a timeline, ask what changed, narrow the scope until the problem is small, and test one hypothesis at a time while writing it down; afterwards, ask "why" repeatedly and fix every contributing factor, not just the trigger.

1 No runbook, and the clock is running

Checkout errors jump from almost zero to **5%**. Nobody has seen this failure. There is no runbook, and every minute costs money. Random guessing wastes the one thing you can't get back during an incident: time. A method beats intuition here, because it works on problems you've never seen.

2 Timeline first, then: what changed?

When exactly did it start? Sudden or gradual? Then the most useful question in debugging: **what changed?** Deploys, config changes, feature flags, traffic shifts, dependency upgrades, certificate expiries. Most incidents start within minutes of a change, so line up the change log against the moment errors began. A deploy history and a config audit log turn this from an hour into seconds.

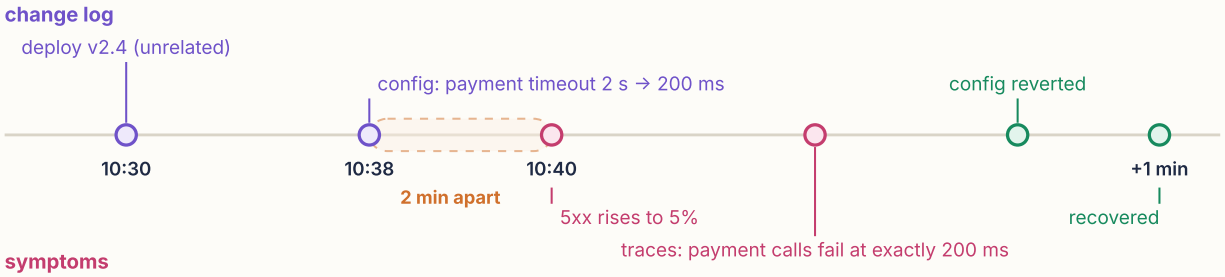


Figure 1. The video's incident, with an unrelated 10:30 deploy added to show the point: the change just before the symptom is the first suspect, not proof.

3 Narrow the scope

Ask	If the answer is "only one"...	Where to look
All endpoints or one route?	code path or dependency of that route	RED dashboard per route (lesson 61)
All regions or one?	regional infra, config or traffic	metrics split by region
All tenants or one?	that tenant's data, plan or limits	logs by tenant_id (lesson 60)
All pods or one node?	bad host, noisy neighbour, disk	metrics by pod and node
Which call fails?	a specific dependency	traces of failed requests (lesson 63)

Each answer cuts the search space roughly in half. Dashboards show where errors concentrate, traces show which call fails, logs show the exact error.

4 Hypotheses, one at a time

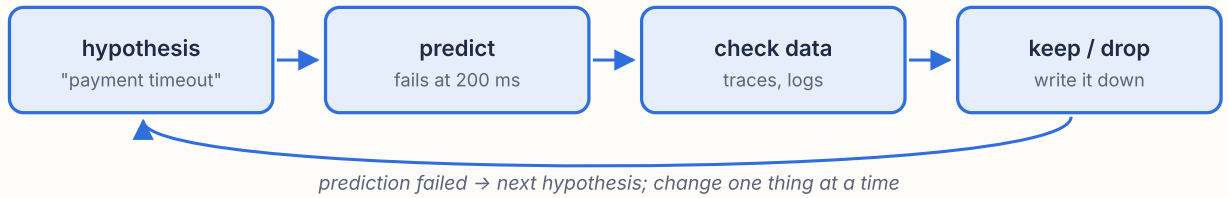


Figure 2. The scientific loop. Post each step in the incident channel so nobody repeats a check that has already been done.

State one possible cause, **predict** what you'd see if it were true, and check. If the prediction fails, drop it. Don't change several things at once: if errors stop, you won't know which change helped, and one of them may cause the next incident. During the incident, mitigation can come before certainty: reverting the suspect change is often both the test and the fix (lesson 70).

5 Five whys and contributing factors

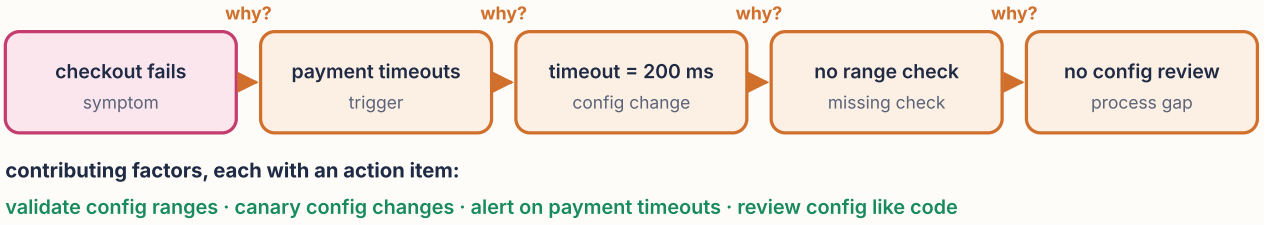


Figure 3. Each "why" goes one level deeper, from symptom to something fixable. Real incidents have several contributing factors, not one root cause.

Stopping at the trigger ("someone set 200 ms") leaves the system just as fragile and invites blame. Keep asking until you reach system properties: missing validation, no canary for config, an alert that didn't fire. Assign an action item to each factor. The goal is a system that is harder to break, not a name in a report.

WORKED EXAMPLE: ELEVATED 5XX ON CHECKOUT

At **10:40** checkout returns elevated 500s. The timeline shows a config change at **10:38** lowering the payment client's timeout from **2 s to 200 ms**. Hypothesis: payments time out. Prediction: failed payment spans end at exactly 200 ms. Traces confirm it. The team reverts the config; errors drop within a minute. Action items: validate config ranges, canary config changes like code, alert on payment timeouts.

COMMON MISTAKES

- Debugging the code before checking the change log.
- Changing three things at once, then not knowing which one fixed it.
- Stopping at "human error": that's where the analysis should start.
- Treating correlation as proof: the deploy at 10:30 also preceded the errors.

6 Key terms

Timeline: ordered record of changes, symptoms and actions

Change log: deploys, config and flag changes with timestamps

Hypothesis: a testable guess with a predicted observation

Trigger: the event that set the failure off

Five whys: asking why repeatedly to reach fixable causes

Contributing factor: a condition that let the trigger cause harm

7 Interview questions

Q1. How do you investigate a production incident you've never seen before? BASIC

Build a timeline, check what changed (deploys, config, flags, traffic), narrow the scope with dashboards, traces and logs, then test hypotheses one at a time, writing every step down.

Q2. What is the five whys? BASIC

Asking why repeatedly, from the symptom down through the trigger to fixable causes and contributing factors, each of which gets an action item.

Q3. Errors started at 14:02; there was a deploy at 13:10 and a config change at 14:00. Where do you start? SCENARIO

With the 14:00 config change: it's closest in time. Predict what it would cause, check traces or logs, and revert it if it fits.

Q4. Errors affect only one tenant. What does that suggest? SCENARIO

Something specific to that tenant: its data, plan limits, a tenant-specific flag or a noisy-neighbour quota. Filter logs and traces by tenant_id.

Q5. Why is "one root cause" a misleading idea? TRICKY

Failures need several conditions at once: a risky change, a missing check, an alert that didn't fire. Fixing only the trigger leaves the other gaps open.

Q6. Should you revert before you're sure the change caused it? TRICKY

Often yes, if the revert is low-risk: it limits impact and doubles as a test. If errors stop, you've confirmed the hypothesis; if not, you've ruled it out.

Next: SD-N-012 · Incident Management backendroadmap.com

Incident Management

one commander, mitigate first, learn after

Watch the 4-min video

In one line: classify by severity, put one incident commander in charge with a comms lead and responders, stop the bleeding before finding the root cause, update on a fixed rhythm, then run a blameless postmortem whose action items have owners and due dates.

1 Ten engineers, nobody in charge

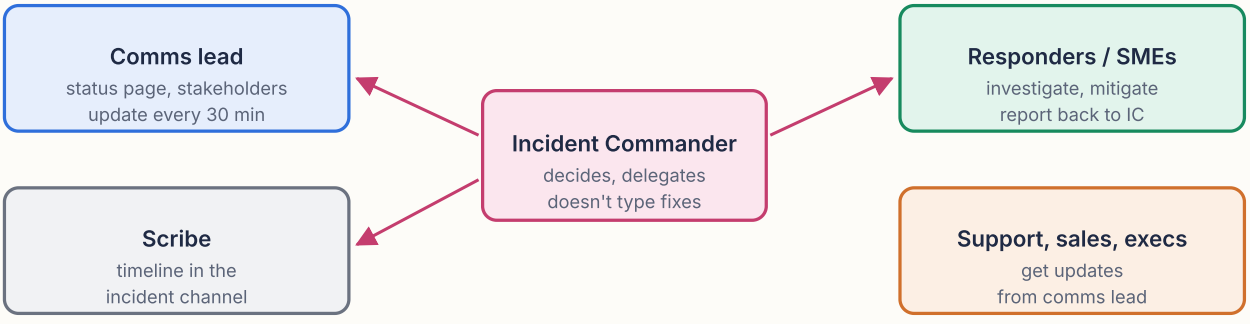
A major outage starts. Ten engineers jump into the same chat. Three restart different services at once. Nobody tells customers anything, and support is flooded. Everyone is busy, but nobody is in charge. An incident process exists to replace that chaos with roles, priorities and a rhythm. Many teams base theirs on the Incident Command System adapted by Google SRE and PagerDuty.

2 Severity levels

Level	Meaning	Example	Response
SEV1	critical user-facing outage	checkout down for everyone	page now, IC assigned, updates every 30 min, status page
SEV2	major feature degraded or some users affected	search slow in one region	page on-call, regular updates
SEV3	minor impact, workaround exists	CSV export fails, retry works	ticket, business hours

Severity decides who is paged, how fast, and how often you communicate. Exact definitions vary by company; what matters is that they are written down and anyone can declare one. When unsure, declare higher and downgrade later: it's cheap to stand down.

3 Roles



one person in charge → no conflicting actions; roles hand over in shifts

Figure 1. The incident commander coordinates; others act and communicate. In a small team one person may hold two roles, but never IC and hands-on fixer in a SEV1.

The **incident commander** (IC) makes decisions, delegates tasks and keeps everyone focused; they don't debug. The **communications lead** posts updates to stakeholders and the status page. **Subject-matter**

experts investigate and act, reporting back to the IC. A **scribe** keeps the timeline. For long incidents, roles hand over in shifts with an explicit handover, so nobody decides things after 18 hours awake.

4 Mitigation before resolution

The first goal is to **stop the bleeding**, not to understand everything. Roll back the deploy (lesson 72), turn off the feature flag (lesson 76), shift traffic to another region (lesson 15), or scale up. Root cause can wait until users stop hurting (lesson 69). Practise rollbacks in calm times so they're fast and boring when it matters.



Figure 2. The incident lifecycle. Mitigation ends the user pain; the postmortem and tracked actions make the next incident less likely.

5 Communication rhythm

Every **30 minutes** for a SEV1 (or as your policy says) the comms lead posts: what is known, what is being tried, and when the next update comes, *even if nothing changed*. One incident channel is the record. Silence makes customers assume the worst; internal teams (support, sales) need updates too, so they know what to tell people.

6 Blameless postmortems

A postmortem asks **how the system allowed the failure**, not who made the mistake. People who fear blame hide information, and the same failure returns. It records the timeline, impact (users, duration, error budget spent), contributing factors and **action items, each with an owner and a due date**. Track them to done, or the postmortem was just paperwork.

WORKED EXAMPLE: A BAD RELEASE, ROLLED BACK

v2.4 goes out and checkout errors spike. Someone declares **SEV1**; an IC takes charge and a comms lead updates the status page. Instead of debugging the new code live, the IC orders a **rollback to v2.3**. Errors recover within **5 minutes**, and the status page says so. Next day, the blameless postmortem finds a missing integration test for the payment path. Action items: add the test (owner: payments team, due Friday) and **automatic rollback on error spikes** in the deploy pipeline (owner: platform team).

COMMON MISTAKES

- The most senior engineer acting as IC *and* debugging, so nobody coordinates.
- Hunting the root cause while users are still failing, when a rollback was available.
- No updates for an hour because "nothing has changed".
- Postmortems that name a person, or action items with no owner and no date.

7 Key terms

SEV level: severity class deciding response speed and scope

Comms lead: owns stakeholder and status-page updates

Blameless postmortem: review of how the system allowed the failure

Incident commander: the one coordinator and decision maker

Mitigation: action that stops user impact without a full fix

Action item: owned, dated fix from a postmortem

8 Interview questions

Q1. Describe an incident process. BASIC

Classify by severity, assign an incident commander, a comms lead and responders, mitigate first (rollback, flag off, failover), communicate on a regular rhythm, then run a postmortem.

Q2. What is a blameless postmortem? BASIC

A review of how the system allowed the failure rather than who to blame, recording timeline, impact and contributing factors, ending in action items with owners and due dates.

Q3. Errors spiked right after a deploy. The author wants 10 minutes to find the bug. What does the IC do? SCENARIO

Roll back first if it's safe: it ends the impact within minutes. Debugging continues afterwards against the bad version in a non-production environment.

Q4. You're unsure if it's a SEV1 or SEV2. Which do you declare? SCENARIO

The higher one: over-declaring costs a little attention; under-declaring delays help and communication. Downgrade when impact is clear.

Q5. Why shouldn't the IC fix things themselves? TRICKY

Hands-on work narrows attention; nobody then tracks parallel efforts, prevents conflicting actions or decides when to escalate and communicate.

Q6. Why can a blameless culture improve reliability? TRICKY

People report mistakes and near-misses honestly, so the real contributing factors are found and fixed instead of hidden.

Next: SD-N-013 · Capacity Planning backendroadmap.com

Capacity Monitoring and Planning

watch the trend, act before the limit

Watch the 4-min video

In one line: track utilisation and saturation as trends against their limits, including cloud quotas you don't control, forecast when each resource will cross a headroom threshold, and add capacity before it does; alert on predictions, which give weeks of warning instead of minutes.

1 The most predictable outage

Black Friday morning. Traffic triples, the database hits its connection limit, checkout falls over. Looking back, the graphs had been creeping up for months; nobody watched the trend. Running out of capacity is one of the most predictable outages there is, if someone looks ahead.

2 Utilisation, saturation and quotas

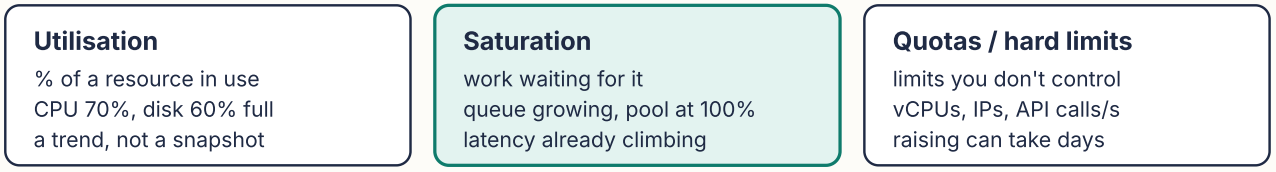


Figure 1. Three things to track for every resource. Saturation is often the first real warning; quotas are the ones most often forgotten.

Utilisation is how much of each resource you use: CPU, memory, disk, network, DB connections, queue depth. Plot it **against its limit** on the same graph, over weeks and months. **Saturation** means work is waiting: a CPU at 70% may be fine, but a connection pool at 100% means requests are already queueing (the USE method, lesson 61). **Quotas**: cloud accounts cap vCPUs per region, public IPs, API calls per second, storage. Hitting one stops scaling instantly, and an increase can take days, so track quota usage like any other resource and request increases early.

3 Sizing from first principles

Little's law: requests in flight = arrival rate × time in system. At 2,000 req/s with 50 ms spent holding a DB connection, you need about 2,000 × 0.05 = **100 connections** busy on average, so a pool of 80 saturates even though CPU looks idle. Measure throughput per instance in a load test, then: instances = peak load ÷ safe per-instance throughput, plus headroom.

4 Forecasting

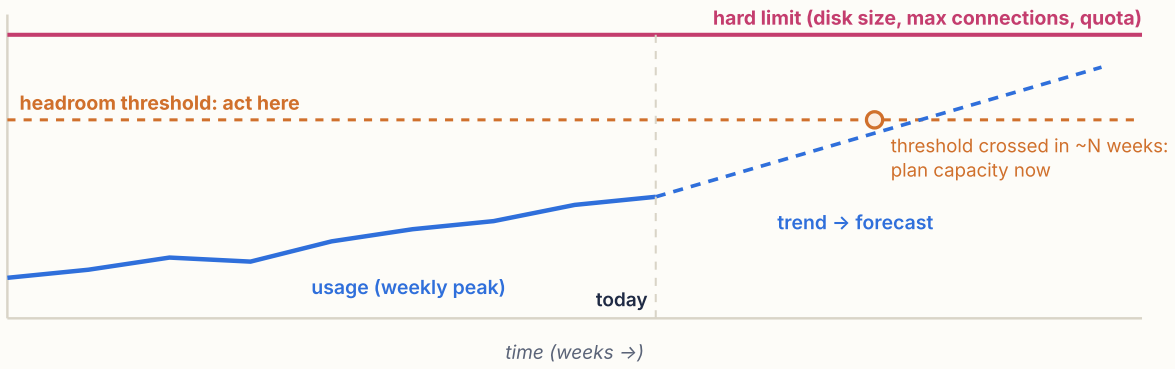


Figure 2. Usage against its limit, with a trend projected forward. The decision point is the headroom threshold, not the hard limit.

Fit a trend to the last few months and project it: if disk grows 2% a week, you know roughly when it fills. Add **known events**: launches, marketing campaigns, seasonal peaks (last year's Black Friday × growth). Forecasts are never exact, but they turn surprises into planned work. Review every quarter and compare last quarter's forecast with reality to calibrate.

5 Headroom policy

Decide how much spare capacity you always hold, e.g. **30–50%**, to absorb spikes and the loss of a zone. With 3 zones, surviving one zone loss means the other two carry 1.5× their load, so each must run below about **66%** at peak. When usage crosses the threshold, add capacity or improve efficiency. Autoscaling covers stateless compute, but databases, quotas and licences still need planning. Too much headroom wastes money: revisit it.

Resource	Leading signal	Lead time to add
Stateless compute	CPU, request rate per pod	minutes (autoscaling)
DB connections	pool wait time, active / max	hours to days (pooler, bigger instance)
Disk / storage	days until full	hours to weeks
Cloud quotas	used / quota	days (support request)

WORKED EXAMPLE: ALERT WHEN THE DISK WILL BE FULL IN 30 DAYS

The orders database has a **2 TB** disk. Instead of alerting at 90% full (minutes of warning on a busy night), the team alerts on a prediction: `predict_linear(node_filesystem_avail_bytes{mountpoint="/data"}[7d], 30*24*3600) < 0`, a linear fit over the last 7 days that asks "will free space hit zero within 30 days?". It opens a **ticket**, not a page. It fires on a calm Tuesday, giving weeks to add storage or archive old orders.

COMMON MISTAKES

- Static 90% thresholds that fire too late on fast growth and nag forever on slow growth.
- Watching CPU while the connection pool or a queue is the real bottleneck.
- Forgetting cloud quotas until a launch-day scale-out fails.
- Planning for average load instead of peak plus a zone failure.

6 Key terms

Utilisation: share of a resource in use

Quota: provider-imposed limit, e.g. vCPUs per region

Forecast: projected usage from trend plus known events

Saturation: work waiting because a resource is full

Headroom: spare capacity held on purpose

predict_linear: PromQL linear extrapolation of a series

7 Interview questions

Q1. How do you know when to add capacity? BASIC

Track utilisation and saturation trends against limits, watch quotas, forecast when each resource crosses its headroom threshold (including launches and peaks), and add capacity before that point.

Q2. Why alert on predictions instead of fixed thresholds? BASIC

They give weeks of warning instead of minutes, and adapt to the growth rate: fast growth alerts early, slow growth doesn't nag.

Q3. CPU is at 40% but latency doubles at peak. What do you check? SCENARIO

Saturation elsewhere: connection-pool wait, thread pools, queue depth, a downstream dependency or lock contention. Low CPU doesn't mean spare capacity.

Q4. A launch will bring 5× traffic in a month. How do you prepare? SCENARIO

Load-test to find per-instance limits and the first bottleneck, check quotas and request increases now, pre-scale stateful tiers, and keep headroom for a zone loss.

Q5. Doesn't autoscaling make capacity planning unnecessary? TRICKY

No: it scales stateless compute within quotas, but databases, connection limits, quotas and cold-start time still need planning ahead.

Q6. Why keep each of three zones below ~66%? TRICKY

If one zone fails, the remaining two take its load: 1.5× each. Above about 66% that pushes them past 100%.

Next: SD-N-014 · CI/CD Pipelines backendroadmap.com

CI/CD Pipelines

build once, test, scan, promote, roll back

Watch the 4-min video

In one line: a pipeline verifies every change automatically (CI), builds one immutable artifact, and promotes that same artifact through staging to production with gates, health checks and a one-click rollback (CD), so releases become small, boring and repeatable.

1 Slow, scary and different every time

A developer builds on their laptop, copies it to a server and restarts it. It works, until the day a step is forgotten or the wrong version goes out, and nobody knows what is actually running. A CI/CD pipeline replaces those manual steps with an automated, repeatable path to production.

2 Continuous integration

Every change is merged often and verified automatically. On each pull request the pipeline builds and runs fast checks: unit tests, linting, type checks. Problems surface in minutes, while the change is fresh. A broken main branch is the team's top priority, because everyone builds on it.

3 Stages and gates

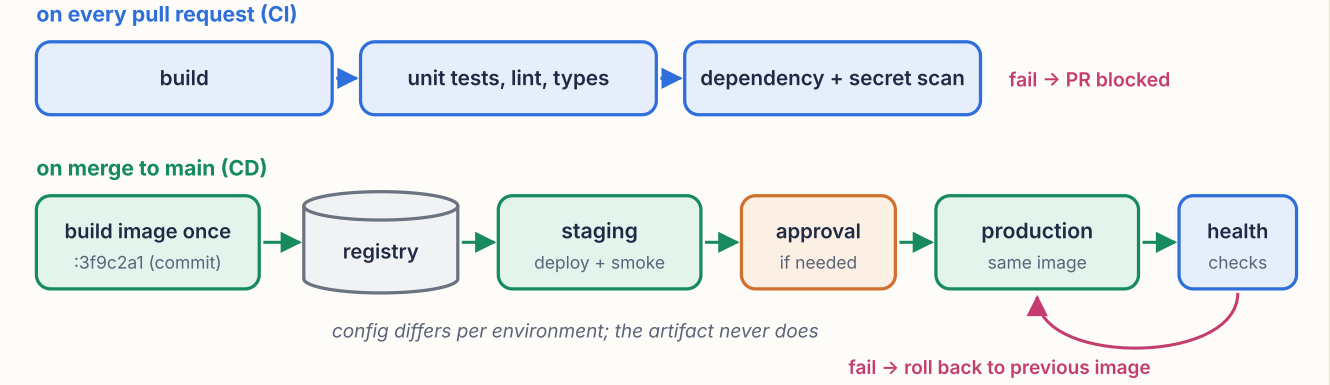


Figure 1. Each stage is a gate: if it fails, nothing moves forward. One image, tagged with the commit, is promoted from staging to production.

Build once, run unit and integration tests, **scan** dependencies for known vulnerabilities and the diff for leaked secrets, then package as an artifact (e.g. a Docker image) and push it to a registry. Later stages deploy, run smoke tests and check health.

4 One immutable artifact

Build the artifact **exactly once**, tag it with the commit (`orders:3f9c2a1`, never only `:latest`), and promote that same artifact from dev to staging to production. Only configuration changes between environments (lesson 38). Rebuilding per environment risks shipping something different from what you tested: a new base image, a moved dependency. The registry keeps every version, so you always know which commit is running and can redeploy the previous one.

5 Delivery vs deployment

	Continuous integration	Continuous delivery	Continuous deployment
Means	merge often, verify every change	every green build *could* be released	every green build *is* released
Prod release	n/a	a human presses the button	automatic, gated by tests and health
Needs	fast tests, trunk discipline	reliable pipeline, rollback	strong tests, monitoring, auto-rollback, flags (lesson 76)

Either way, production deploys need approvals where policy requires, post-deploy health checks, and a fast, **tested** rollback to the previous artifact. Rollout strategies (rolling, blue-green, canary) are lessons 73–75. Small, frequent releases are easier to verify and to undo.

6 The pipeline is production too

- **Pipeline as code** in the repo, reviewed like any change; branch protection and required reviews on main.
- **Short-lived credentials** via OIDC federation instead of long-lived cloud keys stored as secrets.
- **Pin third-party actions** to a commit SHA; a moved tag can run someone else's code with your credentials (lesson 58).
- **Fast:** cache dependencies, run independent jobs in parallel; an hour-long pipeline gets skipped.

```
on: {push: {branches: [main]}}
permissions: {id-token: write, contents: read}      # OIDC, no stored cloud secret
jobs:
  build:
    steps:
      - uses: actions/checkout@<pinned-sha>
      - uses: azure/login@<pinned-sha>                # federated credential
      - run: docker build -t $ACR/orders:${{ github.sha }} . && docker push $ACR/orders:${{ github.sha }}
  deploy-prod:
    needs: [build, deploy-staging]
    environment: production                          # required reviewers = approval gate
    steps:
      - run: kubectl set image deploy/orders orders=$ACR/orders:${{ github.sha }}
      - run: kubectl rollout status deploy/orders --timeout=5m || kubectl rollout undo deploy/orders
```

Figure 2. A sketch of the video's GitHub Actions → ACR → AKS flow (staging job omitted). The environment's required reviewers are the approval gate.

WORKED EXAMPLE: GITHUB ACTIONS TO AZURE KUBERNETES SERVICE

On each **pull request**: tests and a security scan. On **merge to main**: build a Docker image tagged with the commit SHA, push it to Azure Container Registry, deploy to **staging** automatically. After smoke tests pass and a reviewer approves the `production` environment, the **same image** is promoted to production. If `kubectl rollout status` times out or health checks fail, the pipeline runs `kubectl rollout undo` to return to the previous image.

COMMON MISTAKES

- Rebuilding the image for production, so prod runs something never tested.
- Deploying `:latest`: you can't tell what is running or roll back precisely.
- A rollback path that has never been exercised, discovered broken during an incident.
- Long-lived cloud admin keys in CI secrets and unpinned third-party actions.

7 Key terms

CI: merge often; build and test every change automatically

Continuous delivery: every green build is releasable

Continuous deployment: every green build is released automatically

Artifact: built, versioned output, e.g. an image

Promotion: moving the same artifact to the next environment

Gate: a check that must pass before the next stage

8 Interview questions

Q1. What should a production CI/CD pipeline include? BASIC

Build, test and security-scan stages, one immutable artifact promoted from dev to staging to production, approvals where needed, post-deploy health checks and a fast, tested rollback, with the pipeline itself secured and fast.

Q2. Why build the artifact only once? BASIC

So production runs exactly what you tested; rebuilding can pull different dependencies or base images.

Q3. Staging passed but production broke. What would you check in the pipeline? SCENARIO

That the same artifact was promoted, config differences between environments, missing prod-like data or traffic in staging, and whether health checks gate the rollout.

Q4. How do you let CI deploy to Azure without storing a secret? SCENARIO

Use OIDC workload identity federation: the workflow gets a short-lived token from GitHub's OIDC provider and exchanges it for an Azure token scoped to that repo and environment.

Q5. Continuous delivery vs continuous deployment? TRICKY

Delivery keeps every build releasable and a human decides when; deployment releases every passing build automatically.

Q6. Is a fast rollback a substitute for testing? TRICKY

No: it limits damage but users still see the failure, and data changes (migrations, sent emails) may not roll back. You need both.

Next: SD-N-015 · Rolling Deployments backendroadmap.com

Rolling Deployments

a few at a time, old and new side by side

Watch the 4-min video

In one line: a rolling deployment replaces instances a few at a time, gated by readiness, so the service keeps serving throughout; `maxSurge` and `maxUnavailable` set the pace, and because old and new versions run side by side, every change must be backward compatible.

1 Twenty servers, zero downtime

You need to ship a new version of a service running on twenty instances. Stop them all, deploy, start again? That's downtime for every user. A rolling deployment replaces instances a few at a time, so enough healthy ones are always serving. It is the default strategy for a Kubernetes Deployment.

2 How it works

Start a few new instances. Wait until they pass **readiness** (lesson 68) and receive traffic. Stop a few old ones, letting them drain. Repeat until all run the new version. Readiness is what makes it safe: a pod that reports ready too early receives traffic it can't handle, and the rollout keeps going on bad pods. `minReadySeconds` adds a soak time before a new pod counts as available.

3 maxSurge and maxUnavailable

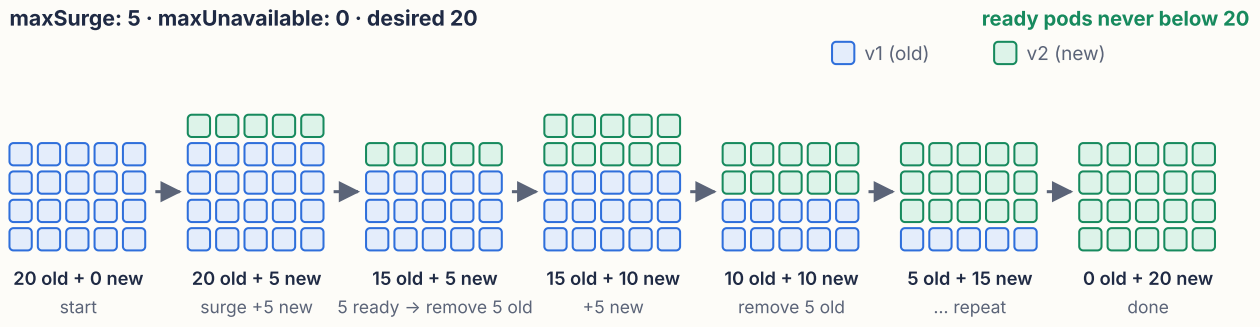


Figure 1. The video's rollout: 5 extra pods at a time, never below 20 ready. Kubernetes doesn't strictly batch; it keeps within both limits as pods become ready.

Setting	Meaning	Default	Trade-off
`maxSurge`	pods allowed **above** desired count	25% (rounded up)	higher = faster, needs spare capacity
`maxUnavailable`	pods allowed **below** desired count	25% (rounded down)	0 = never lose capacity, but needs surge > 0

Both can't be zero. `maxSurge: 0, maxUnavailable: 1` needs no extra capacity but runs one pod short; `maxSurge: 25%, maxUnavailable: 0` keeps full capacity at the cost of temporary extra pods. If new pods don't become ready within `progressDeadlineSeconds` (default 600 s), the rollout is marked as failed, but it is **not rolled back automatically**: your pipeline must act on it.

4 Version skew

During the rollout, old and new versions run side by side, sometimes for many minutes, and one user's requests may hit both. If the new version writes data the old one can't read, sends an API field the old one rejects, or publishes a message format old consumers don't understand, requests fail *in the middle* of the rollout, and again during a rollback.

5 Backward compatibility: expand and contract



Figure 2. Expand and contract (parallel change) spreads a breaking schema change over several releases so every intermediate state is safe.

Add new fields; don't remove or rename old ones in the same release. Readers tolerate unknown fields and missing new ones. For databases, **expand** (add the new column), deploy code that handles both, migrate data, then **contract** (remove the old column) only after no running version needs it. The same rule applies to queues and events.

TRADE-OFFS OF ROLLING DEPLOYMENTS

- **Cheap:** no second environment; only the surge pods are extra.
- **Slow rollback:** undo is just another rolling update, minutes not seconds.
- **Partial exposure:** a bad version serves a growing share of users while it rolls out.
- **Needs automation:** watch error rate and latency, pause or undo on regression; for risky changes use canary (lesson 75) or blue-green (lesson 74).

WORKED EXAMPLE: 20 PODS, MAXSURGE 5, MAXUNAVAILABLE 0

The orders Deployment runs **20 pods** with `strategy.rollingUpdate: {maxSurge: 5, maxUnavailable: 0}`. Kubernetes starts 5 v2 pods, waits for readiness, removes 5 v1 pods, and continues until all 20 are v2 (Figure 1): about four rounds. Capacity never drops below 20. The v2 code reads both the old and new order formats, so no request breaks while versions mix. When error rates rise, `kubectl rollout undo deployment/orders` rolls back the same way.

6 Key terms

Rolling update: replace instances gradually, a few at a time

maxUnavailable: pods allowed missing below desired

Expand and contract: add, migrate, then remove in separate releases

maxSurge: extra pods allowed above desired during rollout

Version skew: old and new versions running together

rollout undo: roll back to the previous ReplicaSet

7 Interview questions

Q1. What compatibility issues arise during a rolling deploy? BASIC

Old and new versions run at the same time, so APIs, message formats and database schemas must stay backward and forward compatible: additive changes and expand-and-contract migrations.

Q2. What do maxSurge and maxUnavailable control? BASIC

How fast the rollout goes (extra pods allowed above desired) and how much capacity it may lose (pods allowed below desired).

Q3. You must rename a database column used by the service. Plan it. SCENARIO

Release 1: add the new column, write both, read either. Backfill. Release 2: read only the new one. Release 3, once no old version runs: drop the old column.

Q4. A cluster has no spare capacity. Which settings? SCENARIO

maxSurge: 0, maxUnavailable: 1 (or a small %): no extra pods, at the cost of running slightly under capacity during the rollout.

Q5. Does Kubernetes roll back automatically if new pods fail readiness? TRICKY

No: the rollout stalls and is marked failed after progressDeadlineSeconds; old pods keep serving. Rollback is a separate action (rollout undo or your pipeline).

Q6. Why is rollback during a rolling deploy also risky? TRICKY

Rolling back creates version skew again, in reverse: if v2 already wrote data or messages v1 can't read, the rollback breaks requests too.

Next: SD-N-016 · Blue-Green Deployments

Blue-Green Deployments

switch in seconds, either direction

Watch the 4-min video

In one line: run two identical production environments, deploy and test the new version on the idle one, then move all traffic at once; switching back is just as fast, as long as both versions can run against the same database (expand and contract) and you pay for two environments.

1 Twenty minutes to roll back

Friday evening, a deploy goes out. Errors spike, and rolling back takes twenty minutes while customers watch, because rollback means another full deployment (lesson 73). What if switching versions took one second, in either direction? That is the promise of blue-green deployment.

2 Two environments, one switch

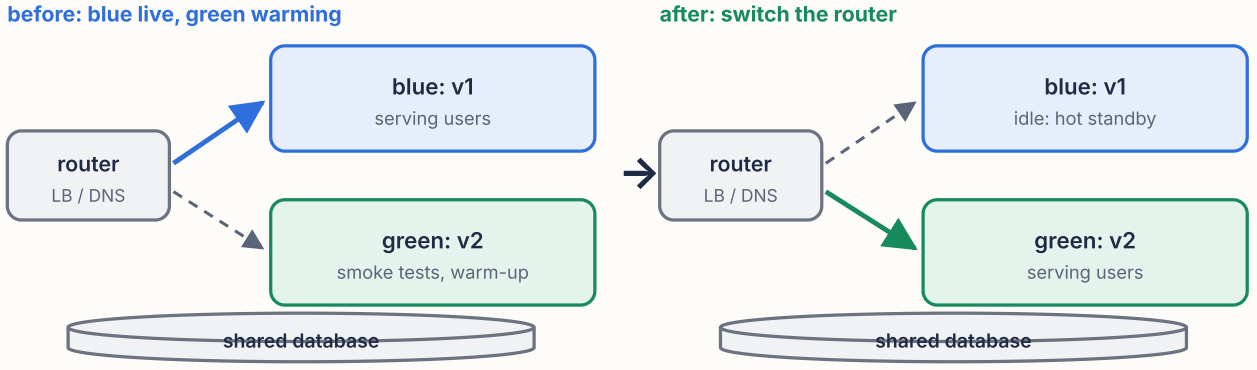


Figure 1. Green is production in every way except one: no customers yet. The cutover is a routing change, so rollback is the same change in reverse.

Blue serves all live traffic. **Green** gets the new version, fully deployed and warmed up, with no user traffic. Test green with smoke tests against real production dependencies. Then the router (load balancer, ingress, or DNS) moves every request to green at once. Blue stays running, untouched, as a hot standby. If green misbehaves, switch back in seconds. Prefer switching at a load balancer: DNS changes are slowed by TTLs and clients that cache.

3 The hard part: the database

Both colours usually share one database, so during and after the switch, old and new code must work with the same schema. Use **expand and contract**: expand the schema in a backward-compatible way while blue still runs, switch, and contract (drop old columns) only in a later release, once rollback to blue is no longer needed.



Figure 2. Splitting a name column across two releases. A destructive migration in the same release as the switch would make instant rollback impossible.

4 Other state

- **Draining:** long requests, sessions and WebSocket connections on blue need time to finish or reconnect; keep sessions in a shared store.
- **Background workers:** queue consumers and cron jobs must not run in both colours at once, or jobs run twice.
- **Caches:** warm green before cutover, or the first minute is slow and hammers the database.
- **Side effects:** smoke tests on green hit real dependencies: use test accounts, not real payments.

5 Comparing strategies

	Rolling (lesson 73)	Blue-green	Canary (lesson 75)
Switch	gradual, by instance	all at once	gradual, by traffic %
Rollback	minutes (another rollout)	seconds (route back)	seconds (weight to 0)
Extra cost	a few surge pods	a second full environment	small canary set + tooling
Blast radius	grows during rollout	everyone, at once	small slice first
Version skew	yes, during rollout	shared DB only	yes, during analysis

TRADE-OFFS

- **Cost:** two full environments can double the bill; many teams keep green small and scale it up only during a release, or create it on demand.
- **All-at-once exposure:** every user hits green together; problems that need real load show up for everyone.
- **Fastest rollback** of any strategy, and a full production-like test before any user sees the change.
- **Schema discipline** is mandatory; without it, the instant rollback is an illusion.

WORKED EXAMPLE: AZURE APP SERVICE DEPLOYMENT SLOTS

The **production** slot serves users; a **staging** slot receives the new build. Before a swap, App Service warms up the staging instances by sending requests to them. Then `az webapp deployment slot swap -g shop -n shop-api --slot staging --target-slot production` exchanges the slots: the new build is production in seconds, with no cold start. Settings marked as **deployment slot settings** (e.g. a staging database connection string) stay with their slot. If anything looks wrong, swap again and the previous version is back. (Slots need the Standard tier or higher.)

6 Key terms

Blue / green: the live and the idle production environments

Hot standby: old environment kept running for instant rollback

Deployment slot: App Service's built-in blue-green environment

Cutover: moving all traffic to the new environment at once

Expand and contract: backward-compatible schema change over releases

Slot setting: config that stays with its slot on swap

7 Interview questions

Q1. How do you handle database schema changes with blue-green? BASIC

Expand and contract: make every change backward compatible so both versions run against the same database, switch, and remove old columns only in a later release once rollback is no longer needed.

Q2. What's the main trade-off of blue-green? BASIC

An instant, all-at-once switch and the fastest rollback, in exchange for running two full environments for a while.

Q3. After cutover, every order is processed twice. Why? SCENARIO

Queue consumers or scheduled jobs are running in both blue and green. Run workers in only the live colour, or use leader election and idempotent processing.

Q4. Why might switching via DNS give a slow, messy cutover? SCENARIO

Resolvers and clients cache records for the TTL or longer, so traffic drains to green gradually and unpredictably; a load-balancer switch is immediate.

Q5. When does blue-green rollback stop being instant? TRICKY

When green has made changes blue can't handle: a dropped column, a new message format, data written in a new shape.

Q6. Blue-green or canary for a risky new algorithm? TRICKY

Canary: blue-green exposes all users at once, while a canary limits the blast radius and compares metrics on real traffic first.

Next: SD-N-017 · Canary Deployments backendroadmap.com

Canary Deployments

ship to 1% first, let the metrics decide

Watch the 4-min video

In one line: send a small, growing slice of real traffic to the new version, compare it with the stable baseline on errors, latency, saturation and business metrics at each step, and let automation promote or roll back, so a hidden problem hits 1% of users for minutes instead of everyone.

1 Offline isn't production

Your team trained a new ranking model and the offline metrics look great. But production has real users, real traffic and real edge cases. Ship it to everyone and a hidden problem hits everyone at once. A canary, named after the canary in the coal mine, lets a small group find out first.

2 Traffic steps

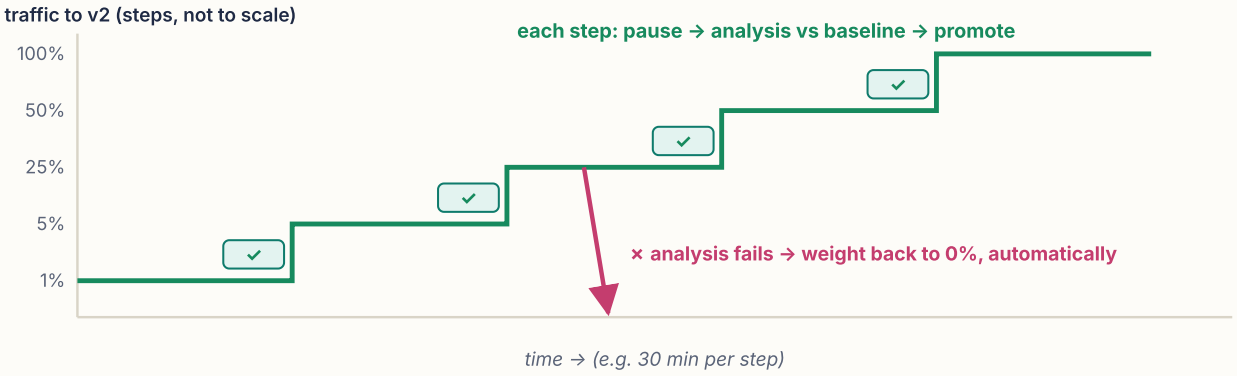


Figure 1. Progressive delivery: 1% → 5% → 25% → 50% → 100%, with a pause and an automated analysis at each step. A failed check snaps traffic back to stable.

Start at **1%**. If healthy, move to 5, 25, 50, then 100%. Each step is a chance to stop before the damage spreads. Weights are set in a router that can split by percentage: a service mesh (lesson 27), an ingress controller, or a cloud load balancer with weighted targets. Without one, Kubernetes can only approximate weights by replica counts (1 canary pod of 10 = 10%).

3 Is the canary healthy?

Don't eyeball dashboards. Compare the canary with the **stable baseline, at the same time, under the same traffic**, so time-of-day and traffic mix cancel out. Some setups (e.g. Spinnaker's Kayenta) start a fresh baseline of the old version at the same size as the canary, so both are equally new.

Signal	Example check	Why
Error rate	canary 5xx ratio < 1% and not worse than baseline	outright breakage
Latency	p99 < 300 ms, within 10% of baseline	slow code, N+1 queries
Saturation	CPU, memory, pool use per request	leaks, heavier code
Business	conversion, click-through vs baseline	"works" but worse for users

4 Automate the decision

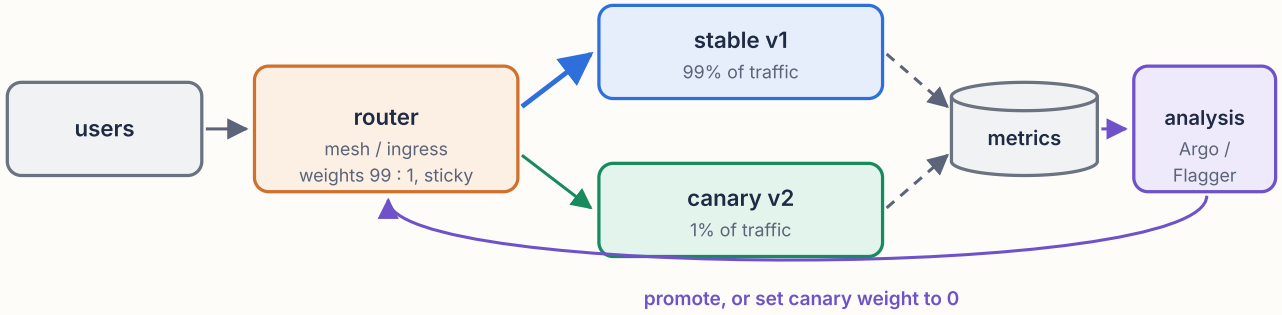


Figure 2. The router splits traffic; both versions report metrics; the analysis controller promotes or aborts. No human is needed for a rollback.

Argo Rollouts and **Flagger** run canaries on Kubernetes: you declare steps and success criteria, they shift traffic via a mesh or ingress and query Prometheus. Cloud platforms offer weighted routing too. Every step, metric and decision is logged, so you can see exactly why a rollout stopped. Rollback becomes a routine non-event, not an incident.

```
strategy:
  canary:
    steps:
    - setWeight: 1
    - pause: {duration: 30m}
    - setWeight: 5
    - pause: {duration: 30m}          # ... 25, 50, then full promotion
    analysis:                         # runs in the background during the steps
      templates: [{templateName: canary-health}]   # error rate < 1%, p99 < 300 ms, CTR vs baseline
```

Figure 3. An Argo Rollouts canary from the video: weights and pauses, with a background analysis that aborts the rollout when a metric fails.

PITFALLS

- **Too little traffic:** at 10 req/s, 30 minutes is ~18,000 requests; a rare error may not show up. Wait longer or use a bigger step.
- **Non-sticky routing:** one user bouncing between versions sees inconsistent results; hash on user ID or a cookie.
- **Compatibility:** both versions run together, so schemas and APIs must be backward compatible (lesson 73).
- **Only technical metrics:** a model can be fast and error-free yet worse for users; include business metrics.

WORKED EXAMPLE: A NEW RANKING MODEL TO 1% OF USERS

The router sends **1%** of search requests to the new model, sticky per user, and keeps 99% on the old one. Analysis compares error rate, p99 latency and click-through rate with the baseline. After **30 minutes**, latency and errors are fine, but **click-through is 3% lower**. The analysis fails, the rollout aborts automatically, and all traffic returns to the old model. Only 1% of users ever saw it, for half an hour; the team investigates offline.

5 Key terms

Canary: new version on a small slice of real traffic

Baseline: stable version measured at the same time for comparison

Progressive delivery: automated step-by-step rollout with analysis

Traffic weight: share of requests routed to a version

Blast radius: how many users a bad change can affect

Abort: automatic return of all traffic to stable

6 Interview questions

Q1. How do you decide a canary is healthy? BASIC

Compare it with the stable baseline running at the same time under the same traffic, on error rate, latency percentiles, saturation and business metrics, with enough traffic and automated pass/fail thresholds.

Q2. Why canary instead of blue-green? BASIC

It limits the blast radius to a small slice of users and catches problems only real traffic reveals; blue-green switches everyone at once.

Q3. A service gets 50 req/s. Is a 1% canary for 10 minutes enough? SCENARIO

No: that's about 300 requests, too few to detect a small error or latency change. Use a larger step (10–25%) or a longer pause.

Q4. Canary errors look slightly higher at 3 a.m. than the baseline was at noon. Problem? SCENARIO

Not a fair comparison: traffic mix differs by time. Compare canary and baseline over the same window.

Q5. Can you canary a database schema change? TRICKY

Not directly: the database is shared, so the schema change must be backward compatible and applied before (expand) or after (contract) the canary of the code.

Q6. Why can't Kubernetes alone do a 1% canary? TRICKY

A plain Service balances across pods, so weights follow replica counts; 1% needs 99 stable pods per canary pod, or a mesh/ingress that splits by percentage.

Next: SD-N-018 · Feature Flags

Feature Flags

deploy is code, release is config

Watch the 4-min video

In one line: feature flags separate deploy from release: code ships switched off and is turned on, targeted, experimented with or killed by a configuration change in seconds; the price is extra code paths and flag debt, so every flag needs an owner, an expiry date and audited changes.

1 The LLM provider is down

2 p.m., your LLM provider has an outage. The AI summary feature times out, and slow requests pile up across the whole app, exhausting threads and connections. You could hot-fix and redeploy: half an hour. Or flip one switch and turn the feature off in seconds.

2 Decoupling deploy from release

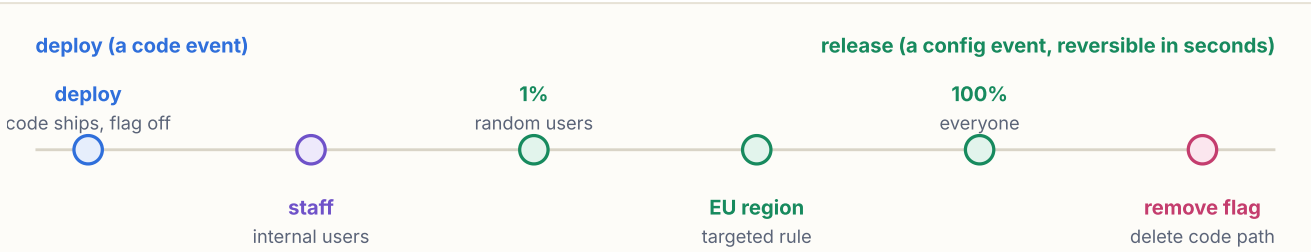


Figure 1. A feature's life behind a flag: deployed dark, released step by step by changing rules, then the flag is removed.

Code ships to production hidden behind a flag. Releasing becomes a configuration change, not a deployment, and rolling back is the same change in reverse. Teams can merge small changes to main every day (trunk-based development), even for unfinished features (lesson 72).

3 Four kinds of flag

Type	Purpose	Lifetime	Example
Release	hide unfinished or unreleased work	days to weeks	new checkout page
Ops / kill switch	turn off expensive or risky features under load or outage	long-lived	`ai_summary_enabled`
Experiment	split users for A/B tests	until the result is in	two ranking models
Permission	features per plan or customer	long-lived	SSO for enterprise plan

This classification follows Pete Hodgson's "Feature Toggles" article on martinfowler.com. Knowing the type tells you who owns it and when to delete it.

4 Targeting and evaluation

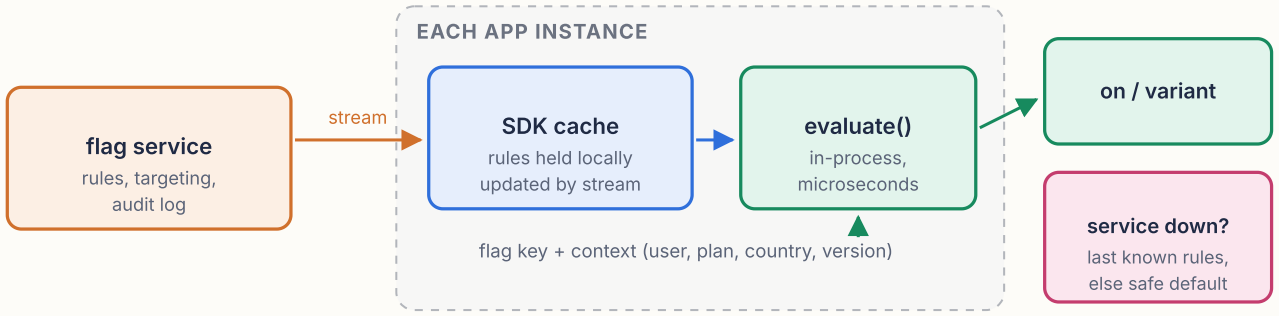


Figure 2. SDKs keep the rules locally and evaluate in-process, so a flag check costs microseconds and survives a flag-service outage.

Targeting decides who gets what: staff first, then 1% of users, then a region, then everyone; any attribute you pass (plan, country, app version) can be used. Percentage rollouts hash a stable key (user ID + flag key) into buckets, so a user stays in the same group. Evaluation must be fast and reliable: SDKs cache rules and update them in the background. If the flag service is unreachable, they use the last known rules, and on a cold start the **default you pass in code**, so choose a safe one.

5 Risks and discipline

- **Combinations:** n boolean flags make 2^n possible states; most were never tested together.
- **Flag debt:** old flags pile up until nobody knows which are safe to delete.
- **Flag changes are production changes:** a mistaken toggle causes outages just like a bad deploy.

So: every release flag gets an **owner and an expiry date** and is removed after full rollout; some teams fail the build when a flag is past expiry. Flag changes are **audited and reviewed** like code. Use a flag service (LaunchDarkly, Unleash, Azure App Configuration) behind the vendor-neutral **OpenFeature** API, rather than ad-hoc `if` statements on environment variables that need a redeploy to change.

```
client = openfeature.api.get_client()
def summary(doc):
    if not client.get_boolean_value("ai_summary_enabled", False):   # safe default: off
        return plain_text(doc)                                       # fallback
    return llm.summarize(doc, timeout=3)
```

Figure 3. The kill switch from the video, written against the OpenFeature API so the flag provider can be swapped.

WORKED EXAMPLE: A KILL SWITCH FOR AN LLM FEATURE

Every call to the summary feature first checks `ai_summary_enabled`. When the provider starts failing, the on-call engineer flips it off (it is in the runbook, lesson 67). Within seconds, every instance's SDK receives the change, stops calling the model and returns the plain text without a summary. Timeouts stop, threads free up, the rest of the app recovers. When the provider is back, the flag goes on again, with no deploy in either direction.

6 Key terms

Feature flag: runtime switch that changes behaviour without a deploy

Kill switch: ops flag that disables a feature fast

Targeting rule: who gets which variation, by attributes

Local evaluation: SDK evaluates cached rules in-process

Flag debt: stale flags and dead code paths left behind

OpenFeature: vendor-neutral feature-flag API (CNCF)

7 Interview questions

Q1. What are the risks of feature flags? BASIC

Untested flag combinations, growing flag debt, more complex code, and flag changes that can cause outages like bad deploys; so give flags owners, expiry dates and audited changes.

Q2. Why use feature flags at all? BASIC

They separate deploy from release, enable gradual rollouts and experiments, and give you instant kill switches.

Q3. A third-party API your feature depends on is down and dragging the app. What do you do? SCENARIO

Flip the feature's kill switch to its fallback, which stops the slow calls at once; combine with timeouts and circuit breakers (lessons 4, 6) so it degrades automatically next time.

Q4. The flag service goes down. What happens to your app? SCENARIO

Nothing visible if designed well: SDKs keep evaluating the last known rules locally; only new instances fall back to code defaults.

Q5. Feature flag or canary for a new version of a service? TRICKY

Canary for the whole binary and infrastructure-level risk; flags for specific features inside it, with per-user targeting and instant off. Many teams use both.

Q6. Why must percentage rollouts hash a stable key instead of random()? TRICKY

Random per request would flip a user between variations on every call; hashing user ID + flag key keeps each user consistently in or out.

Next: Phase 6 · Advanced and Specialized Systems backendroadmap.com